Information-theoretic MPC with Constant Communication Overhead

Ashish Choudhury[1], Ivan Damgård[2], Shravani Patil[3], and Arpita Patra[3]

[1] IIIT Bangalore, India;
[2] Aarhus University, Denmark;
[3] IISc Bangalore, India

Abstract. We study the feasibility of constant communication-overhead information-theoretic Multiparty Computation tolerating malicious adversaries in both the synchronous and asynchronous network settings. A simple observation based on the results of Damgård, Larsen and Neilsen (CRYPTO 2019) allows us to conclude that (a) any statistically-secure MPC with $n = 2t + s$, $s > 0$ requires a communication complexity of $\Theta(\frac{n|C|}{s})$ bits in the synchronous setting; (b) any statistically-secure MPC with $n = 3t + s$, $s > 0$ requires a communication complexity of $\Theta(\frac{n|C|}{s})$ bits in the asynchronous setting; where C denotes the circuit, n denotes the number of parties and t denotes the number of corruptions. Both the results hold even for abort security.

To match the above bounds for $s = \Theta(t)$, we propose two protocols in the synchronous and respectively in the asynchronous setting, both with guaranteed output delivery and a communication complexity of $\mathcal{O}(|C|\kappa)$ bits, for a statistical security parameter κ, a running time of $\mathcal{O}(D)$ for circuit depth D (in expectancy in the asynchronous setting). For this, we require circuits with a SIMD structure. Our constructs are the first constant-overhead statistically-secure MPC protocols with optimal resilience.

Our first technical contributions is a verifiable secret sharing (VSS) protocol with constant-overhead per secret through two-dimensional packing. A second contribution is a degree-reduction (from a higher degree packed sharing to a lower degree packed sharing) protocol with constant overhead.

As a bonus, a simplified version of our statistical MPC protocols, plugged with the VSSs derived from the earlier works of Abraham, Asharov, Patil and Patra (EUROCRYPT 2023) and Choudhury and Patra (IEEE Transactions on Information Theory 2016) allow us to obtain perfectly-secure MPC protocols with a communication cost of $\mathcal{O}(|C|\kappa)$ bits, where $\kappa = \Theta(\log(n))$, and a running time of $\mathcal{O}(D)$ with the resiliency of $n = 3t + \Theta(t)$ in synchronous setting and $n = 4t + \Theta(t)$ in the asynchronous setting. These resiliency are optimal in perfect setting due to the lower bounds of Damgård, Patil, Patra, Roy (Eprint 2025) even for abort security.

Table of Contents

II Asynchronous Setting

1 Introduction

The communication complexity of interactive protocols is one of their most important efficiency measures. Consequently, a huge amount of research has been devoted to characterizing the communication complexity of various distributed tasks (e.g., Byzantine agreement [36,15], and general secure multiparty computation [45,46,39,22,38,35,29,28,26,32]) under different security models.

In this work, we focus on the communication complexity of protocols for multiparty computation that achieve information-theoretic security with negligible error probability (aka. statistical security) in the presence of an active (aka Byzantine or malicious), adaptive, computationally unbounded, rushing adversary. We assume that there are n parties that communicate over *secure point-to-point channels*, and also that they have access to a *broadcast* channel. It is known that the resiliency for perfect security (information-theoretic security with no error probability) is $t < n/3$ [13,19] and $t < n/2$ for statistical security [50].

For protocols computing an arithmetic circuit C of size $|C|$, it was recently shown in [30] that (a) perfectly-secure MPC with $n = 3t + s$, $s > 0$ requires communication complexity $\Theta(\frac{n|C|}{s})$ in the synchronous setting; (b) perfectly-secure MPC with $n = 4t+s$, $s > 0$ requires communication complexity of $\Theta(\frac{n|C|}{s})$ in the asynchronous setting. Both results hold, even for abort security. So, the communication required per gate is $\Theta(n/s)$, and we refer to this as the *overhead*.

It follows that for the minimal value of $s = 1$, the overhead is linear (in the number of parties) for these cases. The protocols of [12,40,1] in synchronous networks and of [4] in the asynchronous setting match this lower bound. When $s = \Theta(t)$, the optimal overhead is constant. We note that in this case, the weak secret sharing protocol of [1] in the synchronous setting and [4] in the asynchronous setting become verifiable secret sharing (VSS) protocols with constant overhead with respectively $3t + \Theta(t)$ and $4t + \Theta(t)$. It is easy to derive MPC protocols from the derived VSS protocols and get the following theorems (we cover these in the Appendix B).

Theorem 1.1 (informal; synchronous)**.** *In the synchronous setting, there exists a perfectly-secure MPC protocol with $n = 3t + \Theta(t)$ and $\mathcal{O}(|D|)$ expected number of rounds with a communication complexity of $\mathcal{O}(|C|)$ field elements for evaluating C in an SIMD fashion over a batch of input size n^6.*

Theorem 1.2 (informal; asynchronous)**.** *In the asynchronous setting, there exists a perfectly-secure MPC protocol with $n = 4t + \Theta(t)$ and $\mathcal{O}(|D|)$ expected number of rounds with a communication complexity of $\mathcal{O}(|C|)$ field elements for evaluating C in an SIMD fashion over a batch of input size n^6.*

These observations almost settle the communication complexity status for perfect security[4]. The goal of this work is to settle the corresponding questions for the communication complexity of statistical MPC protocols. Towards this, we first note that [26] shows the following.

Theorem 1.3 (informal)**.** *Let $n = 2t+s$ with $s > 0$. There exists a function f with circuit complexity $|C|$ such that in any protocol computing f with statistical security against t passive corruptions, the average total communication complexity is at least $\Theta(t|C|/s)$ bits.*

This indicates that the overhead must be $\Theta(n)$ for $s = 1$.[5] For this case, statistical MPC protocols with linear overhead are presented in [14,43] for the synchronous setting. So it is very natural to ask:

> can we have constant-overhead statistical MPC in the synchronous setting with $n = 2t + s$ for $s = \Theta(t)$? What are the communication complexity bounds for asynchronous networks?

Indeed, since the major communication complexity questions for the perfect security settings are resolved, the next goal would be to complete our understanding of the statistical security.

[4] almost, since intermediate cases like $s = \sqrt{t}$ are not covered

[5] To be precise, the bound in [26] is actually in terms of the size of the input to f. However, since f has linear size circuits, we get the result cited. Moreover, since all known MPC constructions do "the same" for any circuit, we expect the overhead to be $\Theta(t/s)$ in general

1.1 Our Results

We present answers to the above questions by presenting a lower bound and two upper bounds. The latter are the first statistical MPC protocols with constant communication overhead with corresponding optimal resiliency, guaranteed output delivery and a run-time that is linear in circuit depth.

Lower Bounds. From Theorem 1.3 recalled above, a simple set of observations lead us to the following corollary, stating that the bound from [26] also holds for security with abort and for asynchronous protocols.

Corollary 1.4 (informal; synchronous and asynchronous)**.** *Let $n = 2t + s$. There exists a function f with circuit complexity $|C|$ such that in any protocol running on a synchronous network computing f with statistical abort security against t active corruptions, the average total communication complexity is at least $\Theta(t|C|/s)$ bits.*

Let $n = 3t+s$. There exists a function f with circuit complexity $|C|$ such that in any protocol running on an asynchronous network computing f with statistical abort security against t active corruptions, the average total communication complexity is at least $\Theta(t|C|/s)$ bits.

Verifiable Secret Sharing. Towards designing our general MPC in the synchronous and asynchronous setting, we first construct a verifiable secret sharing (VSS) protocol in each setting with constant-overhead per secret.

Verifiable secret sharing (VSS) [21] is arguably the most basic primitive in information-theoretic MPC and it is known to be necessary for the construction of general MPC protocols both in the perfect setting (see [8]), and in the statistical setting (see [7]). At a high level, a VSS scheme consists of two phases: a sharing phase, which allows a dealer to share one or more secrets among the parties, and a reconstruction phase, which allows the parties to recover the secret(s). For an honest dealer, VSS has the same guarantees as robust secret sharing, that is, the adversary learns no information about the secrets in the sharing phase (privacy), and the secrets will always be reconstructed properly in the reconstruction phase despite the misbehavior of the adversary (correctness). In addition, for a corrupt dealer, we require commitment, which means that at the end of the sharing phase there are some values that will always be reconstructed in the reconstruction phase.

In our case VSSs will generate multiple packed secret sharings with degree $t+\Theta(t)$ and therefore will hide $\Theta(t)$ secrets. This way, we achieve $\mathcal{O}(1)$ overhead per secret. We obtain the following two results in synchronous and asynchronous setting.

Theorem 1.5 (informal; synchronous)**.** *In the synchronous setting, there exists a constant expected round VSS with $n = 2t + \Theta(t)$ parties and a communication complexity of $\mathcal{O}(1)$ field elements per secret from a field $\mathbb{F}$ with $|\mathbb{F}| > 2^\kappa$ where κ is the statistical security parameter, for a minimum secret size of n^7 field elements.*

Theorem 1.6 (informal; asynchronous)**.** *In the asynchronous setting, there exists a constant expected round [6] VSS with $n = 3t + \Theta(t)$ parties and with a communication complexity of $\mathcal{O}(1)$ field elements per secret from a field $\mathbb{F}$ with $|\mathbb{F}| > 2^\kappa$ where κ is statistical security parameter for a minimum secret size of n^5 field elements.*

These are the first constant-overhead statistical VSSs with the above resiliencies, irrespective of the run-time. As an important additional feature, both of them run in *constant expected rounds*.

In the synchronous setting, our VSS outputs authenticated packed sharings within the same cost budget to allow for robust reconstruction of (possibly linearly-combined) secrets at a later point of time. At a high level, we need authentication because we have $2t + s$ (where $s = \Theta(t)$) parties, but the degree of sharing is $t + s$ to enable packing. Hence, for reconstruction we cannot rely on error correction since the distance of the code reduces to t, which permits error correction of only $t/2$ errors. Therefore, our VSS outputs authenticated packed secret sharings, wherein each share of a party is authenticated with a tag with respect to every other party's key. However, this blows up the communication overhead to $\Theta(n)$. So, we allow computation of a single tag for many shares. This means that the packed secret sharings are

[6] For asynchronous protocols, rounds is not a well-defined notion, what we mean here is that parties need to react to a constant number of events before they are done.

batched and the authentication tags as computed as before but on a batch of shares instead of a single value. This amortizes the cost of tags and we are back to constant-overhead.

General MPC. Utilising our VSS protocols, we arrive at the following results on general MPC in the synchronous and asynchronous settings. Our goal is to securely compute an arithmetic circuit C of size $|C|$. We need to assume that C has a SIMD structure, i.e., it computes the same function an appropriate number of times in parallel (as explained in detail later). Note, however, that using a technique from [25], one can compile any circuit into a new one computing the same function such that the compiled circuit does have the required SIMD structure. This comes at the cost of logarithmic factor overhead in circuit size.

Theorem 1.7 (informal; synchronous)**.** *In the synchronous setting, there exists a statistically-secure MPC protocol with $n = 2t + \Theta(t)$ parties for evaluating any SIMD circuit C with depth D over a batch of input size n^{11}, requiring $\mathcal{O}(D)$ expected number of rounds and a communication complexity of $\mathcal{O}(|C|)$ field elements.*

Theorem 1.8 (informal; asynchronous)**.** *In the asynchronous setting, there exists a statistically-secure MPC protocol with $n = 3t + \Theta(t)$ parties for evaluating any SIMD circuit C with depth D over a batch of input size n^7, requiring $\mathcal{O}(D)$ expected number of rounds and a communication complexity of $\mathcal{O}(|C|)$ field elements.*

In both cases, the VSS is used to generate verifiable triples of shared tuples with a built-in multiplicative relation, which form the primary building block to generate the packed Beaver triples. These are in turn used to evaluate the multiplication gates. Since our VSSs are constant-overhead, so are the protocols for generating packed Beaver triples and the online evaluation of multiplication gates via Beaver triples. Both the above rely on a building block called degree-reduction that allows a bunch of packed secret sharings, shared with a degree say d_1, to be reshared with a lower degree d_2 (i.e. $d_1 > d_2$). [30] demonstrated a method for this, in which the protocol reconstructs towards each party a different linear combination of the set of input packed sharings (of degree d_1). Each party then reshares what she received with degree d_2. This is the "distributed" way of achieving the task, which in earlier work was done by reconstructing to a single party P_{king} [52]. The advantage of the distributed approach is that it works both on synchronous and asynchronous networks and it is easy to make it tolerant against t corrupt parties who may reshare wrong values (via Reed-Solomon error correction).

In our setting, we could in principle replace the error correction by the authentication tags and still do distributed degree reduction. Unfortunately, however, this will not work in general: during the multiplication, we need to multiply (coordinate-wise) two packed secret-shared vectors, say $\mathbf{a}, \mathbf{b}$, and then do degree reduction. One of these vectors does not need to be private, but must be secret-shared for communication efficiency. Now, while shares of $\mathbf{a} \cdot \mathbf{b}$ can be computed by locally multiplying shares, the authentication tags do not support such non-linear operations. To overcome this issue, we drop the tags and do the multiplication and degree reduction over the non-authenticated versions of the packed secret sharings. This makes the reconstruction non-robust and brings other challenges. We solve these robustly without requiring any reruns. We refer to the technical overview section of our paper to see how we resolve the issues and build an MPC protocol.

Our results put in the context of relevant existing results are presented in Table 1.

1.2 Related Works

The communication complexity had remained at the center of interest in the MPC space for a long time. In the regime of perfect security, since the appearance of [13], a series of works improved the communication complexity of MPC protocols calumniating to the linear complexity [12,41,3]. The work of [41] made the complexity linear even in the depth of circuit. The work of [3] achieves a round complexity of $\mathcal{O}(D)$, whereas the other two works requires at least $\mathcal{O}(D + n)$ round complexity, which may be expensive for large n. Over the asynchronous setting, [49] and respectively [4] made the complexity quadratic and linear. The communication complexity lower bound of $\Theta(n|C|/s)$ is proven in [33] for any $n = 3t + s$.

Moving on to the statistical setting, again a series of works finally led to the linear-overhead protocols in [14,43] in the synchronous setting. In the asynchronous setting, the linear VSS protocol of [44] along with the results of [42] presents the linear-overhead protocol.

Setting	Network	Resilience	LB/UB?	Complexity	Security	Reference
Perfect	Synchronous	$n = 3t + s$	LB	$\frac{n\|C\|}{s}$	Malicious, Abort	[31]
		$n = 3t + 1$	UB	$\mathbf{O}(n)$	Malicious, GOD	[1,52]
		$n = 3t + \Theta(t)$	UB	$\mathbf{O}(1)$	Malicious, GOD	This work
	Asynchronous	$n = 4t + s$	LB	$\frac{n\|C\|}{s}$	Malicious, Abort	[31]
		$n = 4t + 1$	UB	$\mathbf{O}(n)$	Malicious, GOD	[4]
		$n = 4t + \Theta(t)$	UB	$\mathbf{O}(1)$	Malicious, GOD	This work
Statistical	Synchronous	$n = 2t + s$	LB	$\frac{n\|C\|}{s}$	Semi-honest	[26]
		$n = 2t + 1$	UB	$\mathbf{O}(n)$	Malicious, GOD	[14,43]
		$n = 2t + \Theta(t)$	UB	$\mathbf{O}(1)$	Malicious, GOD	This work
	Asynchronous	$n = 3t + s$	LB	$\frac{n\|C\|}{s}$	Malicious, Abort	This work
		$n = 3t + 1$	UB	$\mathbf{O}(n)$	Malicious, GOD	[42,44]
		$n = 3t + \Theta(t)$	UB	$\mathbf{O}(1)$	Malicious, GOD	This work

Table 1: Lower and Upper bounds; $s > 0$, C denotes the circuit to be computed. LB denotes lower bound and UB denotes upper bound. The lower bounds are derived for linear (in input) sized circuits. The complexity is in terms of field elements where the field is over which the circuit is defined for lower bounds. The field can be of size $n+t$ for the perfect upper bounds and of size 2^κ, for a statistical security parameter κ, for the statistical upper bounds.

2 Detailed Technical Overview

In this section, we give a detailed technical overview of our protocols. We follow the standard paradigm of designing secret-sharing based MPC in the preprocessing model [11], where parties generate what is known as correlated randomness in a preprocessing phase, which is independent of the circuit to be securely evaluated. The correlated randomness is later utilized to efficiently evaluate the given circuit. Traditionally, in this approach, the entire circuit is evaluated exactly once. We envision a scenario where the same circuit needs to be evaluated in parallel multiple times, in an SIMD fashion, with a hope to achieve $\mathcal{O}(1)$ communication overhead per gate. More specifically, the parties need to evaluate the circuit p number of times in parallel, where p is *packing parameter* and satisfies the following constraints:[7]

- $p = \Theta(n)$;
- $2(t + p) - 1 \leq n$

The idea is then to use packed-secret-sharing (PSS), an idea introduced in [39], where instead of a single value, p number of values are secret-shared through a single sharing polynomial. Namely, let $\mathbf{s} = (s^{(1)}, \ldots, s^{(p)}) \in \mathbb{F}^p$ be a vector of p elements from $\mathbb{F}$. Then $\mathbf{s}$ is said to be degree-ℓ packed-secret-shared where $\ell \geq p - 1$, if there exists a degree-ℓ polynomial $f(X)$ over $\mathbb{F}$ with $f(-k) = s^{(k)}$ for each $k \in \{1, \ldots, p\}$ and each $P_i \in \mathcal{P}$ has a share $s_i = f(i)$ for $\mathbf{s}$. The notation $\langle \mathbf{s} \rangle_\ell$ denotes a degree-ℓ packed-secret-sharing (PSS) of $\mathbf{s}$. Note that like the traditional Shamir's secret-sharing, PSS also enjoys the linearity property. Namely, given vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}^p$ and publicly-known constants c_1 and c_2, the condition $c_1 \cdot \langle \mathbf{a} \rangle_d + c_2 \cdot \langle \mathbf{b} \rangle_d = \langle c_1 \cdot \mathbf{a} + c_2 \cdot \mathbf{b} \rangle_d$ holds; here $c_1 \cdot \mathbf{a}$ denotes the vector $\mathbf{a}$ with each component being multiplied by c_1. The interesting fact about PSS is that each party gets a *single* element from $\mathbb{F}$ as its share for the *entire* vector $\mathbf{s}$ consisting of $\Theta(n)$ elements. So overall the "overhead" here is $\mathcal{O}(1)$ as n parties gets n elements as shares for sharing $\Theta(n)$ elements (this explains why we need $p = \Theta(n)$). Compare this with the standard Shamir's secret-sharing [51], where *only* a single value is shared through a polynomial and the overhead is $\mathcal{O}(n)$. It is easy to see that if $\ell = t + p - 1$ and if $f(X)$ is chosen randomly (with the constraints that $f(-k) = s^{(k)}$), then $\mathbf{s}$ remains random for the adversary. So instead of evaluating the circuit *once* using Shamir's secret-sharing as done in the standard secret-sharing based MPC protocols [27,12], one can evaluate the circuit p number of times in parallel in an SIMD fashion over PSS. The idea has been deployed in several of the recent works which try to design MPC protocols for SIMD circuit based on PSS, with a hope to achieve $\mathcal{O}(1)$ communication overhead per gate [37,53].

[7] The reasons for these constraints will be clear soon.

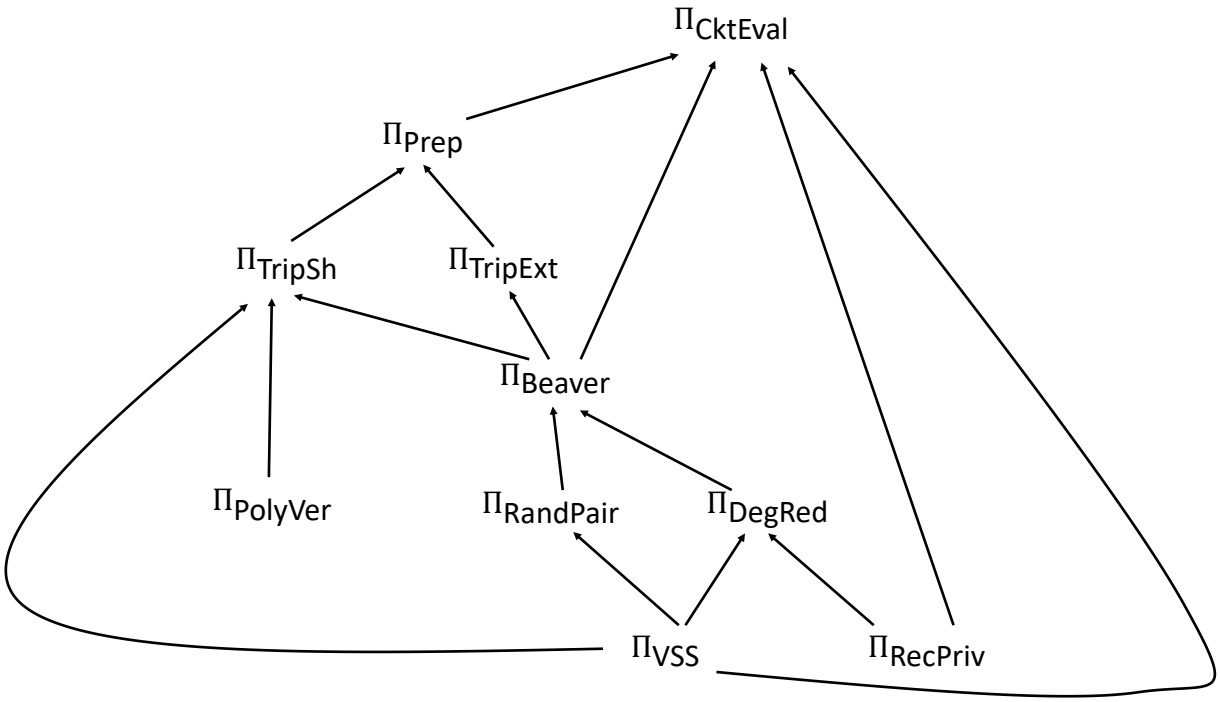

Fig. 1: Pictorial illustration of the the generic blueprint depicting the various building blocks and how they are connected to arrive at our MPC protocol

A Blueprint for the construction of our protocol.

We next outline the generic blueprint we follow to design secret-sharing based MPC for SIMD circuits with $\mathcal{O}(1)$ overhead. The blueprint is illustrated in Fig 1, showing the subprotocols we use and how they are connected to arrive at the MPC (circuit evaluation) protocol on top.

In the following, we start from the top of the diagram in Fig 1 and show how the circuit evaluation protocol can be built from the components immediately below it, and we then move down through the diagram until we reach the $\Pi_{\mathsf{VSS}}, \Pi_{\mathsf{RecPriv}}$ and Π_{PolyVer} protocols. The conclusion is that if one is given instantiations of these protocols with $\mathcal{O}(1)$ communication overhead, one can construct all the other building blocks. Then, in the final part of the technical overview, we explain how to instantiate $\Pi_{\mathsf{VSS}}, \Pi_{\mathsf{RecPriv}}$ and Π_{PolyVer}.

We would like to point out that that the blueprint or parts of it has been used earlier in some of the existing works on MPC based on PSS [37,6,53,30]. However, none of these works have provided instantiation of *all* the building blocks with $\mathcal{O}(1)$ overhead. Moreover, some of these works present *non-robust* instantiation of the building blocks in the dispute control/player elimination framework, resulting in more number of rounds. We instantiate all the underlying building blocks robustly with constant communication overhead.

Constant Overhead MPC from Preprocessing, VSS, Reconstruction and Beaver.

Let $d \stackrel{\mathsf{def}}{=} t + p - 1$. Assume we have the following building blocks to handle vectors of p elements which are packed-secret-shared:

- **A preprocessing phase protocol for PSS with $\mathcal{O}(1)$ overhead.** Protocol Π_{Prep} which generates "sufficiently many" PSS $(\langle \mathbf{a} \rangle_d, \langle \mathbf{b} \rangle_d, \langle \mathbf{c} \rangle_d)$ of vectors of p random multiplication-triples, where $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{F}^p$ are random for the adversary and where $\mathbf{c} = \mathbf{a} \times \mathbf{b}$ component-wise. Moreover, the communication overhead of the protocol is $\mathcal{O}(1)$ per vector of multiplication-triples.
- **A VSS protocol for PSS with $\mathcal{O}(1)$ overhead.** Protocol Π_{VSS}, which lets any designated dealer $D \in \mathcal{P}$ to verifiably generate degree-ℓ PSS of "sufficiently many" vectors of p elements each held by D with a communication overhead of $\mathcal{O}(1)$, where $\ell \in \{p-1, d, d+p-1\}$. The protocol guarantees that if D is *honest* and if $\ell \in \{d, d+p-1\}$, then its vectors remain private while even if D is *corrupt*,

there exist well-defined vectors held by D which such that the parties hold PSS of those vectors with the aforementioned degrees.[8]

- **A reconstruction protocol for PSS with $\mathcal{O}(1)$ overhead.** Protocol Π_{RecPriv} which lets any designated party $P_R \in \mathcal{P}$ to reconstruct $\mathbf{s} \in \mathbb{F}^p$, given that the parties hold $\langle \mathbf{s} \rangle_\ell$ where $\ell \in \{p-1, d, d+p-1\}$, incurring a communication overhead of $\mathcal{O}(1)$.[9] Note that to reconstruct a degree-$d+p-1$ polynomial, we need $d+p$ shares. As the corrupt parties may decide to not provide their shares for reconstruction, this automatically implies that $n - t \geq d + p$ should hold, implying that $2(t+p) - 1 \leq n$ should hold.
- **Beaver's multiplication protocol for PSS with $\mathcal{O}(1)$ overhead.** Protocol Π_{Beaver} which takes input "sufficiently many" $\langle \boldsymbol{x} \rangle_d$ and $\langle \boldsymbol{y} \rangle_d$, along with packed-secret-shared triples $(\langle \mathbf{a} \rangle_d, \langle \mathbf{b} \rangle_d, \langle \mathbf{c} \rangle_d)$ and computes $\langle \boldsymbol{z} \rangle_d$ with a communication overhead of $\mathcal{O}(1)$. It would be guaranteed that $\boldsymbol{z} = \boldsymbol{x} \times \boldsymbol{y}$ component-wise if and only if $\mathbf{c} = \mathbf{a} \times \mathbf{b}$ component-wise. Moreover, $(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z})$ remains random for the adversary provided $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ remains random for the adversary.

Given the above building blocks, one can easily design an MPC protocol Π_{ckteval} for SIMD circuits with $\mathcal{O}(1)$ communication overhead as follows.[10] The idea will be to evaluate the whole ciruit over vectors of inputs of size p using PSS where each vector remains degree-d packed-secret-shared. Namely, the parties first run the protocol Π_{Prep} to generate sufficiently many degree-d PSS of vectors of random multiplication-triples. Each party then generates degree-d PSS of its vectors of inputs for the circuit using protocol Π_{VSS}. After this the parties can evaluate each gate in the circuit over vectors of values in parallel by maintaining the folloing invariant: given degree-d PSS of vectors of gate-inputs, the parties securely compute degree-d PSS of vectors of gate-outputs. Maintaining the invariant will be non-interactive for addition gates in the circuit, thanks to the lineairty property of PSS. For multiplication-gates, the parties can run the protocol Π_{Beaver} by deploying the packed-secret-shared vectors of multiplication-triples generated via Π_{Prep}. Once the output gate is evaluated, the parties can reconstruct the vectors of circuit-outputs by running instances of Π_{RecPriv}. If each of the protocols $\Pi_{\mathsf{Prep}}, \Pi_{\mathsf{VSS}}, \Pi_{\mathsf{RecPriv}}$ and Π_{Beaver} maintains $\mathcal{O}(1)$ communication overhead, then the MPC protocol will also achieve $\mathcal{O}(1)$ communication overhead. The security follows since every vector remains degree-d packed-secret-shared where adversary learns up to t shares for each PSS, where $d = t + p - 1$. Note that the above blueprint is just an extension of the standard secret-sharing based MPC for vectors of values which are degree-d packed-secret-shared.

We next outline the generic blueprint which we follow to design the above building blocks, given the VSS protocol Π_{VSS} and the reconstruction protocol Π_{RecPriv}. We start with protocol Π_{Prep}.

Preprocessing with $\mathcal{O}(1)$ Overhead from Triple Sharing and Extraction.

The work of [23] have shown how to generate t-shared random multiplication-triples generically, using *any* given VSS protocol which generates t-sharing of underlying values and Beaver's multiplication-protocol for multiplying t-shared values. To design protocol Π_{Prep} for generating PSS of vectors of multiplication-triples, our approach is to extend the framework of [23] for PSS. Namely, we rely on the following components for designing Π_{Prep}.

- **A verifiable triple-sharing protocol for PSS with $\mathcal{O}(1)$ overhead.** Protocol Π_{TripSh} allows a designated dealer $D \in \mathcal{P}$ to verifiably generate sufficiently many PSS $(\langle \mathbf{a} \rangle_d, \langle \mathbf{b} \rangle_d, \langle \mathbf{c} \rangle_d)$ of vectors of p multiplication-triples held by D with a communication overhead of $\mathcal{O}(1)$, where $\mathbf{c} = \mathbf{a} \times \mathbf{b}$ component-wise. The protocol guarantees that if D is *honest* then its multiplication-triples remain random for the adversary. The verifiability here guarantees that even if D is *corrupt*, it has generated PSS of vectors of multiplication-triples held by D.
- **A triple-extraction protocol for PSS with $\mathcal{O}(1)$ overhead.** Protocol Π_{TripExt} takes PSS of vectors of random and potentially non-random vectors of multiplication-triples and outputs PSS

[8] Looking ahead, in our protocols we will encounter situations where parties need to verifiably generate PSS of their vectors of inputs with degree being either $p-1, d$ or $d+p-1$. If the degree is $p-1$ then *no* privacy is needed for the underlying values, as adversary may get up to t shares for the underlying degree-$(p-1)$ sharing-polynomials. The privacy will be needed only when the degree of sharing is d or $d+p-1$.

[9] We stress that protocol Π_{RecPriv} may allow robust reconstruction even for other possible degrees of sharing; for our purpose we need robust reconstruction for the said degrees.

[10] The exact input size for the SIMD circuit will depend upon the number of vectors for which protocols $\Pi_{\mathsf{Prep}}, \Pi_{\mathsf{VSS}}$ and Π_{Beaver} achieve $\mathcal{O}(1)$ communication overhead. We do not go into the details now for the current discussion.

of vectors of multiplication-triples which will be random for the adversary, such that the communication overhead of the protocol is $\mathcal{O}(1)$. In more detail, each party $P_i \in \mathcal{P}$ will provide as input sufficiently many PSS $(\langle \mathbf{a}^{(i)} \rangle_d, \langle \mathbf{b}^{(i)} \rangle_d, \langle \mathbf{c}^{(i)} \rangle_d)$ of vectors of p multiplication-triples where $\mathbf{c}^{(i)} = \mathbf{a}^{(i)} \times \mathbf{b}^{(i)}$ component-wise. Moreover, it would be guaranteed that if P_i is *honest*, then $(\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)})$ remains random for the adversary. Let $m \stackrel{\mathsf{def}}{=} \frac{n-1}{2} + 1 - t$. The protocol outputs m PSS $(\langle \boldsymbol{x}^{(j)} \rangle_d, \langle \boldsymbol{y}^{(j)} \rangle_d, \langle \boldsymbol{z}^{(j)} \rangle)$ of vectors of p multiplication-triples where $\boldsymbol{z}^{(j)} = \boldsymbol{x}^{(j)} \times \boldsymbol{y}^{(j)}$ component-wise and where the multiplication-triples $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}, \boldsymbol{z}^{(i)})$ remains random for the adversary for each $j \in \{1, \ldots, m\}$. The protocol guarantees that if $m = \Theta(n)$, then the communication overhead of the protocol is $\mathcal{O}(1)$.

Given the above two building-blocks, one can design protocol Π_{Prep} very easily as follows. Each $P_i \in \mathcal{P}$ picks (sufficiently many) vectors $(\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)})$ of random multiplication-triples and generates $(\langle \mathbf{a}^{(i)} \rangle_d, \langle \mathbf{b}^{(i)} \rangle_d, \langle \mathbf{c}^{(i)} \rangle_d)$ using protocol Π_{TripSh}. This is followed by the parties running protocol Π_{TripExt} on the vectors of multiplication-triples shared by the different parties and extracting m PSS of vectors of truly random multiplication-triples, where $m = \frac{n-1}{2} + 1 - t$. If $m = \Theta(n)$ (which will be guaranteed for our case), then the communication overhead will be $\mathcal{O}(1)$. Depending upon the number of multiplication-triples needed for the circuit-evaluation, the parties can run the above steps in parallel the required number of times.

We next outline the generic blueprint which we follow to design protocols Π_{TripSh} and Π_{TripExt} with $\mathcal{O}(1)$ communication overhead.

Verifiable Triple-sharing with $\mathcal{O}(1)$ Overhead from Polynomial-Verification.

The work of [23] have shown how to verifiably generate t-shared multiplication-triples held by a dealer generically, using *any* given VSS protocol which generates t-sharing of underlying values, Beaver's multiplication-protocol for multiplying t-shared values and a protocol to verify the "multiplicative relationship" among a triplet of secret-shared polynomials. We extend the above approach for PSS. Namely, we rely on the following component for designing Π_{TripSh}.

- **A polynomial-verification for vectors of polynomials with $\mathcal{O}(1)$ overhead.** Protocol Π_{PolyVer} takes input sufficiently many vectors $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ of p number of polynomials of degree ℓ_1, ℓ_1 and ℓ_2 respectively, where $\ell_1 \geq t$. That is, each $\boldsymbol{X}(\cdot) = (f^{(1)}(\cdot), \ldots, f^{(p)}(\cdot))$, each $\boldsymbol{Y}(\cdot) = (g^{(1)}(\cdot), \ldots, g^{(p)}(\cdot))$ and each $\boldsymbol{Z}(\cdot) = (h^{(1)}(\cdot), \ldots, h^{(p)}(\cdot))$, where each $f^{(k)}(\cdot), g^{(k)}(\cdot)$ and $h^{(k)}(\cdot)$ polynomial is of degree-ℓ_1, ℓ_1 and degree-ℓ_2 respectively. Let $\boldsymbol{X}(j), \boldsymbol{Y}(j)$ and $\boldsymbol{Z}(j)$ denote the vectors of j^{th} points on the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ respectively. That is, $\boldsymbol{X}(j) = (f^{(1)}(j), \ldots, f^{(p)}(j))$, $\boldsymbol{Y}(j) = (g^{(1)}(j), \ldots, g^{(p)}(j))$ and $\boldsymbol{Z}(j) = (h^{(1)}(j), \ldots, h^{(p)}(j))$. It would be guaranteed that the parties hold PSS $\langle \boldsymbol{X}(j) \rangle_d, \langle \boldsymbol{Y}(j) \rangle_d$ and $\langle \boldsymbol{Y}(j) \rangle_d$, for each $j \in \{1, \ldots, n\}$. The goal of the protocol is to verify if component-wise, $\boldsymbol{X}(\cdot) \cdot \boldsymbol{Y}(\cdot) = \boldsymbol{Z}(\cdot)$ holds. That is, we want to verify if $f^{(k)}(\cdot) \cdot g^{(k)}(\cdot) = h^{(k)}(\cdot)$ holds, for each $k \in \{1, \ldots, p\}$. Protocol Π_{PolyVer} achieves this by revealing at most t additional points on the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$, with a communication overhead of $\mathcal{O}(1)$.

Given protocols $\Pi_{\mathsf{VSS}}, \Pi_{\mathsf{PolyVer}}$ and Π_{Beaver}, we can design Π_{TripSh} as follows: D picks n vectors of random multiplication-triples $(\mathbf{u}^{(i)}, \mathbf{v}^{(i)}, \mathbf{w}^{(i)})$ and generates $(\langle \mathbf{u}^{(i)} \rangle_d, \langle \mathbf{v}^{(i)} \rangle_d, \langle \mathbf{w}^{(i)} \rangle_d)$ using protocol Π_{VSS}. To verify if D has secret-shared multiplication-triples, the parties "transform" the secret-shared triples $(\langle \mathbf{u}^{(i)} \rangle_d, \langle \mathbf{v}^{(i)} \rangle_d, \langle \mathbf{w}^{(i)} \rangle_d)$ into "correlated" secret-shared triples $(\langle \boldsymbol{x}^{(i)} \rangle_d, \langle \boldsymbol{y}^{(i)} \rangle_d, \langle \boldsymbol{z}^{(i)} \rangle_d)$ such that the following hold:

- There exist vectors of polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ of degree-$\frac{n-1}{2}$, $\frac{n-1}{2}$ and degree-$(n-1)$ respectively, such that $\boldsymbol{X}(i) = \boldsymbol{x}^{(i)}, \boldsymbol{Y}(i) = \boldsymbol{y}^{(i)}$ and $\boldsymbol{Z}(i) = \boldsymbol{z}^{(i)}$ for each $i \in \{1, \ldots, n\}$.
- The vectors of polynomials $(\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot))$ satisfy the multiplicative-relationship $\boldsymbol{Z}(\cdot) = \boldsymbol{X}(\cdot) \cdot \boldsymbol{Y}(\cdot)$ if and only if all the input triples $(\mathbf{u}^{(i)}, \mathbf{v}^{(i)}, \mathbf{w}^{(i)})$ are multiplication-triples.

The transformation works as follows: the first $\frac{n-1}{2} + 1$ transformed triples are set to be the first $\frac{n-1}{2} + 1$ input triples. That is, for each $i \in \{1, \ldots, \frac{n-1}{2} + 1\}$, the parties set $\langle \boldsymbol{x}^{(i)} \rangle_d = \langle \mathbf{u}^{(i)} \rangle_d$, $\langle \boldsymbol{y}^{(i)} \rangle_d = \langle \mathbf{v}^{(i)} \rangle_d$ and $\langle \boldsymbol{z}^{(i)} \rangle_d = \langle \mathbf{w}^{(i)} \rangle_d$. The polynomials in $\boldsymbol{X}(\cdot)$ and $\boldsymbol{Y}(\cdot)$ are then defined through the first and second components of the first $\frac{n-1}{2} + 1$ transformed triples. That is, let $\boldsymbol{X}(\cdot)$ and $\boldsymbol{Y}(\cdot)$ be the vectors of degree-$\frac{n-1}{2}$ polynomials, passing through the vectors of $\frac{n-1}{2} + 1$ points $\{(i, \mathbf{u}^{(i)})\}$ and $\{(i, \mathbf{v}^{(i)})\}$, where $i \in$

$\{1, \ldots, \frac{n-1}{2} + 1\}$. The polynomials in $\boldsymbol{X}(\cdot)$ and $\boldsymbol{Y}(\cdot)$ are next "extended" for $\frac{n-1}{2}$ new points in a secret-shared fashion, which are then set to be the first and second components of the remaining $\frac{n-1}{2}$ transformed triples. That is, for each $i \in \{\frac{n-1}{2} + 2, \ldots, n\}$, let $\boldsymbol{x}^{(i)} = \boldsymbol{X}(i)$ and $\boldsymbol{y}^{(i)} = \boldsymbol{Y}(i)$. Note that each of these $\boldsymbol{x}^{(i)}$ and $\boldsymbol{x}^{(i)}$ is a publicly-known linear combination of the vectors $\{\boldsymbol{x}^{(j)}\}_{j \in \{1,\ldots,\frac{n-1}{2}+1\}}$ and $\{\boldsymbol{y}^{(j)}\}_{j \in \{1,\ldots,\frac{n-1}{2}+1\}}$ respectively. So using the linearity property of PSS, the parties can non-interactively compute $\langle \boldsymbol{x}^{(i)} \rangle_d$ and $\langle \boldsymbol{y}^{(i)} \rangle_d$ from $\{\langle \boldsymbol{x}^{(j)} \rangle_d\}_{j \in \{1,\ldots,\frac{n-1}{2}+1\}}$ and $\{\langle \boldsymbol{y}^{(j)} \rangle_d\}_{j \in \{1,\ldots,\frac{n-1}{2}+1\}}$ respectively. Finally, the parties try to compute the product of these extended points, using the remaining triples shared by D (which are not touched till now). That is, for every $j \in \{\frac{n-1}{2} + 2, \ldots, n\}$, the parties run the protocol Π_{Beaver} on inputs $\langle \boldsymbol{x}^{(j)} \rangle_d$ and $\langle \boldsymbol{y}^{(j)} \rangle_d$, using the shared triples $(\langle \mathbf{u}^{(j)} \rangle_d, \langle \mathbf{v}^{(j)} \rangle_d, \langle \mathbf{w}^{(j)} \rangle_d)$ as the auxiliary triple and compute $\langle \boldsymbol{z}^{(j)} \rangle_d$. It follows that $\boldsymbol{z}^{(j)} = \boldsymbol{x}^{(j)} \times \boldsymbol{y}^{(j)}$ component-wise if and only if $(\mathbf{a}^{(j)}, \mathbf{b}^{(j)}, \mathbf{c}^{(j)})$ constitute vector of multiplication-triples. The polynomials in $\boldsymbol{Z}(\cdot)$ are then defined through the third components of all the n transformed triples. That is, $\boldsymbol{Z}(i) = \boldsymbol{z}^{(i)}$, for every $i \in \{1, \ldots, n\}$. Note that if D is *honest*, then throughout this transformation, the privacy of the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ is maintained.

From the above transformation, it follows that $\boldsymbol{Z}(\cdot) = \boldsymbol{X}(\cdot) \cdot \boldsymbol{Y}(\cdot)$ holds if and only if all the n vectors of triples shared by D are multiplication-triples. Hence to verify the triples of D, it is sufficient to verify the multiplicative-relationship among the polynomials. For this, the parties run an instance of Π_{PolyVer}. If D is *honest*, then the output of Π_{PolyVer} will be "positive" and at most t points on the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ might be revealed, leaving $\frac{n-1}{2} + 1 - t$ "degree of freedom" in the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$. Consequently, the parties (locally) compute PSS of $\frac{n-1}{2} + 1 - t$ new points on the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$, which are taken as the output on behalf of D. Note that these vectors of points will be multiplication-triples and known to D, as the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ are completely determined by the vectors of input triples of D. We also note that if $\frac{n-1}{2} + 1 - t = \Theta(n)$, then the communication overhead of the protocol will be $\mathcal{O}(1)$.

We will next see the generic bluebrint which we follow to design the protocol Π_{TripExt} with $\mathcal{O}(1)$ communication overhead.

Triple-Extraction with $\mathcal{O}(1)$ Overhead from Beaver Multiplication.

Protocol Π_{TripExt} can be designed very easily, given protocol Π_{Beaver}. Let $(\langle \mathbf{a}^{(i)} \rangle_d, \langle \mathbf{b}^{(i)} \rangle_d, \langle \mathbf{c}^{(i)} \rangle_d)$ be the secret-shared vectors of input multiplication-triples, where P_i knows the multiplication-triples $(\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)})$, which will be random if P_i is *honest*. To extract $m = \frac{n-1}{2} + 1 - t$ number of packed-secret-shared vectors of multiplication-triples, parties transform $\{(\langle \mathbf{a}^{(i)} \rangle_d, \langle \mathbf{b}^{(i)} \rangle_d, \langle \mathbf{c}^{(i)} \rangle_d)\}_{i \in \{1,\ldots,n\}}$ into correlated vectors of secret-shared triples $\{(\langle \boldsymbol{x}^{(i)} \rangle_d, \langle \boldsymbol{y}^{(i)} \rangle_d, \langle \boldsymbol{z}^{(i)} \rangle_d)\}_{i \in \{1,\ldots,n\}}$ in exactly the same way as done in protocol Π_{TripSh} discussed above, by executing instances of Π_{Beaver}. Note that all the transformed triples will be multiplication-triples, as all the input triples will be guaranteed to be multiplication-triples. Consequently, the underlying vectors of polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot)$ defined through the transformed triples will satisfy the multiplicative-relationship. Moreover, adversary will be knowing at most t points on the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$, leaving m "degree of freedom" in these polynomials. Hence, the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot)$ can be evaluated at m new points and their PSS can be taken as the extracted multiplication-triples which will be random for the adversary.

From the discussion till now, we find that protocols $\Pi_{\mathsf{VSS}}, \Pi_{\mathsf{RecPriv}}, \Pi_{\mathsf{Beaver}}$ and Π_{PolyVer} constitute the primary building blocks for designing protocol Π_{ckteval} and to get $\mathcal{O}(1)$ communication overhead for protocol Π_{ckteval}, we need each of these building blocks to have $\mathcal{O}(1)$ communication overhead. We next discuss the generic blueprint which we follow for designing protocol Π_{Beaver}.

Packed Beaver's Multiplication with $\mathcal{O}(1)$ Overhead from Random Shared Pairs and Degree Reduction.

Traditionally, Beaver's multiplication protocol proceeds as follows for computing product of t-shared values. Let x and y be t-shared and the goal is to compute a t-sharing of xy, for which the protocol is also provided an auxiliary t-shared triple (a, b, c) as input, which is supposed to be a multiplication-triple. In the protocol, parties first compute a t-sharing of $x + a$ and $y + b$ and *publicly reconstruct* $x + a$ and $y + b$. It then follows that if (a, b, c) is indeed a multiplication-triple, then a t-sharing of xy can be locally computed as a linear combination of t-sharing of a, b and c where $x + a$ and $y + b$ serves as the linear

combiners. Namely, the parties locally compute a t-sharing of $(x+a)(y+b)-(x+a)\cdot b-(y+b)\cdot a+c$ which will be a t-sharing of xy if and only if $c=ab$. The privacy of inputs x and y will be maintained if (a,b,c) remains random for the adversary. In the protocol, the only communication happens to publicly reconstruct $x+a$ and $y+b$.

A naive extension of the above approach for PSS of vectors will proceed as follows: given inputs $\langle \boldsymbol{x}\rangle_d$ and $\langle \boldsymbol{y}\rangle_d$ and auxiliary inputs $(\langle \mathbf{a}\rangle_d, \langle \mathbf{b}\rangle_d, \langle \mathbf{c}\rangle_d)$, the parties first locally compute $\langle \boldsymbol{x}+\mathbf{a}\rangle_d$ and $\langle \boldsymbol{y}+\mathbf{b}\rangle_d$, followed by *publicly* reconstructing $\boldsymbol{x}+\mathbf{a}$ and $\boldsymbol{y}+\mathbf{b}$. Now notice that the following holds over vectors if component-wise $\mathbf{c}=\mathbf{a}\times\mathbf{b}$:

$$\boldsymbol{x}\times\boldsymbol{y}=(\boldsymbol{x}+\mathbf{a})\times(\boldsymbol{y}+\mathbf{b})-(\boldsymbol{x}+\mathbf{a})\times\mathbf{b}-(\boldsymbol{y}+\mathbf{b})\times\mathbf{a}+\mathbf{c}.$$

Unfortunately, the above operation *cannot* be performed over PSS of vectors. That is,

$$\langle \boldsymbol{x}\times\boldsymbol{y}\rangle_d \neq (\boldsymbol{x}+\mathbf{a})\times(\boldsymbol{y}+\mathbf{b})-(\boldsymbol{x}+\mathbf{a})\times\langle \mathbf{b}\rangle_d-(\boldsymbol{y}+\mathbf{b})\times\langle \mathbf{a}\rangle_d+\langle \mathbf{c}\rangle_d.$$

This is because the operations $(\boldsymbol{x}+\mathbf{a})\times\langle \mathbf{b}\rangle_d$ and $(\boldsymbol{y}+\mathbf{b})\times\langle \mathbf{a}\rangle_d$ are not defined. For instance, $\boldsymbol{x}+\mathbf{a}$ is a vector of p values while each party has a single share as part of $\langle \mathbf{b}\rangle_d$. Another issue here is that publicly reconstructing $\boldsymbol{x}+\mathbf{a}$ (and $\boldsymbol{y}+\mathbf{b}$) will have an overhead of $\mathcal{O}(n)$, since we need to run n instances of Π_{RecPriv}. Hence we have to use a different approach.

Suppose we have the following additional ingredianets for the protocol Π_{Beaver}.

- **Double-packed-secret-shared random values.** A vector $\mathbf{r}$ of p random values, which is packed-secret-shared, both with degree d as well as $d+p-1$. That is, the parties hold $(\langle \mathbf{r}\rangle_d, \langle \mathbf{r}\rangle_{d+p-1})$. We call such a sharing as a double-packed-secret-sharing (DPSS). For the moment assume there exists a protocol Π_{RandPair} which can generate such DPSS of random vectors with $\mathcal{O}(1)$ communication overhead. Such a protocol can be easily designed given protocol Π_{VSS}. Namely each party selects sufficiently many random vectors and generates PSS of their vectors with degrees d as well as $d+p-1$ using instances of Π_{VSS}. Note that corrupt parties may share different vectors and to verify the same, each party gives a "zero-knowledge" proof that it shared the same vectors through degree d as well as $d+p-1$. Finally, the parties apply some existing randomness-extraction techniques such as [27,12] on the PSS of the vectors of the parties whose proofs are correct and extract DPSS of random vectors.
- **A degree-reduction protocol with $\mathcal{O}(1)$ overhead.** Protocol Π_{DegRed} which takes inputs sufficiently many PSS $\langle \mathbf{u}\rangle_\ell$ where $\ell\in\{d, d+p-1\}$ and outputs $\langle \mathbf{u}\rangle_{p-1}$, incurring a communication overhead of $\mathcal{O}(1)$. Note that the privacy of $\mathbf{u}$ will not be maintained as adversary may learn $\mathbf{u}$ from the output $\langle \mathbf{u}\rangle_{p-1}$, however that will be fine for our purpose.

Protocol Π_{Beaver} now proceeds as follows.

1. Parties first locally compute $\langle \boldsymbol{x}+\mathbf{a}\rangle_d$ and $\langle \boldsymbol{y}+\mathbf{b}\rangle_d$.
2. Parties then execute instances of protocol Π_{DegRed} with inputs $\langle \boldsymbol{x}+\mathbf{a}\rangle_d$ and $\langle \boldsymbol{y}+\mathbf{b}\rangle_d$ to obtain $\langle \boldsymbol{x}+\mathbf{a}\rangle_{p-1}$ and $\langle \boldsymbol{y}+\mathbf{b}\rangle_{p-1}$ respectively. Note that adversary at this stage may learn $\boldsymbol{x}+\mathbf{a}$ and $\boldsymbol{y}+\mathbf{b}$. However, the privacy of $\boldsymbol{x}$ and $\boldsymbol{y}$ will be maintained if $(\mathbf{a},\mathbf{b},\mathbf{c})$ remains random.
3. Parties then locally compute the following:

$$\langle \boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{d+p-1}=(\boldsymbol{x}+\mathbf{a})\times(\boldsymbol{y}+\mathbf{b})-\langle \boldsymbol{x}+\mathbf{a}\rangle_{p-1}\langle \mathbf{b}\rangle_d-\langle \boldsymbol{y}+\mathbf{b}\rangle_{p-1}\langle \mathbf{a}\rangle_d+\langle \mathbf{c}\rangle_d+\langle \mathbf{r}\rangle_{d+p-1}.$$

4. Parties then execute an instance of the protocol Π_{DegRed} with input $\langle \boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_d$ to obtain $\langle \boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{p-1}$. Note that adversary may learn $\boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}$. However, the privacy of $\boldsymbol{x}$ and $\boldsymbol{y}$ will be still maintained since $\mathbf{r}$ remains random.
5. Parties then locally compute the output $\langle \boldsymbol{x}\times\boldsymbol{y}\rangle_d=\langle \boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{p-1}-\langle \mathbf{r}\rangle_d$.

From the above discussion, it follows that protocol Π_{DegRed} constitute the primary component for the protocol Π_{Beaver}. Interestingly, one can design protocol Π_{DegRed} generically using protocols Π_{VSS} and Π_{RecPriv} and that too with $\mathcal{O}(1)$ communication overhead by using the overview discussed next.

Packed Degree Reduction with $\mathcal{O}(1)$ Overhead from Private Reconstruction.

Protocol Π_{DegRed} takes input degree-ℓ PSS of vectors $\mathbf{s}^{(1)},\ldots,\mathbf{s}^{(k)}$ and outputs degree-$(p-1)$ PSS of the same vectors, where $\ell\in\{d, d+p-1\}$. Here k is a parameter, satisfying the conditions $k=\Theta(n)$

and $n-(k-1) \geq 2t$. In the protocol, parties first locally "expand" the k input PSS into n number of PSS using Reed-Solomon (RS) linear error-correcting code that can error-correct up to t errors. Note that the expanded sharings can be computed locally from the input sharings as it involves performing linear operations on input sharings. Namely, let $\mathbf{S}(X)$ be the vectors of p polynomials of degree-$(k-1)$ each where $\mathbf{S}(X) = \mathbf{s}^{(1)} + \mathbf{s}^{(1)} \cdot X + \ldots + \mathbf{s}^{(k)} \cdot X^{k-1}$. That is, the coefficients of X^j in the polynomials in $\mathbf{S}(X)$ are the elements of the vector $\mathbf{s}^{(j+1)}$. Then consider the vectors of points $\mathbf{S}(1), \ldots, \mathbf{S}(n)$ on the polynomials in $\mathbf{S}(X)$. Each of these vectors $\mathbf{S}(i)$ is a publicly-known linear combination of the input vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(k-1)}$, where the linear combiners are the Lagrange's interpolation coefficients. Consequently, the parties can locally compute $\langle \mathbf{S}(i) \rangle_\ell$ from the input PSSs $\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(k)} \rangle_\ell$.

Next, each party robustly reconstructs one of the designated expanded sharings, followed by redistributing a degree-$(p-1)$ PSS of the reconstructed vector. Namely, each P_i reconstructs the vector $\mathbf{S}(i)$ from $\langle \mathbf{S}(i) \rangle_\ell$ using Π_{RecPriv}, followed by generating $\langle \mathbf{S}(i) \rangle_{p-1}$ by using an instance of Π_{VSS}. Note that up to t corrupt parties may generate PSS of *incorrect vectors*. However, by leveraging the linearity property of RS error-correction procedure, parties can error-correct these errors on PSS, thus obtaining degree-$(p-1)$ PSS of $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(k)}$ as desired. The observation here is that the RS error-correction procedure performs linear operations on the vectors of points redistributed by the individual parties to error-correct up to t incorrect vectors and outputs the polynomials in $\mathbf{S}(X)$. Consequently, the same linear operations can instead be performed on the degree-$(p-1)$ PSS of the redistributed vectors of points to obtain degree-$(p-1)$ PSS of the coefficients of the polynomials in $\mathbf{S}(X)$. The constant overhead follows from the fact that $k = \Theta(n)$ and there are n instances of Π_{RecPriv} involved, each having a constant overhead.

2.1 Statistically-Secure $\Pi_{\mathsf{VSS}}, \Pi_{\mathsf{RecPriv}}$ and Π_{PolyVer} with $\mathcal{O}(1)$ Overhead

For the ease of explanation, we assume $n = 3t+1$ and instantiate the blueprint of Fig 1 with statistical security; however we stress that we present protocols for *any* $n = 2t + s$ where $s = \Theta(n)$. We set $p = \frac{t}{4} + 1$, implying that $d = t + \frac{t}{4}$. Note that this satisfies the constraints $p = \Theta(n)$ and $2(t + p) - 1 \leq n$. The first immediate problem that we face in this case that we cannot any more robustly reconstruct a degree-d PSS $\langle \mathbf{s} \rangle_\ell$ using RS error-correction, if $n = 3t+1$ and $\ell > t$. As a way out, we can alternatively use an "authenticated" version of PSS called authenticated-packed-secret-sharing (APSS), where each share is authenticated by pair-wise information-theoretically secure message-authentication-codes (MAC). Namely, let $\mathbf{s} \in \mathbb{F}^p$. Then we augment $\langle \mathbf{s} \rangle_\ell$ where each share s_i is further authenticated as follows: every party P_j will have a MAC-key K_{ji} corresponding to P_i and party P_i will hold a MAC-tag τ_{ij} for the share s_i, computed under the key K_{ji}. Given such an augmented PSS, reconstructing a vector from its PSS can proceed as follows: let P_j be the designated party to reconstruct $\mathbf{s}$ which is degree-ℓ packed-secret-shared with authentication, where $\ell \leq n-t-1$. Every party P_i then sends its shares s_i as well as the tag τ_{ij} to P_j, who verifies the authenticity of the share s_i, using its key K_{ji} and the received tag. If the verification passes, then it is guaranteed that s_i is the correct share, unless a corrupt P_i can forge the MAC-tag on an incorrect share, which it can do with a negligible probability. Once P_j receives $\ell+1$ verified shares, it can reconstruct $\mathbf{s}$. This will constitute the protocol Π_{RecPriv}.

From the above discussion it follows that instead of PSS, the statistically-secure MPC protocol for SIMD circuits will work over *authenticated* PSS (APSS). Note that we will require the linearity property from APSS, which is guaranteed if the underlying MAC satisfies the linearity property. The standard information-theoretically secure MAC, used in previous statistically-secure MPC protocols works as follows: a random pair $\mathsf{K} = (u, v)$ is selected as the MAC-key, where $u, v \in \mathbb{F}$. The MAC-tag on a value $s \in \mathbb{F}$ is defined as $\mathsf{MAC}_{\mathsf{K}}(s) = \tau \stackrel{\mathsf{def}}{=} u \cdot s + v$. For the sake of efficiency, instead of authenticating *single* field element, we will be authenticating vectors of field elements, as done in [14,44]. Here to authenticate a vector $\mathbf{s} \in \mathbb{F}^L$, a random pair $\mathsf{K} = (\boldsymbol{\mu}, v)$ is selected as the MAC-key, where $v \in \mathbb{F}$ and $\boldsymbol{\mu} \in \mathbb{F}^L$. The MAC-tag on $\mathbf{s} \in \mathbb{F}$ is then defined as $\mathsf{MAC}_{\mathsf{K}}(\mathbf{s}) = \tau \stackrel{\mathsf{def}}{=} \boldsymbol{\mu} \cdot \mathbf{s} + v$, where $\cdot$ here denotes the inner-product operation. The advantage of authenticating vectors over a single element is that we can reuse $\boldsymbol{\mu}$ for authenticating multiple vectors by just sampling a random v every time. Namely, if $\mathbf{s}, \mathbf{s}'$ are two vectors to be authenticated, then one can use the keys $\mathsf{K} = (\boldsymbol{\mu}, v)$ and $\mathsf{K}' = (\boldsymbol{\mu}, v')$ for authenticating $\mathbf{s}$ and $\mathbf{s}'$ respectively. Consequently, once $\boldsymbol{\mu}$ is selected, the key-size for subsequent authentications will be *independent* of the vector-size L which needs to be authenticated. Reusing the same $\boldsymbol{\mu}$ also ensures

that the MAC satisfies the linearity properties. That is, let $\tau = \mathsf{MAC}_{\mathsf{K}}(\mathbf{s})$ and $\tau' = \mathsf{MAC}_{\mathsf{K}'}(\mathbf{s}')$. Then the MAC-tag and the corresponding MAC-key for any publicly-known linear combination of $\mathbf{s}$ and $\mathbf{s}'$ can be computed from τ, τ' and K, K' respectively (see Section 4.1 for the formal details).

Consider two-dimensional vectors $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$, where each vector $\mathbf{s}^{(k)}$ consists of p elements from $\mathbb{F}$, where $m \geq 1$. Then a degree-d APSS of $\mathbf{S}$ consists of degree-d PSS $\langle \mathbf{s}^{(k)} \rangle_d$ of each vector $\mathbf{s}^{(k)}$ in $\mathbf{S}$. Additionally, the share-vector of every party will be authenticated with information-theoretic MAC. In more detail, let $\mathbf{s}_i \in \mathbb{F}^m$ be the vector of shares of P_i for $\langle \mathbf{s}^{(1)} \rangle_d, \ldots, \langle \mathbf{s}^{(M)} \rangle_d$. Then every party P_j will hold a MAC-key $\mathsf{K}_{ji} = (\boldsymbol{\mu}_{ji}, v_{ji})$ and P_i will hold the MAC-tag $\tau_{ij} = \boldsymbol{\mu}_{ji} \cdot \mathbf{s}_i + v_{ji}$. The notation ${}^{A}\langle \mathbf{S} \rangle_d$ will denote an APSS of $\mathbf{S}$. We will say that ${}^{A}\langle \mathbf{S} \rangle_d$ and ${}^{A}\langle \mathbf{S}' \rangle_d$ are *key-consistent* if for every pair of parties P_i, P_j, party P_j has the same $\boldsymbol{\mu}_{ji}$ for authenticating P_i's share-vectors for both ${}^{A}\langle \mathbf{S} \rangle_d$ and ${}^{A}\langle \mathbf{S}' \rangle_d$. Note that key-consistent APSS satisfies the linearity properties, which follows from the lineairty properties of PSS and MACs.

We extend APSS to another dimension. Let $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)})$ be three-dimensional vectors, where each individual vector consists of p elements from $\mathbb{F}$. Then a degree-d APSS of $\mathbb{S}$ consists of APSS of every $\mathbf{S}^{(k)}$ from $\mathbb{S}$. If ${}^{A}\langle \mathbb{S} \rangle_d$ and ${}^{A}\langle \mathbb{S}' \rangle_d$ are key-consistent, then an APSS of any publicly-known linear combination of the vectors in $\mathbb{S}$ and $\mathbb{S}'$ can be non-interactively computed from ${}^{A}\langle \mathbb{S} \rangle_d$ and ${}^{A}\langle \mathbb{S}' \rangle_d$ (see Section 4.2.1 for the formal exposition). The reason for working with two-dimensional vectors of APSS is that our statistically-secure VSS protocol with $\mathcal{O}(1)$ overhead generates ${}^{A}\langle \mathbb{S} \rangle_d$ for a vectors of vectors of elements $\mathbb{S}$ held by the designated dealer, for suitable sizes of L and M. We next review our VSS protocol.

2.1.1 Overview of Our Statistically-Secure VSS with $\mathcal{O}(1)$ Overhead. Let $D \in \mathcal{P}$ be the designated dealer with vectors of vectors $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)})$ as input, where each $\mathbf{S}^{(k)}$ further consists of Mp vectors each consisting of p elements from $\mathbb{F}$; hence $\mathbb{S}$ has LMp^2 elements from $\mathbb{F}$. Our VSS protocol then generates a degree-ℓ APSS of $\mathbb{S}$ (i.e. ${}^{A}\langle \mathbb{S} \rangle_\ell$), where $\ell \in \{p-1, d, d+p-1\}$ with $\mathcal{O}(1)$ communication overhead, provided $L = n^3$ and $M = n^2$ (hence $\mathbb{S}$ consists of $n^5 p^2 = \Theta(n^7)$ field elements); recall that $d = t + p - 1$. For simplifying the exposition, we consider the case when $\ell = d$ and $\mathbb{S}$ has a single vector of vectors $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(Mp)})$, where each vector $\mathbf{s}^{(k)} \in \mathbb{F}^p$. The protocol consists of a sequence of following 4 phases. We first summarize the purpose of each phase and then subsequently go into more technical details of each phase.

- **Sharing phase**: In this phase, D distributes shares of its inputs and publicly identifies a set of Core parties, who publicly confirms the receipt of their shares from D, where we require Core to have at least $d+t+1 = 2t+p$ parties, which will guarantee that Core has at least $d+1$ *honest* parties, whose shares will define degree-d PSS. Note that this phase *need not* be always successful, as there can be only $2t+1$ honest parties and the corrupt parties may deny to acknowledge the receipt of shares, even if D is *honest*. Consequently, if D fails to identify a desired Core set, the parties restart the protocol, where the parties who failed to make it to Core previously are considered to be in "dispute" with D and included in a publicly-known set Dp, such that D sets the shares of the parties in Dp to 0 when it restarts the protocol. This will ensure that if the parties restart the sharing phase, then an honest D is bound to get a Core set of size at least $2t+p$, since the parties in Dp will be by default considered to be part of Core and $|\mathsf{Dp}| \geq t - p + 1$ will hold, where $p = \frac{t}{4}$. Note that "fixing" the shares of the parties in Dp to 0 will not violate the privacy for an *honest* D since in this case the parties in Dp are guaranteed to be *corrupt*.
- **Verification phase**: Once a Core set is identified, the parties publicly check if the shares of the (honest) parties in the set $\mathsf{Core} \cup \mathsf{Dp}$ are "consistent" and define degree-d PSS, by asking D to publicly open a random linear combination of the shares that it distributed to the parties in $\mathsf{Core} \cup \mathsf{Dp}$, where the shares of the parties in Dp are expected to be 0.
- **Non-Core share-reconstruction phase**: Note that if D is *corrupt* then Core may not have all honest parties. Consequently, once the "consistency" of the shares of the (honest) parties in $\mathsf{Core} \cup \mathsf{Dp}$ is confirmed, the parties in Core help the parties outside in Core to recompute their shares, which are consistent with the secrets "defined" by the shares of the honest parties in $\mathsf{Core} \cup \mathsf{Dp}$.
- **Pairwise tag-generation phase**: The last phase is the tag-generation phase, which consists of two stages. During the first stage, the parties compute the tags where if a party fails to compute its tags then it publicly complaints against the parties whom it accuses for corrupt behaviour. All such complaints are then publicly resolved during the second stage. Here either all the complaints

are resolved successfully and tag-computations are done or the parties publicly identify a set Cr of *corrupt* parties of size at least $t - p + 1$ and restart the protocol. In the latter case, D will set the shares of the parties in Cr (apart from the parties in Dp) to 0 during the rerun. The protocol makes sure that the parties in Cr cannot "disrupt" the tag-generation phase during the rerun.

From the above discussion it follows that the parties need to restart the protocol at most *twice*, once due to the failure of sharing phase where first time D fails to identify a Core set of size at least $2t + p$ and once due to the failure of tag-generation phase. After this, the next run of the protocol is bound to generate its desired outcome. We note that the asynchrnous VSS protocol of [44] also has the same structure as our VSS. However, in their protocol the parties *need not* have to restart the protocol since they *do not* generate PSS. Dealing with the restarts bring in additional technicalities in our context. We now elaborate upon the exact details of each individual phase.

Sharing phase. The first phase is the sharing phase, where D distributes shares of its input $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(Mp)})$. Namely, it picks Mp random degree-d sharing-polynomials, where vth polynomial embeds the elements in $\mathbf{s}^{(u,v)}$. To prove that it has picked d-degree sharing-polynomials, D embeds these polynomials in bivariate polynomials of degree d in each variable, where each bivariate polynomial embeds p number of sharing-polynomials. Precisely, D fixes M bivariate polynomials $F^{(1)}(X, Y), \ldots, F^{(M)}(X, Y)$, each of degree d in both variables. Each polynomial $F^{(v)}(X, Y)$ is a random bivariate polynomial, except that the polynomials $F^{(u,v)}(X, -1), \ldots, F^{(u,v)}(X, -p)$ constitute the sharing-polynomials for the vectors $\langle \mathbf{s}^{(u,(v-1)p+1)} \rangle_d, \ldots, \langle \mathbf{s}^{(u,vp)} \rangle_d$ respectively. Let $F^{(v)}(X, i)$ and $F^{(v)}(i, Y)$ denote the i^{th} row and column-polynomial of $F^{(v)}(X, Y)$. The dealer then distributes the i^{th} row and column-polynomial of every bivariate polynomial to party P_i. Note that the row and column-polynomial of every party in $\mathsf{Dp} \cup \mathsf{Cr}$ will be set to 0, if the parties have restarted the protocol due to the failure of sharing or tag-generation phase earlier and so D need not have to explicitly distribute these polynomials.

Note that this distribution of information does not reveal any additional information about $\mathbf{S}$ if D is *honest*, since for every bivariate polynomial, the adversary will learn at most t rows and columns, leaving $(t+p)^2 - t \cdot (t+p) - t \cdot p = p^2$ "degree of freedom" in the polynomial. Hence the adversary's view will be independent of the p^2 elements from $\mathbf{S}$ which are embedded in that bivariate polynomial. If D has distributed consistent row and column polynomials, then every $P_i \notin \mathsf{Dp} \cup \mathsf{Cr}$ can compute its shares corresponding to $\langle \mathbf{S} \rangle_d$ by evaluating each of its column-polynomial $F^{(v)}(i, Y)$ at $Y = -1, \ldots, -p$. Note that for the parties in $\mathsf{Dp} \cup \mathsf{Cr}$, their shares are by default set to 0, since D has implicitly set their row and column polynomials to 0.

Looking ahead, in the remaining phases of the protocol, D will be asked to publicly open random linear combinations of its bivariate polynomials. To prevent a potentially *corrupt* D from using different bivariate polynomials than what it has used for distributing the rows and column-polynomials (to the honest parties), once party P_i receives its row and column polynomials, it gives *information-checking signatures* (IC-signatures) on the points of the received column-polynomials to D. Informally, IC-signatures are information-theoretic analogue of digital signatures, used for authenticating data, which can be generated using an *information-checking protocol* (ICP) [50]. For the sake of efficiency, we present a new ICP in this paper (see Section 5). D checks if it has received back signed column polynomials from a set of parties Core such that $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + p$. If such a Core exists then D publicly announces the same. The idea here is that if a Core with the above condition exists, then it implies that there are at least $t + p = d + 1$ honest parties in $\mathsf{Core} \cup \mathsf{Dp}$, who have received d-degree column polynomials from D. Further, the honest parties in Dp (if any) have 0 shares. So, the column polynomials of the *first* $t + p$ honest parties in $\mathsf{Core} \cup \mathsf{Dp}$ define valid bivariate polynomials of degree-d in each variable, with the column polynomials of the parties in Dp being supposedly 0.

Note that during the *first* run of the protocol, D *may not* find such a Core set, even if it is *honest*. This is because there may be only $2t + 1$ honest parties who will send back the signed column polynomials to D, but up to $t - p + 1$ *corrupt* parties many not send back the correct signed column polynomials. In this case, D gets into dispute with these parties and the parties restart the protocol with the set Dp. Note that it is not necessary that the parties in Dp are corrupt, since a potentially *corrupt* D may get into dispute even with honest parties and accuse them of not sending back the correct signed column polynomials. However, if the parties restart the sharing phase, then an *honest* D is bound to get a Core set such that $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + p$.

Verification phase. In this phase, the parties publicly verify if the rows and column-polynomials of the (honest) parties in $\mathsf{Core} \cup \mathsf{Dp}$ lie on degree-d bivariate polynomials, such that the row and column polynomials of the parties in $\mathsf{Cr} \cup \mathsf{Dp}$ are 0. For this D is challenged to make public a linear combination of its bivariate polynomials, where the linear combiners are randomly and jointly selected by the parties. To prevent a *corrupt* D from making public linear combination of incorrect bivariate polynomials, D is also asked to publicly reveal the IC-signature of every party in Core on the column polynomials of the linear combination of the bivariate polynomials. The parties then verify the signatures and check if these signed values lie on the polynomials made public by D. Additionally, it is also verified that if the rows and column polynomials of the parties in Dp and Cr in the publicly-revealed bivariates are all 0. If all these checks are successful then it is guaranteed that except with a negligible probability, D has distributed consistent rows and column polynomials to the honest parties in $\mathsf{Core} \cup \mathsf{Dp}$.

Note that asking D to publicly reveal linear combination of its bivariate polynomials will leak information about $\mathbf{S}$. A simple fix to this issue is to let D use additional random "masking data" independent of $\mathbf{S}$, which has the same number of vectors as $\mathbf{S}$ (namely Mp) and distribute shares of those vectors during the sharing phase, by embedding those vectors in bivariate polynomials in the same way as done for $\mathbf{S}$. Throughout the protocol, D will be asked to make public linear combination of its bivariate polynomials for $\mathbf{S}$ *four* times and so it is sufficient for D to pick four batches of random masking vectors.

Non-Core share reconstruction phase. If D is *honest* then all honest parties will be present in Core. On the other hand, if D is *corrupt*, then even though the row and column polynomials of the *honest* parties in $\mathsf{Core} \cup \mathsf{Dp}$ define degree-d bivariate polynomials, there still might be some honest parties outside $\mathsf{Core} \cup \mathsf{Dp}$, whose row and column polynomials might be inconsistent with the bivariate polynomials "committed" by D to the parties in $\mathsf{Core} \cup \mathsf{Dp}$. To help such parties get their supposed row and column polynomials, the parties in $\mathsf{Core} \cup \mathsf{Dp}$ provide the supposedly common points on their row and column polynomials to the outside parties; while the parties in Core send these common points explicitly, for the parties in $\mathsf{Dp} \cup \mathsf{Cr}$ the supposedly common points are implicitly taken as 0. While the honest parties in Core will provide the supposedly common points, corrupt parties in Core may not do so. Moreover, we *cannot* use RS error-correction to error-correct incorrect common points, since each row and column polynomial is of degree d. The way out is to ask D to help out identifying the correct and incorrect common points. For this, we use a similar idea used in the verification phase. Namely, we ask D to publicly reveal a random linear combination of its committed bivariate polynomials, where we again use the IC-signature trick to prevent a corrupt D from revealing a linear combination of incorrect bivariate polynomials. Every party outside $\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr}$ then apples the same linear combination on the supposedly common points that it received from the parties in Core and check if they lie on the bivariate polynomials made public by D. The idea here is that if a *corrupt* party in Core sends *incorrect* common points, then they will be discarded with a high probability. This is because the adversary will *not* be knowing beforehand the random linear combination which will be used to verify the common points sent by the corrupt parties. Moreover, even if D is *corrupt*, it is bound to reveal a correct linear combination of the bivariate polynomials which it committed to the honest parties in $\mathsf{Core} \cup \mathsf{Dp}$.

Once every party outside $\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr}$ verifiably receives $d+1$ correct common points on their supposed row and column polynomials, it interpolates them to recompute its row and column polynomials lying on the the same bivariate polynomials which D has committed to the honest parties in $\mathsf{Core} \cup \mathsf{Dp}$. Note that every party outside $\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr}$ will indeed receive at least $d+1$ correct common points, since there are at least $d+1$ honest parties in $\mathsf{Core} \cup \mathsf{Dp}$, as $|\mathsf{Core} \cup \mathsf{Dp}| \geq d+t+1$.

Pairwise tag-generation phase. This is the last phase of the protocol where the parties compute pairwise tags. Note that at the end of the previous phase, parties would have computed $\langle \mathbf{S} \rangle_d$, where every P_i would have computed its share-vector $\mathbf{s}_i = (g_i^{(1)}(-1), \ldots, g_i^{(1)}(-p), \ldots, \ldots g_i^{(M)}(-1), \ldots, g_i^{(M)}(-p))$, where $g_i^{(1)}(Y), \ldots, g_i^{(M)}(Y)$ are the column polynomials held by P_i at the end of the previous phase. The computation of the pair-wise tags will transform $\langle \mathbf{S} \rangle_d$ into ${}^A\langle \mathbf{S} \rangle_d$. As mentioned earlier during the summary of various phases, this phase may fail at most once, in which case the parties publicly identify a set Cr of at least $t-p+1$ corrupt parties and restart the protocol in such a way that in the next run this phase is *guaranteed* to be successful. We first discuss the overview of this phase assuming that the parties are running this phase for the first time and $\mathsf{Cr} = \emptyset$. We will then discuss the additional steps which need to be incorporated if this phase fails and a set Cr is identified so that the next run of this phase is guaranteed to be successful.

Consider an arbitrary pair of parties (P_i, P_j). Then the goal of this phase is to let P_i compute the tag $\tau_{ij} \stackrel{\mathsf{def}}{=} \mathbf{s}_i \cdot \boldsymbol{\mu}_{ji} + v_{ji}$, such that P_j holds the MAC-key $\mathsf{K}_{ji} \stackrel{\mathsf{def}}{=} (\boldsymbol{\mu}_{ji}, v_{ji})$. The standard way to achieve this would be via a "mini-MPC" protocol, where P_i secret-shares its share-vector $\mathbf{s}_i$ and P_j secret-shares its key-component v_{ji}, let the parties compute a secret-sharing of τ_{ij} and reconstruct it towards P_i. However, based on an idea observed in [14,44], we note that a degree-d twisted-sharing of $\mathbf{s}_i$ with respect to P_i is now *already* available among the parties.[11] Namely the parties already hold $\lceil \mathbf{s}_i \rfloor_d^i$ where every party P_l except P_i *already* hold its respective twisted share corresponding to $\lceil \mathbf{s}_i \rfloor_d^i$, and the share of P_l is actually the vector $\mathbf{s}_l$. This is because the polynomials $((F^{(1)}(X,-1),\ldots,F^{(1)}(X,-p)),\ldots,(F^{(M)}(X,-1),\ldots,F^{(M)}(X,-p)))$ constitute degree-d polynomials whose i^{th} points are the elements of $\mathbf{s}_i$ and every party $P_l \neq P_i$ has the l^{th} points on these polynomials which are elements of the vector $\mathbf{s}_l$. Note that P_i *will not* have its twisted-shares corresponding to $\lceil \mathbf{s}_i \rfloor_d^i$, since they are the 0^{th} points on the same polynomials, which are *not* available with P_i. Based on this observation, the idea is to let parties compute the following

$$\lceil \tau_{ij} \rfloor_{t+d}^i = \lceil \mathbf{s}_i \rfloor_d^i \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor_t^i + \lceil v_{ji} \rfloor_t^i,$$

where $\lceil v_{ji} \rfloor_t^i$ is a random twisted t-sharing of v_{ji} held by P_j. By setting the twisted-shares of P_i for $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ to $\mathbf{0}$, we make sure that the twisted-share of P_i for $\lceil \mathbf{s}_i \rfloor_d^i \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ is $\mathbf{0}$ too. So P_i can *also* compute its share corresponding to $\lceil \tau_{ij} \rfloor_{t+d}^i$. It now follows that all the parties can compute their respective shares of $\lceil \tau_{ij} \rfloor_{t+d}^i$ and send to P_i. However directly computing and sending the shares of $\lceil \tau_{ij} \rfloor_{t+d}^i$ as above will breach the privacy, since $\lceil \tau_{ij} \rfloor_{t+d}^i$ is *not* a random degree-$(t+d)$ twisted-sharing of τ_{ij}. The simple fix will be to randomize the above sharing *without* affecting the underlying secret τ_{ij}, which is embedded at the i^{th} point. This could be done by masking the above sharing with a random degree-$(t+d)$ twisted sharing $\lceil 0 \rfloor_{t+d}^i$ of 0. Following the ideas in [37,44], such a secret-sharing can be easily generated from a *publicly-known* linear combination of several random t-sharings. Based on the above discussion, the parties can compute their respective shares of $[\tau_{ij}]_{t+d}^i$ and send them to P_i. Since the corrupt parties can send incorrect tag-shares of τ_{ij}, we need to provide a mechanism for P_i to verify the tag-shares that it received from every party P_l. The verification mechanism is to let the parties compute a twisted t-sharing with respect to P_i, whose l^{th} share is same as the supposed l^{th} tag-share of τ_{ij} sent by P_l to P_i. If such a twisted t-sharing is computable then the parties can enable P_i to reconstruct the complete twisted t-sharing, who can then compare the tag-shares received from P_l with the l^{th} share of the reconstructed t-sharing. Such a twisted t-sharing can be computed as follows: consider the random sharing $\lceil 0 \rfloor_{t+d}^i$, for which P_l would have computed its share by taking a *publicly-known* linear combination of the random t-sharings which are used to compute $\lceil 0 \rfloor_{t+d}^i$. If all the parties in $\mathcal{P}$ apply the same linear combination (as P_ℓ), then it will lead to a t-sharing whose ℓ^{th} share will be the same as share of P_ℓ for $\lceil 0 \rfloor_{t+d}^i$. What now remains is to find a t-sharing whose l^{th} share will be exactly the same $\mathbf{s}_l \cdot \boldsymbol{\mu}_{ji,l} + v_{ji,l}$. For a moment, let us forget about the privacy of $\mathbf{S}$ and ask D to make public the share $\mathbf{s}_l$ in a verifiable way (we will see how this can be achieved). The parties can then compute the degree-0 constant twisted-sharing $\lceil \mathbf{s}_l \rfloor_0^i$ with respect to P_i. Then consider the degree-t twisted-sharing $\lceil \mathbf{s}_l \rfloor_0^i \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor_t^i + \lceil v_{ji} \rfloor_t^i$. It turns out that l^{th} share of this t-sharing is exactly the same as $\mathbf{s}_l \cdot \boldsymbol{\mu}_{ji,l} + v_{ji,l}$. Hence we have a t-sharing whose l^{th} share is exactly the same as P_l's supposed tag-share $\tau_{ij,l}$.

Two issues which are not addressed above are how to enforce a potentially *corrupt* D to make public the correct $\mathbf{s}_l$ and how to maintain the privacy of $\mathbf{S}$. Both these issues are addressed simultaneously as follows. Instead of applying the above idea only for $\mathbf{S}$, we apply it on a random linear combination of $\mathbf{S}$ and the masking-vectors used by D. Namely, we let D to publicly reveal a random linear combination of its committed bivariate polynomials (both for and masking-vectors) in a verifiable way, through IC-signatures (as done in previous phases). Party P_i applies the same linear combination on all the tag-shares that P_i has received from P_l. The parties then compute a twisted t-sharing whose l^{th} share will be same as the supposed linear combination of tag-shares of P_l. This twisted t-sharing is then reconstructed towards P_i who can then verify *all* the tag-shares sent by P_l in one go. The above process maintains the privacy of D's input $\mathbb{S}$ because of the masking vectors. One final note about the above process is that directly reconstructing the final twisted t-sharing as above will reveal additional information about the

[11] A vector $\mathbf{a}$ is said to be degree-ℓ twisted-shared with respect to P_i, denoted by $\lceil \mathbf{s}_i \rfloor_\ell^i$, if for every element $a \in \mathbf{a}$, there is a degree-ℓ polynomial, say $f_a(X)$, such that $f_a(i) = a$ and every $P_j \in \mathcal{P} \setminus \{P_i\}$ holds the share $f(j)$ and party P_i holds the share $f(0)$. In essence a twisted-sharing (with respect to P_i) is same as the regular degree-ℓ sharing, where the slots for embedding the secret and the share of P_i are "swapped".

honest parties' shares. The simple way out is to mask the final twisted t-sharing with a random twisted t-sharing whose l^{th} share is guaranteed to be 0 and then reconstruct it towards P_i.

If party P_i gets correct tag-shares from at least $t+d = 2t+p$ parties then it can reconstruct the the supposed tag τ_{ij}. Else, it publicly complaints against those parties P_l whom it alleges to send incorrect tag-shares. Such complaints are then *publicly* resolved by first letting P_l make public all its tag-shares for P_i. This is followed by D making public a random linear combination of its committed bivariate polynomials and then parties computing a twisted t-sharing whose l^{th} share will be the corresponding linear combination of all the tag-shares which P_l has made public. By publicly reconstructing this t-sharing, the parties can then publicly identify if P_l made public the correct tag shares. In the process either a corrupt P_l gets caught publicly and included in the set Cr or P_i gets correct tag-shares from P_l. The idea here is that if P_i complaints against P_l for providing incorrect tag-shares, then at least one of them is corrupt. So it is fine to let P_l make public all its tag-shares (which it provided earlier only to P_i) and also publicly reconstruct the corresponding t-sharing which was used earlier by P_i to verify P_l's tag-shares.

If every party gets $2t+p$ tag-shares for every tag that it is supposed to compute, then the protocol terminates here. Otherwise the protocol restarts but with a publicly known set of corrupt parties Cr who are caught publicly to provide incorrect tag-shares during the complaint resolution phase. Note that $|\mathsf{Cr}| \geq t-p+1$. To ensure that during the rerun the parties in Cr do not "disrupt" the tag-computation phase, we ensure that for every $P_l \in \mathsf{Cr}$, the tag-shares of P_l are "set" to 0 during all tag computations. For this, D fixes the row and column polynomials of all the parties in Cr to $\mathbf{0}$ while selecting the bivariate polynomials (and the parties verify the same everytime D is asked to make public a random linear combination of its bivariate polynomials). This will ensure that the shares of every $P_l \in \mathsf{Cr}$ are $\mathbf{0}$ and thus the twisted-shares of P_l for the twisted-sharings of every party's share is also $\mathbf{0}$. Furthermore, for every pair of parties (P_i, P_j), while generating the sharings $\lceil v_{ji} \rfloor_t^i$ and the random t-sharings which are used to compute the random degree-$(t+d)$ twisted-sharings of 0 used during the tag-generation phase, we ensure that the shares of P_l are 0. With these modifications, it would be ensured that P_l's tag-shares for every τ_{ij} is 0 and hence no communication of tag-share is required from P_l during the rerun.

The cost of the VSS protocol is dominated by the instances of ICP involved. It turns out that to achieve $\mathcal{O}(1)$ communication overhead, D needs to execute the protocol with n^3 batches $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(n^3)}$ of secrets, where each batch $\mathbf{S}^{(k)}$ consists of $n^2 p$ vectors, each consisting of p elements from $\mathbb{F}$. We refer to Section 6 for the complete formal details.

2.1.2 Overview of Statistically-Secure Multiplication with $\mathcal{O}(1)$ Overhead. Once we have a VSS protocol which can verifiably generate ${}^{A}\langle\cdot\rangle_{\ell}$-sharing of the underlying vectors of inputs for any given $\ell \in \{p-1, d, d+p-1\}$, it seems that we can easily get a statistically-secure MPC for SIMD circuits with $\mathcal{O}(1)$ overhead, by following the generic blueprint depicted in Fig 1. Unfortunately, we face a very subtle issue during protocol Π_{Beaver}, if we operate with ${}^{A}\langle\cdot\rangle_{\ell}$-sharings, due to the non-linearity of MACs. To understand the issue, let us instantiate the steps of the generic protocol Π_{Beaver}, assuming that the inputs are ${}^{A}\langle\cdot\rangle_{\ell}$-sharings. For the ease of exposition, let the inputs be ${}^{A}\langle\boldsymbol{x}\rangle_d$ and ${}^{A}\langle\boldsymbol{y}\rangle_d$ and the goal is to compute ${}^{A}\langle\boldsymbol{x}\times\boldsymbol{y}\rangle_d$. To achieve this goal, the auxiliary inputs for the protocol will be $({}^{A}\langle\mathbf{a}\rangle_d, {}^{A}\langle\mathbf{b}\rangle_d, {}^{A}\langle\mathbf{c}\rangle_d)$ and $({}^{A}\langle\mathbf{r}\rangle_d, {}^{A}\langle\mathbf{r}\rangle_{d+p-1})$, where $(\mathbf{a}, \mathbf{b}, \mathbf{c})$ is supposed to be a vector of multiplication-triples. Each of the vectors above consists of p elements from $\mathbb{F}$. Protocol Π_{Beaver} then proceeds as follows.

1. Parties first locally compute ${}^{A}\langle\boldsymbol{x}+\mathbf{a}\rangle_d$ and ${}^{A}\langle\boldsymbol{y}+\mathbf{b}\rangle_d$. This step can be executed due to the linearity property of ${}^{A}\langle\cdot\rangle$.-sharings.
2. Parties execute instances of protocol Π_{DegRed} with inputs ${}^{A}\langle\boldsymbol{x}+\mathbf{a}\rangle_d$ and ${}^{A}\langle\boldsymbol{y}+\mathbf{b}\rangle_d$ to obtain ${}^{A}\langle\boldsymbol{x}+\mathbf{a}\rangle_{p-1}$ and ${}^{A}\langle\boldsymbol{y}+\mathbf{b}\rangle_{p-1}$ respectively. A closer look into the protocol Π_{DegRed} when instantiated for ${}^{A}\langle\cdot\rangle$.-sharings reveals that this step also works, as it involves robust reconstruction of ${}^{A}\langle\cdot\rangle$.-sharings towards designated parties who then redistribute ${}^{A}\langle\cdot\rangle_{p-1}$-sharings of reconstructed vectors using our VSS, followed by executing the steps of RS error-correction on these redistributed ${}^{A}\langle\cdot\rangle_{p-1}$-sharings.
3. The next step will be to let the parties locally compute ${}^{A}\langle\boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{d+p-1}$, followed by running an instance of Π_{DegRed} on ${}^{A}\langle\boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{d+p-1}$ to get ${}^{A}\langle\boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{p-1}$. Unfortunately, the parties *cannot* locally compute ${}^{A}\langle\boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{d+p-1}$. This is because while the following holds for unauthenticated sharing,

$$\langle\boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{d+p-1} = \langle\boldsymbol{x}+\mathbf{a}\rangle_{p-1}\langle\boldsymbol{y}+\mathbf{b}\rangle_{p-1} - \langle\boldsymbol{x}+\mathbf{a}\rangle_{p-1}\langle\mathbf{b}\rangle_d - \langle\boldsymbol{y}+\mathbf{b}\rangle_{p-1}\langle\mathbf{a}\rangle_d + \langle\mathbf{c}\rangle_d + \langle\mathbf{r}\rangle_{d+p-1},$$

over authenticated sharings, the following holds:

$$^{A}\langle \boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{d+p-1} \neq {}^{A}\langle \boldsymbol{x}+\mathbf{a}\rangle_{p-1}{}^{A}\langle \boldsymbol{y}+\mathbf{b}\rangle_{p-1} - {}^{A}\langle \boldsymbol{x}+\mathbf{a}\rangle_{p-1}\langle \mathbf{b}\rangle_{d} - {}^{A}\langle \boldsymbol{y}+\mathbf{b}\rangle_{p-1}{}^{A}\langle \mathbf{a}\rangle_{d} + {}^{A}\langle \mathbf{c}\rangle_{d} + \langle \mathbf{r}\rangle_{d+p-1},$$

The reason is that the MACs *do not* support non-linearity. For instance, the parties *cannot* locally compute the MAC-tags and MAC-keys corresponding to $^{A}\langle(\boldsymbol{x}+\mathbf{a})\times\mathbf{b}\rangle_{d+p-1}$, given $^{A}\langle \boldsymbol{x}+\mathbf{a}\rangle_{p-1}$ and $^{A}\langle \mathbf{b}\rangle_{d}$. Hence, while the parties *can* compute their respective shares for $\langle \boldsymbol{x}\times\boldsymbol{y}+\mathbf{r}\rangle_{d+p-1}$, they *cannot* compute their respective MAC-tags for their shares (and the corresponding MAC-keys). In the absence of the MAC-tags, parties *cannot* identify the incorrect shares and robustly reconstruct degree-$(d+p-1)$-sharings, required as part of protocol Π_{DegRed}.

To deal with the above issue, we note that even though the parties *cannot* robustly reconstruct a degree-$(d+p-1)$ PSS, they can *detect* the presence of more than $\frac{t}{2}$ errors, since $d+p-1 = t+\frac{t}{2}, n = 3t+1$ and the degree of the corresponding RS code will be $t+\frac{t}{2}$. If the parties complaint about the presence of more than $\frac{t}{2}$ errors while reconstructing their designated degree-$(d+p-1)$ PSS as part of the instance of Π_{DegRed}, we can publicly open all the underlying $^{A}\langle\cdot\rangle_{p-1}$-sharings and all the underlying $^{A}\langle\cdot\rangle_{d}$-sharings, which are involved in the computation of the underlying degree-$(d+p-1)$ PSSs. From this, the complaining parties can locally identify more than $\frac{t}{2}$ corrupt parties, after which they will be able to robustly reconstruct their designated degree-$(d+p-1)$ PSSs and complete the remaining steps of Π_{DegRed}. We stress that a somewhat similar idea is also used in [53]. However, their protocol requires the parties to restart the protocol once the complaining parties identify the corrupt parties. Our protocol *do not* require any such restart. The details follow.

For $i \in \{1,\ldots,t+1\}$, let parties have the input $^{A}\langle \boldsymbol{x}^{(i)}\rangle_{d}$ and $^{A}\langle \boldsymbol{y}^{(i)}\rangle_{d}$ and the goal is to compute $^{A}\langle \boldsymbol{x}^{(i)}\times\boldsymbol{y}^{(i)}\rangle_{d}$ via protocol Π_{Beaver}, where each vector consists of $p-1$ elements from $\mathbb{F}$. The reason for computing $t+1$ products instead of one product is that it will lead to $\mathcal{O}(1)$ communication overhead. In fact, looking ahead, we will actually need n^3 batches of $t+1$ products to be computed for "several" vectors of elements to achieve $\mathcal{O}(1)$ communication overhead. However, for our current description, we focus only on one such batch of $t+1$ vectors. The parties will hold auxiliary inputs $(^{A}\langle \mathbf{a}^{(i)}\rangle_{d}, {}^{A}\langle \mathbf{b}^{(i)}\rangle_{d}, {}^{A}\langle \mathbf{c}^{(i)}\rangle_{d})$ and $(^{A}\langle \mathbf{r}^{(i)}\rangle_{d}, {}^{A}\langle \mathbf{r}^{(i)}\rangle_{d+p-1})$, where $(\mathbf{a}^{(i)},\mathbf{b}^{(i)},\mathbf{c}^{(i)})$ is supposed to be a vector of multiplication-triples. To deal with the problem discussed earlier, the parties now will hold *additional* auxiliary information $(^{A}\langle \hat{\mathbf{a}}^{(i)}\rangle_{d}, {}^{A}\langle \hat{\mathbf{b}}^{(i)}\rangle_{d}, {}^{A}\langle \hat{\mathbf{c}}^{(i)}\rangle_{d})$ and $(^{A}\langle \hat{\mathbf{r}}^{(i)}\rangle_{d}, {}^{A}\langle \hat{\mathbf{r}}^{(i)}\rangle_{d+p-1})$. Note that $(\hat{\mathbf{a}}^{(i)},\hat{\mathbf{b}}^{(i)},\hat{\mathbf{c}}^{(i)})$ *need not* be a vector of multiplication-triples. We refer to this additional auxiliary information as "sarficing randomness" as their privacy will be lost to identify the corrupt parties.

The parties will follow the first two steps outline earlier. During the third step, the parties locally compute

$$\langle \mathbf{s}^{(i)}\rangle_{d+p-1} = \langle \boldsymbol{x}^{(i)}+\mathbf{a}^{(i)}\rangle_{p-1}\langle \boldsymbol{y}^{(i)}+\mathbf{b}^{(i)}\rangle_{p-1} - \langle \boldsymbol{x}^{(i)}+\mathbf{a}^{(i)}\rangle_{p-1}\langle \mathbf{b}^{(i)}\rangle_{d} - \langle \boldsymbol{y}^{(i)}+\mathbf{b}^{(i)}\rangle_{p-1}\langle \mathbf{a}^{(i)}\rangle_{d} + \langle \mathbf{c}^{(i)}\rangle_{d} + \langle \mathbf{r}^{(i)}\rangle_{d+p-1},$$

where $\mathbf{s}^{(i)} = \boldsymbol{x}^{(i)}\times\boldsymbol{y}^{(i)}+\mathbf{r}^{(i)}$. The parties then proceed with the steps of Π_{DegRed} as follows: let $\mathbf{S}(X)$ be the vectors of p polynomials of degree-t each where $\mathbf{S}(X) = \mathbf{s}^{(1)} + \mathbf{s}^{(2)}\cdot X + \ldots + \mathbf{s}^{(t+1)}\cdot X^{t}$. That is, the coefficients of X^{j} in the polynomials in $\mathbf{S}(X)$ are the elements of the vector $\mathbf{s}^{(j+1)}$. For $i \in \{1,\ldots,n\}$, the parties locally compute $\langle \mathbf{S}(i)\rangle_{d+p-1}$ from $\langle \mathbf{s}^{(1)}\rangle_{d+p-1},\ldots,\langle \mathbf{s}^{(t+1)}\rangle_{d+p-1}$ and send their shares of $\langle \mathbf{S}(i)\rangle_{d+p-1}$ to P_i. Party P_i error-correct $\frac{t}{2}$ errors, assuming at most $\frac{t}{2}$ incorrect shares. If P_i fails to reconstruct any $\mathbf{S}(i)$ through error-correction, then it knows that more than $\frac{t}{2}$ corrupt parties have sent incorrect shares and so it complains publicly. On the other hand, if error-correction outputs any vector for P_i then it known for sure that it is the correct $\mathbf{S}(i)$. This is because the minimum distance of the underlying RS code (defined by degree-$(d+p-1)$ polynomials) is $t+\frac{t}{2}+1$ if $p = \frac{t}{4}+1$ and $n = 3t+1$. So even if adversary introduces t errors in the shares of $\langle \mathbf{S}(i)\rangle_{d+p-1}$, the received vector of shares will be at a distance of at least $\frac{t}{2}+1$ from the RS codeword of any other $\mathbf{S}'(i)$ where $\mathbf{S}'(i) \neq \mathbf{S}(i)$. Consequently, error-correction will *not* output any $\mathbf{S}'(i)$ different from $\mathbf{S}(i)$ for P_i. If there are no complaints from any party then the parties can complete the remaining steps of Π_{DegRed}. Otherwise, the parties proceed to resolve the complaints as follows by deploying the sacrificing randomness. For each $i \in \{1,\ldots,t+1\}$, the parties first locally compute

$$\langle \hat{\mathbf{s}}^{(i)}\rangle_{d+p-1} = \langle \boldsymbol{x}^{(i)}+\mathbf{a}^{(i)}\rangle_{p-1}\langle \boldsymbol{y}^{(i)}+\mathbf{b}^{(i)}\rangle_{p-1} - \langle \boldsymbol{x}^{(i)}+\mathbf{a}^{(i)}\rangle_{p-1}\langle \hat{\mathbf{b}}^{(i)}\rangle_{d} - \langle \boldsymbol{y}^{(i)}+\mathbf{b}^{(i)}\rangle_{p-1}\langle \hat{\mathbf{a}}^{(i)}\rangle_{d} + \langle \hat{\mathbf{c}}^{(i)}\rangle_{d} + \langle \hat{\mathbf{r}}^{(i)}\rangle_{d+p-1}.$$

Let $\hat{\mathbf{S}}(X)$ be the vectors of p polynomials of degree-t each where $\hat{\mathbf{S}}(X) = \hat{\mathbf{s}}^{(1)} + \hat{\mathbf{s}}^{(2)}\cdot X + \ldots + \hat{\mathbf{s}}^{(t+1)}\cdot X^{t}$. For $i \in \{1,\ldots,n\}$, the parties locally compute $\langle \hat{\mathbf{S}}(i)\rangle_{d+p-1}$ from $\langle \hat{\mathbf{s}}^{(1)}\rangle_{d+p-1},\ldots,\langle \hat{\mathbf{s}}^{(t+1)}\rangle_{d+p-1}$ and send

their shares of $\langle \hat{\mathbf{S}}(i) \rangle_{d+p-1}$ to P_i. Note that if more than $\frac{t}{2}$ corrupt parties send incorrect shares to P_i, then P_i *cannot* reconstruct the vector $\hat{\mathbf{S}}(i)$. The parties next collectively generate a random value, say R. For each $j \in \{1, \ldots, t+1\}$, consider the vector $\mathbf{s}^{(j)} + R \cdot \hat{\mathbf{s}}^{(j)}$, which is same as as

$$(R+1)\cdot(\boldsymbol{x}^{(j)}+\mathbf{a}^{(j)})\times(\boldsymbol{y}^{(j)}+\mathbf{b}^{(j)})-(\boldsymbol{x}^{(j)}+\mathbf{a}^{(j)})\times(\mathbf{b}^{(j)}+R\cdot\hat{\mathbf{b}}^{(j)})-(\boldsymbol{y}^{(j)}+\mathbf{b}^{(j)})\times(\mathbf{a}^{(j)}+R\cdot\hat{\mathbf{a}}^{(j)})+(\mathbf{c}^{(j)}+R\cdot\hat{\mathbf{c}}^{(j)})+(\mathbf{r}^{(j)}+R\cdot\hat{\mathbf{r}}^{(j)}).$$

It then turns out that the vector of points $\mathbf{S}(i) + R \cdot \hat{\mathbf{S}}(i)$ on the polynomials in $\mathbf{S}(X) + R \cdot \hat{\mathbf{S}}(X)$ is a publicly-known linear combination of the vectors $\{\mathbf{s}^{(j)} + R \cdot \hat{\mathbf{s}}^{(j)}\}_{j \in \{1,\ldots,t+1\}}$. So if we reconstruct the vectors $\{\mathbf{s}^{(j)} + R \cdot \hat{\mathbf{s}}^{(j)}\}_{j \in \{1,\ldots,t+1\}}$ for the parties P_i who complained about not being able to reconstruct their supposed vector $\mathbf{S}(i)$ (and also possibly $\hat{\mathbf{S}}(i)$), then P_i can recompute $\langle \mathbf{S}(i) + R \cdot \hat{\mathbf{S}}(i) \rangle_{d+p-1}$ and find out the identity of more than $\frac{t}{2}$ corrupt parties who sent incorrect shares for $\langle \mathbf{S}(i) \rangle_{d+p-1}$ (and also possibly for $\langle \hat{\mathbf{S}}(i) \rangle_{d+p-1}$). Then by ignoring the shares of those corrupt parties, P_i will be left with sufficiently many shares to error-correct the remaining incorrect shares (which are at most $\frac{t}{2} - 1$) on degree-$(d + p - 1)$ polynomials to robustly reconstruct the vector $\mathbf{S}(i)$ and complete the rest of the steps of the protocol Π_{DegRed}. The idea here is that the corrupt parties while sending incorrect shares for $\langle \mathbf{S}(i) \rangle_{d+p-1}$ (and also possibly for $\langle \hat{\mathbf{S}}(i) \rangle_{d+p-1}$) to P_i will have no information about the random R, which is generated later. If we can ensure that P_i computes $\langle \mathbf{S}(i) + R \cdot \hat{\mathbf{S}}(i) \rangle_{d+p-1}$ correctly, then with a very high probability, P_i will be able to identify the incorrect shares.

To enable P_i compute the vectors $\{\mathbf{s}^{(j)}+R\cdot\hat{\mathbf{s}}^{(j)}\}_{j\in\{1,\ldots,t+1\}}$, it is sufficient for P_i to robustly reconstruct the vectors $\{(\boldsymbol{x}^{(j)} + \mathbf{a}^{(j)}), (\boldsymbol{y}^{(j)} + \mathbf{b}^{(j)}), (\mathbf{b}^{(j)} + R \cdot \hat{\mathbf{b}}^{(j)}), (\mathbf{a}^{(j)} + R \cdot \hat{\mathbf{a}}^{(j)}), (\mathbf{c}^{(j)} + R \cdot \hat{\mathbf{c}}^{(j)}), (\mathbf{r}^{(j)} + R \cdot \hat{\mathbf{r}}^{(j)})\}_{j \in \{1,\ldots,t+1\}}$. This is possible since the parties can locally compute a ${}^{A}\langle \cdot \rangle_d$-sharing of each of these vectors (as R will be known publicly) and hence reconstruct these ${}^{A}\langle \cdot \rangle_d$-sharings towards P_i. Note that the privacy of multiplicands $\boldsymbol{x}^{(j)}$ and $\boldsymbol{y}^{(j)}$ is maintained, as they are masked with random $\mathbf{a}^{(j)}$ and $\mathbf{b}^{(j)}$. Moreover, the privacy of $\mathbf{a}^{(j)}, \mathbf{b}^{(j)}, \mathbf{c}^{(j)}$ and $\mathbf{r}^{(j)}$ is preserved, as they are further masked by $\hat{\mathbf{a}}^{(j)}, \hat{\mathbf{b}}^{(j)}, \hat{\mathbf{c}}^{(j)}$ and $\hat{\mathbf{r}}^{(j)}$ respectively.

A small issue above is that we may need to reconstruct $\Theta(n)$ ${}^{A}\langle \cdot \rangle_d$-sharings for $\Theta(n)$ complaining parties to resolve the complaints and complete the steps of Π_{DegRed}. This will lead to a communication overhead of $\mathcal{O}(n)$. A simple fix will be to run the steps of Π_{Beaver} for "multiple" batches in parallel, where in each batch we perform multiplications of $t + 1$ pairs of vectors. If any complaint arises during the instances of Π_{DegRed}, then we just pick one batch and resolve the complaints of that batch only, through which the complaining parties will identify more than $\frac{t}{2}$ corrupt parties, whose shares they can ignore for all the batches. Looking ahead, considering n^3 batches will help us to achieve $\mathcal{O}(1)$ communication overhead.

2.1.3 Statistically-Secure Polynomial Verification with $\mathcal{O}(1)$ Overhead.

We can instantiate protocol Π_{PolyVer} with statistical security very easily by verifying the vectors of polynomials at a randomly generated point and checking if the multiplicative relationship among the vectors of polynomials is satisfied. Since the random point will not be known apriori, if the test passes then it will be guaranteed that with a high probability, the vectors of polynomials indeed satisfy the multiplicative relationship.

2.2 Perfectly-Secure $\Pi_{\mathsf{VSS}}, \Pi_{\mathsf{RecPriv}}$ and Π_{PolyVer} with $\mathcal{O}(1)$ Overhead

We now discuss how we can instantiate the blueprint of Fig 1 with perfect security. While our protocol works for *any* $n = 3t + s$ where $s = \Theta(n)$, for the ease of explaining the technical overview, we assume $n = 3t + \frac{t}{2} + 1$. Hence we can fix the packing parameter $p = \frac{t}{4} + 1$ in this case as well. Consequently, $d = t + p - 1 = t + \frac{t}{4}$. The first thing to note here is that unlike the case of $n = 3t+1$, one can robustly reconstruct degree-d as well as degree-$d + p - 1$ PSS easily using standard RS error-correction. Consequently, we *do not* need authenticated-secret-sharings for robust reconstruction and protocol Π_{RecPriv} can be instantiated using RS error-correction.

To instantiate Π_{VSS} with perfect security, we look into the work of [1]. With $n = 3t+1$, they provided a protocol which can generate degree-ℓ PSS of a vector of p values, where $\ell \in \{p - 1, d, d + p - 1\}$. The protocol incurs a communication overhead of $\mathcal{O}(1)$, if sufficiently many vectors have to be packed-secret-shared.

Protocol Π_{PolyVer} can be instantiated by extending the existing perfectly-secure protocol for polynomial verification of a single triplet of polynomials [23] to vectors of triplets of polynomials. The idea will

be to designate each party P_i to verify a designated *distinct* vector of points on the vectors of polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot)$ for which we want to check the multiplicative relationship and check if those vectors of points satisfy the multiplicative relationship and complaint publicly if that is not the case. If any party P_i complaints then the complaint is resolved publicly by making public the vectors of points which P_i supposedly verify and doing the verification on behalf of P_i publicly. The complaint is then discarded if the public verification is successful, else the parties conclude that the vectors of polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot)$ *do not* satisfy the multiplicative relationship. The idea here is that if there are no complaints or if all the complaints are publicly discarded, then depending upon the degree of the polynomials in polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot)$, we can conclude that they satisfy the multiplicative relationship. Specifically, if the degree of polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ are $\frac{n-1-t}{2}, \frac{n-1-t}{2}$ and $n-1-t$ respectively and if all the complaints (if any) are discarded, then it follows that the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot)$ satisfy the multiplicative relationship for at least $n-t$ distinct points, corresponding to the honest parties. Consequently, the polynomials do satisfy the multiplicative relationship.

Since all the building blocks for instantiating the blueprint of Fig 1 with perfect security can be obtained by extending the existing perfectly-secure building blocks, we give the details of our perfectly-secure MPC protocol for SIMD circuits with a constant overhead in Appendix B.

3 An observation on lower bounds

In [26], the following result was shown:

Theorem 3.1. *Let $n = 2t + s$ with $s > 0$. There exists a function $\widehat{IP}_{I,n}$ with circuit complexity $O(nI)$ such that in any protocol computing $\widehat{IP}_{I,n}$ with statistical security against t passive corruptions, the average total communication complexity is at least $\Theta(ntI/s)$*

Clearly, Theorem 3.1 also holds for protocols that are secure against active corruptions, even if we only require security with abort. This is because an adversary could corrupt t parties but let them play according to the protocol, and a non-trivial abort secure protocol must terminate normally if all players follow the protocol. So, such a protocol implies a passively secure protocol for the same function and we can apply the theorem[12].

If we consider instead an asynchronous network, we can get a similar result, assuming abort security against t active corruptions and $n = 3t + s$ players. This is because the following three scenarios are indistinguishable: 1) all players are honest but the adversary delays the messages from t players. 2) t players are corrupt and never send messages. 3) t players are corrupt but play according to the protocol, and the adversary delays messages from t honest players. Now, an abort secure protocol must terminate normally in scenario 1) since all players are honest, and it cannot wait for the t delayed players as it would then deadlock in scenario 2. So, the protocol must terminate normally and be secure in scenario 3 as well. It therefore implies a passively secure protocol for $n = 2t + s$ players and we can apply the theorem.

Summarizing, we have

Corollary 3.2. *Let $n = 2t + s$. There exists a function $\widehat{IP}_{I,n}$ with circuit complexity $O(nI)$ such that in any protocol running on a synchronous network computing $\widehat{IP}_{I,n}$ with statistical abort security against t active corruptions, the average total communication complexity is at least $\Theta(ntI/s)$.*

Let $n = 3t + s$. There exists a function $\widehat{IP}_{I,n}$ with circuit complexity $O(nI)$ such that in any protocol running on an asynchronous network computing $\widehat{IP}_{I,n}$ with statistical abort security against t active corruptions, the average total communication complexity is at least $\Theta(ntI/s)$.

[12] In some contrived cases, active security does not imply passive security, but this is not the case for the functions considered in the theorem, where all input bit may potentially influence the output.

Part I

Synchronous Setting

4 Preliminaries

We assume a set of parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ connected by pairwise private and authentic channels (namely the secure-channel model), where t is the maximum number of parties, which can be under the control of a computationally-unbounded malicious (Byzantine) adversary $\mathcal{A}$. We describe our protocols assuming $n = 3t+1$ which allows for clean protocol presentation. Later, we will discuss how to modify our building-blocks and the protocol for any $n = 2t+\Theta(t)$. In this part of the paper, we assume a *synchronous* communication network, where the parties are synchronized by a global clock and every sent message is guaranteed to be delivered within some publicly known time. Consequently, the protocols operate as a sequence of communication rounds. The communication complexity of any protocol is defined to be the total number of bits sent by the *honest* parties, namely the parties which are not under adversary's control.

Let κ be the statistical security parameter. We consider functions that can be represented as arithmetic circuits over a finite field $\mathbb{F}$ where $|\mathbb{F}| \geq 2^\kappa$, consisting of input, addition, multiplication and output gates. For any circuit $\texttt{ckt}$ satisfying the above conditions, we have $|\mathbb{F}| \geq |\texttt{ckt}| + n$. This is because $|\mathbb{F}| \geq 2^\kappa$ and both $|\texttt{ckt}|$ and n are polynomial in κ. This also implies that $|\mathbb{F}| \geq 2n$. For simplicity and without loss of generality we assume that the elements $1, \ldots, n \in \mathbb{F}$ (and consequently $-1, \ldots, -n \in \mathbb{F}$ as well). Looking ahead, we will use the points $1, \ldots, n$ to determine the shares of $P_1, \ldots, P_n$ respectively, while secrets will be embedded at $-1, \ldots, -n$.

We will be operating on vectors of values in our protocols for which we use various fonts. Variables like $\mathbf{a}, \mathbf{b}, \mathbf{c}$ will denote one-dimensional vectors of values over $\mathbb{F}$. Notations like $\mathbf{A}, \mathbf{B}, \mathbf{C}$ will denote two-dimensional vectors (matrices) over $\mathbb{F}$. In other words, these denote a collection of one-dimensional vectors of values over $\mathbb{F}$. Finally, variables like $\mathbb{A}, \mathbb{B}, \mathbb{C}$ will denote three-dimensional vectors over $\mathbb{F}$. That is they denote a collection of two-dimensional vectors over $\mathbb{F}$.

In our protocols, we will be using the following standard property of Reed-Solomon codes.

Theorem 4.1 ([47]). *Let C be an Reed-Solomon (RS) code word of length N, corresponding to a k-degree polynomial (containing $k+1$ coefficients) such that the distance of the code is $d = N-k$. Let $\bar{C}$ be a word of length N such that the distance between C and $\bar{C}$ can be at most T. Moreover, let e be parameter such that $e \leq T$ and $d > T+e$. Furthermore, suppose we apply the RS decoding algorithm to $\bar{C}$ to error-correct e errors. Then the following hold.*

- *If the decoding algorithm outputs a codeword C' then $C' = C$;*
- *Else the decoding algorithm detects the presence of more than e errors in $\bar{C}$.*

4.1 Information-Theoretic Message Authentication Codes

We will use information-theoretically secure message-authentication codes (MAC). Here a random pair $\mathsf{K} = (\boldsymbol{\mu}, v)$ is selected as the MAC-key, where $\boldsymbol{\mu} \in \mathbb{F}^L$ and $v \in \mathbb{F}$. The MAC-tag on a vector of values $\mathbf{s} \in \mathbb{F}^L$ is defined as $\mathsf{MAC}_\mathsf{K}(\mathbf{s}) = \tau \stackrel{\mathsf{def}}{=} \boldsymbol{\mu} \cdot \mathbf{s} + v$, where $\boldsymbol{\mu} \cdot \mathbf{s}$ denotes the vector dot-product of $\boldsymbol{\mu}$ and $\mathbf{s}$. The MACs will be used as follows: a party P_i will hold some value $\mathbf{s} \in \mathbb{F}^L$ and a MAC tag τ, while party P_j will hold the key K. Later, when P_i wants to reveal $\mathbf{s}$ to P_j, it sends $\mathbf{s}$, along with τ. Party P_j then verifies the revealed data and the tag, by checking their consistency with respect to the key K held by P_j. One can show that if each component of $\mathsf{K} = (\boldsymbol{\mu}, v)$ is selected uniformly at random, then the probability that a *corrupt* P_i can find a *different* $(\mathbf{s}', \tau')$, such that $\tau' = \boldsymbol{\mu} \cdot \mathbf{s}' + v$ holds is at most $2^{-\Omega(\kappa)}$.

We call two MAC keys $\mathsf{K} = (\boldsymbol{\mu}, v)$ and $\mathsf{K}' = (\boldsymbol{\mu}', v')$ as *consistent* if $\boldsymbol{\mu} = \boldsymbol{\mu}'$ holds. Given two consistent MAC keys $\mathsf{K} = (\boldsymbol{\mu}, v)$ and $\mathsf{K}' = (\boldsymbol{\mu}, v')$ and a publicly-known constant $c \in \mathbb{F}$, we define the following operations on the MAC keys:

$$\mathsf{K} + \mathsf{K}' \stackrel{\mathsf{def}}{=} (\boldsymbol{\mu}, v + v'), \quad \mathsf{K} + c \stackrel{\mathsf{def}}{=} (\boldsymbol{\mu}, v + \boldsymbol{\mu} \cdot \mathbf{c}) \quad \text{and} \quad c \cdot \mathsf{K} \stackrel{\mathsf{def}}{=} (\boldsymbol{\mu}, cv),$$

where $\mathbf{c} \stackrel{\mathsf{def}}{=} (c, \ldots, c$ (L times)). Given consistent MAC keys K, K' and a publicly-known constant $c \in \mathbb{F}$, the following linearity property holds for the MAC, for any $\mathbf{s}, \mathbf{s}' \in \mathbb{F}^L$.

- **Addition**:

$$\mathsf{MAC}_\mathsf{K}(\mathbf{s}) + \mathsf{MAC}_{\mathsf{K}'}(\mathbf{s}') = \mathsf{MAC}_{\mathsf{K}+\mathsf{K}'}(\mathbf{s} + \mathbf{s}').$$

– **Addition/Subtraction by a Constant**: Let $\mathbf{c} \stackrel{\mathsf{def}}{=} (c, \ldots, c \ (L \text{ times}))$. Then:

$$\mathsf{MAC}_{\mathsf{K}-c}(\mathbf{s}+\mathbf{c}) = \mathsf{MAC}_{\mathsf{K}}(\mathbf{s}) \quad \text{and} \quad \mathsf{MAC}_{\mathsf{K}+c}(\mathbf{s}-\mathbf{c}) = \mathsf{MAC}_{\mathsf{K}}(\mathbf{s}).$$

– **Multiplication by a Constant**:

$$c \cdot \mathsf{MAC}_{\mathsf{K}}(\mathbf{s}) = \mathsf{MAC}_{c\cdot\mathsf{K}}(c \cdot \mathbf{s}).$$

4.2 Various Sharing Semantics and their Properties

In this section, we discuss the various kind of secret-sharings used in our protocols. The basic secret-sharing is the standard degree-ℓ Shamir sharing [51], where the secret is embedded in the constant term of a degree-ℓ polynomial and every party has a distinct point on the polynomial as its share.

Definition 4.2 (ℓ**-sharing).** *A value $s \in \mathbb{F}$ is said to be ℓ-shared, if there exists some degree-ℓ polynomial over $\mathbb{F}$, say $f(\cdot)$, with $f(0) = s$, such that each (honest) party P_i has a share $s_i \stackrel{\mathsf{def}}{=} f(i)$ of s. A vector of shares of a ℓ-sharing of s corresponding to the (honest) parties is denoted by $[s]_\ell$.*

Let $\mathbf{s} = \left(s^{(1)}, \ldots, s^{(M)}\right)$ be a vector of values over $\mathbb{F}^M$, where $M \geq 1$ and known publicly. We say that $\mathbf{s}$ is ℓ-shared, provided each element $s^{(m)} \in \mathbf{s}$ is ℓ-shared. A vector of shares of a ℓ-sharing of $\mathbf{s}$ corresponding to the honest parties is denoted as $[\mathbf{s}]_\ell$. That is, $[\mathbf{s}]_\ell \stackrel{\mathsf{def}}{=} \left[s^{(1)}\right]_\ell, \ldots, \left[s^{(M)}\right]_\ell$.

The next secret-sharing is a "twisted" version of ℓ-sharing, where the slots for the secret and the share of a designated party in the underlying sharing polynomial are "exchanged".

Definition 4.3 (Twisted ℓ-sharing). *A value $s \in \mathbb{F}$ is said to be twisted ℓ-shared with respect to a designated party $P_i \in \mathcal{P}$, provided there exists some degree-ℓ polynomial, say $f(\cdot)$, where $f(i) = s$ and every (honest) party $P_j \in \mathcal{P} \setminus \{P_i\}$ holds its twisted-share $s_j \stackrel{\mathsf{def}}{=} f(j)$. The twisted-share s_i of P_i is defined to be $s_i \stackrel{\mathsf{def}}{=} f(0)$. A vector of twisted-shares of a twisted ℓ-sharing of s corresponding to the honest parties is denoted as $\lceil s \rfloor_\ell^i$.*

A vector of values $\mathbf{s} = \left(s^{(1)}, \ldots, s^{(M)}\right) \in \mathbb{F}^M$ is said to be twisted ℓ-shared with respect to P_i, provided each element $s^{(m)} \in \mathbf{s}$ is twisted ℓ-shared with respect to P_i. A vector of twisted-shares of a twisted ℓ-sharing of $\mathbf{s}$ with respect to P_i, corresponding to the honest parties is denoted as $\lceil \mathbf{s} \rfloor_\ell^i$. That is, $\lceil \mathbf{s} \rfloor_\ell^i \stackrel{\mathsf{def}}{=} \lceil s^{(1)} \rfloor_\ell^i, \ldots, \lceil s^{(M)} \rfloor_\ell^i$.

Typically for ℓ-sharing the evaluation point 0 is used to store the underlying secret. We will be also using a variant of ℓ-sharing (and twisted ℓ-sharing) where the underlying secret will be stored at different evaluation points.

Definition 4.4 (ℓ**-sharing with Different Secret Slots** [37]). *For any $i \in \{1, p+1\}$, we use $[s||i]_\ell$ to denote a ℓ-sharing of s where s is stored at the evaluation point $-i$. That is, if $f(\cdot)$ is underlying degree-ℓ sharing polynomial then $f(-i) = s$ holds. The notation $\lceil s||i \rfloor_\ell^i$ will denote a degree-ℓ twisted-sharing of s where s is stored at the evaluation point $-i$.*

The next secret-sharing is packed secret-sharing, a concept introduced in [39], which is an extension of the Shamir's sharing, where multiple values are shared through a single sharing-polynomial.

Definition 4.5 (Packed Secret Sharing (PSS) [39]). *Let $\mathbf{s} = (s^{(1)}, \ldots, s^{(k)}) \in \mathbb{F}^k$. We say that $\mathbf{s}$ is degree-ℓ packed-secret-shared where $\ell \geq k-1$, if there exists some degree-ℓ polynomial over $\mathbb{F}$, say $f(\cdot)$, where $f(-1) = s^{(1)}, \ldots, f(-k) = s^{(k)}$ holds and each (honest) party $P_i \in \mathcal{P}$ has its share $s_i \stackrel{\mathsf{def}}{=} f(i)$. A vector of shares for a degree-ℓ packed secret-sharing of $\mathbf{s}$, corresponding to the honest parties is denoted by $\langle \mathbf{s} \rangle_\ell$.*

We extend the above for m batches of k values. Let $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)})$, where $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)} \in \mathbb{F}^k$. Then $\langle \mathbf{S} \rangle_\ell \stackrel{\mathsf{def}}{=} \langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(m)} \rangle_\ell$.

We extend the above in another dimension. Let $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(\ell)})$ be vectors of values where each $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}), \ldots, \mathbf{s}^{(u,m)})$ and where each $\mathbf{s}^{(u,v)} \in \mathbb{F}^k$. Hence, each $\mathbf{S}^{(u)} \in \mathbb{F}^{mk}$. Then $\langle \mathbb{S} \rangle_\ell \stackrel{\mathsf{def}}{=} \langle \mathbf{S}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{S}^{(\ell)} \rangle_\ell$.

Note that in $\langle \mathbf{s} \rangle_\ell$, if the underlying sharing polynomial is a random degree-ℓ polynomial, then the probability distribution of any subset of up to $\ell + 1 - k$ shares is independent of $\mathbf{s}$. This implies that if $\ell \geq t + k - 1$, the probability distribution of any subset of up to t shares is independent of $\mathbf{s}$.

Next, in an authenticated packed-secret-sharing of $\langle \mathbf{S} \rangle_\ell$, the parties hold an MAC on their respective vector of shares.

Definition 4.6 (Authenticated Packed Secret Sharing (APSS)). *Let $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)})$, where $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)} \in \mathbb{F}^k$. We say that $\mathbf{S}$ is degree-ℓ packed-secret-shared with authentication where $\ell \geq k - 1$, if all the following hold.*

- *Parties hold $\langle \mathbf{S} \rangle_\ell$.*
- *Every party $P_i \in \mathcal{P}$ holds a MAC tag on all its shares for a key held by every party P_j. That is, let $\mathbf{s}_i \in \mathbb{F}^m$ be the vector of shares of P_i, corresponding to $\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(m)} \rangle_\ell$. Then there exists a MAC key $\mathsf{K}_{ji} = (\boldsymbol{\mu}_{ji}, v_{ji}) \in \mathbb{F}^{m+1}$ held by P_j, such that P_i holds the MAC tag $\tau_{ij} \stackrel{\mathsf{def}}{=} \mathsf{MAC}_{\mathsf{K}_{ji}}(\mathbf{s}_i)$.*

We denote by ${}^A\langle \mathbf{S} \rangle_\ell$ the vector of shares, MAC keys and the MAC tags of $\mathbf{S}$ corresponding to all the (honest) parties in $\mathcal{P}$.[13] *That is:*

$$ {}^A\langle \mathbf{S} \rangle_\ell \stackrel{\mathsf{def}}{=} \Big\{ \mathbf{s}_i, (\tau_{i1}, \ldots, \tau_{in}), (\mathsf{K}_{i1}, \ldots, \mathsf{K}_{in}) \Big\}_{i=1}^{n}. $$

We extend the above in another dimension. Let $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)})$ be vectors of values where each $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}), \ldots, \mathbf{s}^{(u,m)})$ and where each $\mathbf{s}^{(u,v)} \in \mathbb{F}^k$. Hence, each $\mathbf{S}^{(u)} \in \mathbb{F}^{mk}$. Then ${}^A\langle \mathbb{S} \rangle_\ell \stackrel{\mathsf{def}}{=} {}^A\langle \mathbf{S}^{(1)} \rangle_\ell, \ldots, {}^A\langle \mathbf{S}^{(L)} \rangle_\ell$.

Two sharings ${}^A\langle \mathbf{S} \rangle_{\ell_1}$ and ${}^A\langle \mathbf{S}' \rangle_{\ell_2}$, where $|\mathbf{S}| = |\mathbf{S}'|$ are said to be key-consistent if every party $P_i \in \mathcal{P}$ holds consistent MAC keys for every $P_j \in \mathcal{P}$ across both ${}^A\langle \mathbf{S} \rangle_{\ell_1}$ and ${}^A\langle \mathbf{S}' \rangle_{\ell_2}$.

Two sharings ${}^A\langle \mathbb{S} \rangle_{\ell_1}$ and ${}^A\langle \mathbb{S}' \rangle_{\ell_2}$, where $|\mathbb{S}| = |\mathbb{S}'|$ are said to be key-consistent if every party $P_i \in \mathcal{P}$ holds consistent MAC keys for every $P_j \in \mathcal{P}$ across both ${}^A\langle \mathbb{S} \rangle_{\ell_1}$ and ${}^A\langle \mathbb{S}' \rangle_{\ell_2}$.

For a pictorial illustration of ${}^A\langle \mathbf{S} \rangle_\ell$, see Fig 2.

The next secret-sharing is a special kind of PSSA where the same values are secret-shared through two different degrees.

Definition 4.7 (Double Authenticated Packed Secret Sharing (APSS)). *Let $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)})$, where $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)} \in \mathbb{F}^p$. Here p is a given packing parameter and let $d = t + p - 1$. We say that $\mathbf{S}$ is double-packed-secret-shared with authentication, if the parties hold ${}^A\langle \mathbf{S} \rangle_d$ as well as ${}^A\langle \mathbf{S} \rangle_{d+p-1}$.*

Let $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)})$ be vectors of values where each $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}), \ldots, \mathbf{s}^{(u,m)})$ and where each $\mathbf{s}^{(u,v)} \in \mathbb{F}^p$. Then $\mathbb{S}$ is said to be double-packed-secret-shared with authentication, if the parties hold ${}^A\langle \mathbf{S}^{(u)} \rangle_d$ as well as ${}^A\langle \mathbf{S}^{(u)} \rangle_{d+p-1}$, for each $u \in \{1, \ldots, L\}$.

The definition of key-consistent sharing extends naturally for double-packed-secret-sharing with authentication.

The final secret sharing is a twisted variant of packed secret sharing.

Definition 4.8 (degree-ℓ Degenerate Packed Twisted Sharing [44]). *Let $\mathbf{s} = (s^{(0)}, \ldots, s^{(\ell)}) \in \mathbb{F}^{\ell+1}$. A degree-$\ell$ degenerate packed secret-sharing of $\mathbf{s}$ with respect to a designated party $P_i \in \mathcal{P}$ consists of n shares $s_1, \ldots, s_n \in \mathbb{F}$, such that the following hold:*

- *There exists a degree-ℓ sharing polynomial $f(X)$ over $\mathbb{F}$ such that $f(i) = s^{(0)}$ and $f(-1) = s^{(1)}, \ldots, f(-(i-1)) = s^{(i-1)}, f(-(i+1)) = s^{(i+1)}, \ldots, f(-\ell)) = s^{(\ell)}$.*
- *Every party $P_j \in \mathcal{P} \setminus \{P_i\}$ has the share $s_j = f(j)$.*
- *Party P_i has the share $s_i = f(0)$.*

We denote by $[[\mathbf{s}]]_\ell^i$ the vector of shares $(s_1, \ldots, s_n)$ corresponding to a degree-ℓ degenerate packed secret-sharing of $\mathbf{s}$ with respect to P_i. Note that the vector $[[\mathbf{s}]]_\ell^i$ is completely determined by the vector $\mathbf{s}$.

Looking ahead, our VSS protocol will generate ${}^A\langle \cdot \rangle_\ell$-sharings with $\mathcal{O}(1)$ communication overhead and our circuit evaluation will happen over ${}^A\langle \cdot \rangle_\ell$-shared values. The linear gates are evaluated non-interactively because of linearity of our secret-sharings and key-consistency among the sharings. In the next section we discuss linearity properties of our various sharings semantics for completeness.

[13] The subscript A in the notation ${}^A\langle \cdot \rangle$ denotes authentication (MACs), which is the additional component compared to $\langle \cdot \rangle$.

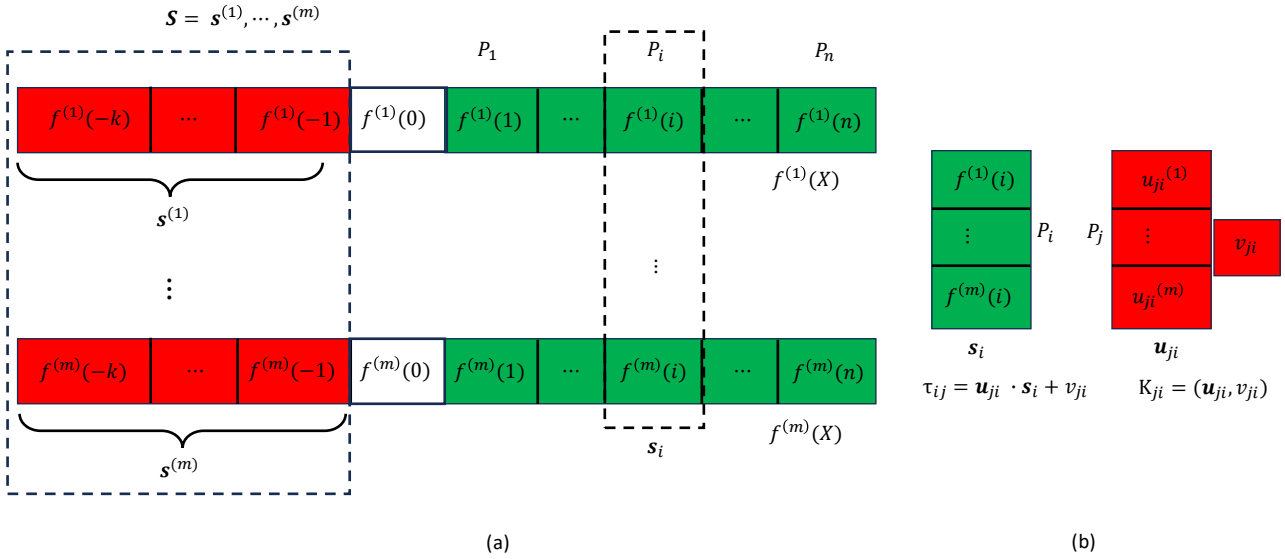

Fig. 2: Pictorial illustration of ${}^{A}\langle \mathbf{S} \rangle_{\ell}$, a degree-$\ell$ authenticated packed-secret-sharing of $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)})$. As shown in part (a), $\mathbf{S}$ consists of vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)}$, each consisting of k secrets, which are shown in red color. The elements in these vectors are embedded in degree-ℓ polynomials $f^{(1)}(X), \ldots, f^{(m)}(X)$ respectively, where $\ell \geq k-1$. The secrets are embedded in the polynomials at points $-1, \ldots, -k$. As a share, each party P_i gets the i^{th} point on the m polynomials. Vector $\mathbf{s}_i$ denotes the vector of m shares held by P_i. Part (b) denotes the authentication information held by the parties. For every pair of parties (P_i, P_j), party P_j will have a MAC key $\mathsf{K}_{ji} = (\boldsymbol{\mu}_{ji}, v_{ji})$, where $\boldsymbol{\mu}_{ji} \in \mathbb{F}^m$. Party P_i will have the tag $\tau_{ij} = \mathsf{MAC}_{\mathsf{K}_{ji}}(\mathbf{s}_i) = \boldsymbol{\mu}_{ji} \cdot \mathbf{s}_i + v_{ji}$

4.2.1 Properties of the various Sharings

In this section, we overview the linearity and non-linearity properties of the secret sharings.

Properties of ℓ-sharing. Given $[a]_{\ell_1} = \{a_i\}_{i=1}^n$ and $[b]_{\ell_2} = \{b_i\}_{i=1}^n$ where $\ell_1 \geq \ell_2$ and public constants $c_1, c_2 \in \mathbb{F}$, we have:

- *Linearity*: To compute $[c_1 a + c_2 b]_{\ell_1}$, each $P_i \in \mathcal{P}$ needs to locally compute $c_1 a_i + c_2 b_i$,

$$c_1 \cdot [a]_{\ell_1} + c_2 \cdot [b]_{\ell_2} = [c_1 a + c_2 b]_{\ell_1} = \{c_1 a_i + c_2 b_i\}_{i=1}^n.$$

- *Multiplicativity*: To compute $[ab]_{\ell_1+\ell_2}$ such that $\ell_1 + \ell_2 < n$, each $P_i \in \mathcal{P}$ needs to locally compute $a_i b_i$,

$$[a]_{\ell_1} \cdot [b]_{\ell_2} = [ab]_{\ell_1+\ell_2} = \{a_i b_i\}_{i=1}^n.$$

Similarly, given $[\mathbf{a}]_{\ell_1}$ and $[\mathbf{b}]_{\ell_2}$, where $\mathbf{a} = (a^{(1)}, \ldots, a^{(M)})$ and $\mathbf{b} = (b^{(1)}, \ldots, b^{(M)})$ and $\ell_1 \geq \ell_2$, along with public constants $c_1, c_2 \in \mathbb{F}$, we have:

- *Linearity*: The parties can locally compute $[c_1 \cdot \mathbf{a} + c_2 \cdot \mathbf{b}]_{\ell_1}$ as

$$c_1 \cdot [\mathbf{a}]_{\ell_1} + c_2 \cdot [\mathbf{b}]_{\ell_2} = [c_1 \cdot \mathbf{a} + c_2 \cdot \mathbf{b}]_{\ell_1}.$$

- *Multiplicativity*: The parties can locally compute $[\mathbf{a} \cdot \mathbf{b}]_{\ell_1+\ell_2}$, where $\ell_1 + \ell_2 < n$ as

$$[\mathbf{a}]_{\ell_1} \cdot [\mathbf{b}]_{\ell_2} = [\mathbf{a} \cdot \mathbf{b}]_{\ell_1+\ell_2}.$$

Properties of twisted ℓ-sharing. Since twisted ℓ-sharing is just a variant of ℓ-sharing, the above stated properties of ℓ-sharings hold for twisted ℓ-sharing as well and so we do not review them formally.

Properties of packed secret sharing. Let $\mathbf{a} = (a^{(1)}, \ldots, a^{(k)})$ and $\mathbf{b} = (b^{(1)}, \ldots, b^{(k)})$. Given $\langle \mathbf{a} \rangle_{\ell_1} = \{a_i\}_{i=1}^n$ and $\langle \mathbf{b} \rangle_{\ell_1} = \{b_i\}_{i=1}^n$ where $\ell_1 \geq \ell_2$ and $\ell_1 + \ell_2 \leq n$ and public constants $c_1, c_2 \in \mathbb{F}$, we have:

- *Linearity*: Let $\mathbf{c} = c_1 \cdot \mathbf{a} + c_2 \cdot \mathbf{b} = (c_1 \cdot a^{(1)} + c_2 \cdot b^{(1)}, \ldots, c_1 \cdot a^{(k)} + c_2 \cdot b^{(k)})$. To compute $\langle \mathbf{c} \rangle_{\ell_1}$, each P_i locally needs to compute $c_1 a_i + c_2 b_i$,

$$c_1 \cdot \langle \mathbf{a} \rangle_{\ell_1} + c_2 \cdot \langle \mathbf{b} \rangle_{\ell_2} = \langle \mathbf{c} \rangle_{\ell_1} = \{c_1 a_i + c_2 b_i\}_{i=1}^n.$$

- *Multiplicativity*: Let $\mathbf{c} = \mathbf{a} \times \mathbf{b} = (a^{(1)} b^{(1)}, \ldots, a^{(k)} b^{(k)})$ be the component-wise product of the elements of $\mathbf{a}$ and $\mathbf{b}$. To compute $\langle \mathbf{c} \rangle_{\ell_1 + \ell_2}$, each P_i needs to locally compute $a_i b_i$,

$$\langle \mathbf{a} \rangle_{\ell_1} \times \langle \mathbf{b} \rangle_{\ell_2} = \langle \mathbf{c} \rangle_{\ell_1 + \ell_2} = \{a_i b_i\}_{i=1}^n.$$

The above properties carry over for $\langle \mathbf{S} \rangle_\ell$ and $\langle \mathbb{S} \rangle_\ell$.

Properties of Authenticated packed secret-sharing. Let $\mathbf{A} = (\mathbf{a}^{(1)}, \ldots, \mathbf{a}^{(m)})$ and $\mathbf{B} = (\mathbf{b}^{(1)}, \ldots, \mathbf{b}^{(m)})$, where each $\mathbf{a}^{(j)}, \mathbf{b}^{(j)} \in \mathbb{F}^k$. Given ${}^A\langle \mathbf{A} \rangle_{\ell_1} = \left\{ \mathbf{a}_i, (\tau_{i1}, \ldots, \tau_{in}), (\mathsf{K}_{i1}, \ldots, \mathsf{K}_{in}) \right\}_{i=1}^n$ and ${}^A\langle \mathbf{B} \rangle_{\ell_2} = \Big\{ \mathbf{b}_i, (\tau'_{i1}, \ldots, \tau'_{in}), (\mathsf{K}'_{i1}, \ldots, \mathsf{K}'_{in}) \Big\}_{i=1}^n$ that are key-consistent and publicly-known constants c_1, c_2, we have:

- *Linearity*: Let $\mathbf{C} = c_1 \cdot \mathbf{A} + c_2 \cdot \mathbf{B} = (\mathbf{c}^{(1)}, \ldots, \mathbf{c}^{(m)})$, where each $\mathbf{c}^{(j)} = c_1 \cdot \mathbf{a}^{(j)} + c_2 \cdot \mathbf{b}^{(j)}$. To compute ${}^A\langle \mathbf{C} \rangle_{\ell_1}$, every party $P_i \in \mathcal{P}$ needs to locally compute $c_1 \cdot \mathbf{a}_i + c_2 \cdot \mathbf{b}_i$ as its share-vector, $\left\{ c_1 \cdot \tau_{ij} + c_2 \cdot \tau'_{ij} \right\}_{j=1}^n$ as tags on its share-vector and $\left\{ c_1 \cdot \mathsf{K}_{ij} + c_2 \cdot \mathsf{K}'_{ij} \right\}_{j=1}^n$ as keys for authenticating other party's share-vector. We will write this operation as

$$c_1 \cdot {}^A\langle \mathbf{A} \rangle_{\ell_1} + c_2 \cdot {}^A\langle \mathbf{B} \rangle_{\ell_2} = {}^A\langle \mathbf{C} \rangle_{\ell_1}.$$

We stress that the multiplicativity property *does not* hold for authenticated packed secret-sharing. Namely, if $\mathbf{C} = (\mathbf{c}^{(1)}, \ldots, \mathbf{c}^{(m)}) = \mathbf{A} \times \mathbf{B}$, where each $\mathbf{c}^{(j)} = \mathbf{a}^{(j)} \times \mathbf{b}^{(j)}$, then

$${}^A\langle \mathbf{A} \rangle_{\ell_1} \times {}^A\langle \mathbf{B} \rangle_{\ell_2} \neq {}^A\langle \mathbf{C} \rangle_{\ell_1 + \ell_2}.$$

This is because the parties *cannot* locally compute their tags and MAC-keys corresponding to ${}^A\langle \mathbf{C} \rangle_{\ell_1 + \ell_2}$, since MACs *do not* support any multiplicative property.

The above discussion easily extend for ${}^A\langle \mathbb{A} \rangle_\ell$ and ${}^A\langle \mathbb{B} \rangle_\ell$.

4.3 Reconstruction Protocols

In our protocols, we will encounter situations where we need to reconstruct different types of secret-shared values. We now discuss these reconstruction mechanisms.

Reconstructing $[\cdot]_t$-shared and $[\cdot]_t^i$-shared values. To enable a designated party $P_R \in \mathcal{P}$ reconstruct a t-shared value s, every party can send its share corresponding to $[s]_t$ to P_R, who applies the Reed-Solomon (RS) error-correction procedure on the received shares, error-corrects up to t incorrect shares and reconstructs s. Note that the protocol allows P_R to reconstruct the entire underlying degree-t sharing-polynomial and hence $[s]_t$. The protocol requires one round and incurs a communication of $\mathcal{O}(n\kappa)$ bits. To publicly reconstruct s, one can run n instances of the above protocol, where each instance is for one designated party from $\mathcal{P}$ to reconstruct s. This incurs a communication of $\mathcal{O}(n^2\kappa)$ bits.

Note that the above protocol(s) work even for twisted t-shared values.

Reconstructing $\langle \cdot \rangle_\ell$-shared values. Let $\mathbf{s} = (s^{(1)}, \ldots, s^{(k)})$ such that the parties hold $\langle \mathbf{s} \rangle_\ell$, where $\ell \geq k - 1$ and $\ell \leq t$. To enable a designated party $P_R \in \mathcal{P}$ reconstruct $\mathbf{s}$, every party can send its share corresponding to $\langle \mathbf{s} \rangle_\ell$ to P_R, who applies the Reed-Solomon (RS) error-correction procedure on the received shares, error-corrects up to t incorrect shares, reconstructs the underlying degree-ℓ sharing polynomial and outputs $\mathbf{s}$, whose elements are embedded in the underlying sharing polynomial at

$-1, \ldots, -k$. The protocol requires one round and incurs a communication of $\mathcal{O}(n\kappa)$ bits. Consequently, it has an overhead of $\mathcal{O}(1)$ bits, since $|\mathbf{s}| = \Theta(n\kappa)$ bits.

The above procedure can be used by P_R to even reconstruct any $\mathbb{S}$, given the parties hold $\langle \mathbb{S} \rangle_\ell$, provided each vector in $\mathbb{S}$ consists of k elements where k and ℓ satisfies the above constraints. This is because the parties can execute the above protocol to let P_R robustly reconstruct each vector in $\mathbb{S}$. The protocol requires one round and has a communication overhead of $\mathcal{O}(1)$ bits.

Reconstructing $^{A}\langle \cdot \rangle_\ell$-shared values. Let $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(m)})$, with each $\mathbf{s}^{(v)} \in \mathbb{F}^k$, such that the parties hold $^{A}\langle \mathbf{S} \rangle_\ell$, where $\ell \geq k-1$ and where $n - t > \ell$. To enable a designated P_j reconstruct $\mathbf{S}$, each $P_i \in \mathcal{P}$ can send its share-vector $\mathbf{s}_i$ corresponding to $^{A}\langle \mathbf{S} \rangle_\ell$, along with the MAC-tag τ_{ij} on $\mathbf{s}_i$, available as part of $^{A}\langle \mathbf{S} \rangle_\ell$. Party P_j on receiving $\mathbf{s}_i$ and τ_{ij} can verify them using its MAC-key K_{ji} available as part of $^{A}\langle \mathbf{S} \rangle_\ell$. If the verification passes, then except with probability $2^{-\Omega(\kappa)}$, share $\mathbf{s}_i$ is the correct share-vector. Once P_j has $\ell + 1$ correct share-vectors for $\mathbf{S}$, it can use them to reconstruct the vectors $\mathbf{s}^{(v)}$ and hence $\mathbf{S}$. Note that there will be at least $n - t$ honest parties and so P_j will have $\ell + 1$ correct share-vectors, even if corrupt parties send incorrect shares. The protocol will require one round and involves a communication of $\mathcal{O}(nm\kappa)$ bits, as each share-vector $\mathbf{s}_i$ consists of $m\kappa$ bits. We also note that $|\mathbf{S}| = \Theta(mk\kappa)$ bits. Hence if $k = \Theta(n)$ (which will be ensured by our packing parameter), then the protocol will have a communication overhead of $\mathcal{O}(1)$ bits.

Note that apart from robustly reconstructing $\mathbf{S}$, if needed then party P_j can recompute the shares of *every* party in $^{A}\langle \mathbf{S} \rangle_\ell$. That is, P_i can recompute $\langle \mathbf{S} \rangle_\ell$ corresponding to $^{A}\langle \mathbf{S} \rangle_\ell$. This is because while reconstructing each vector $\mathbf{s}^{(v)}$, party P_j will be learning the entire underlying degree-ℓ sharing-polynomial used for packing the vector $\mathbf{s}^{(v)}$.

Given $^{A}\langle \mathbb{S} \rangle_\ell$ where each vector in $\mathbb{S}$ consists of k elements and where k and ℓ satisfies the above constraints (i.e. $\ell \geq k-1$ and $n-t > \ell$), it follows that any designated P_j can reconstruct $\mathbb{S}$ by following the above procedure to reconstruct each $\mathbf{S}^{(u)} \in \mathbb{S}$. Moreover, if required, party P_j can recompute $\langle \mathbb{S} \rangle_\ell$. The protocol will require one round. Moreover, it will have a communication overhead of $\mathcal{O}(1)$ bits, provided $k = \Theta(n)$

To enable the parties in $\mathcal{P}$ to publicly reconstruct $\mathbb{S}$, the parties can run the steps of the above protocol in parallel, once on behalf of each designated P_j. The protocol will then have a communication overhead of $\mathcal{O}(n)$ bits, provided $k = \Theta(n)$.

4.4 Existing Primitives

In this section, we briefly discuss the existing primitives used in our protocols.

4.4.1 Reliable Broadcast. Informally, a *reliable broadcast* (BCast) protocol allows a designated *sender* $P_S \in \mathcal{P}$ with input $m \in \{0,1\}^\star$ to identically send m to all the parties, even if P_S is potentially corrupt. The ideal functionality $\mathcal{F}_{\mathsf{BC}}$ capturing the requirements for reliable broadcast is presented in Functionality 4.9.

Functionality 4.9: $\mathcal{F}_{\mathsf{BC}}$ — The ideal functionality for reliable broadcast

The functionality is parametrized by a designated sender P_S. The functionality interacts with an adversary $\mathcal{S}$ and the parties in $\mathcal{P}$ as follows.

– Upon receiving m from P_S, the functionality sends m to all the parties in $\mathcal{P}$.

To securely realize the functionality $\mathcal{F}_{\mathsf{BC}}$, we use the perfectly-secure broadcast protocol of [9] with $t < n/3$. The protocol requires an expected constant number of rounds and achieves a communication complexity of $\mathcal{O}(n\ell)$ bits plus expected $\mathcal{O}(n^3 \log^2 n)$ bits for broadcasting a message of size ℓ bits. The notation $N \times \mathcal{BC}(\ell)$ will denote the communication complexity of N calls to the broadcast channel with a message of size ℓ bits.

4.4.2 Generation of t-sharing. A protocol for generating t-sharing allows a designated *dealer* $D \in \mathcal{P}$ to verifiably generate t-sharing of its inputs, even if D is potentially *corrupt*. The ideal functionality for this is presented in Functionality 4.10.

Functionality 4.10: $\mathcal{F}_{[\cdot]_t}$ — The ideal functionality for generating $[\cdot]_t$

The functionality is parametrized by a designated dealer $D \in \mathcal{P}$ and a publicly known value N. The functionality interacts with an adversary $\mathcal{S}$ and the parties $\mathcal{P}$ as follows.

– Upon receiving the polynomials $q_1(\cdot), \ldots, q_N(\cdot)$ from D, send $q_1(k), \ldots, q_N(k)$ to every $P_k \in \mathcal{P}$, provided $q_1(\cdot), \ldots, q_N(\cdot)$ are all atmost t-degree polynomials over $\mathbb{F}$.

To securely realize $\mathcal{F}_{[\cdot]_t}$, we use the perfectly-secure VSS protocol of [1]. The protocol incurs a communication of $\mathcal{O}(N \cdot n\kappa + n^4 \log n)$ bits and requires $\mathcal{O}(1)$ expected rounds.

4.4.3 Generating a Random Value. Functionality $\mathcal{F}_{\mathsf{Rand}}$ (see Functionality 4.11) generates a uniformly random value r from $\mathbb{F}$ for all the parties. The functionality does so only when invoked by at least one honest party; this ensures that the adversary will not be knowing the output of $\mathcal{F}_{\mathsf{Rand}}$ beforehand.

Functionality 4.11: $\mathcal{F}_{\mathsf{Rand}}$ — The ideal functionality for generating a random value

The functionality proceeds as follows, interacting with the parties in $\mathcal{P}$ and an adversary $\mathcal{S}$.

– If (Rand, P_i) has been received from at least $t+1$ parties P_i, then pick a random value $r \in \mathbb{F}$ and send r to every party in $\mathcal{P}$.

There are several standard ways to securely and efficiently realize the functionality $\mathcal{F}_{\mathsf{Rand}}$. For instance, a naive way will be to let each party P_i pick a random value $r_i \in \mathbb{F}$ and t-share it through an instance of VSS. The parties can then set $r \stackrel{\mathsf{def}}{=} r_1 + \ldots + r_n$ and publicly reconstruct the value r. This will require a constant expected number of rounds and incur a communication of $\mathcal{O}(n^5 \log n)$ bits.

4.4.4 Generating Twisted-Sharing with Zero Shares for Designated Parties. We assume the existence of a protocol, that allows a designated dealer $D \in \mathcal{P}$, who can be potentially *corrupt*, to verifiably generate twisted t-sharing of its inputs with respect to a designated party P_i, such that the twisted-shares of a designated subset of parties of size up to $t+1$ (possibly including P_i) for all the inputs are 0. The ideal functionality for this purpose is presented in Functionality 4.12 (here ZS stands for zero-share).

Functionality 4.12: $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ — Ideal functionality for generating $\lceil\cdot\rfloor_t^i$ with 0 as the designated shares

The functionality is parametrized by a designated dealer $D \in \mathcal{P}$, a designated party P_i (which could be same as D) and a publicly known value N. The functionality interacts with an adversary $\mathcal{S}$ and the parties $\mathcal{P}$ as follows.

– Upon receiving a set $\mathcal{T}$ and polynomials $q_1(\cdot), \ldots, q_N(\cdot)$ from D, send $\{q_1(j), \ldots, q_N(j)\}$ to every $P_j \in \mathcal{P} \setminus \{P_i\}$ and $\{q_1(0), \ldots, q_N(0)\}$ to P_i, provided all the following hold.
  - $|\{P_i\} \cup \mathcal{T}| \leq t+1$.
  - $q_1(\cdot), \ldots, q_N(\cdot)$ are all t-degree polynomials over $\mathbb{F}$.
  - $q_1(k) = \ldots = q_N(k) = 0$ holds for every $P_k \in \mathcal{T} \setminus \{P_i\}$.
  - If $P_i \in \mathcal{T}$ then $q_1(i) = \ldots = q_N(i) = 0$ holds

Given a secure realization of $\mathcal{F}_{[\cdot]_t}$, a secure realization of $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ is straightforward. The idea is to let D first secret-share its inputs using $\mathcal{F}_{[\cdot]_t}$ and then publicly checking a random linear combination of these sharings to check if D has selected its polynomials satisfying the given constraints.

In more detail, on having input t-degree polynomials $q_1(\cdot), \ldots, q_N(\cdot)$ satisfying the constraints of $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$, the dealer D picks an additional t-degree random "masking polynomial" $q_{N+1}(\cdot)$ satisfying the same constraints and sends these $N+1$ polynomials to $\mathcal{F}_{[\cdot]_\mathsf{t}}$, for distributing the shares of these polynomials to the respective parties.[14] The parties then generate a uniformly random value r by calling $\mathcal{F}_{\mathsf{Rand}}$ and publicly reconstruct the t-degree masked polynomial $M(\cdot) \stackrel{\mathsf{def}}{=} q_{N+1}(\cdot) + rq_1(\cdot) + \ldots + r^N q_N(\cdot)$ and verify if all the following hold.

- $M(k) = 0$, for every $P_k \in \mathcal{T} \setminus \{P_i\}$.
- $M(0) = 0$, if $P_i \in \mathcal{T}$.

One can easily show that if a potentially *corrupt* D has not selected correctly any of its $N+1$ input polynomials, then the above verifications fail, except with probability $2^{-\Omega(\kappa)}$. Hence if the verifications pass then the parties output their respective twisted-shares of the polynomials $q_1(\cdot), \ldots, q_N(\cdot)$. One can easily show that this protocol securely realizes the functionality $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ with statistical security. The protocol will incur a communication of $\mathcal{O}(N \cdot n\kappa + n^4 \log n)$ bits, apart from one invocation of $\mathcal{F}_{\mathsf{Rand}}$ and requires $\mathcal{O}(1)$ expected number of rounds.

4.4.5 Generating Random Twisted t-Sharings with Zero Shares for Designated Parties.

Functionality $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor_t^i}$ (Fig 4.13) generate random twisted degree-t sharings (with respect to a designated party) based on the corrupted parties' shares and then distribute the honest parties' shares to them, such that the twisted-shares of a designated subset of parties of size up to $t+1$ are 0. Moreover, the underlying secrets of the generated sharings will be embedded at designated evaluation point $-i$ (instead of 0), where $i \in \{1, \ldots, p+1\}$.

The functionality can be securely realized using standard techniques and a protocol for securely realizing $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$. Namely, we let each party verifiably twisted t-share sufficiently many random secrets using $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ where the shares of the designated parties is 0 and where the secrets are stored at the designated evaluation point instead of 0. Then the random sharings can be extracted from the sharings generated by the parties using known randomness extraction techniques from [27]. By using the VSS protocol of [4] to securely realize $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$, one gets a protocol for securely realizing $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor_t^i}$ that will incur $\mathcal{O}(1)$ expected number of rounds and incur a communication of bits $\mathcal{O}(N \cdot n\kappa + n^5 \log n)$ bits, where N is the number of random t-sharings generated..

Functionality 4.13: $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor_t^i}$ — The ideal functionality for generating N random twisted t-sharings where secrets are stored at a designated evaluation point

The functionality parametrized by a publicly known value $N \geq 1$ proceeds as follows, interacting with the parties in $\mathcal{P}$ and an adversary $\mathcal{S}$.

- Upon receiving $(\mathsf{RandShare}, \mathcal{T}, P_i, \mathsf{position})$ from the honest parties where $|\mathcal{T}| \leq t$ and $\mathsf{position} \in \{1, \ldots, p+1\}$, the functionality randomly selects $r_1, \ldots, r_N \in \mathbb{F}$ and does the following:
  - The functionality receives a set of shares for the corrupt parties from $\mathcal{S}$ and then samples random degree-t twisted sharings $\lceil r_1 || \mathsf{position} \rfloor_t^i, \ldots, \lceil r_N || \mathsf{position} \rfloor_t^i$, based on the shares of the corrupt parties and the secrets $r_1, \ldots, r_N$ respectively, such that the shares of the parties in $\mathcal{T}$ are 0.
  - To every honest party P_j, the functionality sends P_j's shares of $\lceil r_1 || \mathsf{position} \rfloor_t^i, \ldots, \lceil r_N || \mathsf{position} \rfloor_t^i$.

5 Information-Checking Protocol (ICP)

In this section, we present the instantiation of ICP which is an adaptation of the asynchronous ICP of [48] to the synchronous setting. An ICP involves three entities: a designated *signer* $S \in \mathcal{P}$, a designated *intermediary* $I \in \mathcal{P}$ and the set of parties $\mathcal{P}$ acting as *verifiers* (note that S and I also plays

[14] Note that as per the semantics of $\mathcal{F}_{[\cdot]_\mathsf{t}}$ (see Functionality 4.10), the share of P_i is the i^{th} points on the polynomials received by $\mathcal{F}_{[\cdot]_\mathsf{t}}$. However, since in $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ we are generating twisted-sharings with respect to P_i, the share of P_i will be the 0^{th} point on D's polynomials. Consequently, upon receiving the polynomials $q_1(\cdot), \ldots, q_{N+1}(\cdot)$, instead of the i^{th} point on these polynomials, the functionality $\mathcal{F}_{[\cdot]_\mathsf{t}}$ distributes the 0^{th} point on these polynomials to P_i as its share.

the role of verifiers). Party S has vectors of private inputs $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^{\ell}$. An ICP can be considered an information-theoretically secure analogue of digital signatures, where S gives a "signature" on $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ to I, who later reveals it publicly, claiming that it got the signature from S. If S and I are *honest*, then the signature is accepted by all (honest) parties (correctness). Moreover, the signed values remain private (privacy) till they are revealed. If S is *honest*, then a *corrupt* I cannot forge S's signature (unforgeability). And finally, if S is *corrupt* and gave signed values to an *honest* I, then I can later reveal it (non-repudiation).[15]

An ICP consists of a sequence of three phases as described below, each of which is implemented by a dedicated sub-protocol.

- **Generation Phase**: Executed through a protocol Gen, where S sends $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ to I along with some *auxiliary information* and to each verifier, S gives some *verification information*.
- **Authentication Phase**: Executed by the parties in $\mathcal{P}$ through a protocol Auth, to verify whether S distributed "consistent" information to I and the verifiers. Upon successful verification I sets a Boolean variable $V_{S,I}$ to 1 and the information held by I is considered as the *information-checking signature* (IC-signature) on $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$, denoted as $\mathsf{Sig}(S \rightarrow I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$. The notation $S \rightarrow I$ signifies that the signature is *given by* S *to* I.
- **Public Reveal Phase**: Executed by I and the verifiers by running a protocol Rev, where I publicly reveals $\mathsf{Sig}(S \rightarrow I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$, which gets verified by the parties, upon which the parties either output $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ or reject.

Definition 5.1 (ICP). *A triplet of protocols* $(\mathsf{Gen}, \mathsf{Auth}, \mathsf{Rev})$ *where* S *has a private input* $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^{\ell}$ *for* Gen *is called a secure ICP, if all the following requirements hold for every possible adversary corrupting up to* t *parties.*[16]

- **Correctness***: If* S *and* I *are honest, then* I *sets* $V_{S,I}$ *to* 1 *during* Auth*. Moreover, every honest party outputs* $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ *during* Rev*.*
- **Privacy***: If* S *and* I *are honest, then the view of the adversary remains independent of* $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ *during* Gen *and* Auth*.*
- **Unforgeability***: If* S *is honest and* I *reveals* $\mathsf{Sig}(S \rightarrow I, \bar{\mathbf{s}}^{(1)}, \ldots, \bar{\mathbf{s}}^{(M)})$ *during* Rev *and if the honest parties output* $\bar{\mathbf{s}}^{(1)}, \ldots, \bar{\mathbf{s}}^{(M)}$*, then except with a negligible probability, the condition* $\bar{\mathbf{s}}^{(m)} = \mathbf{s}^{(m)}$ *holds, for* $m = 1, \ldots, M$*.*
- **Non-repudiation***: If* S *is corrupt and* I *is honest and if* I *sets* $V_{S,I}$ *to* 1 *holding* $\mathsf{Sig}(S \rightarrow I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$ *during* Auth*, then except with a negligible probability, all honest parties output* $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ *during* Rev*.*

Note that we *do not* capture the requirements of ICP through an ideal functionality. Looking ahead, we will show how the steps of ICP can be simulated while giving the simulation-based security proofs for the higher-level protocols which use the ICP as a primitive.

5.1 The construct

Our instantiation of ICP is presented in Protocol 5.2. During protocol Gen, the signer S hides its inputs $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^{\ell}$ in the higher order ℓ coefficients of random $(\ell + t - 1)$-degree polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$ respectively and sends these polynomials to I. In parallel, each verifier P_i receives a random evaluation-point α_i from S and the value of the polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$ at α_i as verification-tags. Note that this distribution of information guarantees *privacy*, since for every polynomial, the adversary may learn at most t distinct points, while the degree of each polynomial is $(\ell + t - 1)$. Later, during Rev, I makes public the polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$. In response, each verifier P_i locally verifies these polynomial at α_i, with respect to their verification-tags and publicly announce the result of their local verification. If at least $n - 2t$ verifiers respond positively, implying that there is at

[15] We stress that the original notion of ICP as formulated in [50] considers a setting where I reveals the signature only to a single designated receiver. In [48] this notion was extended to the case where all the parties play the role of receivers, as well as verifiers and where the signature is *publicly* revealed (the same setting as ours). The modification suits best for our VSS.

[16] Here $L, M \geq 1$ and will be publicly-known.

least one honest verifier with a positive response, then it is very likely that I revealed the correct polynomials. This is because a potentially *corrupt* I will *not* know the evaluation-point of the honest verifiers and the verification-tags held by them. Consequently, the parties output the higher order ℓ coefficients of the publicly-revealed polynomials. Hence the above distribution of information also guarantees the *unforgeability* property.

Unfortunately, *non-repudiation* is not guaranteed yet. A potentially corrupt S can distribute "inconsistent" polynomials and verification-tags to an honest I and an honest verifier respectively. During the protocol Auth, the verifiers and I interact in a "zero-knowledge" fashion to check the consistency of the information distributed by S, without revealing any additional information about their respective data. To enable the zero-knowledge verification, S distributes an additional random $(\ell+t-1)$-degree masking polynomial $R(X)$ to I during Gen and each verifier P_i is also provided the evaluation of $R(X)$ at α_i as part of its verification-tag. Then during protocol Auth, the intermediary I checks the consistency of its polynomials by making public a random linear combination of the polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$, masked with $R(X)$, where the random linear combiners are selected by I. In response, each verifier P_i locally computes the corresponding linear combination of its verification-tags and verifies if it lies on the linear combination of the polynomials made public by I. If the verification is positive, then P_i announces it publicly. Finally, I checks if a set $\mathcal{R}$ of at least $n-t$ verifiers have responded positively, in which case it is convinced that very likely, the polynomials held by it are consistent with the evaluation-point and verification-tags of all the honest verifiers in $\mathcal{R}$. This is because a potentially *corrupt* S while distributing inconsistent information during Gen will not know the random linear combiners which will be used later by I during Auth for verifying the consistency of the distributed information. Hence, I sets $f^{(1)}(X), \ldots, f^{(M)}(X)$ as the IC-signature and that completes the protocol Auth.

Protocol 5.2: ICP — The information-checking protocol in the $\mathcal{F}_{\mathsf{BC}}$-hybrid model

Generation Phase: Protocol $\mathsf{Gen}(S, I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$

Round 1: On having the input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in F^{\ell}$, the sender S does the following.

- Select random $(\ell+t-1)$-degree polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$ over $\mathbb{F}$, where for $m = 1, \ldots, M$, the higher order ℓ coefficients of $f^{(m)}(X)$ are elements of $\mathbf{s}^{(m)}$.
- Select a random $(\ell+t-1)$-degree *masking-polynomial* $R(x)$ over $\mathbb{F}$.
- Corresponding to each verifier P_i, select a random *evaluation-point* $\alpha_i \in \mathbb{F}$.
- Send $R(X), f^{(1)}(X), \ldots, f^{(M)}(X)$ to I.
- For $i = 1, \ldots, n$, send $\alpha_i, r_i, v_{1,i}, \ldots, v_{M,i}$ to P_i, where $r_i = R(\alpha_i)$ and $v_{m,i} = f^{(m)}(\alpha_i)$, for $m = 1, \ldots, M$.

Authentication Phase: Protocol $\mathsf{Auth}(S, I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$

Round 1: I on receiving $(\ell+t-1)$-degree polynomials $R(X), f^{(1)}(X), \ldots, f^{(M)}(X)$ from S during the protocol Gen, initializes a set $\mathcal{R}$ to $\emptyset$ and does the following.

- Select a random $d \in \mathbb{F}$ and compute the *masked-polynomial* $M(X) \stackrel{\mathsf{def}}{=} R(X) + df^{(1)}(X) + \ldots + d^M f^{(M)}(X)$.
- Send $d, M(X)$ to $\mathcal{F}_{\mathsf{BC}}$.

Round 2: Every party $P_i \in \mathcal{P}$ (including S and I) upon receiving $d, M(X)$ from $\mathcal{F}_{\mathsf{BC}}$ where I is the sender, does the following.

- Send $\mathtt{Accept}_i$ to $\mathcal{F}_{\mathsf{BC}}$, provided $M(\alpha_i) = r_i + dv_{1,i} + \ldots + d^M v_{M,i}$ holds.

Local Computation by I: I does the following.

- Upon receiving $\mathtt{Accept}_i$ from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender, include P_i in the set $\mathcal{R}$.
- For $m = 1, \ldots, M$, let $\mathbf{s}^{(m)} \in \mathbb{F}^{\ell}$ be the higher order ℓ coefficients of the polynomial $f^{(m)}(X)$, where $f^{(1)}, \ldots, f^{(M)}(X)$ has been received by I from S. If $|\mathcal{R}| \geq n-t$, then set $V_{S,I} = 1$ and set $\mathsf{Sig}(\mathsf{sid}, S \rightarrow I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}) = (f^{(1)}(X), \ldots, f^{(M)}(X))$.

Public Reveal Phase: Protocol $\mathsf{Rev}(S, I, \mathcal{P}, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$

Round 1: If $V_{\mathsf{sid},S,I} = 1$ during the protocol Auth, then I sends $f^{(1)}(X), \ldots, f^{(M)}(X)$ to $\mathcal{F}_{\mathsf{BC}}$.

Round 2: Every party $P_i \in \mathcal{P}$ (including S and I) upon receiving $(\ell + t - 1)$-degree polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$ $\mathcal{F}_{\mathsf{BC}}$ where I is the sender, sends $\mathtt{Accept}_i$ to $\mathcal{F}_{\mathsf{BC}}$, provided all the following hold.

- P_i has sent $\mathtt{Accept}_i$ to $\mathcal{F}_{\mathsf{BC}}$ during the protocol Auth.
- The condition $f^{(m)}(\alpha_i) = v_{m,i}$ holds, for $m = 1, \ldots, M$.

Local Computation: Every $P_j \in \mathcal{P}$ (including S and I) does the following.

- If there are at least $n - 2t$ parties P_i such that $\mathtt{Accept}_i$ is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender during Round 2 of the protocol Rev, then output $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$, where for $m = 1, \ldots, M$, the elements of $\mathbf{s}^{(m)}$ are the higher order ℓ coefficients of $f^{(m)}(X)$.

We now proceed to prove the properties of our ICP.

Lemma 5.3 (Correctness)**.** *Let $S, I \in \mathcal{H}$ and let S has input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^\ell$ for the protocol Gen. Then I sets $V_{S,I}$ to 1 during the protocol Auth. Moreover, every $P_j \in \mathcal{H}$ outputs $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ during Rev.*

Proof. Let $S, I \in \mathcal{H}$ and let S has input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^\ell$ for the protocol Gen. Then during protocol Gen, the sender S distributes consistent polynomials and verification tags to I and each verifier respectively. Namely, $r_i = R(\alpha_i)$ and for $m = 1, \ldots, M$, the condition $v_{m,i} = f^{(m)}(\alpha_i)$ will hold for every $P_i \in \mathcal{P}$. Consequently, irrespective of the value of d selected by I, the condition $M(\alpha_i) = r_i + dv_{1,i} + \ldots + d^M v_{M,i}$ will hold for every $P_i \in \mathcal{P}$. This implies that every $P_i \in \mathcal{H}$ will broadcast $\mathtt{Accept}_i$ during the protocol Auth and hence $\mathcal{H} \subseteq \mathcal{R}$. As $|\mathcal{H}| \geq n - t$, this implies that I will set $V_{S,I}$ to 1. During the protocol Rev, intermediary I will broadcast the polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$ and each $P_i \in \mathcal{H}$ will broadcast $\mathtt{Accept}_i$. Since $|\mathcal{H}| \geq n - t > n - 2t$, it follows that each P_j will output the higher order ℓ coefficients of $f^{(1)}, \ldots, f^{(M)}(X)$, namely the vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ respectively. □

Lemma 5.4 (Privacy)**.** *Let $S, I \in \mathcal{H}$ and let S has input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^\ell$ for the protocol Gen. Then the view of the adversary remains independent of $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ during the protocols Gen and Auth.*

Proof. Let $S, I \in \mathcal{H}$ and let S has input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^\ell$ for the protocol Gen. For simplicity and without loss of generality, let $P_1, \ldots, P_t$ be under the control of the adversary. During the protocol Gen, the adversary learns the verification-tags of the parties $P_1, \ldots, P_t$. Namely, corresponding to each $P_i \in \{P_1, \ldots, P_t\}$, the adversary learns the evaluation-point α_i and the value of the polynomials $R(X), f^{((1))}, \ldots, f^{(M)}(X)$ at α_i. However, each of these polynomials is a random $(\ell + t - 1)$-degree polynomial and through the verification-tags of the corrupt parties, the adversary can learn at most t distinct points on each of these polynomials. Hence the view of the adversary during Gen is independent of the higher order ℓ coefficients of the polynomials $R(X), f^{(1)}(X), \ldots, f^{(M)}(X)$, selected by D. Namely, for every candidate $\mathbf{r} \in \mathbb{F}^\ell$, there exists a unique $(\ell + t - 1)$-degree polynomial $R(X)$, whose higher order ℓ coefficients are elements of $\mathbf{r}$, such that $R(\alpha_i) = r_i$ holds for every $P_i \in \{P_1, \ldots, P_t\}$. Similarly, for $m = 1, \ldots, M$, for every candidate $\mathbf{s}^{(m)} \in \mathbb{F}^\ell$, there exists a unique $(\ell + t - 1)$-degree polynomial $f^{(m)}(X)$, whose higher order ℓ coefficients are elements of $\mathbf{s}^{(m)}$, such that $f^{(m)}(\alpha_i) = v_{m,i}$ holds for every $P_i \in \{P_1, \ldots, P_t\}$. Now since the polynomials $R(X)$ and $f^{(m)}(X)$ are randomly chosen by D, it follows that the view of the adversary during protocol Gen remains independent of $\mathbf{r}$ and $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$.

Next, consider the protocol Auth where the adversary learns d, along with the masked-polynomial $M(X) = R(X) + df^{(1)}(X) + \ldots + d^M f^{(M)}(X)$. However, since the adversary's view during Gen is independent of the higher order ℓ coefficients of $R(X)$ and $f^{(1)}(X), \ldots, f^{(M)}(X)$, it follows that even after learning the polynomial $M(X)$, the adversary's view remains independent of the higher order ℓ coefficients of $f^{(1)}(X), \ldots, f^{(M)}(X)$. Namely, for every candidate $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^\ell$, there is a corresponding unique $\mathbf{r} \in \mathbb{F}^\ell$, such that all the following hold:

- For $m = 1, \ldots, M$, $f^{(m)}(\alpha_i) = v_{m,i}$, for every $P_i \in \{P_1, \ldots, P_t\}$.
- $R(\alpha_i) = r_i$, for every $P_i \in \{P_1, \ldots, P_t\}$.
- For $m = 1, \ldots, M$, the higher order ℓ coefficients of $f^{(m)}(X)$ are the elements of $\mathbf{s}^m$.

– The higher order ℓ coefficients of $R(X)$ are the elements of $\mathbf{r}$.
– The condition $M(X) = R(X) + df^{(1)}(X) + \ldots + d^M f^{(M)}(X)$ holds.

Now since the polynomial $R(X)$ selected by D is completely random, it holds that the view of the adversary at the end of Auth remains independent of $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$. □

Lemma 5.5 (Non-repudiation)**.** *Let S be corrupt and let $I \in \mathcal{H}$. If I sets $V_{S,I}$ to 1 and holds $\mathsf{Sig}(S \to I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$ at the end of protocol Auth, then except with probability $2^{-\Omega(\kappa)}$, every party $P_j \in \mathcal{H}$ outputs $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ during Rev, where $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^\ell$.*

Proof. Let S be corrupt and let $I \in \mathcal{H}$ such that I has set $V_{S,I}$ to 1 during protocol Auth. This further implies that there is a set $\mathcal{R}$ of size at least $n-t$, such that every $P_i \in \mathcal{R}$ has broadcasted $\mathtt{Accept}_i$ during Auth, in response to the polynomial $M(X)$ broadcasted by I. Let $\mathcal{H}_\mathcal{R}$ be the set of honest parties in $\mathcal{R}$, where $|\mathcal{H}_\mathcal{R}| \geq n-2t$. Moreover, let $R(X)$ and $f^{(1)}(X), \ldots, f^{(M)}(X)$ be the $(\ell+t-1)$-degree polynomials received by I from S during Gen. We claim that for every $P_i \in \mathcal{H}_\mathcal{R}$, the condition $r_i = R(\alpha_i)$ and the condition $f^{(m)}(\alpha_i) = v_{m,i}$ holds, for $m = 1, \ldots, M$, except with a negligible probability. Here α_i and $r_i, v_{1,i}, \ldots, v_{M,i}$ are received by P_i from S during Gen. If the claim is true, then it follows that every $P_i \in \mathcal{H}_\mathcal{R}$ will again broadcast $\mathtt{Accept}_i$ during Rev. This is because I will set $\mathsf{Sig}(S \to I, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$ to $f^{(1)}(X), \ldots, f^{(M)}(X)$ during Auth and will broadcast the polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$ during Rev for revealing the IC-signature.

We prove the claim through a contradiction. Let there exist some $P_i \in \mathcal{H}_\mathcal{R}$, such that D distributes points to P_i which are inconsistent with respect to the polynomials distributed to I. Namely, α_i, r_i and $v_{1,i}, \ldots, v_{M,i}$ received by P_i satisfy the following conditions.

– $r_i = R(\alpha_i) + \delta_i$
– $v_{m,i} = f^{(m)}(\alpha_i) + \Delta_{m,i}$, for $m = 1, \ldots, M$.

Here $\delta_i, \Delta_{1,i}, \ldots, \Delta_{M,i} \in \mathbb{F}$. Moreover, at least two of these values are not zero.[17] We now compute the probability with which $M(\alpha_i) = r_i + dv_{1,i} + \ldots + d^M v_{M,i}$ holds, under the above conditions. Substituting the values of $M(X)$ and $r_i, v_{1,i}, \ldots, v_{M,i}$, if $M(\alpha_i) = r_i + dv_{1,i} + \ldots + d^M v_{M,i}$, then the following holds:

$$R(\alpha_i) + df^{(1)}(\alpha_i) + \ldots + d^M f^{(M)}(\alpha_i) = (R(\alpha_i) + \delta_i) + d(f^{(1)}(\alpha_i) + \Delta_{1,i}) + \ldots + d^M(f^{(M)}(\alpha_i) + \Delta_{M,i}).$$

By rearranging the terms we get that the following holds:

$$\delta_i + d\Delta_{1,i} + \ldots + d^M \Delta_{M,i} = 0.$$

This further implies that d should be the root of the polynomial $D(X) \stackrel{\mathsf{def}}{=} \delta_i + \Delta_{1,i}X + \ldots + \Delta_{M,i}X^M$. Notice that the polynomial $D(X)$ is a non-zero polynomial, with at least two of its coefficients being non-zero. We also note that S will not know the value of d beforehand while distributing the polynomials $R(X), f^{(1)}(X), \ldots, f^{(M)}(X)$ to I and $\alpha_i, r_i, v_{1,i}, \ldots, v_{M,i}$ to P_i during Gen, since d is selected during Auth. Moreover, I selects d uniformly at random from $\mathbb{F}$. So the probability that a randomly chosen d turns out to be the root of the polynomial $D(X)$ is at most $\frac{M}{|\mathbb{F}|} \approx 2^{-\Omega(\kappa)}$. Hence, if S distributed inconsistent information to I and P_i during Gen, then except with probability $2^{-\Omega(\kappa)}$, P_i will not broadcast $\mathtt{Accept}_i$ during Auth, which is a contradiction. □

Lemma 5.6 (Unforgeability)**.** *Let S be honest and let S have input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^\ell$ for protocol Gen. If during Rev, I reveals $\mathsf{Sig}(S \to I, \overline{\mathbf{s}}^{(1)}, \ldots, \overline{\mathbf{s}}^{(M)})$ and if the parties in $\mathcal{H}$ output $\overline{\mathbf{s}}^{(1)}, \ldots, \overline{\mathbf{s}}^{(M)}$, then except with probability $2^{-\Omega(\kappa)}$, the condition $\overline{\mathbf{s}}^{(m)} = \mathbf{s}^{(m)}$ holds, for $m = 1, \ldots, M$.*

Proof. Let $S \in \mathcal{H}$. If $I \in \mathcal{H}$, then the lemma is true trivially with probability 1, which follows from the correctness of the ICP (see Lemma 5.3). So we consider the case where I is corrupt.

Let I reveal $\mathsf{Sig}(S \to I, \overline{\mathbf{s}}^{(1)}, \ldots, \overline{\mathbf{s}}^{(M)})$ during Rev by broadcasting $(\ell + t - 1)$-degree polynomials $\overline{f}^{(1)}(X), \ldots, \overline{f}^{(M)}(X)$, where for $m = 1, \ldots, M$, the higher order ℓ coefficients of $\overline{f}^{(m)}(X)$ are the elements of $\overline{\mathbf{s}}^{(m)}$. Moreover, there exist some $m \in \{1, \ldots, M\}$, such that $\mathbf{s}^{(m)} \neq \mathbf{s}^{(m)}$, further implying that

[17] On contrary, if exactly one of the values $\delta_i, \Delta_{1,i}, \ldots, \Delta_{M,i}$ is non-zero, then $M(\alpha_i) \neq r_i + dv_{1,i} + \ldots + d^M v_{M,i}$ will hold for P_i and hence P_i will never broadcast $\mathtt{Accept}_i$ during Auth, which is a contradiction.

$\overline{f}^{(m)}(X) \neq \overline{f}^{(m)}(X)$. We claim that in this case, except with probability at most $\frac{n}{|\mathbb{F}|} \approx 2^{-\Omega(\kappa)}$, no party $P_i \in \mathcal{H}$ will broadcast $\texttt{Accept}_i$ during Rev. This will further imply that no party in $\mathcal{H}$ will output $\overline{\mathbf{s}}^{(1)}, \ldots, \overline{\mathbf{s}}^{(M)}$ during Rev.

To prove our claim, consider an arbitrary $P_i \in \mathcal{H}$. We argue that throughout protocol Gen and Auth, the adversary does not learn any additional information about the evaluation-point α_i held by P_i. This is trivially true for the protocol Gen, since only S interacts with the parties. Similarly, during Auth, intermediary I broadcasts the polynomial $M(X)$ and every $P_i \in \mathcal{H}$ may broadcast $\texttt{Accept}_i$, again revealing no additional information about α_i. Now consider an $m \in \{1, \ldots, M\}$, such that I broadcasts $\overline{f}^{(m)}(X)$ during Rev where $\overline{f}^{(m)}(X) \neq f^{(m)}(X)$. In this case, P_i will broadcast $\texttt{Accept}_i$ during Rev only if $\overline{f}^{(m)}(\alpha_i) = v_{m,i}$ holds. However, since $S \in \mathcal{H}$, it follows that $v_{m,i} = f^{(m)}(\alpha_i)$ holds. This further implies that $\overline{f}^{(m)}(\alpha_i) = f_m(\alpha_i)$ holds. However, α_i is randomly chosen from by S from $\mathbb{F}$ and not known to the adversary. So in order to ensure that $\overline{f}^{(m)}(\alpha_i) = f_m(\alpha_i)$ holds, the adversary has to correctly guess the value of α_i, which it can do with probability $\frac{1}{|\mathbb{F}|}$. Since we considered an arbitrary $P_i \in \mathcal{H}$ and since $|\mathcal{H}| \leq n$, it follows from the union bound that the probability that any party in $\mathcal{H}$ broadcasts $\texttt{Accept}_i$ in response to the incorrect polynomials of I during Rev is at most $\frac{n}{|\mathbb{F}|} \approx 2^{-\Omega(\kappa)}$. □

Lemma 5.7 (Complexity)**.** *Let S has input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)} \in \mathbb{F}^{\ell}$. Then the following hold in the ICP presented in Protocol 5.2.*

- *Protocol* Gen *requires one round and involves a communication of $\mathcal{O}((\ell M + nM)\kappa)$ bits.*
- *Protocol* Auth *involves a communication of $\mathcal{BC}((\ell + n)\kappa)$ bits and requires $\mathcal{O}(1)$ expected rounds.*
- *Protocol* Rev *involves a communication of $\mathcal{BC}((\ell M + nM)\kappa)$ bits and requires $\mathcal{O}(1)$ expected rounds.*

Proof. The round complexity of Gen follows easily from the protocol description. During Auth and Rev, the parties need to call $\mathcal{F}_{\mathsf{BC}}$ for two rounds each. Realizing the calls to $\mathcal{F}_{\mathsf{BC}}$ through the broadcast protocol of [9], this will require a constant expected number of rounds.

During Gen, the sender has to distribute $M + 1$ polynomials to I over $\mathbb{F}$, each of degree $(\ell + t - 1)$. In addition, each verifier P_i receives a random evaluation-point over $\mathbb{F}$ and the value of the polynomials at this evaluation point. During Auth, the intermediary I has to broadcast a linear combination of the $M + 1$ polynomials received from S, where the linearly combined polynomial is of degree $(\ell + t - 1)$. In response, each party has to broadcast a single bit in the form of an $\texttt{Accept}$ message. Finally, in the protocol Rev, the intermediary I has to broadcast M number of $(\ell + t - 1)$-degree polynomials that it received from S and in response, each party may broadcast a bit in the form of an $\texttt{Accept}$ message. The communication complexity of the various protocols now follow from the fact that each element from $\mathbb{F}$ can be represented by κ bits. □

Note that in our ICP, any party from $\mathcal{P}$ can be designated to play the role of S and I and the parties will be knowing the identity of the designated S and I. For convenience, we will use the following notations while using our ICP.

Notation 5.7: (Terminologies for Using ICP)
While using our ICP, we say that:

- "P_i *gives* $\mathsf{Sig}(P_i \rightarrow P_j, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$ *to* P_j" to mean that P_i acts as S and invokes an instance of Gen with input $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$, where P_j plays the role of I.
- "P_j *receives* $\mathsf{Sig}(P_i \rightarrow P_j, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$ *from* P_i" to mean that P_j as I has set V_{P_i,P_j} to 1 during the instance of Auth where P_i is S and P_j holds $(\ell + t - 1)$-degree polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$, where for $m = 1, \ldots, L$, the higher order ℓ coefficients of $f^{(m)}(X)$ are the elements of $\mathbf{s}^{(m)}$.
- "P_j *publicly reveals* $\mathsf{Sig}(P_i \rightarrow P_j, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$" to mean that P_j as I invokes an instance of Rev, where P_i is designated as S.
- "P_k *accepts* $\mathsf{Sig}(P_i \rightarrow P_j, \mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$ " to mean that P_k outputs $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)}$ during the instance of Rev, invoked by P_j as I where P_i is designated as S.

5.2 Linearity Property

Our ICP satisfies the linearity property provided "special care" is taken while generating the IC-signatures during the corresponding instances of Gen and Auth. In more detail, consider a *fixed* S and I.

Let $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(N)}$ be vectors of values, where for $k = 1, \ldots, N$, we have $\mathbf{S}^{(k)} = (\mathbf{s}^{(k,1)}, \ldots, \mathbf{s}^{(k,M)})$, with each $\mathbf{s}^{(k,1)}, \ldots, \mathbf{s}^{(k,M)} \in \mathbb{F}^{\ell}$. Hence each $\mathbf{S}^{(k)}$ consists of ℓM elements from $\mathbb{F}$. Let I receive $\mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(1)}), \ldots, \mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(N)})$ from S (see Notation 5.7). Moreover, for $k = 1, \ldots, N$, let $\mathsf{Gen}^{(k)}$ and $\mathsf{Auth}^{(k)}$ be the corresponding instances of Gen and Auth respectively, which are used to generate $\mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(k)})$, such that all the following conditions are satisfied.

- Corresponding to every $P_i \in \mathcal{P}$, sender S has used the same evaluation-point α_i during the instances $\mathsf{Gen}^{(1)}, \ldots, \mathsf{Gen}^{(N)}$.
- Let $\mathcal{R}_1, \ldots, \mathcal{R}_N$ be the instances of $\mathcal{R}$, computed by I during the instances $\mathsf{Auth}^{(1)}, \ldots, \mathsf{Auth}^{(N)}$ respectively, where each $|\mathcal{R}_k| \geq n - t$, for $k = 1, \ldots, N$. Then $|\mathcal{R}_1 \cap \ldots \cap \mathcal{R}_N| \geq n - t$ holds.

Let $\mathbf{S} \stackrel{\mathsf{def}}{=} c_1\mathbf{S}^{(1)} + \ldots + c_N\mathbf{S}^{(N)}$, where $c_1, \ldots, c_N \in \mathbb{F}$ and known publicly. It then follows that if the above conditions are satisfied, then I can locally compute $\mathsf{Sig}(S \rightarrow I, \mathbf{S})$ from $\mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(1)}), \ldots, \mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(N)})$.

In more detail, for $k = 1, \ldots, N$, let $\mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(k)}) = (f^{(k,1)}(X), \ldots, f^{(k,M)}(X))$, where each of these polynomials $f^{(k,1)(X)}, \ldots, f^{(k,M)}(X)$ is a $(\ell + t - 1)$-degree polynomial over $\mathbb{F}$, with the higher order ℓ coefficients being elements of $\mathbf{S}^{(k)}$, namely $\mathbf{s}^{(k,1)}, \ldots, \mathbf{s}^{(k,M)}$ respectively. Let $\mathbf{S} = (\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(M)})$, where for $m = 1, \ldots, M$, we have $\mathbf{s}^{(m)} = c_1\mathbf{s}^{(1,m)} + \ldots + c_N\mathbf{s}^{(N,m)}$. Then consider the $(\ell + t - 1)$-degree polynomial $f^{(m)}(X) \stackrel{\mathsf{def}}{=} c_1 f^{(1,m)}(X) + \ldots + c_N f^{(N,m)}(X)$. It follows that the higher order ℓ coefficients of the polynomial $f^{(m)}(X)$ are the elements of $\mathbf{s}^{(m)}$. Hence I can locally compute the polynomials $f^{(1)}(X), \ldots, f^{(M)}(X)$ and set $\mathsf{Sig}(S \rightarrow I, \mathbf{S}) = (f^{(1)}(X), \ldots, f^{(M)}(X))$. For a pictorial illustration of the above explanation, see part (a) of Fig 3.

Similarly, if the conditions mentioned previously are satisfied during the instances of Gen and Auth, then every party in $\mathcal{R}_1 \cap \ldots \cap \mathcal{R}_N$ can locally compute its verification-tags corresponding to $\mathsf{Sig}(S \rightarrow I, \mathbf{S})$, from its verification-tags corresponding to $\mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(1)}), \ldots, \mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(N)})$.

In more detail, let $\mathcal{R}_{\mathsf{com}} \stackrel{\mathsf{def}}{=} \mathcal{R}_1 \cap \ldots \cap \mathcal{R}_N$. Then every $P_i \in \mathcal{R}_{\mathsf{com}}$ has a common evaluation-point α_i across all the N instances of Gen and Auth. Moreover, for $k = 1, \ldots, N$, during the instance $\mathsf{Gen}^{(k)}$, party P_i would receive the verification-tags, say $v_{1,i}^{(k)}, \ldots, v_{M,i}^{(k)}$ from S. Then for $m = 1, \ldots, M$, party P_i can locally compute $v_{m,i} \stackrel{\mathsf{def}}{=} c_1 v_{m,i}^{(1)} + \ldots + c_N v_{m,i}^{(N)}$. Party P_i can then set $v_{1,i}, \ldots, v_{M,i}$ as the verification-tags corresponding to $\mathsf{Sig}(S \rightarrow I, \mathbf{S})$. For a pictorial illustration of the above explanation, see part (b) of Fig 3.

Later, if I is asked to reveal $\mathsf{Sig}(S \rightarrow I, \mathbf{S})$, then it can invoke the protocol Rev to reveal the polynomials $f^{(1)}, \ldots, f^{(M)}(X)$, defined as above. And each $P_i \in \mathcal{R}_{\mathsf{com}}$ can verify these polynomials with respect to α_i and the verification-tags $v_{1,i}, \ldots, v_{M,i}$.

If S and I are honest, then the correctness property of the instances $(\mathsf{Gen}^{(1)}, \mathsf{Auth}^{(1)}), \ldots, (\mathsf{Gen}^{(N)}, \mathsf{Auth}^{(N)})$ will guarantee that every honest party will accept $\mathbf{S}$. This is because there will be at least $n - 2t$ honest parties in $\mathcal{R}_{\mathsf{com}}$. And for every honest $P_i \in \mathcal{R}_{\mathsf{com}}$, the condition $f^{(m)}(\alpha_i) = v_{m,i}$ will hold, for $m = 1, \ldots, M$. Consequently, at least $n - 2t$ parties will broadcast $\mathtt{Accept}$ during Rev.

On the other hand, even if S is corrupt and I is honest, all honest parties will accept $\mathbf{S}$. This is because the non-repudiation property of the instances $(\mathsf{Gen}^{(1)}, \mathsf{Auth}^{(1)}), \ldots, (\mathsf{Gen}^{(N)}, \mathsf{Auth}^{(N)})$ would ensure that the verification-tags receives by every honest $P_i \in \mathcal{R}_{\mathsf{com}}$ are consistent with the polynomials held by I. Namely, if I holds the polynomials $f^{(k,1)}(X), \ldots, f^{(k,M)}(X)$ and P_i holds the verification-tags $v_{1,i}^{(k)}, \ldots, v_{M,i}^{(k)}$ from $\mathsf{Gen}^{(k)}$, then $\mathsf{Auth}^{(k)}$ would guarantee that except with probability $2^{-\Omega(\kappa)}$, the condition $v_{m,i}^{(k)} = f^{(k,m)}(\alpha_i)$ holds, for $m = 1, \ldots, M$. This will further imply that the condition $v_{m,i} = f^{(m)}(\alpha_i)$ will also hold and so every honest $P_i \in \mathcal{R}_{\mathsf{com}}$ will broadcast $\mathtt{Accept}_i$ during Rev.

Looking ahead, we will require the linearity property from ICP when used in our VSS protocol, where there will be multiple instances of Gen and Auth, involving the same S and I for generating IC-signatures and later I will be asked to linearly combine these IC-signatures and reveal through an instance of Rev. To achieve this, in all the instances of Gen and Auth involving the same S and I, it will be ensured that S uses a common random evaluation-point, say α_{S,I,P_i}, for the party P_i to compute its verification-tags. Similarly, I should check that there exists a common subset of parties $\mathcal{R}_{\mathsf{com}}$ of size at least $n - t$, present across the candidate $\mathcal{R}$ sets across all the instances of Auth and then only it should set $V_{S,I}$ to 1 in all the Auth instances. Notice that if S and I are honest, then I will always find such a common $\mathcal{R}_{\mathsf{com}}$ set, as the set of honest parties $\mathcal{H}$ always constitute a candidate $\mathcal{R}_{\mathsf{com}}$ set. In the rest of the paper, we will use the following terminologies while invoking the linearity property of ICP.

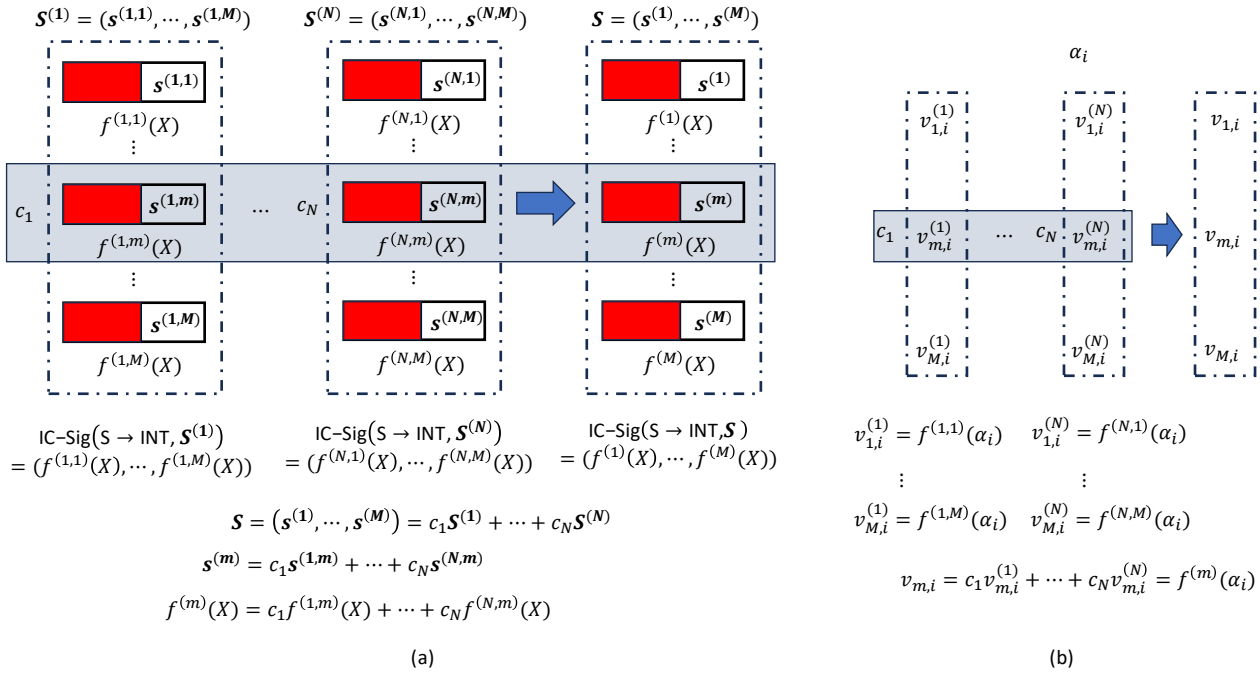

Fig. 3: Pictorial illustration of the linearity of our ICP. The information held by I is depicted in part (a), while part (b) depicts the verification-tags held by each verifier P_i. I holds IC-signature on $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(N)}$, where each $\mathbf{S}^{(k)} = (\mathbf{s}^{(k,1)}, \ldots, \mathbf{s}^{(k,M)})$ and each $\mathbf{s}^{(k,m)} \in \mathbb{F}^{\ell}$. As part of IC-signature on $\mathbf{S}^{(k)}$, I holds $(\ell + t - 1)$-degree polynomials $f^{(k,1)}(X), \ldots, f^{(k,M)}(X)$, where the higher order ℓ coefficients of these polynomials are elements of $\mathbf{s}^{(k,1)}, \ldots, \mathbf{s}^{(k,M)}$ respectively. The red part in each polynomial depicts the remaining t coefficients which are random elements of $\mathbb{F}$. Party P_i holds an evaluation-point α_i and the value of all the polynomials at α_i. I can linearly combine the polynomials to get IC-signature on any linear combination of $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(N)}$. Party P_i can also apply the same linear combination on the corresponding evaluations and compute its verification tags

Notation 5.7: (Terminologies related to linearity of ICP)
While invoking our ICP, we will say that:

- *"Parties follow the linearity property while generating IC-signatures"*, to mean that the underlying instances of Gen and Auth are invoked as above.
- *"I locally computes* $\mathsf{Sig}(S \rightarrow I, \mathbf{S})$ *from* $\mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(1)}), \ldots, \mathsf{Sig}(S \rightarrow I, \mathbf{S}^{(N)})$*"* to mean that I locally computes $\mathsf{Sig}(S \rightarrow I, \mathbf{S})$ as above and every (honest) party in $\mathcal{R}_{\mathsf{com}}$ locally computes its verification-tags as above.

6 VSS for Generating $^{A}\langle\cdot\rangle$-Sharing with a Constant Overhead

In this section, we present our VSS protocol $\Pi_{^{A}\langle\cdot\rangle}$. In the protocol, $D \in \mathcal{P}$ is a designated dealer with a private input $\mathbb{S}$, where $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)})$, with each $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,Mp)})$ and where each $\mathbf{s}^{(u,v)} = (s^{(u,v,1)}, \ldots, s^{(u,v,p)}) \in \mathbb{F}^{p}$. Hence $\mathbb{S} \in \mathbb{F}^{LMp^2}$, where $L, M \geq 1$. If D is *honest*, then at the end of the protocol parties hold $^{A}\langle\mathbb{S}\rangle_d = \left(^{A}\langle\mathbf{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbf{S}^{(L)}\rangle_d\right)$, such that the view of the adversary remains independent of $\mathbb{S}$. Here $p = \frac{t}{4} + 1$ and $d = t + p - 1$. The verifiability here guarantees that even if D is *corrupt*, at the end of the protocol there is a well-defined $\mathbb{S} \in \mathbb{F}^{LMp^2}$ (held by D), such that the parties hold $^{A}\langle\mathbb{S}\rangle_d$. The protocol incurs a constant communication overhead, provided $L = n^3$ and $M = n^2$, implying that $\mathbb{S}$ contains $\Theta(n^7)$ field elements.

We ensure that all the $^{A}\langle\cdot\rangle_d$-sharings generated via our VSS protocol even under different dealers are key-consistent with each other. For this, it would be ensured that the parties apriori hold a set-up for

using consistent MAC keys. Namely, it would be ensured that for every pair of parties (P_i, P_j), there exists a vector $\boldsymbol{\mu}_{ji} = (u_{ji}^{(1)}, \ldots, u_{ji}^{(Mp)}) \in \mathbb{F}^{Mp}$ held by P_j, such that the parties hold their respective twisted-shares for the twisted-sharing $[\boldsymbol{\mu}_{ji}]_t^i$, where the twisted-shares of P_i are 0. Moreover, if P_j is *honest* and P_i is *corrupt*, then $\boldsymbol{\mu}_{ji}$ will be random and unknown to P_i, prohibiting it to forge a different MAC and cheat. Interestingly, if both P_i and P_j are *honest* then adversary will completely know $\boldsymbol{\mu}_{ji}$, since it will have t shares and also the share of P_i which is 0. However, this is fine since an honest P_i will never forge an incorrect MAC to P_j.

The high-level overview of the protocol has been already discussed in Section 2. For a pictorial illustration of how D embeds its inputs into bivariate polynomials and how column-polynomials are signed by the parties through ICP instances, see Fig 4.

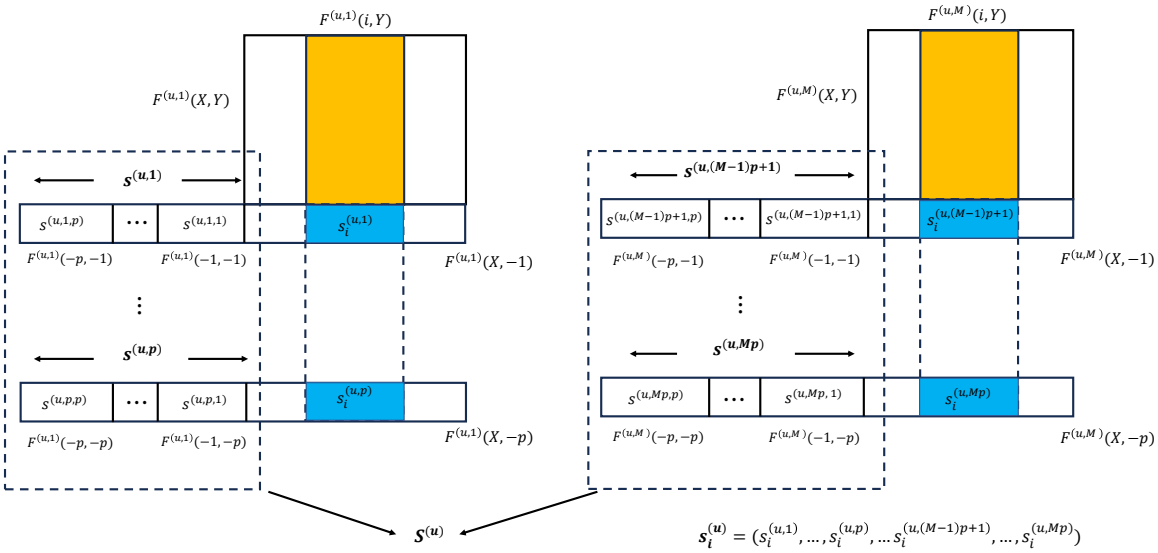

Fig. 4: Pictorial illustration of the sharing phase of protocol $\Pi_{A\langle\cdot\rangle}$. The figure shows the way elements of $\mathbf{S}^{(u)}$ are embedded. D picks M bivariate polynomials $F^{(u,1)}(X,Y), \ldots, F^{(u,M)}(X,Y)$, each of degree (d,d). Each bivariate polynomial hides p number of vectors from $\mathbf{S}^{(u)}$. For instance, the polynomial $F^{(u,v)}(X,Y)$ embeds the elements of the vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)}, \ldots, \mathbf{s}^{(u,v\cdot p)}$, in the polynomials $F^{(u,v)}(X,-1), \ldots, F^{(u,v)}(X,-p)$ respectively. The individual elements of the vectors are embedded at $X = -1, \ldots, -p$. Party P_i gets M column polynomials which are highlighted. Party P_i can compute its share $\mathbf{s}_i^{(u)}$ of the elements in $\mathbf{S}^{(u)}$ by evaluating its column polynomials at $Y = -1, \ldots, -p$. The elements of $\mathbf{s}_i^{(u)}$ are highlighted in blue color. Note that $\mathbf{s}_i^{(u)}$ is degree-d twisted-shared with respect to P_i through d-degree polynomials $F^{(u,1)}(X,-1), \ldots, F^{(u,1)}(X,-p), \ldots, F^{(u,M)}(X,-1), \ldots, F^{(u,M)}(X,-p)$, where the twisted-share of P_l is the same as $\mathbf{s}_l^{(u)}$. Party P_i signs M column polynomials (namely distinct points on these polynomials as highlighted in yellow color) through an instance of ICP for D.

Throughout the protocol, D will be asked to publicly open random linear combination of all its bivariate polynomials *four* times. To maintain the privacy of $\mathbb{S}$ while opening these polynomials, D additionally pick 4 batches of random "masking data" $\mathbf{S}^{(L+1)}, \ldots, \mathbf{S}^{(L+4)}$, each consisting of Mp^2 random elements (similar to $\mathbf{S}^{(u)}$ for any $u \in \{1, \ldots, L\}$). For a pictorial illustration of how the linear combinations of D's bivariate polynomials are opened during the verification phase, see Fig 5.

Duirng the tag-generation phase, for every pair of parties (P_i, P_j) and for every batch $u \in \{1, \ldots, L+4\}$, we want the parties to compute the sharing $\lceil \tau_{ij}^{(u)} \rfloor_{t+d}$ and reconstruct the MAC-tag $\tau_{ij}^{(u)}$ towards P_i, where $\tau_{ij}^{(u)} = \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$. To randomize the above sharing, we mask it with the following sharing:

$$\lceil m \rfloor_{t+d}^i = [[\mathbf{e}_1]]_d^i \cdot \lceil r_{ji}^{(u,1)} || 1 \rfloor_t^i + \ldots + [[\mathbf{e}_d]]_d^i \cdot \lceil r_{ji}^{(u,d)} || d \rfloor_t^i.$$

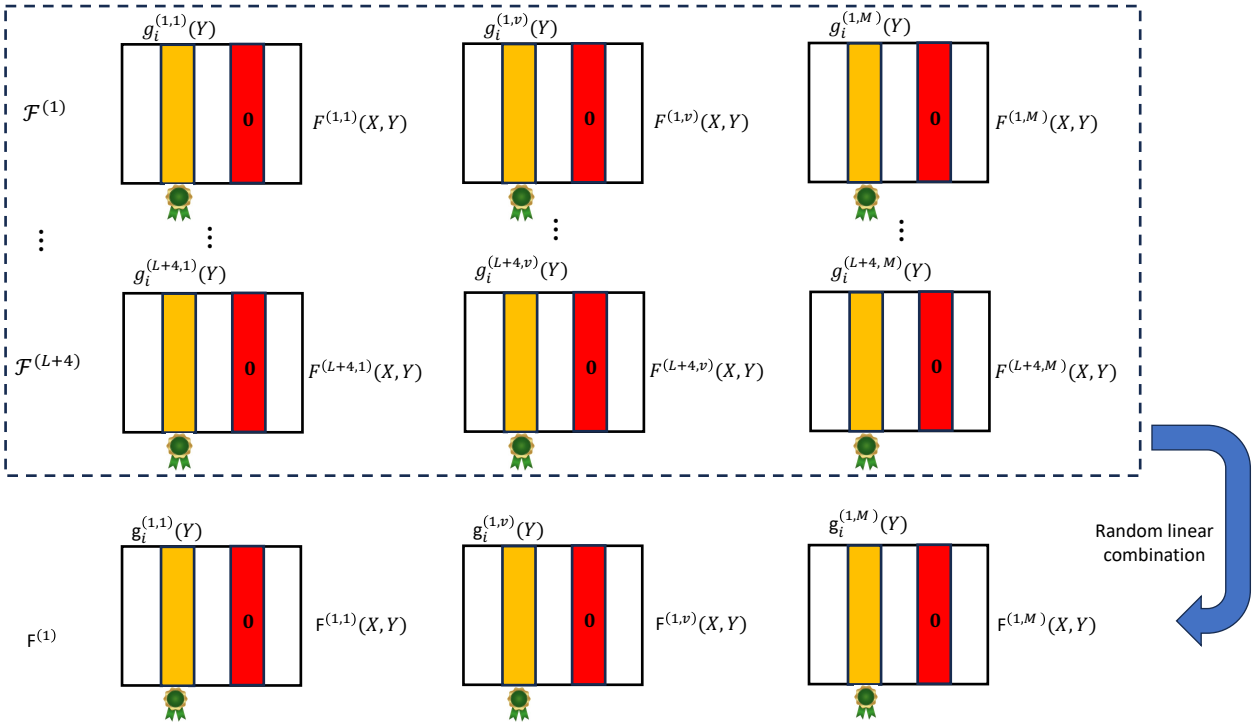

Fig. 5: Pictorial illustration of the verification phase of protocol $\Pi_{A\langle\cdot,\cdot\rangle}$. There are $L+4$ batches $\mathcal{F}^{(1)},\ldots,\mathcal{F}^{(L+4)}$ of bivariate polynomials, whose row and column polynomials have been distributed by D. Each batch has M such bivariate polynomials. The degree of each bivariate polynomial is supposed to be (d,d). Moreover, D is supposed to set the column (and row) polynomials of the parties in Dp (and Cr) to $\mathbf{0}$ in all the bivariate polynomials, as indicated by red color. For each batch of bivariate polynomials, party $P_i \in$ Core has signed all its column polynomials for that batch for D, as indicated with a badge sign. To verify if D has indeed selected the bivariate polynomials in each batch satisfying the above conditions, the parties challenge D to open a random linear combination $\mathsf{F}^{(1)}$ of the polynomials in $\mathcal{F}^{(1)},\ldots,\mathcal{F}^{(L+4)}$ and check if the polynomials in $\mathsf{F}^{(1)}$ satisfy the same conditions as expected from the polynomials in $\mathcal{F}^{(1)},\ldots,\mathcal{F}^{(L+4)}$. To prevent a corrupt D from opening incorrect linear combination, D is also challenged to produce IC-signatures of the parties in Core on their supposed column polynomials of the bivariate polynomials in $\mathsf{F}^{(1)}$, which D computes homomorphically from the IC-signatures of the parties in Core on their column polynomials for the bivariate polynomials in $\mathcal{F}^{(1)},\ldots,\mathcal{F}^{(L+4)}$

Here $\lceil r_{ji}^{(u,1)}||1\rfloor_t^i,\ldots,\lceil r_{ji}^{(u,d)}||d\rfloor_t^i$ are random twisted sharings of random values $r_{ji}^{(u,1)},\ldots,r_{ji}^{(u,d)}$, where $r_{ji}^{(u,1)},\ldots,r_{ji}^{(u,d)}$ are embedded at the points $-1,\ldots,-d$ respectively in the underlying sharing polynomials. Moreover, $\mathbf{e}_1,\ldots,\mathbf{e}_d$ are *publicly known* vectors, each containing $d+1$ elements from $\mathbb{F}$, where $\mathbf{e}_1=(0,1,0,0,\ldots,0),\mathbf{e}_2=(0,0,1,0,\ldots,0),\ldots\mathbf{e}_p=(0,\ldots,0,1)$. Furthermore, $[[\mathbf{e}_1]]_d^i,\ldots,[[\mathbf{e}_d]]_d^i$ denotes publicly-known degree-d degenerate packed twisted-sharing of $\mathbf{e}_1,\ldots,\mathbf{e}_d$ respectively with respect to P_i. In essence, for each $[[\mathbf{e}_k]]_d^i$ the underlying sharing polynomial evaluates to 0 at i, evaluates to 1 at $-k$ and evaluates to 0 at every $j\in\{-1,\ldots,-d\}\setminus\{-k\}$. It is easy to see that the underlying sharing polynomial $f_m(X)$ for $[m]_{t+d}^i$ evaluates to 0 at $X=i$. Hence $m=0$. Moreover, $f_m(X)$ evaluates to $r_{ji}^{(u,1)},\ldots,r_{ji}^{(u,d+1)}$ at $X=-1,\ldots,-d$ respectively. Since $r_{ji}^{(u,1)},\ldots,r_{ji}^{(u,d)}$ are random and adversary will know at most $t+1$ points on $f_m(X)$ incluing the shares of corrupt parties and the i^{th} point which is 0, this will leave d degree-of-freedom in $f_m(X)$ as desired.

We note that P_l's tag-share $\tau_{ij,l}^{(u)}$ for $\lceil\tau_{ij}^{(u)}\rfloor_{t+d}^i$ is supposed to be:

$$\mathbf{s}_l^{(u)}\cdot\boldsymbol{\mu}_{ji,l}+v_{ji,l}^{(u)}+\mathbf{e}_{1,l}\cdot r_{ji,l}^{(u,1)}+\ldots+\mathbf{e}_{d,l}\cdot r_{ji,l}^{(u,d)},$$

where $\boldsymbol{\mu}_{ji,l}$ and $v^{(u)}_{ji,l}$ denote P_l's shares corresponding to $\lceil \boldsymbol{\mu}_{ji} \rfloor^i_t$ and $\lceil v^{(u)}_{ji} \rfloor^i_t$ respectively. And $\mathbf{e}_{k,l}$ and $r^{(u,k)}_{ji,l}$ denotes P_l's shares corresponding to $[[\mathbf{e}_k]]^i_d$ and $\lceil r^{(u,k)}_{ji} || k \rfloor^i_t$ respectively. To verify P_l's tag-share $\tau^{(u)}_{ij,l}$, a naive (privacy-free) solution could be to let D make public the share $\mathbf{s}^{(u)}_l$ in a verifiable way. Then consider the t-sharing

$$\lceil \mathbf{s}^{(u)}_l \rfloor^i_0 \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor^i_t + \lceil v^{(u)}_{ji} \rfloor^i_t + \mathbf{e}_{1,l} \cdot [r^{(u,1)}_{ji} || 1]^i_t + \ldots + \mathbf{e}_{d,l} \cdot [r^{(u,d)}_{ji} || d]^i_t.$$

It is easy to see that P_l's share in the above t-sharing is exactly the same as $\tau^{(u)}_{ij,l}$. So by reconstructing the above t-sharing towards P_i, we can let P_i to verify the tag-share received from P_l. To maintain privacy, instead of applying the above idea on each individual batch $u \in \{1, \ldots, L+4\}$, we apply the idea on a random linear combination of $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L+4)}$.

Discarding a corrupt dealer in the protocol. In the protocol, there will be certain steps where a corrupt D might be publicly identified to be cheating. In that case, the parties terminate the protocol and output ${}^{A}\langle \mathbb{S} \rangle_d$ on behalf of D by setting $\mathbb{S} = 0^{LMp^2}$. As part of ${}^{A}\langle 0^{LMp^2} \rangle_d$, every party P_i sets all its shares as 0 and all its MAC-tags as 0. Correspondingly, every party P_j sets the second component of all its MAC-keys for P_i to 0.

Protocol 6.1: VSS for Generating ${}^{A}\langle \cdot \rangle_d$-sharing – $\Pi_{{}^{A}\langle \cdot \rangle}$

Initialization: Parties hold two common sets dispute set $\mathsf{Dp} \subset \mathcal{P}$ and corrupt set $\mathsf{Cr} \subset \mathcal{P}$. Both initialized to $\emptyset$ and is carried over if the parties restart the protocol. They also maintain two common Boolean variables flag_1 and flag_2, indicating whether the sharing-phase and the pairwise tag-generation phase of protocol is restarted or not (for the same D). Both these variables are initialized to 0.

Input: D has LMp^2 elements from $\mathbb{F}$ which are arranged as a three-dimensional vector $\mathbb{S}$ which consists of L two-dimensional vectors $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)}$. For $u = 1, \ldots, L$, $\mathbf{S}^{(u)}$ consists of Mp one-dimensional vectors, each consisting of p elements. That is, $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,Mp)})$, where each $\mathbf{s}^{(u,v)} \in \mathbb{F}^p$, for $v = 1, \ldots, Mp$ and $\mathbf{s}^{(u,v)} = (s^{(u,v,1)}, \ldots, s^{(u,v,p)})$.

Prior setup: For every pair of parties (P_i, P_j), there exists a $\boldsymbol{\mu}_{ji} = (u^{(1)}_{ji}, \ldots, u^{(Mp)}_{ji}) \in \mathbb{F}^{Mp}$ held by P_j, such that the parties hold $\lceil \boldsymbol{\mu}_{ji} \rfloor^i_t$, where the twisted-shares of P_i are 0.

(Sharing Phase)

1. D selects 4 batches of random *masking-vectors* $\mathbf{S}^{(L+1)}, \ldots, \mathbf{S}^{(L+4)}$, each consisting of Mp^2 uniformly random elements from $\mathbb{F}$. For $u = L+1, \ldots, L+4$, $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,Mp)})$, where each vector $\mathbf{s}^{(u,v)} \in \mathbb{F}^p$, for $v = 1, \ldots, Mp$. The vector $\mathbf{s}^{(u,v)} = (s^{(u,v,1)}, \ldots, s^{(u,v,p)})$.
2. D picks $(L+4)M$ bivariate polynomials $F^{(u,v)}(X, Y)$ over $\mathbb{F}$ for $u = 1, \ldots, L+4$ and $v = 1, \ldots, M$, where each of these polynomials is otherwise a random (d, d)-degree bivariate polynomial over $\mathbb{F}$ with the following constraints, where $d \stackrel{\mathsf{def}}{=} t + p - 1$:
   - $F^{(u,v)}(X, Y)$ embeds $\mathbf{s}^{(u,(v-1)\cdot p+1)}, \ldots, \mathbf{s}^{(u,v\cdot p)}$ at $F^{(u,v)}(X, -1), \ldots, F^{(u,v)}(X, -p)$ respectively. Namely, $\mathbf{s}^{(u,(v-1)\cdot p+1)}$ is placed between indices $-1, \ldots, -p$ for X of $F^{(u,v)}(X, -1)$ and likewise for other polynomials.
   - For every $P_i \in \mathsf{Dp} \cup \mathsf{Cr}$, the condition $F^{(u,v)}(X, i) = F^{(u,v)}(i, Y) = \mathbf{0}$ holds, where $\mathbf{0}$ denotes an all-0 polynomial.
3. For every $P_i \notin \mathsf{Dp} \cup \mathsf{Cr}$, let $f^{(u,v)}_i(X) \stackrel{\mathsf{def}}{=} F^{(u,v)}(X, i)$ and $g^{(u,v)}_i(Y) \stackrel{\mathsf{def}}{=} F^{(u,v)}(i, Y)$. D sends the polynomials $\{f^{(u,v)}_i(X), g^{(u,v)}_i(Y)\}_{u=1,\ldots,L+4, v=1,\ldots,M}$ to P_i.
4. Every $P_i \notin \mathsf{Dp} \cup \mathsf{Cr}$ upon receiving the polynomials $\{f^{(u,v)}_i(X), g^{(u,v)}_i(Y)\}_{u=1,\ldots,L+4, v=1,\ldots,M}$ from D does the following if each of these polynomials is a d-degree polynomial.
   - Send OK_i to $\mathcal{F}_{\mathsf{BC}}$.
   - For $u = 1, \ldots, L+4$, give $\mathsf{Sig}(P_i \to D, \mathbf{g}^{(u,1)}_i, \ldots, \mathbf{g}^{(u,M)}_i)$ to D, *where the parties follow the linearity property while generating IC-signatures* (see Notation 5.7). Here for $v = 1, \ldots, M$, $\mathbf{g}^{(u,v)}_i \stackrel{\mathsf{def}}{=} (g^{(u,v)}_i(1), \ldots, g^{(u,v)}_i(n))$. %. Here P_i is invoking $L+4$ instances of ICP, where in each instance, it is signing M vectors for D, each consisting of n values.

5. For $u = 1, \ldots, L+4$, upon receiving $\mathsf{Sig}(P_i \to D, \mathbf{g}_i^{(u,1)}, \ldots, \mathbf{g}_i^{(u,M)})$ from $P_i \notin \mathsf{Dp} \cup \mathsf{Cr}$ and upon receiving OK_i from the $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender, D includes P_i to Core (initialized to $\emptyset$), provided $\mathbf{g}_i^{(u,v)} = (g_i^{(u,v)}(1), \ldots, g_i^{(u,v)}(n))$ holds, for $v = 1, \ldots, M$.
   - If $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + p$, then D sends Core to $\mathcal{F}_{\mathsf{BC}}$.
   - Else D sets $\mathsf{Dp} = (\mathcal{P} \setminus \mathsf{Core})$ and sends Dp to $\mathcal{F}_{\mathsf{BC}}$.
6. The parties in $\mathcal{P}$ do the following.
   - If Core is received from $\mathcal{F}_{\mathsf{BC}}$ with D as the sender where $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + p$ then do:
     - If OK_i is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender corresponding to each $P_i \in \mathsf{Core}$, then proceed to the next phase.
     - Else discard D and terminate.
   - Else if Dp is received from $\mathcal{F}_{\mathsf{BC}}$ where D is the sender then do:
     - If $\mathsf{flag}_1 = 0$ and $t - p + 1 \leq |\mathsf{Dp}| \leq t$, then set $\mathsf{flag}_1 = 1$, update Dp and restart the protocol.
     - Else discard D and terminate.
   - Else discard D and terminate the protocol.

(Verification Phase)

1. Every $P_i \in \mathcal{P}$ sends Rand to $\mathcal{F}_{\mathsf{Rand}}$ and receives r_1 from $\mathcal{F}_{\mathsf{Rand}}$.
2. The parties invoke $\mathsf{Ver}(r_1)$ (see below the steps of Ver). If D is not discarded after executing the steps of $\mathsf{Ver}(r_1)$, then proceed to the next phase. % The code of Ver will be invoked 4 times within this protocol. So to avoid repetition, we will invoke Ver every time with different randomness.

Verification Process $\mathsf{Ver}(r_\alpha) : \alpha \in \{1, \ldots, 4\}$

1. For $v = 1, \ldots, M$, D does the following.
   - $\mathsf{F}^{(\alpha,v)}(X, Y) \stackrel{\mathsf{def}}{=} F^{(1,v)}(X, Y) + r_\alpha \cdot F^{(2,v)}(X, Y) + \ldots + r_\alpha^{L+3} \cdot F^{(L+4,v)}(X, Y)$.
   - For every $P_i \in \mathsf{Core}$, $\mathsf{G}_i^{(\alpha,v)}(y) \stackrel{\mathsf{def}}{=} \mathsf{F}^{(\alpha,v)}(i, Y)$. Let $\mathsf{G}_i^{(\alpha,v)} \stackrel{\mathsf{def}}{=} (\mathsf{G}_i^{(\alpha,v)}(1), \ldots, \mathsf{G}_i^{(\alpha,v)}(n))$.
   - For every $P_i \in \mathsf{Core}$, D locally computes the signature $\mathsf{Sig}(P_i \to D, \mathsf{G}_i^{(1,1)}, \ldots, \mathsf{G}_i^{(1,M)})$ from $\mathsf{Sig}(P_i \to D, \mathbf{g}_i^{(1,1)}, \ldots, \mathbf{g}_i^{(1,M)}), \mathsf{Sig}(P_i \to D, \mathbf{g}_i^{(2,1)}, \ldots, \mathbf{g}_i^{(2,M)}), \ldots, \mathsf{Sig}(P_i \to D, \mathbf{g}_i^{(L+4,1)}, \ldots, \mathbf{g}_i^{(L+4,M)})$.

   D sends $(\mathsf{F}^{(\alpha,1)}(X, Y), \ldots, \mathsf{F}^{(\alpha,M)}(X, Y))$ to $\mathcal{F}_{\mathsf{BC}}$. For every $P_i \in \mathsf{Core}$, D publicly reveals $\mathsf{Sig}(P_i \to D, \mathsf{G}_i^{(1,1)}, \ldots, \mathsf{G}_i^{(1,M)})$.
2. Every $P_k \in \mathcal{P}$ discards D if any of the following holds.
   - $(\mathsf{F}^{(\alpha,1)}(X, Y), \ldots, \mathsf{F}^{(\alpha,M)}(X, Y)))$ is received from $\mathcal{F}_{\mathsf{BC}}$ where D is the sender and there exists some $v \in \{1, \ldots, M\}$, such that one of the following hold.
     - $\mathsf{F}^{(\alpha,v)}(X, Y)$ is not a (d, d)-degree bivariate polynomial.
     - $\mathsf{F}^{(\alpha,v)}(X, i) \neq \mathbf{0}$ or $\mathsf{F}^{(\alpha,v)}(i, Y) \neq \mathbf{0}$ for some $P_i \in \mathsf{Dp} \cup \mathsf{Cr}$.
   - There exists some $P_i \in \mathsf{Core}$, such that P_k does not accept $\mathsf{Sig}(P_i \to D, \mathsf{G}_i^{(1,1)}, \ldots, \mathsf{G}_i^{(1,M)})$.
   - There exists some $P_i \in \mathsf{Core}$ and some $v \in \{1, \ldots, M\}$, such that the values in $\mathsf{G}_i^{(\alpha,v)}$ do not lie on $\mathsf{F}^{(\alpha,v)}(i, Y)$.

(Non-Core Row and Column Reconstruction Phase)

1. Each $P_i \in \mathsf{Core}$ sends $\{(f_i^{(u,v)}(j), g_i^{(u,v)}(j))\}_{u=1,\ldots,L+4, v=1,\ldots,M}$ to each $P_j \in \mathcal{P} \setminus (\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr})$.
2. Every $P_i \in \mathcal{P}$ sends Rand to $\mathcal{F}_{\mathsf{Rand}}$ and receives r_2 from it. Invoke $\mathsf{Ver}(r_2)$. % 2nd call of Ver.
3. Every $P_j \in \mathcal{P} \setminus (\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr})$ does the following, provided D is not discarded after $\mathsf{Ver}(r_2)$.
   - On receiving $\{(f_i^{(u,v)}(j), g_i^{(u,v)}(j))\}_{u=1,\ldots,L+4, v=1,\ldots,M}$ from $P_i \in \mathsf{Core}$, include P_i to a set $\mathcal{S}_j$ (initialized to $\emptyset$), provided both the following hold for every $v \in \{1, \ldots, M\}$:

$$\mathsf{F}^{(2,v)}(i, j) = \sum_{u=1}^{L+4} (r_2)^{u-1} g_i^{(u,v)}(j), \quad \mathsf{F}^{(2,v)}(j, i) = \sum_{u=1}^{L+4} (r_2)^{u-1} f_i^{(u,v)}(j).$$

   - For every $u \in \{1, \ldots, L+4\}$ and $v \in \{1, \ldots, M\}$, interpolate the points $\{(i, f_i^{(u,v)}(j))\}_{P_i \in \mathcal{S}_j} \cup \{(i, 0)\}_{P_i \in \mathsf{Dp}}$ to recompute the polynomial $g_i^{(u,v)}(Y)$ and interpolate the points $\{(i, g_i^{(u,v)}(j))\}_{P_i \in \mathcal{S}_j} \cup \{(i, 0)\}_{P_i \in \mathsf{Dp}}$ to recompute the polynomial $f_i^{(u,v)}(X)$.

(Pairwise Tag Generation)

1. **(Setup)** For every ordered pair (P_i, P_j) such that $P_i \notin \mathsf{Cr}$, parties prepare the following.
   (a) Parties send $(\mathsf{RandShare}, \mathsf{Cr}, P_i, 0), (\mathsf{RandShare}, \mathsf{Cr}, P_i, 1), \ldots, (\mathsf{RandShare}, \mathsf{Cr}, P_i, d)$ to $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor^i_t}$ to generate random twisted t-sharings $\lceil v^{(u)}_{ji}\rfloor^i_t, \lceil r^{(u,1)}_{ji}||1\rfloor^i_t, \lceil r^{(u,2)}_{ji}||2\rfloor^i_t, \ldots, \lceil r^{(u,d)}_{ji}||d\rfloor^i_t$ with respect to P_i, where the twisted-shares of the parties in Cr are 0 in all these sharings, where $u \in \{1, \ldots, L+4\}$.
   (b) For every $P_l \notin \mathsf{Cr}$, parties send $(\mathsf{RandShare}, \{P_l\}, P_i, 1)$ to $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor^i_t}$ to generate random twisted t-sharings $\lceil \mathsf{mask}^{(l)}_{ji}||1\rfloor^i_t$ and $\lceil \mathtt{mask}^{(l)}_{ji}||1\rfloor^i_t$ with respect to P_i, where the shares of P_l are 0.
2. **(Tag Share Computation)**
   (a) For every ordered pair (P_i, P_j) where $P_i \notin \mathsf{Cr}$, every $P_l \notin \mathsf{Cr}$ computes its share of $(t+d)$-degree sharing of the MAC-tag $\tau^{(u)}_{ij} \stackrel{\mathsf{def}}{=} \mathbf{s}^{(u)}_i \cdot \boldsymbol{\mu}_{ji} + v^{(u)}_{ji}$, for each $u \in \{1, \ldots, L+4\}$ as follows:

$$\lceil \tau^{(u)}_{ij}\rfloor^i_{t+d} = \lceil \mathbf{s}^{(u)}_i\rfloor^i_d \cdot \lceil \boldsymbol{\mu}_{ji}\rfloor^i_t + \lceil v^{(u)}_{ji}\rfloor^i_t + \sum_{k=1}^{d} [[\mathbf{e}_k]]^i_d \cdot \lceil r^{(u,k)}_{ji}||k\rfloor^i_t,$$

   where $\mathbf{s}^{(u)}_i \stackrel{\mathsf{def}}{=} (g^{(u,1)}_i(-1), \ldots, g^{(u,1)}_i(-p), \ldots, g^{(u,M)}_i(-1), \ldots, g^{(u,M)}_i(-p)) \in \mathbb{F}^{M\cdot p}$. Here $\mathbf{e}_1, \ldots, \mathbf{e}_d \in \mathbb{F}^{d+1}$ are publicly-known vectors, where $\mathbf{e}_1 = (0, 1, 0, \ldots, 0), \mathbf{e}_2 = (0, 0, 1, 0, \ldots, 0), \ldots, \mathbf{e}_d = (0, \ldots, 0, 1)$. Moreover, $[[\mathbf{e}_1]]^i_d, \ldots, [[\mathbf{e}_d]]^i_d$ denote degree-d degenerate packed twisted sharing of $\mathbf{e}_1, \ldots, \mathbf{e}_d$ with respect to P_i (Definition 4.8).
   (b) For every ordered pair (P_i, P_j) where $P_i \notin \mathsf{Cr}$, every $P_l \notin \mathsf{Cr}$ sends its tag-shares $(\tau^{(1)}_{ij,l}, \ldots, \tau^{(L+4)}_{ij,l})$ to P_i, where $\tau^{(u)}_{ij,l}$ denotes P_l's share of $\lceil \tau^{(u)}_{ij}\rfloor^i_{t+d}$.
3. **(Tag Share Verification)**
   (a) Every $P_i \in \mathcal{P}$ sends Rand to $\mathcal{F}_{\mathsf{Rand}}$ and receives r_3 from it. Invoke $\mathsf{Ver}(r_3)$.% 3rd call of Ver.
   (b) If D is not discarded during $\mathsf{Ver}(r_3)$, then for every ordered triplet (P_i, P_j, P_l) where $P_i, P_l \notin \mathsf{Cr}$, the parties do the following.
   – Let $\mathbf{s}_l \stackrel{\mathsf{def}}{=} (\mathsf{F}^{(3,1)}(l, -1), \ldots, \mathsf{F}^{(3,1)}(l, -p), \ldots, \mathsf{F}^{(3,M)}(l, -1), \mathsf{F}^{(3,M)}(l, -1), \ldots, \mathsf{F}^{(3,M)}(l, -p)) \in \mathbb{F}^{M\cdot p}$. Parties compute

$$\lceil \beta_{ij,l}\rfloor^i_t = \lceil \mathbf{s}_l\rfloor^i_0 \cdot \lceil \boldsymbol{\mu}_{ji}\rfloor^i_t + \sum_{\nu=1}^{L+3} (r_3)^{\nu-1} \left(\lceil v^{(\nu)}_{ji}\rfloor^i_t + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r^{(\nu,k)}_{ji}||k\rfloor^i_t \right) + \lceil \mathsf{mask}^{(l)}_{ji}\rfloor^i_t$$

     Here $[\mathbf{s}_l]^i_0$ denotes the default degree-0 twisted sharing of $\mathbf{s}_l$ with respect to P_i where the twisted-share of all the parties is $\mathbf{s}_l$. And $e_{1,l}, \ldots, e_{d,l}$ denotes P_l's shares of $[[\mathbf{e}_1]]^i_d, \ldots, [[\mathbf{e}_d]]^i_d$, which are known publicly. The parties then reconstruct the sharing $\lceil \beta_{ij,l}\rfloor^i_t$ towards P_i.
   – Party P_i upon receiving the tag-shares $(\tau^{(1)}_{ij,l}, \ldots, \tau^{(L+4)}_{ij,l})$ from P_l, includes P_l to the set $\mathcal{R}_{ij}$, initialized to $\emptyset$, provided P_l's share in the reconstructed sharing $\lceil \beta_{ij,l}\rfloor^i_t$ is same as $\tau^{(1)}_{ij,l} + r_3 \cdot \tau^{(2)}_{ij,l} + \ldots + r_3^{L+3} \cdot \tau^{(L+4)}_{ij,l}$.
   (c) Every $P_i \notin \mathsf{Cr}$ does the following.
   – If for every P_j, the condition $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t+p$ holds, then for $u = 1, \ldots, L+4$, interpolate the points $\{(l, \tau^{(u)}_{ij,l})\}_{P_l \in \mathcal{R}_{ij}} \cup \{(l, 0)\}_{P_l \in \mathsf{Cr}} \cup (0, \tau^{(u)}_{ij,i})$ to reconstruct the sharing $\lceil \tau^{(u)}_{ij}\rfloor^i_{t+d}$, compute $\tau^{(u)}_{ij}$ and send Tag_i to $\mathcal{F}_{\mathsf{BC}}$.
   – Else include P_l to a set Dp_i, initialized to $\emptyset$, provided there exists some P_j such that $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \leq 2t + p$ and $P_l \notin \mathcal{R}_{ij} \cup \mathsf{Cr}$. Send $(\mathsf{NOK}, \mathsf{Dp}_i)$ to $\mathcal{F}_{\mathsf{BC}}$.
   (d) If $\mathsf{flag}_2 = 1$ OR if Tag_k is received from $\mathcal{F}_{\mathsf{BC}}$ where P_k is the sender corresponding to every $P_k \notin \mathsf{Cr}$, then the parties do the following.
   – For every ordered pair (P_i, P_j) where $P_i \notin \mathsf{Cr}$, parties reconstruct $\lceil v^{(u)}_{ji}\rfloor^i_t$ towards P_j, for every $u \in \{1, \ldots, L\}$.
   – For $u = 1, \ldots, L$, each $P_i \notin \mathsf{Cr}$ outputs $\mathbf{s}^{(u)}_i$ as its shares, $(\tau^{(u)}_{i1}, \ldots, \tau^{(u)}_{in})$ as MAC-tags and $(\mathsf{K}^{(u)}_{i1}, \ldots, \mathsf{K}^{(u)}_{in})$ as MAC-keys corresponding to ${}^A\langle \mathbf{S}^{(u)}\rangle_d$. Moreover, based on its information corresponding to ${}^A\langle \mathbf{S}^{(1)}\rangle_d, \ldots, {}^A\langle \mathbf{S}^{(L)}\rangle_d$, party P_i outputs its respective information corresponding to ${}^A\langle \mathbb{S}\rangle_d$. Here $\mathsf{K}^{(u)}_{ij} = (\boldsymbol{\mu}_{ij}, v^{(u)}_{ij})$ for every $P_j \notin \mathsf{Cr}$, while $\mathsf{K}^{(u)}_{ij} = (\boldsymbol{\mu}_{ij}, 0)$ for every $P_j \in \mathsf{Cr}$.

- For $u = 1, \ldots, L$, each $P_i \in \mathsf{Cr}$ outputs 0^{Mp} as its shares, 0^n as MAC-tags and $(\mathsf{K}_{i1}^{(u)}, \ldots, \mathsf{K}_{in}^{(u)})$ as MAC-keys corresponding to ${}^{A}\langle \mathbf{S}^{(u)} \rangle_d$. Moreover, based on its information corresponding to ${}^{A}\langle \mathbf{S}^{(1)} \rangle_d, \ldots, {}^{A}\langle \mathbf{S}^{(L)} \rangle_d$, party P_i outputs its respective information corresponding to ${}^{A}\langle \mathbb{S} \rangle_d$. Here $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, v_{ij}^{(u)})$ for every $P_j \notin \mathsf{Cr}$, while $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, 0)$ for every $P_j \in \mathsf{Cr}$.

(e) Else the parties proceed to the next phase.

4. **(Tag Share Complaint Resolution)**

(a) If $(\mathsf{NOK}, \mathsf{Dp}_i)$ is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender such that $|\mathsf{Dp}_i| \leq t$, then every $P_l \in \mathsf{Dp}_i$ sends $\{(\tau_{ij,l}^{(1)}, \ldots, \tau_{ij,l}^{(L+4)})\}_{j=1,\ldots,n}$ to $\mathcal{F}_{\mathsf{BC}}$.

(b) Every $P_i \in \mathcal{P}$ sends Rand to $\mathcal{F}_{\mathsf{Rand}}$ and receives r_4 from it. Invoke $\mathsf{Ver}(r_4)$. % 4th call of Ver.

(c) If D is not discarded during $\mathsf{Ver}(r_4)$, then for every pair (P_i, P_l) where $(\mathsf{NOK}, \mathsf{Dp}_i)$ is received from $\mathcal{F}_{\mathsf{BC}}$ with P_i being the sender and where $P_l \in \mathsf{Dp}_i$, the parties do the following upon receiving $(\mathsf{tagshare}, P_i, \{\tau_{ij,l}^{(1)}, \ldots, \tau_{ij,l}^{(L+4)}\}_{j=1,\ldots,n})$ from $\mathcal{F}_{\mathsf{BC}}$ where P_l is the sender.

- Let $\mathsf{s}_l \stackrel{\mathsf{def}}{=} (\mathsf{F}^{(4,1)}(l,-1), \ldots, \mathsf{F}^{(4,1)}(l,-p), \ldots, \mathsf{F}^{(4,M)}(l,0), \mathsf{F}^{(4,M)}(l,-1), \ldots, \mathsf{F}^{(4,M)}(l,-p)) \in \mathbb{F}^{M \cdot p}$. For $j = 1, \ldots, n$, parties compute

$$\lceil \gamma_{ij,l} \rfloor_t^i = \lceil \mathsf{s}_l \rfloor_0^i \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor_t^i + \sum_{\nu=1}^{L+3} (r_4)^{\nu-1} \left(\lceil v_{ji}^{(\nu)} \rfloor_t^i + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r_{ji}^{(\nu,k)} || k \rfloor_t^i \right) + \lceil \mathsf{mask}_{ji}^{(l)} \rfloor_t^i$$

  Here $\lceil \mathsf{s}_l \rfloor_0^i$ is degree-0 twisted-sharing of s_l where the twisted-share of all the parties is s_l. For $j = 1, \ldots, n$, the parties publicly reconstruct the sharings $\lceil \gamma_{i1,l} \rfloor_t^i, \ldots, \lceil \gamma_{in,l} \rfloor_t^i$.
- If for some $j \in \{1, \ldots, n\}$, party P_l's share in the reconstructed sharing $[\gamma_{ij,l}]_t^i$ is different from $\tau_{ij,l}^{(1)} + r_4 \cdot \tau_{ij,l}^{(2)} + \ldots + r_4^{L+3} \cdot \tau_{ij,l}^{(L+4)}$, then include P_l to the set Cr.
- Else P_i removes P_l from Dp_i and includes P_l to the set $\mathcal{R}_{ij}$, for $j = 1, \ldots, n$.

(d) If $|\mathsf{Cr}| \geq t - p + 1$, then the parties set $\mathsf{flag}_2 = 1$ and restart the protocol.

(e) Else the parties do the following.

- For every order pair of parties (P_i, P_j) where $P_i \notin \mathsf{Cr}$, parties reconstruct $\lceil v_{ji}^{(u)} \rfloor_t^i$ towards P_j, for every $u \in \{1, \ldots, L\}$.
- Each $P_i \notin \mathsf{Cr}$ does the following.
  - If for some $P_j \in \mathcal{P}$ the MAC-tags are not yet computed then for $u = 1, \ldots, L+4$, interpolate the points $\{(l, \tau_{ij,l}^{(u)})\}_{P_l \in \mathcal{R}_{ij}} \cup \{(l,0)\}_{P_l \in \mathsf{Cr}} \cup (0, \tau_{ij,i}^{(u)})$ to reconstruct the sharing $\lceil \tau_{ij}^{(u)} \rfloor_{t+d}^i$ and compute $\tau_{ij}^{(u)}$. Here if P_l is added to $\mathcal{R}_{ij}$ in this phase, then consider $\tau_{ij,l}^{(u)}$ broadcasted by P_l in this phase.
  - For $u = 1, \ldots, L$, party P_i outputs $\mathbf{s}_i^{(u)}$ as its shares, $(\tau_{i1}^{(u)}, \ldots, \tau_{in}^{(u)})$ as MAC-tags and $(\mathsf{K}_{i1}^{(u)}, \ldots, \mathsf{K}_{in}^{(u)})$ as MAC-keys corresponding to ${}^{A}\langle \mathbf{S}^{(u)} \rangle_d$. Moreover, based on its information corresponding to ${}^{A}\langle \mathbf{S}^{(1)} \rangle_d, \ldots, {}^{A}\langle \mathbf{S}^{(L)} \rangle_d$, each P_i outputs its respective information corresponding to ${}^{A}\langle \mathbb{S} \rangle_d$. Here $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, v_{ij}^{(u)})$ for every $P_j \notin \mathsf{Cr}$, while $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, 0)$ for every $P_j \in \mathsf{Cr}$.
- For $u = 1, \ldots, L$, each $P_i \in \mathsf{Cr}$ outputs 0^{Mp} as its shares, 0^n as MAC-tags and $(\mathsf{K}_{i1}^{(u)}, \ldots, \mathsf{K}_{in}^{(u)})$ as MAC-keys corresponding to ${}^{A}\langle \mathbf{S}^{(u)} \rangle_d$. Moreover, based on its information corresponding to ${}^{A}\langle \mathbf{S}^{(1)} \rangle_d, \ldots, {}^{A}\langle \mathbf{S}^{(L)} \rangle_d$, party P_i outputs its respective information corresponding to ${}^{A}\langle \mathbb{S} \rangle_d$. Here $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, v_{ij}^{(u)})$ for every $P_j \notin \mathsf{Cr}$, while $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, 0)$ for every $P_j \in \mathsf{Cr}$.

We now prove the properties of the protocol $\Pi_{A\langle\cdot\rangle}$. We stress that we *do not* model the security requirements of $\Pi_{A\langle\cdot\rangle}$ through an ideal functionality. Instead, we prove the standard security properties as expected from any VSS, namely privacy, correctness and commitment. Looking ahead, we will show how to simulate the steps of $\Pi_{A\langle\cdot\rangle}$ in the protocols where $\Pi_{A\langle\cdot\rangle}$ is used as a primitive. While proving the properties of $\Pi_{A\langle\cdot\rangle}$ it will be assumed that the parties will have a prior setup of consistent MAC-keys. Namely, for every pair of parties (P_i, P_j), there will be a vector $\boldsymbol{\mu}_{ji} \in \mathbb{F}^{Mp}$ held by P_j, such that the parties hold $[\boldsymbol{\mu}_{ji}]_t^i$, where the twisted-shares of P_i are 0. Moreover, if P_j is *honest* and P_i is *corrupt* then $\boldsymbol{\mu}_{ji}$ remains random for the adversary. Looking ahead, it would be ensured that the parties have this apriori setup when $\Pi_{A\langle\cdot\rangle}$ will be used as a primitive in any of our protocols.

To begin with, we first consider an *honest* D and prove the properties of the protocol which hold for an honest D. We first show that an honest D will *not* be discarded in the protocol.

Claim 6.2. *If D is honest, then except with a negligible probability, D will not be discarded in the protocol $\Pi_{A\langle\cdot\rangle}$.*

Proof. We analyze the various conditions in the protocol under which D could be discarded and show that none of those conditions will be true for an honest D, except with a negligible probability.

During the sharing phase, an honest D will not be discarded, since it will broadcast a valid Core set, either during the first run of the protocol or during the second run of the protocol. This is because if D is unable to find a Core of size $2t+p$ during the first run of the protocol, then it will include at least $t-p+2$ *corrupt* parties in Dp. Subsequently, when the parties restart the protocol, at least $2t+1$ honest parties will be included in Core and consequently $|\mathsf{Core} \cup \mathsf{Dp}| \geq (2t+1)+t-p+2 \geq 2t+p$ will be satisfied, as $p = \frac{t}{4}+1$.

For the rest of the phases, D gets discarded if it fails to reveal the IC-signature of any party from Core. An honest D will always be able to reveal IC-signature of every honest party from Core, which follows from the correctness of ICP (Lemma 5.3). On the other hand hand, the non-repudiation property of ICP (Lemma 5.5) guarantees that except with a negligible probability, an *honest* D will be able to reveal the IC-signature of every potential corrupt party in Core. As there are polynomially many instances of ICP, it follows that D will not be discarded due to this condition, except with a negligible probability. □

We next claim that if D is honest then every party in Cr is corrupt.

Claim 6.3. *In protocol $\Pi_{A\langle\cdot\rangle}$, if D is honest and $\mathsf{Cr} \neq \emptyset$, then every party in Cr is corrupt.*

Proof. Let D be honest and let $\mathsf{Cr} \neq \emptyset$. Consider an arbitrary party $P_l \in \mathsf{Cr}$. We want to show that P_l is *corrupt*. On contrary, let P_l be honest. Since $\mathsf{Cr} \neq \emptyset$, this implies that D is *not* discarded. Next, consider an arbitrary pair of parties (P_i, P_j), such that P_i broadcasts $(\mathsf{NOK}, \mathsf{Dp}_i)$ in the protocol where $P_l \in \mathsf{Dp}_i$. Since P_l is honest, it correctly computes and sends the tag-share $\tau_{ij,l}^{(u)}$ to P_i, for every $u \in \{1, \ldots, L+4\}$, where the following holds:

$$\tau_{ij,l}^{(u)} = \mathbf{s}_l^{(u)} \cdot \boldsymbol{\mu}_{ji,l} + v_{ji,l}^{(u)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(u,k)}.$$

where $r_{ji,l}^{(u,k)}$ denotes P_l's share of $\lceil r_{ji}^{(u,k)} || k \rfloor_t^i$. We will show that if this is the case then P_l will not be included to Cr, which is a contradiction. Since D is *honest*, it implies that for each $v \in \{1, \ldots, M\}$, the following holds:

$$\mathsf{F}^{(3,v)} = F^{(1,v)}(X,Y) + r_3 \cdot F^{(2,v)}(X,Y) + \ldots + r_3^{L+3} \cdot F^{(L+4,v)}(X,Y).$$

This further implies that $\mathbf{s}_l = \mathbf{s}_l^{(1)} + r_3 \cdot \mathbf{s}_l^{(2)} + \ldots + r_3^{L+3} \cdot \mathbf{s}_l^{(L+4)}$ holds. Now consider the following linear combination of the tags:

$$\tau_{ij,l}^{(1)} + r_3 \cdot \tau_{ij,l}^{(2)} + \ldots + r_3^{L+3} \cdot \tau_{ij,l}^{(L+4)}.$$

The above linear combination will be the same as

$$\mathbf{s}_l \cdot \boldsymbol{\mu}_{ji,l} + \left((v_{ji,l}^{(1)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(1,k)}) + r_3 \cdot (v_{ji,l}^{(2)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(2,k)}) + \ldots + r_3^{L+3} \cdot (v_{ji,l}^{(L+4)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(L+4,k)}) \right)$$

Now consider the twisted t-sharing:

$$[\beta_{ij,l}]_t^i = [\mathbf{s}_l]_0^i \cdot [\boldsymbol{\mu}_{ji}]_t^i + \left(\lceil v_{ji}^{(1)} \rfloor_t^i + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r_{ji}^{(1,k)} || k \rfloor_t^i \right) + r_3 \cdot \left(\lceil v_{ji}^{(2)} \rfloor_t^i + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r_{ji}^{(2,k)} || k \rfloor_t^i \right) + \ldots$$
$$+ r_3^{L+3} \cdot \left(\lceil v_{ji}^{(L+4)} \rfloor_t^i + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r_{ji}^{(L+4,k)} || k \rfloor_t^i \right) + [\mathsf{mask}_{ji}^{(l)}]_t^i$$

It is easy to see that P_l's share in $[\mathbf{s}_l]_0^i \cdot [\boldsymbol{\mu}_{ji}]_t^i$. will be $\mathbf{s}_l \cdot \boldsymbol{\mu}_{ji,l}$. We also know that P_l's share in $[\mathsf{mask}_{ji}^{(l)}]_t^i$ will be 0, as $[\mathsf{mask}_{ji}^{(l)}]_t^i$ is generated by D, who sets the share of P_l to 0. It then turns out that the share of P_l in the rest of twisted t-sharing above will be the same as $(v_{ji,l}^{(1)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(1,k)}) + r_3 \cdot (v_{ji,l}^{(2)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(2,k)})$

$+ \ldots + r_3^{L+3} \cdot (v_{ji,l}^{(L+4)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(L+4,k)})$. Consequently, the share of P_l in $[\beta_{ij,l}]_t^i$ will be the same as $\tau_{ij,l}^{(1)} + r_3 \cdot \tau_{ij,l}^{(2)} + \ldots + r_3^{L+3} \cdot \tau_{ij,l}^{(L+4)}$. From the protocol steps, P_i will robustly reconstruct the entire sharing $[\beta_{ij,l}]_t^i$ and find that the share of P_l is the same as $\tau_{ij,l}^{(1)} + r_3 \cdot \tau_{ij,l}^{(2)} + \ldots + r_3^{L+3} \cdot \tau_{ij,l}^{(L+4)}$. Now there are two possible cases, depending upon whether P_i is *honest* or *corrupt*.

If P_i is honest, then P_i will *not* include P_l to Dp_i, which is a contradiction. On the other hand, if P_i is *corrupt*, then it may include P_l to Dp_i. Then consider the tag-share complaint resolution phase. By using similar arguments as above, it can be shown that P_l's share in $[\gamma_{ij,l}]_t^i$ will be the same as $\tau_{ij,l}^{(1)} + r_4 \cdot \tau_{ij,l}^{(2)} + \ldots + r_4^{L+3} \cdot \tau_{ij,l}^{(L+4)}$. Consequently, P_l will not be included to Cr in this case as well, which is a contradiction. □

We next show that if D is honest then adversary's view remains independent of D's input.

Lemma 6.4 (Privacy). *If D is honest then adversary's view remains independent of D's input $\mathbb{S}$.*

Proof. Let D be *honest*. To prove the lemma, we show that throughout the protocol, adversary may learn at most t row and column polynomials on each bivariate polynomial $F^{(u,v)}(X,Y)$, where $u \in \{1,\ldots,L\}$ and $v \in \{1,\ldots,M\}$. This will imply that adversary will learn at most $t(t+p)+tp$ distinct points on each of these polynomials, as each row and column polynomial is of degree $d = t+p-1$. Since each of these bivariate polynomials is a random (d,d)-degree polynomial, this will leave p^2 degree-of-freedom in these polynomials and consequently, adversary will not learn the elements of $\mathbb{S}$ which are embedded in these polynomials. That is, for every candidate vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,v\cdot p)} \in \mathbb{F}^p$, there exists a unique degree-(d,d) bivariate polynomial $F^{(u,v)}(X,Y)$, embedding the elements of $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,v\cdot p)}$ at $F^{(u,v)}(X,-1),\ldots,F^{(u,v)}(X,-p)$ respectively, which will be consistent with the t row and column polynomials learnt by the adversary for $F^{(u,v)}(X,Y)$. Since $F^{(u,v)}(X,Y)$ would have been chosen randomly by D, it follows that adversary's view will remain independent of the elements of $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,v\cdot p)}$.

We next compute the amount of information for each bivariate polynomial $F^{(u,v)}(X,Y)$ learnt by the adversary during various steps of the protocol.

- **Sharing phase**: Consider any arbitrary polynomial $F^{(u,v)}(X,Y)$. Then adversary might receive at most t row and column polynomials. Moreover, the row and column polynomials corresponding to the parties in $\mathsf{Dp} \cup \mathsf{Cr}$ will be set to $\mathbf{0}$. However, from Claim 6.3 and from the proof of Claim 6.2, if D is honest then the parties in $\mathsf{Dp} \cup \mathsf{Cr}$ are corrupt and consequently $|\mathsf{Dp} \cup \mathsf{Cr}| \leq t$. Next, consider an arbitrary *honest* party P_i. From the privacy property of ICP (Lemma 5.4), adversary does not learn any additional information about the points $\mathbf{g}_i^{(u,v)}$ during the instances of ICP invoked by P_i. So overall in this phase, the adversary learns at most t row and column polynomials on $F^{(u,v)}(X,Y)$. Note that this is true even if parties restart the protocol, since D picks independent bivariate polynomials during the rerun.
- **Publicly revealing linear combinations of bivariate polynomials**: In the worst case, D makes public the linear combinations $\mathsf{F}^{(1,v)}(X,Y), \mathsf{F}^{(2,v)}(X,Y), \mathsf{F}^{(3,v)}(X,Y)$ and $\mathsf{F}^{(4,v)}(X,Y)$ of the bivariate polynomials $F^{(1,v)}(X,Y),\ldots,F^{(L+4,v)}(X,Y)$ during the remaining phases of the protocol. Let $U_1 = \{1,\ldots,L\}$ and $U_2 = \{L+1,\ldots,L+4\}$. As shown above, during the sharing phase, for *every* $F^{(u,v)}(X,Y)$ where $u \in U_1 \cup U_2$ adversary learnt at most $t(t+p)+tp$ distinct points and its view remains independent of the vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,v\cdot p)} \in \mathbb{F}^p$ embedded in this bivariate polynomial. It then follows that even after learning $\mathsf{F}^{(1,v)}(X,Y), \mathsf{F}^{(2,v)}(X,Y), \mathsf{F}^{(3,v)}(X,Y)$ and $\mathsf{F}^{(4,v)}(X,Y)$, adversary does not learn any additional information about the bivariates $F^{(u,v)}(X,Y)$ where $u \in U_1$. That is from adversary's point of view, for every candidate vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,v\cdot p)} \in \mathbb{F}^p$ embedded in $F^{(u,v)}(X,Y)$ where $u \in U_1$, there exist unique masking vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,v\cdot p)} \in \mathbb{F}^p$ embedded in $F^{(u,v)}(X,Y)$ where $u \in U_2$ and unique bivariate polynomials $F^{(u,v)}(X,Y)$ where $u \in U_1 \cup U_2$, which will be consistent with $\mathsf{F}^{(1,v)}(X,Y), \mathsf{F}^{(2,v)}(X,Y), \mathsf{F}^{(3,v)}(X,Y), \mathsf{F}^{(4,v)}(X,Y)$ and with the row and column polynomials learnt by the adversary throughout. Since for every $F^{(u,v)}(X,Y)$ where $u \in U_2$, the underlying embedded vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,v\cdot p)} \in \mathbb{F}^p$ are completely random, as they are part of the random masking-vectors $\mathbf{S}^{(u)}$, it follows that adversary's view still consists of t rows and columns on the bivariates $F^{(u,v)}(X,Y)$ for every $u \in U_1$.
  Note that while making public the linear combinations $\mathsf{F}^{(1,v)}(X,Y), \mathsf{F}^{(2,v)}(X,Y), \mathsf{F}^{(3,v)}(X,Y)$ and $\mathsf{F}^{(4,v)}(X,Y)$, dealer also publicly reveal the IC-signature on the points of the column polynomials

of $\mathsf{F}^{(1,v)}(X,Y), \mathsf{F}^{(2,v)}(X,Y), \mathsf{F}^{(3,v)}(X,Y)$ and $\mathsf{F}^{(4,v)}(X,Y)$. However, these points will be already in adversary's view and *does not* add any new information to adversary's view.

– **Tag Generation Phase**: In the setup stage of the pairwise tag generation phase, parties generate random t-sharings. For each of these t-sharings, adversary may learn at most t shares and hence the underlying shared values remain random for the adversary. While computing the tags, each party learns a degree-$(d+t)$ sharing of each its supposed MAC-tag. The degree-$(d+t)$ sharing $\lceil \mathbf{s}_i^{(u)} \rfloor_d^i \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor_t^i + \lceil v_{ji}^{(u)} \rfloor_t^i$ is masked by the random degree-$(d+t)$ polynomial corresponding to $\sum_{k=1}^{d} [[\mathbf{e}_k]]_d^i \cdot \lceil r_{ji}^{(u,k)} || k \rfloor_t^i$ thus ensuring the view of the adversary remains independent of the inputs. While verifying the tag-shares and resolving complaints, the randomness of `mask` and `mask` prevents the adversary from learning additional information about the bivariate polynomials.

Thus, in each phase of the protocol $\Pi_{A\langle\cdot\rangle}$, the view of the adversary is independent of the inputs if D is *honest*. □

The next lemma shows that the (honest) parties always terminate the protocol $\Pi_{A\langle\cdot\rangle}$, irrespective of the behaviour of the corrupt parties.

Lemma 6.5. *The parties need to run the protocol $\Pi_{A\langle\cdot\rangle}$ at most 3 times to get an output.*

Proof. From the protocol steps, it follow that the parties need to restart the protocol, either due to the failure of tag-generation phase or due to the failure of sharing phase. Let us first consider the case when the parties restart the protocol due to the failure of tag-generation phase. In this case, the parties include at least $t-p+1$ *corrupt* parties in Cr. We now show that once at least $t-p+1$ corrupt parties are included in Cr, then next time the parties restart the protocol, the tag-generation phase will be successful for an *honest* dealer. This will also imply that if the tag-generation phase of the next run of the protocol fails, then D is *corrupt* and the parties pre-maturely terminate the protocol after discarding D and taking default output on behalf of D.

Note that $|\mathcal{H}| \geq 2t+1$, where $\mathcal{H}$ is the set of honest parties in $\mathcal{P}$. During the tag-generation phase of the next run of the protocol, every $P_i \in \mathcal{H}$ is bound to get an $\mathcal{R}_{ij}$ set for every $P_j \in \mathcal{P}$ such that $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t+p$ holds. This is because the set $\mathcal{H}$ always constitutes such a candidate $\mathcal{R}_{ij}$ set, as $p = \frac{t}{4}+1$. Consequently, if P_i complaints about not having an $\mathcal{R}_{ij}$ set satisfying the above condition during the next run of the protocol, the parties will ignore the complaint and do not proceed with the complaint resolution, since P_i is guanateed to be corrupt in this case.

Let us now consider the case when the parties restart the protocol due to the failure of sharing phase, implying that D fails to obtain a Core set of size at least $2t+p$. In this case, the parties who are not part of Core will be included in the set Dp, where $|\mathsf{Dp}| \geq t-p+1$ and the parties restart the protocol. Now in the next run of the protocol, an *honest* D is guaranteed to obtain a candidate Core set, satisfying the condition $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t+p$, since the set $\mathcal{H}$ always constitute a candidate Core set. Consequently, for an *honest* D, the sharing phase of the next run of the protocol will not fail and if at all D fails to produce a Core set where $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t+p$, it will imply that D is *corrupt* and the parties will terminate the protocol with a default output.

From the above discussion it follows that the worst case occurs when the sharing-phase fails once and when the tag-generation phase fails once, resulting in $|\mathsf{Dp}| \geq t-p+1$ and $|\mathsf{Cr}| \geq t-p+1$. This is followed by the third run of the protocol, where either all the phases of the protocol succeed or otherwise D is publicly discarded. □

We next show that if D is honest, then except with a negligible probability, the honest parties output ${}^{A}\langle \mathbb{S} \rangle_d$.

Lemma 6.6 (Correctness). *In protocol $\Pi_{A\langle\cdot\rangle}$ if D is honest and participates with input $\mathbb{S} \in \mathbb{F}^{LMp^2}$, then except with probability $2^{-\Omega(\kappa)}$, all (honest) parties output their respective information corresponding to ${}^{A}\langle \mathbb{S} \rangle_d$.*

Proof. Let D be honest. To prove the lemma, first observe that the protocol can terminate only in two ways, either the dealer is discarded and parties output the default sharing of all-zero values or parties output the shares dealt by the dealer along with its authentication information. From the protocol description, it is easy to see that parties may compute the output only at Step 6 of the sharing phase, Step 2 of the verification phase, Step 2 of the non-Core row and column reconstruction phase, Steps 3a,

3d of tag share verification, and Steps 4b, 4e of tag share complaint resolution. By observation, excluding Step 3d of tag share verification and Step 4e of tag share complaint resolution, all the cases correspond to the dealer being discarded and parties computing a sharing of the default all-zero vectors. From Claim 6.2, it follows that an honest dealer is never discarded in the protocol, except with a negligible probability. Hence, the parties must compute an output during either Step 3d of tag share verification or Step 4e of tag share complaint resolution. For these cases, it is easy to see that the following holds:

- **Sharing Phase:** During the sharing phase, an honest dealer always sends shares consistent with its bivariate polynomials to all the parties. Moreover, it receives signatures on shares from all honest parties. Hence, each honest party is always included in Core. It is easy to see that throughout the protocol, each honest P_i holds $\mathbf{s}_i^{(u)}$ as its share that is received from the dealer and does not update it. Moreover, the share $\mathbf{s}_i^{(u)} = 0^{Mp}$ for each $P_i \in \mathsf{Dp} \cup \mathsf{Cr}$ is also consistent with the dealer's bivariate polynomials used during the sharing phase and does not change during the protocol execution.
- **Non-Core Row and Column Reconstruction Phase:** The only other instance in the protocol where parties may update their shares is during the non-Core row and column reconstruction phase. Moreover, only parties outside $\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr}$ update their shares during this step. Observe that each honest $P_i \in \mathsf{Core}$ sends $\{(f_i^{(u,v)}(j), g_i^{(u,v)}(j))\}_{u=1,\ldots,L+4, v=1,\ldots,M}$ to each $P_j \in \mathcal{P} \setminus (\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr})$. During the verification, due to the correctness (Lemma 5.3) and non-repudiation (Lemma 5.5) of ICP, except with negligible probability, the dealer successfully reveals $\mathsf{F}^{(2,v)}(X,Y) = F^{(1,v)}(X,Y) + r_2 \cdot F^{(2,v)}(X,Y) + \ldots + r_2^{L+3} \cdot F^{(L+4,v)}(X,Y)$ such that $\mathsf{F}^{(2,v)}(i,j) = \sum_{u=1}^{L+4}(r_2)^{u-1} g_i^{(u,v)}(j), \mathsf{F}^{(2,v)}(j,i) = \sum_{u=1}^{L+4}(r_2)^{u-1} f_i^{(u,v)}(j)$ holds for each honest P_i. This guarantees that each honest $P_i \in \mathcal{S}_j$. Hence, except with negligible probability, the polynomials $f_j^{(u,v)}(Y)$ and $g_j^{(u,v)}(Y)$ interpolated by P_j are pairwise consistent with (at least) $2t+1 > d+1$ honest parties, and consequently are consistent with the dealer's bivariate polynomials.

Parties do not update their shares at any other step in the protocol. It is therefore established that all the parties hold shares that are consistent with the dealer's bivariate polynomials that embed the secrets. It remains to show that parties output these shares together with consistent authentication information.

Pairwise Tag Generation: Recall that parties compute their output either during Step 3d of tag share verification or Step 4e of tag share complaint resolution.

1. **Case 1 - Parties compute their output during Step 3d of tag share verification:** In this case, it must hold that either every party $P_i \notin \mathsf{Cr}$ broadcasted Tag_i or $\mathsf{flag}_2 = 1$. In the former case, it implies that each P_i has $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t + p$ and is able to compute $\tau_{ij}^{(u)}$ corresponding to each P_j. In the latter case, since $\mathsf{flag}_2 = 1$, it must hold that the protocol identified a set of at least $t - p + 1 > p$ corrupt parties Cr (Claim 6.3) in the previous iteration and restarted with their shares set to all-zero vectors. Moreover, since the honest parties always send correct their tag shares, $\mathcal{R}_{ij}$ of each honest P_i will include all the honest parties. Hence, in this case as well, it must hold that $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t + 1 + (t - p + 1) \geq 2t + p$ (as $p = \frac{t}{4} + 1$) ensuring that each $P_i \notin \mathsf{Cr}$ can compute its tag corresponding to each P_j.
2. **Case 2 - Parties compute their output during Step 4e of tag share complaint resolution:** Firstly , this implies that the protocol has not identified a set of corrupt parties $|\mathsf{Cr}| \geq t - p + 1$. Otherwise, it would restart with $\mathsf{flag}_2 = 1$ and parties would compute their output via Step 3d of tag share verification as discussed in the prior case, which is a contradiction. This implies that (at least) $3t + 1 - |\mathsf{Cr}| \geq 2t + p$ parties P_l are outside Cr and indeed reveal their tag shares $(\tau_{ij,l}^{(1)}, \ldots, \tau_{ij,l}^{(L+4)})$ to each $P_i \notin \mathsf{Cr}$, either during tag share computation (Step 2b) or during tag share complaint resolution (Step 4a). Hence, each $P_i \notin \mathsf{Cr}$ has $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t + p$ and is able to reconstruct $\tau_{ij}^{(u)}$ corresponding to each P_j.

The correctness of the tags computed can be easily established as follows. Consider a pair of parties (P_i, P_j). During this phase, each $P_l \notin \mathsf{Cr}$ sends its share of $\lceil \tau_{ij}^{(u)} \rfloor_{t+d}^{i}$ to each $P_i \notin \mathsf{Cr}$. If a party P_l's tag shares $\tau_{ij,l}^{(u)}$ sent to P_i are accepted by an (honest) P_i during reconstruction of $\tau_{ij}^{(u)}$, that is P_i includes

P_l in $\mathcal{R}_{ij}$, then the following must hold for each $u \in \{1, \ldots, L+4\}$:

$$\tau_{ij,l}^{(u)} = \mathbf{s}_l^{(u)} \cdot \boldsymbol{\mu}_{ji,l} + v_{ji,l}^{(u)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(u,k)}.$$

where $r_{ji,l}^{(u,k)}$ denotes P_l's share of $\lceil r_{ji}^{(u,k)} || k \rfloor_t^i$.

Since the dealer is not discarded, it must also hold that $\mathsf{F}^{(3,1)}(X,Y), \ldots, \mathsf{F}^{(3,M)}(X,Y)$ are received from $\mathcal{F}_{\mathsf{BC}}$ where dealer D is the sender and for each $v \in \{1, \ldots, M\}$, the following holds:

$$\mathsf{F}^{(3,v)} = F^{(1,v)}(X,Y) + r_3 \cdot F^{(2,v)}(X,Y) + \ldots + r_3^{L+3} \cdot F^{(L+4,v)}(X,Y).$$

This further implies that $\mathbf{s}_l = \mathbf{s}_l^{(1)} + r_3 \cdot \mathbf{s}_l^{(2)} + \ldots + r_3^{L+3} \cdot \mathbf{s}_l^{(L+4)}$ holds. Now consider the following linear combination of the tags:

$$\tau_{ij,l} = \tau_{ij,l}^{(1)} + r_3 \cdot \tau_{ij,l}^{(2)} + \ldots + r_3^{L+3} \cdot \tau_{ij,l}^{(L+4)}.$$

If $\tau_{ij,l}^{(u)} \neq \mathbf{s}_l^{(u)} \cdot \boldsymbol{\mu}_{ji,l} + v_{ji,l}^{(u)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(u,k)}$ for some u, then except with negligible probability, the following holds true:

$$\tau_{ij,l} \neq \mathbf{s}_l \cdot \boldsymbol{\mu}_{ji,l} + \left((v_{ji,l}^{(1)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(1,k)}) + r_3 \cdot (v_{ji,l}^{(2)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(2,k)}) + \ldots + r_3^{L+3} \cdot (v_{ji,l}^{(L+4)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(L+4,k)}) \right).$$

Now consider the twisted t-sharing:

$$[\beta_{ij,l}]_t^i = [\mathbf{s}_l]_0^i \cdot [\boldsymbol{\mu}_{ji}]_t^i + \left(\lceil v_{ji}^{(1)} \rfloor_t^i + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r_{ji}^{(1,k)} || k \rfloor_t^i \right) + r_3 \cdot \left(\lceil v_{ji}^{(2)} \rfloor_t^i + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r_{ji}^{(2,k)} || k \rfloor_t^i \right) + \ldots$$
$$+ r_3^{L+3} \cdot \left(\lceil v_{ji}^{(L+4)} \rfloor_t^i + \sum_{k=1}^{d} e_{k,l} \cdot \lceil r_{ji}^{(L+4,k)} || k \rfloor_t^i \right) + [\mathsf{mask}_{ji}^{(l)}]_t^i$$

It is easy to see that P_l's share in $[\mathbf{s}_l]_0^i \cdot [\boldsymbol{\mu}_{ji}]_t^i$ will be $\mathbf{s}_l \cdot \boldsymbol{\mu}_{ji,l}$. We also know that P_l's share in $[\mathsf{mask}_{ji}^{(l)}]_t^i$ will be 0, as $[\mathsf{mask}_{ji}^{(l)}]_t^i$ is generated using $\mathcal{F}_{\lceil \cdot \rfloor_t^i}^{\mathsf{R}}$ that sets the share of P_l to 0. It then turns out that the share of P_l in the rest of twisted t-sharing above will be the same as $(v_{ji,l}^{(1)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(1,k)}) + r_3 \cdot (v_{ji,l}^{(2)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(2,k)}) + \ldots + r_3^{L+3} \cdot (v_{ji,l}^{(L+4)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(L+4,k)})$. Consequently, the share of P_l in $[\beta_{ij,l}]_t^i$ will not be the same as $\tau_{ij,l}^{(1)} + r_3 \cdot \tau_{ij,l}^{(2)} + \ldots + r_3^{L+3} \cdot \tau_{ij,l}^{(L+4)}$. From the protocol steps, P_i will robustly reconstruct the entire sharing $[\beta_{ij,l}]_t^i$ and find that the share of P_l is not the same as $\tau_{ij,l}^{(1)} + r_3 \cdot \tau_{ij,l}^{(2)} + \ldots + r_3^{L+3} \cdot \tau_{ij,l}^{(L+4)}$. Hence, an (honest) P_i will include P_l in Dp_i and not accept its tag shares.

Then consider the tag-share complaint resolution phase. By using similar arguments as above, it can be shown that P_l's share in $[\gamma_{ij,l}]_t^i$ will not be the same as $\tau_{ij,l}^{(1)} + r_4 \cdot \tau_{ij,l}^{(2)} + \ldots + r_4^{L+3} \cdot \tau_{ij,l}^{(L+4)}$. Hence, except with a negligible probability, P_l will not get included in $\mathcal{R}_{ij}$ and its shares will not be used for reconstructing the sharing $\lceil \tau_{ij}^{(u)} \rfloor_{t+d}^i$ and compute $\tau_{ij}^{(u)} = \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$. However, this is a contradiction, and hence we have that $\tau_{ij,l}^{(u)} = \mathbf{s}_l^{(u)} \cdot \boldsymbol{\mu}_{ji,l} + v_{ji,l}^{(u)} + \sum_{k=1}^{d} e_{k,l} \cdot r_{ji,l}^{(u,k)}$ for each $u \in \{1, \ldots, L+4\}$. Finally, P_i reconstructs its tag $\tau_{ij}^{(u)}$ from the shares of parties for which the above condition holds. Hence, we are guaranteed that it reconstructs the sharing corresponding to $\lceil \tau_{ij}^{(u)} \rfloor_{t+d}^i = \lceil \mathbf{s}_i^{(u)} \rfloor_d^i \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor_t^i + \lceil v_{ji}^{(u)} \rfloor_t^i + \sum_{k=1}^{d} [[\mathbf{e}_k]]_d^i \cdot \lceil r_{ji}^{(u,k)} || k \rfloor_t^i$. It is easy to see that the polynomial corresponding to $\sum_{k=1}^{d} [[\mathbf{e}_k]]_d^i \cdot \lceil r_{ji}^{(u,k)} || k \rfloor_t^i$ evaluates to 0 at i due to the degenerate packed twisted sharing of $\mathbf{e}_1, \ldots, \mathbf{e}_d$. This ensures that $\tau_{ij}^{(u)}$ which is embedded at i in the underlying polynomial for $\lceil \tau_{ij}^{(u)} \rfloor_{t+d}^i$ is indeed $\mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$.

In conclusion, when the dealer is not discarded, except with negligible probability, for each party $P_i \notin \mathsf{Cr}$ we have that:

- P_i outputs $\mathbf{s}_i^{(u)}$ as its shares of the secrets. Moreover, from Claim 6.2, for each honest P_i, it holds that $P_i \notin \mathsf{Cr}$.

– Also, P_i outputs $(\mathsf{K}_{i1}^{(u)}, \ldots, \mathsf{K}_{in}^{(u)})$ as the MAC-keys, where each $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, v_{ij}^{(u)})$. Of these, $\boldsymbol{\mu}_{ij}$ is known to P_i at the onset of the protocol. Whereas, $v_{ij}^{(u)}$ for each $P_j \notin \mathsf{Cr}$ is robustly reconstructed from $\lceil v_{ij}^{(u)} \rfloor_t^j$ used during tag computation, and $v_{ij}^{(u)} = 0$ for each $P_j \in \mathsf{Cr}$.
– Finally, P_i outputs $(\tau_{i1}^{(u)}, \ldots, \tau_{in}^{(u)})$ as MAC-tags, where $\tau_{ij}^{(u)} = \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$ holds.

Whereas, except with negligible probability, for each $P_i \in \mathsf{Cr}$ it can be easily observed that:

– P_i has 0^{Mp} as its shares of the secrets that is exactly the same as that used by the dealer while choosing its polynomials for sharing the secrets.
– P_i has $(\mathsf{K}_{i1}^{(u)}, \ldots, \mathsf{K}_{in}^{(u)})$ as the MAC-keys, where each $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, v_{ij}^{(u)})$. Of these, $\boldsymbol{\mu}_{ij}$ is known to P_i at the onset of the protocol. Whereas, $v_{ij}^{(u)}$ for each $P_j \notin \mathsf{Cr}$ is robustly reconstructed from $\lceil v_{ij}^{(u)} \rfloor_t^j$ used during tag computation, and $v_{ij}^{(u)} = 0$ for each $P_j \in \mathsf{Cr}$.
– P_i has 0^n as MAC-tags, where $\tau_{ij}^{(u)} = \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$ holds since $\mathbf{s}_i^{(u)} = 0^n$ and $v_{ji}^{(u)} = 0^n$ is ensured by the protocol description.

This implies that except with a negligible probability, all (honest) parties output their respective information corresponding to ${}^{A}\langle \mathbb{S} \rangle_d$. □

We next show that even if D is *corrupt*, at the end of the protocol $\Pi_{A\langle\cdot\rangle}$, there exists some $\mathbb{S} \in \mathbb{F}^{LMp^2}$ known to D, such that except with a negligible probability, the honest parties output ${}^{A}\langle \mathbb{S} \rangle_d$.

Lemma 6.7 (Commitment). *In protocol $\Pi_{A\langle\cdot\rangle}$ if D is corrupt, then except with probability $2^{-\Omega(\kappa)}$, there exists some $\mathbb{S} \in \mathbb{F}^{LMp^2}$ known to D, such that at the end of the protocol, all (honest) parties output their respective information corresponding to ${}^{A}\langle \mathbb{S} \rangle_d$.*

Proof. As seen in the proof of Lemma 6.6, the protocol can terminate in only two ways, one where the parties discard the dealer and output a default sharing and the other where parties output shares obtained during the protocol execution. It is easy to observe that parties discard the dealer based on publicly broadcasted information. Hence we have the guarantee that if there exists some honest party that discards the dealer, then all the honest parties would have discarded the dealer. In this case, the lemma holds trivially. We now consider the case when the dealer is not discarded and analyze the information held by the (honest) parties in each phase of the protocol.

– **Sharing Phase:** Since the dealer is not discarded, it must hold that either in the first or the second run of the protocol, the dealer broadcasted a Core such that $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + p$ and OK_i is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender for each $P_i \in \mathsf{Core}$. This further implies that there are at least $|\mathsf{Core} \cup \mathsf{Dp}| - t \geq t + p \geq d + 1$ honest parties in $\mathsf{Core} \cup \mathsf{Dp}$, where the honest parties in Core have received degree-d column polynomials $g_i^{(u,v)}(Y)$ from D and gave the signatures on the points on their column-polynomials to D. On the other hand, the column-polynomials of the honest parties in Dp are supposed to be 0. For each $u \in \{1, \ldots, L\}$ and $v \in \{1, \ldots, M\}$, let $F^{(u,v)}(X, Y)$ be the degree-(d, d) bivariate polynomial, defined by the column-polynomials of the first $d+1$ honest parties in $\mathsf{Core} \cup \mathsf{Dp}$, with the column-polynomials of the honest parties in Dp supposedly being 0. Moreover, let $\mathbf{s}^{(u,(v-1)\cdot p)}, \ldots, \mathbf{s}^{(u,v\cdot p)}$ be the vectors, each consisting of p elements, which are embedded in $F^{(u,v)}(X, Y)$ at $F^{(u,v)}(X, -1), \ldots, F^{(u,v)}(X, -p)$ respectively. Furthermore, let $\mathbb{S} \stackrel{\mathsf{def}}{=} (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)})$, where $\mathbf{S}^{(u)} \stackrel{\mathsf{def}}{=} (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,Mp)})$.
– **Verification Phase:** Again, since the dealer is not discarded, it must hold that $\mathsf{F}^{(1,1)}(X, Y), \ldots, \mathsf{F}^{(1,M)}(X, Y)$ are received from $\mathcal{F}_{\mathsf{BC}}$ where dealer D is the sender and that D successfully revealed $\mathsf{Sig}(P_i \to D, \mathsf{G}_i^{(1,1)}, \ldots, \mathsf{G}_i^{(1,M)})$ publicly, for every $P_i \in \mathsf{Core}$. By the correctness (Lemma 5.3) and unforgeability property of ICP (Lemma 5.6), except with negligible probability, it must hold that $\mathsf{G}_i^{(1,v)}(y) = g_i^{(1,v)}(Y) + r_1 \cdot g_i^{(2,v)}(i, Y) + \ldots + r_1^{L+3} \cdot g_i^{(L+4,v)}(i, Y)$ for every honest $P_i \in \mathsf{Core}$ where $g_i^{(u,v)}(Y)$ is held by P_i as its column-polynomial from the previous phase. Moreover, for every honest $P_i \in \mathsf{Dp}$, it must hold that $\mathsf{F}^{(1,v)}(X, i) = \mathbf{0}$ and $\mathsf{F}^{(1,v)}(i, Y) = \mathbf{0}$. Further, since all the honest parties in Core hold row and column-polynomials consistent with the same linear combination of degree-d bivariate polynomials $\mathsf{F}^{(1,1)}(X, Y), \ldots, \mathsf{F}^{(1,M)}(X, Y)$, except with negligible probability, it implies that they all hold polynomials $f_i^{(u,v)}(Y), g_i^{(u,v)}(Y)$ consistent with the defined bivariate polynomial $F^{(u,v)}(X, Y)$.

– **Non-Core Row and Column Reconstruction Phase:** Each honest $P_i \in \mathsf{Core}$ sends its common points $\{(f_i^{(u,v)}(j), g_i^{(u,v)}(j))\}_{u=1,\ldots,L+4,v=1,\ldots,M}$ to each $P_j \in \mathcal{P} \setminus (\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr})$. Moreover, due to the same argument as above, except with negligible probability, $\mathsf{F}^{(2,v)}(i,j) = \sum_{u=1}^{L+4}(r_2)^{u-1} g_i^{(u,v)}(j)$ and $\mathsf{F}^{(2,v)}(j,i) = \sum_{u=1}^{L+4}(r_2)^{u-1} f_i^{(u,v)}(j)$ holds for each honest $P_i \in \mathsf{Core}$. Hence, each (honest) $P_j \notin \mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr}$ has $P_i \in \mathcal{S}_j$. Since P_j interpolates $f_j^{(u,v)}(Y)$ and $g_j^{(u,v)}(Y)$ from points obtained from parties in $\mathcal{S}_j$ and parties in Dp and due to the guarantee that $|\mathsf{Core} \cup \mathsf{Dp}| - t \geq t + p$, we have that P_j's updated row and column-polynomials are consistent with the polynomials of the honest parties in $\mathsf{Core} \cup \mathsf{Dp}$. Hence, by the end of this phase, if the dealer is not discarded, all honest parties hold row and column-polynomials that are consistent and define unique degree-(d,d) bivariate polynomials, with the rows and column-polynomials of the parties in Dp being 0.
– **Pairwise Tag Generation:** It remains to show that parties output shares together with consistent authentication information corresponding to ${}^{A}\langle \mathbb{S} \rangle_d$. Recall that parties compute their output either during Step 3d of tag share verification or Step 4e of tag share complaint resolution.

1. **Case 1 - Parties compute their output during Step 3d of tag share verification:** In this case, it must hold that either every party $P_i \notin \mathsf{Cr}$ broadcasted Tag_i or $\mathsf{flag}_2 = 1$. In the former case, it implies that each P_i has $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t + p$ and is able to compute $\tau_{ij}^{(u)}$ corresponding to each P_j. In the latter case, since $\mathsf{flag}_2 = 1$, it must hold that the protocol identified a set of at least $t - p + 1 > p$ parties and included them in Cr in the previous iteration and restarted with their shares set to all-zero vectors. Further, since by this phase, all the honest parties hold their shares and the dealer is not discarded, by an argument similar to Claim 6.3, except with negligible probability, an honest party is not added to Cr. Moreover, since the honest parties always send correct their tag shares, $\mathcal{R}_{ij}$ of each honest P_i will include all the honest parties. Hence, in this case as well, it must hold that $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t + 1 + (t - p + 1) \geq 2t + p$ ensuring that each $P_i \notin \mathsf{Cr}$ can compute its tag corresponding to each P_j.
2. **Case 2 - Parties compute their output during Step 4e of tag share complaint resolution:** Firstly , this implies that the protocol has not identified a set of corrupt parties $|\mathsf{Cr}| \geq t - p + 1$. Otherwise, it would restart with $\mathsf{flag}_2 = 1$ and parties would compute their output via Step 3d of tag share verification as discussed in the prior case, which is a contradiction. This implies that (at least) $3t + 1 - |\mathsf{Cr}| \geq 2t + p$ parties P_l are outside Cr and indeed reveal their correct tag shares $(\tau_{ij,l}^{(1)}, \ldots, \tau_{ij,l}^{(L+4)})$ to each $P_i \notin \mathsf{Cr}$, either during tag share computation (Step 2b) or during tag share complaint resolution (Step 4a). Hence, each $P_i \notin \mathsf{Cr}$ has $|\mathcal{R}_{ij} \cup \mathsf{Cr}| \geq 2t + p$ and is able to reconstruct $\tau_{ij}^{(u)}$ corresponding to each P_j.

The correctness of tags can be established similarly as described in Lemma 6.6.
In conclusion, when the dealer is not discarded, except with negligible probability, for each party $P_i \notin \mathsf{Cr}$ we have that:

- P_i outputs $\mathbf{s}_i^{(u)}$ as its shares of the secrets held by the dealer.
- Also, P_i outputs $(\mathsf{K}_{i1}^{(u)}, \ldots, \mathsf{K}_{in}^{(u)})$ as the MAC-keys, where each $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, v_{ij}^{(u)})$. Of these, $\boldsymbol{\mu}_{ij}$ is known to P_i at the onset of the protocol. Whereas, $v_{ij}^{(u)}$ for each $P_j \notin \mathsf{Cr}$ is robustly reconstructed from $\lceil v_{ij}^{(u)} \rfloor_t^j$ used during tag computation, and $v_{ij}^{(u)} = 0$ for each $P_j \in \mathsf{Cr}$.
- Finally, P_i outputs $(\tau_{i1}^{(u)}, \ldots, \tau_{in}^{(u)})$ as MAC-tags, where $\tau_{ij}^{(u)} = \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$ holds.

Whereas, except with negligible probability, for each $P_i \in \mathsf{Cr}$ it can be easily observed that:

- P_i has 0^{Mp} as its shares of the secrets that is exactly the same as that used by the dealer while choosing its polynomials for sharing the secrets.
- Also, P_i has $(\mathsf{K}_{i1}^{(u)}, \ldots, \mathsf{K}_{in}^{(u)})$ as the MAC-keys, where each $\mathsf{K}_{ij}^{(u)} = (\boldsymbol{\mu}_{ij}, v_{ij}^{(u)})$. Of these, $\boldsymbol{\mu}_{ij}$ is known to P_i at the onset of the protocol. Whereas, $v_{ij}^{(u)}$ for each $P_j \notin \mathsf{Cr}$ is robustly reconstructed from $\lceil v_{ij}^{(u)} \rfloor_t^j$ used during tag computation, and $v_{ij}^{(u)} = 0$ for each $P_j \in \mathsf{Cr}$.
- P_i has 0^n as MAC-tags, where $\tau_{ij}^{(u)} = \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$ holds since $\mathbf{s}_i^{(u)} = 0^n$ and $v_{ji}^{(u)} = 0^n$ is ensured by the protocol description.

□

We next determine the round and communication complexity of the protocol $\Pi_{{}^{A}\langle\cdot\rangle}$.

Lemma 6.8. *Protocol $\Pi_{A\langle\cdot\rangle}$ requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^7\kappa)$ bits, apart from 15 calls to $\mathcal{F}_{\mathsf{Rand}}$.*

Proof. We first do the analysis assuming that the parties do not restart the protocol.

During the sharing phase, D distributes $\mathcal{O}(LM)$ rows and column polynomials to each party, where each polynomial is a d-degree polynomial and hence consists of $\mathcal{O}(n)$ field elements as coefficients. This requires one round and incurs a total communication of $\mathcal{O}(n^2LM)$ field elements. Every party upon receiving its rows and column polynomials may broadcast an OK message, so this step requires a communication of $n \times \mathcal{BC}(1)$ bits. Additionally each P_i may sign its column polynomials for D, for which $\mathcal{O}(L)$ instances of Gen and Auth protocols are executed, where in each instance, P_i as a signer has M vectors for signing, each consisting of n field elements. From Lemma 5.7, by substituting $l = n$, this step requires a total communication of $\mathcal{O}(n^2LM)$ field elements plus $\mathcal{BC}(n^2L)$ field elements.[18] Finally, D may broadcast the identity of a Core set, which requires a communication of $\mathcal{BC}(n)$ bits. Now securely realizing the calls to $\mathcal{F}_{\mathsf{BC}}$ through the broadcast protocol of [9], the sharing phase will require a constant expected number of rounds and incurs a communication of $\mathcal{O}((n^2LM + n^3L)\kappa + n^3\log^2 n)$ bits, since each field element is represented by κ bits.

During verification phase, the parties call $\mathcal{F}_{\mathsf{Rand}}$ to generate the random value r_1. The dealer D broadcasts M bivariate polynomials, each of degree (d, d). This requires a communication of $\mathcal{BC}(n^2M)$ field elements. Additionally, D may invoke n instances of Rev to publicly reveal IC-signatures on M vectors, each consisting of n field elements. Form Lemma 5.7, by substituting $l = n$, this step requires two rounds and incurs a communication of $\mathcal{BC}(n^2M)$ field elements.[19] By securely realizing the calls to $\mathcal{F}_{\mathsf{BC}}$ through the broadcast protocol of [9], the verification phase will require a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^3M\kappa + n^3\log^2 n)$ bits.

Next consider the non-Core row and column reconstruction phase. Here every party in Core may send the supposedly common points on their row and column polynomials to the parties outside Core. This requires one round and a total communication of $\mathcal{O}(n^2LM)$ field elements. The parties then call $\mathcal{F}_{\mathsf{Rand}}$ to generate the random value r_2. This is followed by similar steps as in the verification phase, where D may broadcast M bivariate polynomials and publicly reveals IC-signatures of up to n parties on M vectors, each consisting of n field elements. So overall this phase will require a constant expected number of rounds and incur a communication of $\mathcal{O}((n^2LM + n^3M)\kappa + n^3\log^2 n)$ bits.

The next phase is the pairwise tag generation phase. As part of the setup, for every pair of parties (P_i, P_j), parties have to generate twisted t-sharing of $\mathcal{O}(nL)$ random values. So there will be total $\mathcal{O}(n^2)$ calls to $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor^i_t}$, where in each call we generate $\mathcal{O}(nL)$ random t-sharings. By securely realizing the calls to $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor^i_t}$, this will require a constant expected number of rounds and a total communication of $\mathcal{O}(n^4L\kappa)$ bits. Additionally, for every triplet of parties (P_i, P_j, P_l), the parties generate twisted t-sharing of two random masks. For this, the parties make $\mathcal{O}(n^2)$ calls to $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor^i_t}$, where in each call $\mathcal{O}(n)$ random t-sharings are generated. By securely realizing the calls to $\mathcal{F}^{\mathsf{R}}_{\lceil\cdot\rfloor^i_t}$, this requires a constant expected number of rounds and a total communication of $\mathcal{O}(n^4\kappa + n^6\log n)$ bits. Next, for every triplet of parties (P_i, P_j, P_l), party P_l has to send $\mathcal{O}(L)$ tag-shares to P_i. This incurs a total communication of $\mathcal{O}(n^3L\kappa)$ bits. To verify the tag-shares, the parties first call $\mathcal{F}_{\mathsf{Rand}}$ to generate a random value r_3. This is followed by D making public M bivariate polynomials and IC-signatures of up to n parties on M vectors, each consisting of n field elements. This requires a constant expected number of rounds and incurs a communication of $\mathcal{O}((n^2LM + n^3M)\kappa + n^3\log^2 n)$ bits. Then for every triplet of parties (P_i, P_j, P_l), party P_i reconstructs a twisted t-sharing. This requires one round and incurs a communication of $\mathcal{O}(n^4\kappa)$ bits. Every party may then broadcast the list of disputed parties from which it supposedly received incorrect tag-shares. This requires a constant expected number of rounds and a communication of $\mathcal{O}(n^4\log^2 n)$ bits. The final step is the tag-share complaint resolution phase. Here in the worst case, for up to $\mathcal{O}(n^2)$ pairs of parties (P_i, P_l), party P_l may broadcast $\mathcal{O}(nL)$ tag-shares. This requires a constant expected number of rounds and a communication of $\mathcal{O}(n^4L\kappa)$ bits. The parties then call $\mathcal{F}_{\mathsf{Rand}}$ to generate a random value r_4. This is followed by D making making public M bivariate polynomials and IC-signatures of up to n parties on M vectors, each consisting of n field elements. This requires a constant expected

[18] Since in all the nL instances of Gen and Auth, the dealer D is playing the role of I, it can broadcast all the data across all the nL ICP instances through a single call to $\mathcal{F}_{\mathsf{BC}}$.

[19] Since D is playing the role of I in all the Rev, it can broadcast all the data across these Rev instances through a single call to $\mathcal{F}_{\mathsf{BC}}$.

number of rounds and a communication of $\mathcal{O}((n^2LM + n^3M)\kappa + n^3 \log^2 n)$ bits. Finally, in the worst case, for up to $\mathcal{O}(n^2)$ pairs of parties (P_i, P_l), there will be n number of twisted t-sharings which will be publicly reconstructed. This will require one round and a communication of $\mathcal{O}(n^5\kappa)$ bits. So overall the tag-generation phase will require a constant expected number of rounds and incur a communication of $\mathcal{O}((n^2LM + n^3M + n^4L)\kappa + n^3 \log^2 n)$ bits.

Hence, if the protocol does not restart, then it requires a constant expected number of rounds and incurs a communication of $\mathcal{O}((n^2LM + n^3M + n^4L)\kappa + n^3 \log^2 n)$ bits. Additionally, there will be four calls to $\mathcal{F}_{\mathsf{Rand}}$ to generate the random values r_1, r_2, r_3 and r_4 which are used by D to make public its linear combinations of bivariate polynomials. Additionally, for all the instances of $\mathcal{F}_{\lceil \cdot \rfloor_t^i}$, we can use a single call to $\mathcal{F}_{\mathsf{Rand}}$ to generate a single random value, which can be used to check the correctness of all twisted-sharings generated by various parties (as dealer) during the setup stage of the tag-generation phase.

Next consider the case when the parties restart the protocol. As shown in Lemma 6.5, the parties need to rerun the protocol at most 3 times to get an output. Since each run requires a constant expected number of rounds, it follows that protocol $\Pi_{A\langle\cdot\rangle}$ will overall require a constant expected number of rounds to generate output for the parties. Moreover, $\mathcal{O}((n^2LM + n^3M + n^4L)\kappa + n^3 \log^2 n)$ bits will be communicated by the (honest) parties, apart from 15 calls to $\mathcal{F}_{\mathsf{Rand}}$. □

Recall that in the protocol $\Pi_{A\langle\cdot\rangle}$, dealer D has LMp^2 elements from $\mathbb{F}$ (and hence $\Theta(LMp^2\kappa)$ bits) to secret-share. If we set $L = n^3$ and $M = n^2$, then D's input consists of $\Theta(n^7)$ field elements (since $p = \Theta(n)$), incurring a communication of $\mathcal{O}(n^7)$ field elements, apart from at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$. So ignoring the calls to $\mathcal{F}_{\mathsf{Rand}}$, the communication overhead of the protocol will be a constant.

If D has multiple groups of n^5p^2 elements to be ${}^{A}\langle\cdot\rangle_d$-shared with a constant overhead, then it can run an instance of $\Pi_{A\langle\cdot\rangle}$ for each group. The calls to $\mathcal{F}_{\mathsf{Rand}}$ can be reused to generate the random challenges for D across all the instances of $\Pi_{A\langle\cdot\rangle}$ during various phases. For instance, during the verification phase of all the instances of $\Pi_{A\langle\cdot\rangle}$, a single call to $\mathcal{F}_{\mathsf{Rand}}$ is sufficient to generate the random challenge r_1, which has to be used by D to compute its linear combinations of bivariate polynomals across all the instances of $\Pi_{A\langle\cdot\rangle}$. This will ensure that the total number of calls to $\mathcal{F}_{\mathsf{Rand}}$ is upper bounded by 15, irrespective of the number of instances of $\Pi_{A\langle\cdot\rangle}$ run by D. Based on the discussion till now, we can state the following theorem for $\Pi_{A\langle\cdot\rangle}$.

Theorem 6.9. *Let $D \in \mathcal{P}$ be a designated party with inputs $\mathbb{S}^{(1)}, \ldots, \mathbb{S}^{(N)} \in \mathbb{F}^{n^5p^2}$, where $N \geq 1$ and known publicly. Then there exists a protocol in the $\mathcal{F}_{\mathsf{Rand}}$-hybrid model, which generates key-consistent sharings ${}^{A}\langle\mathbb{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{S}^{(N)}\rangle_d$, such that the view of the adversary remains independent of $\mathbb{S}^{(1)}, \ldots, \mathbb{S}^{(N)}$ if D is honest. Moreover, even if D is corrupt, there exist some $\mathbb{S}^{(1)}, \ldots, \mathbb{S}^{(N)} \in \mathbb{F}^{n^5p^2}$ known to D, such that the protocol generates ${}^{A}\langle\mathbb{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{S}^{(N)}\rangle_d$. Here $p = \frac{t}{4} + 1$ and $d = t + p - 1$.*

The protocol requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^7N\kappa)$ bits, apart from at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$. Asymptotically, for sufficiently large N, the communication overhead of the protocol is $\mathcal{O}(1)$.

Looking ahead, we will come across scenarios where multiple parties have to act as a dealer and invoke instances of $\Pi_{A\langle\cdot\rangle}$ at the same "time". For efficiency, the calls to $\mathcal{F}_{\mathsf{Rand}}$ can be "clubbed" across all the instances of $\Pi_{A\langle\cdot\rangle}$ invoked by different dealers. This will be possible because the underlying network is synchronous and all the instances of $\Pi_{A\langle\cdot\rangle}$ (even by different dealers) will be executed at the same time. Hence the maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ will be still 15.

6.1 Extending Protocol $\Pi_{A\langle\cdot\rangle}$ for Degrees $d + p - 1$ and $p - 1$

Looking ahead, in our MPC protocol we will come across situations where a dealer has to generate ${}^{A}\langle\cdot\rangle_\ell$-sharing of its inputs where $\ell \in \{p - 1, d, d + p - 1\}$. While we already discussed how protocol $\Pi_{A\langle\cdot\rangle}$ can handle the case when $\ell = d$, we now show the modifications needed to handle the remaining two cases. We stress that the communication complexity (and the number of calls to $\mathcal{F}_{\mathsf{Rand}}$) remains the same as earlier.

Extending protocol $\Pi_{A\langle\cdot\rangle}$ for degree $d + p - 1$. Let D have an input $\mathbb{S} \in \mathbb{F}^{LMp^2}$ such that $\mathbb{S} = \{s^{(u,v,w)}\}_{u\in\{1,\ldots,L\},v\in\{1,\ldots,Mp\},w\in\{1,\ldots,p\}}$. Let the elements of $\mathbb{S}$ be structured into L batches $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)}$ of vectors, where each batch $\mathbf{S}^{(u)}$ consists of Mp vectors $\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,Mp)}$ and where each vector $\mathbf{s}^{(u,v)}$

consists of the p elements $(s^{(u,v,1)}, \ldots, s^{(u,v,p)})$. The goal is to now generate ${}^{A}\langle \mathbf{S}^{(1)} \rangle_{d+p-1}, \ldots, {}^{A}\langle \mathbf{S}^{(L)} \rangle_{d+p-1}$, hence resulting in ${}^{A}\langle \mathbb{S} \rangle_{d+p-1}$.

- **Sharing phase**: In this phase, the dealer as before shares its inputs, but now the secrets are embedded in polynomials of degree-$(d+p-1)$ instead of d. Namely, the elements of each vector $\mathbf{s}^{(u,v)}$ are embedded in a random $(d+p-1)$-degree univariate polynomial (at $-1, \ldots, -p$). And the sharing-polynomials for the vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)}, \ldots, \mathbf{s}^{(u,v\cdot p)}$ are embedded in a random $(d+p-1, d+p-1)$-degree bivariate polynomial $F^{(u,v)}(X,Y)$ at $Y = -1, \ldots, -p$. We now require D to find a Core set such that $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + 2p - 1 = d + p + t$. An *honest* dealer may *not* be able to get such a Core set in the first attempt, in which case D gets into a dispute with at least $t/2+1$ parties (since $p = \frac{t}{4} + 1$). Moreover, these parties are bound to be corrupt when the dealer is honest, and hence the subsequent instance of the sharing phase after a restart is guaranteed to succeed.
- **Verification phase**: The verification phase proceeds similarly to before, with parties verifying that the rows and columns of the parties in Core are consistent with each other, rows and columns of the parties in $\mathsf{Dp} \cup \mathsf{Cr}$ are 0 and that all these polynomials lie on $(d+p-1, d+p-1)$-degree bivariate polynomials.
- **Non-Core row and column reconstruction phase**: Note that once the "consistency" of the polynomials of the (honest) parties in $\mathsf{Core} \cup \mathsf{Dp}$ is confirmed, similar to the case of $\ell = d$, presence of (at least) $d+p$ honest parties in $\mathsf{Core} \cup \mathsf{Dp}$ ensures that parties outside Core can compute their rows and column polynomials lying on dealer's committed bivariate polynomials successfully.
- **Pairwise tag-generation phase**: This phase also proceeds similar to the case of $\ell = d$ with minor changes. Specifically, during the computation of the tag $\tau_{ij}^{(u)} \stackrel{\mathsf{def}}{=} \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$, parties will hold $\lceil \tau_{ij}^{(u)} \rfloor_{2d}^{i}$. This is because $\mathbf{s}_i^{(u)}$ will be degree-$(d+p-1)$-twisted-shared and $\boldsymbol{\mu}_{ji}$ will be degree-t-twisted-shared and hence $[\mathbf{s}_i^{(u)}]_{d+p-1}^{i} \cdot [\boldsymbol{\mu}_{ji}]_t^i = [\mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji}]_{2d}^{i}$. To randomize the degree-$2d$ sharing of $\tau_{ij}^{(u)}$, parties now need to generate degree-t twisted-sharing of $2d - t = d + p - 1$ random values. Moreover, the publicly known vectors used in tag computation will now be $\mathbf{e}_1, \ldots, \mathbf{e}_{d+p-1}$, each containing $d+p$ elements from $\mathbb{F}$, where $\mathbf{e}_1 = (0,1,0,0,\ldots,0), \mathbf{e}_2 = (0,0,1,0,\ldots,0), \ldots \mathbf{e}_{d+p-1} = (0,\ldots,0,1)$. If the tag computation fails for some party, then the public resolution will identify at least $t/2+1$ corrupt parties, and similar to the previous case, the subsequent run of the protocol is guaranteed to succeed.

Extending protocol $\Pi_{A\langle\cdot\rangle}$ for degree $p-1$ (sharing non-private values). During our MPC protocol, we will come across situations where there exists a dealer $D \in \mathcal{P}$ with input $\mathbb{S} \in \mathbb{F}^{LMp^2}$, such that $\mathbb{S}$ needs to be learned publicly (hence the privacy of $\mathbb{S}$ is *not* needed). To maintain $\mathcal{O}(1)$ communication overhead, D would like to generate ${}^{A}\langle \mathbb{S} \rangle_{\frac{t}{4}}$, where the elements of $\mathbb{S}$ are structured as earlier. That is, $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)})$, where $\mathbf{S}^{(u)} = \{\mathbf{s}^{(u,v)}\}_{v \in \{1,\ldots,Mp\}}$ and where each vector $\mathbf{s}^{(u,v)} = \{s^{(u,v,w)}\}_{w \in \{1,\ldots,p\}}$. To achieve the above goal, D embeds each vector $\mathbf{s}^{(u,v)}$ in a $p-1$-degree polynomial. Note that these polynomials are no longer randomly chosen, since each $\mathbf{s}^{(u,v)}$ has p elements. The sharing-polynomials are further embedded in $(p-1, p-1)$-degree bivariate polynomials. These bivariate polynomials are *no* longer randomly chosen. This is because each bivariate polynomial $F^{(u,v)}(X,Y)$ embeds p^2 elements from the vectors $\{\mathbf{s}^{(u,(v-1)\cdot p)+1)}, \ldots, \mathbf{s}^{(u,v\cdot p)}\}$.

Before proceeding, we observe that since each vector $\mathbf{s}^{(u,v)}$ is $(p-1)$-packed-secret-shared, we could avoid the tag-generation phase of the protocol $\Pi_{A\langle\cdot\rangle}$ when $\ell = p-1$. This is because the MAC-tags are needed only when the degree of sharing is *more* than t for robust reconstruction. But with $n = 3t+1$, we can always reconstruct a $(p-1)$-degree polynomial robustly using RS error-correction (since $p = \frac{t}{4}+1$). However, we generate the authentication tags following the steps of the protocol $\Pi_{A\langle\cdot\rangle}$ even for the case $\ell = p-1$ to maintain a generic protocol. The changes in the protocol to handle $(p-1)$-degree polynomials are described below.

- **Sharing phase**: In this phase, D picks $(p-1, p-1)$-degree bivariate polynomials as described above. We now require D to find a Core set such that $|\mathsf{Core}| \geq t + p$. Note that *unlike* the case where $\ell = d$ or $\ell = d + p - 1$, an *honest* D will certainly be able to identify such a Core in the first run of the protocol, since we have at least $2t+1$ honest parties. Hence we *do not* have any reruns for this phase and the parties *need not* maintain any Dp set.
- **Verification phase**: This phase proceeds similarly as earlier, with parties verifying that the rows and columns of the (honest) parties in Core lying on consistent $(p-1, p-1)$-degree bivariate polynomials.

– **Non-Core row and column reconstruction phase**: Once the "consistency" of the row and column polynomials of the (honest) parties in Core is confirmed, similar to the prior cases, presence of (at least) p honest parties in Core ensures that (honest) parties outside Core can compute their corresponding row and column polynomials successfully.
– **Pairwise tag-generation phase**: As mentioned earlier, we *do not* require MAC-tags to reconstruct values shared via $(p-1)$-degree polynomials, however, we follow the protocol steps described earlier (with minor changes) to maintain the generality of the protocol. Specifically, during the computation of the MAC-tag $\tau_{ij}^{(u)} \stackrel{\mathsf{def}}{=} \mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji} + v_{ji}^{(u)}$, the value $\tau_{ij}^{(u)}$ will be secret-shared via $t+p-1=d$-degree polynomial, that is, parties will hold $\lceil \tau_{ij}^{(u)} \rfloor_d^i$. This is because $\mathbf{s}_i^{(u)}$ will be now $(p-1)$-degree twisted-shared and hence $\lceil \mathbf{s}_i^{(u)} \rfloor_{p-1}^i \cdot \lceil \boldsymbol{\mu}_{ji} \rfloor_t^i = [\mathbf{s}_i^{(u)} \cdot \boldsymbol{\mu}_{ji}]_d^i$. To randomize the degree-d sharing of $\tau_{ij}^{(u)}$, parties now need to generate degree-t twisted-sharing of $p-1$ random values. Moreover, the publicly known vectors used in tag computation will now be $\mathbf{e}_1, \ldots, \mathbf{e}_{p-1}$ are *publicly known* vectors, each containing p elements from $\mathbb{F}$, where $\mathbf{e}_1 = (0,1,0,0,\ldots,0), \mathbf{e}_2 = (0,0,1,0,\ldots,0), \ldots \mathbf{e}_{p-1} = (0,\ldots,0,1)$. Finally, we note that due to the presence of (at least) $2t+1$ honest parties, the tag computation phase always succeeds. Hence, this phase does not require any re-executions.

As mentioned earlier, we now describe the simulation steps for $\Pi_{\mathcal{A}\langle\cdot\rangle}$ which will be used when the protocol is used as a primitive.

Simulator 6.10: Simulator for the Protocol $\Pi_{\mathcal{A}\langle\cdot\rangle}$

We assume that the simulator has access to the setup information (this is guaranteed while simulating the outer protocol Π_{Prep} since the simulator itself emulates the role of the setup functionality $\mathcal{F}_{\mathsf{MacKeys}}$, Similarly, it is ensured while simulating the outer protocol Π_{ckteval} since the simulator itself emulates the functionality $\mathcal{F}_{\mathsf{PREP}}$ which establishes the setup information.

– **Case 1: Honest Dealer**

- On behalf of the honest parties, initialize $\mathsf{Dp}, \mathsf{Cr} = \phi$.
- Select $L+4$ random $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L+4)} \in \mathbb{F}^{Mp^2}$ random two-dimensional vectors on behalf of an honest dealer.
- Choose $(L+4)M$ bivariate polynomials $F^{(u,v)}(X,Y)$ over $\mathbb{F}$ for $u = 1, \ldots, L+4$ and $v = 1, \ldots, M$, where each is a (d,d)-degree bivariate polynomial over $\mathbb{F}$ which embeds the secrets from $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L+4)}$ as in the protocol specification.
- For every $P_k \notin \mathsf{Dp} \cup \mathsf{Cr}$ that is under the control of $\mathcal{A}$, let $f_k^{(u,v)}(X) \stackrel{\mathsf{def}}{=} F^{(u,v)}(X,k)$ and $g_k^{(u,v)}(Y) \stackrel{\mathsf{def}}{=} F^{(u,v)}(k,Y)$. On behalf of the dealer, send the polynomials $\{f_k^{(u,v)}(X), g_k^{(u,v)}(Y)\}_{u=1,\ldots,L+4,v=1,\ldots,M}$ to $\mathcal{A}$.
- On behalf of each honest party P_i not under the control of $\mathcal{A}$, emulate $\mathcal{F}_{\mathsf{BC}}$ and send OK_i to $\mathcal{A}$.
- For some P_k controlled by $\mathcal{A}$, if OK_k is received from $\mathcal{A}$, then emulate $\mathcal{F}_{\mathsf{BC}}$ and send OK_k to all. Additionally, simulate the Gen algorithm of ICP if $\mathcal{A}$ gives $\mathsf{Sig}(P_k \to D, \mathbf{g}_k^{(u,1)}, \ldots, \mathbf{g}_k^{(u,M)})$ for $u = 1, \ldots, L+4$ corresponding to some P_k controlled by it by receiving the corresponding values on behalf of the honest dealer and all the honest parties.
- On behalf of the dealer, include each honest P_i not under the control of $\mathcal{A}$ in the set Core. For each P_k under the control of $\mathcal{A}$ that sent the signature, simulate the Auth algorithm of ICP. For each P_k under the control of $\mathcal{A}$, if $\mathcal{A}$ sent OK_k and the signature of P_k is received in the simulation of ICP, then include P_k in Core.
- If $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t+p$, on behalf of the dealer assume Core to be sent to $\mathcal{F}_{\mathsf{BC}}$, emulate $\mathcal{F}_{\mathsf{BC}}$ and send Core to the $\mathcal{A}$. Otherwise, set $\mathsf{Dp} = \mathcal{P} \setminus \mathsf{Core}$. On behalf of the dealer assume Dp to be sent to $\mathcal{F}_{\mathsf{BC}}$, emulate $\mathcal{F}_{\mathsf{BC}}$ and send Dp to the $\mathcal{A}$.
- On behalf of the honest parties, set flag_1 as an honest party would, and if required, restart the simulation steps from the beginning.

- Emulate $\mathcal{F}_{\mathsf{Rand}}$ by receiving Rand from $\mathcal{A}$ and sending r_1 to $\mathcal{A}$.
- Perform the computation as per $\mathsf{Ver}(r_1)$ on behalf of the honest dealer. Emulate $\mathcal{F}_{\mathsf{BC}}$ by sending $(\mathsf{F}^{(1,1)}(X,Y), \ldots, \mathsf{F}^{(1,M)}(X,Y))$ to $\mathcal{A}$, and simulate the Rev algorithm of ICP for each party in Core.

- On behalf of each honest party $P_i \in \mathsf{Core}$, send the points $\{(f_i^{(u,v)}(k), g_i^{(u,v)}(k))\}_{u=1,\ldots,L+4,v=1,\ldots,M}$ to $\mathcal{A}$ for each $P_k \in \mathcal{P} \setminus (\mathsf{Core} \cup \mathsf{Dp} \cup \mathsf{Cr})$ under the control of $\mathcal{A}$.

- Receive Rand from the $\mathcal{A}$ and emulate $\mathcal{F}_{\mathsf{Rand}}$ by sending a random r_2 to all.
- Perform the computation as per $\mathsf{Ver}(r_2)$ on behalf of the honest dealer. Emulate $\mathcal{F}_{\mathsf{BC}}$ by sending $(\mathsf{F}^{(2,1)}(X,Y),\ldots,\mathsf{F}^{(2,M)}(X,Y))$ to $\mathcal{A}$, and simulate the Rev algorithm of ICP for each party in Core.

- For every ordered pair of parties (P_i,P_j) such that $P_i \notin \mathsf{Cr}$, emulate $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ and communicate the appropriate shares to each party P_k under the control of $\mathcal{A}$.
- On behalf of each honest party P_l not under the control of $\mathcal{A}$, compute the sharing of the MAC-tag for every ordered pair (P_i,P_j) where $P_i \notin \mathsf{Cr}$ as per the protocol steps. For each P_k under the control of $\mathcal{A}$, on behalf of every honest P_l, send the tag-shares $(\tau_{kj,l}^{(1)},\ldots,\tau_{kj,l}^{(L+4)})$ corresponding to every ordered pair (P_k,P_j) where $P_k \notin \mathsf{Cr}$. On behalf of each honest P_i, for every ordered pair (P_i,P_j) receive the shares $(\tau_{ij,k}^{(1)},\ldots,\tau_{ij,k}^{(L+4)})$ from $\mathcal{A}$ corresponding to each $P_k \notin \mathsf{Cr}$.
- Receive Rand from the $\mathcal{A}$ and emulate $\mathcal{F}_{\mathsf{Rand}}$ by sending a random r_3 to all.
- Perform the computation as per $\mathsf{Ver}(r_3)$ on behalf of the honest dealer. Emulate $\mathcal{F}_{\mathsf{BC}}$ by sending $(\mathsf{F}^{(3,1)}(X,Y),\ldots,\mathsf{F}^{(3,M)}(X,Y))$ to $\mathcal{A}$, and simulate the Rev algorithm of ICP for each party in Core.
- For every ordered triplet (P_k,P_j,P_l) such that $P_k,P_l \notin \mathsf{Cr}$, compute the shares of honest parties corresponding to $\lceil\beta_{kj,l}\rfloor_t^i$ and send them to $\mathcal{A}$ for every party P_k under the control of $\mathcal{A}$. On behalf of each honest P_i not under the control of $\mathcal{A}$, for every ordered triplet (P_i,P_j,P_l), receive shares of $\lceil\beta_{ij,l}\rfloor_t^i$ from $\mathcal{A}$ corresponding to each P_k controlled by $\mathcal{A}$.
- On behalf of each honest P_i, define the set $\mathcal{R}_{ij}$ as per the protocol specifications, and emulate $\mathcal{F}_{\mathsf{BC}}$ on behalf of each honest P_i and send Tag_i or $(\mathsf{NOK},\mathsf{Dp}_i)$ to $\mathcal{A}$ based on the protocol steps.
- Receive from the $\mathcal{A}$, either Tag_k or $(\mathsf{NOK},\mathsf{Dp}_k)$ corresponding to each $P_k \notin \mathsf{Cr}$ that is under the control of $\mathcal{A}$ and send the same to all.
- For every ordered pair (P_i,P_j) where $P_i \notin \mathsf{Cr}$, receive shares of $\lceil v_{ji}^{(u)}\rfloor_t^i$ for every $u \in \{1,\ldots,L\}$ on behalf of every honest P_j from $\mathcal{A}$. On behalf of the honest parties, for every ordered pair (P_i,P_k) where $P_i \notin \mathsf{Cr}$, send shares of $\lceil v_{ki}^{(u)}\rfloor_t^i$ to $\mathcal{A}$ corresponding to each P_k under the control of $\mathcal{A}$.
- Emulate the role of $\mathcal{F}_{\mathsf{BC}}$ for each P_i which sends $(\mathsf{NOK},\mathsf{Dp}_i)$ as per the protocol steps.
- Receive Rand from the $\mathcal{A}$ and emulate $\mathcal{F}_{\mathsf{Rand}}$ by sending a random r_4 to all.
- Perform the computation as per $\mathsf{Ver}(r_4)$ on behalf of the honest dealer. Emulate $\mathcal{F}_{\mathsf{BC}}$ by sending $(\mathsf{F}^{(4,1)}(X,Y),\ldots,\mathsf{F}^{(4,M)}(X,Y))$ to $\mathcal{A}$, and simulate the Rev algorithm of ICP for each party in Core.
- For each P_i such that $(\mathsf{NOK},\mathsf{Dp}_i)$ was for broadcast, on behalf of honest parties, send shares of $\lceil\gamma_{i1,l}\rfloor_t^i,\ldots,\lceil\gamma_{in,l}\rfloor_t^i$ to all for construction corresponding to each $P_l \in \mathsf{Dp}_i$.
- Populate the set Cr (and restart if necessary), and on behalf of each honest P_i, update $\mathcal{R}_{ij}$ as per protocol steps.
- For every order pair of parties (P_i,P_j) where $P_i \notin \mathsf{Cr}$, receive shares of $\lceil v_{ji}^{(u)}\rfloor_t^i$, for every $u \in \{1,\ldots,L\}$ from $\mathcal{A}$ on behalf of every honest P_j. On behalf of the honest parties, for every ordered pair (P_i,P_k) where $P_i \notin \mathsf{Cr}$, send shares of $\lceil v_{ki}^{(u)}\rfloor_t^i$ to $\mathcal{A}$ corresponding to each P_k under the control of $\mathcal{A}$.

– **Case 2: Corrupt Dealer**

- On behalf of every honest $P_i \notin \mathsf{Dp}\cup\mathsf{Cr}$, receive the polynomials $\{f_i^{(u,v)}(X),g_i^{(u,v)}(Y)\}_{u=1,\ldots,L+4,v=1,\ldots,M}$ from $\mathcal{A}$.
- On behalf of each honest party P_i, verify the received polynomials as per protocol specifications and if the conditions pass, it emulates $\mathcal{F}_{\mathsf{BC}}$ sending OK_i to $\mathcal{A}$. It also simulates the steps of Gen algorithm of ICP on behalf of all the parties.
- Emulate $\mathcal{F}_{\mathsf{BC}}$ and receive either Core or Dp from $\mathcal{A}$ and send it to all.
- On behalf of the honest parties, perform the local verification of Core or Dp and if it fails, terminate. Otherwise continue to simulate the remaining protocol steps as below.

- Receive Rand from the $\mathcal{A}$ and emulate $\mathcal{F}_{\mathsf{Rand}}$ by sending a random r_1 to all.
- Perform the computation as per $\mathsf{Ver}(r_4)$. Emulate $\mathcal{F}_{\mathsf{BC}}$ by receiving $(\mathsf{F}^{(1,1)}(X,Y),\ldots,\mathsf{F}^{(1,M)}(X,Y))$ from the $\mathcal{A}$ and send it to all. Simulate the Rev algorithm of ICP for each party in Core.
- On behalf of the honest parties, discard the dealer and terminate if any of the conditions fail as per the protocol specification.

- On behalf of the honest parties, simulate the rest of the protocol as per the protocol steps as in the case of an honest dealer.

7 On the Size of SIMD Circuit

From the previous section, it follows that our VSS protocol can achieve a constant overhead only if there are "sufficiently many" values available for secret-sharing. More precisely, given a set of elements $\mathbb{S}$, our VSS can generate ${}^{A}\langle \mathbb{S} \rangle_{\ell}$ where $\ell \in \{p-1, d, d+p-1\}$ with a constant overhead only if $|\mathbb{S}| = n^5p^2$, where $d = t+p-1$. We will require the values in $\mathbb{S}$ to be arranged into n^3 two-dimensional vectors $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(n^3)}$. Hence $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(n^3)})$. For $u \in \{1, \ldots, n^3\}$, each $\mathbf{S}^{(u)}$ will further consists of n^2p one-dimensional vectors, with each one-dimensional vector having p elements from $\mathbb{F}$. Namely, the elements of $\mathbf{S}^{(u)}$ will be arranged as $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,n^2p)})$, where each vector $\mathbf{s}^{(u,v)} = (s^{(u,v,1)}, \ldots, s^{(u,v,p)})$. So $\mathbb{S} = \{s^{(u,v,w)}\}_{u \in \{1,\ldots,n^3\}, v \in \{1,\ldots,n^2p\}, w \in \{1,\ldots,p\}}$ and our VSS will generate ${}^{A}\langle \mathbb{S} \rangle_{\ell}$ with parties holding ${}^{A}\langle \mathbf{S}^{(1)} \rangle_{\ell}, \ldots, {}^{A}\langle \mathbf{S}^{(n^3)} \rangle_{\ell}$. In summary, *each one-dimensional vector* $\mathbf{s}$ *will contain* p *elements, each two-dimensional vector* $\mathbf{S}$ *will contain* n^2p^2 *elements and each three-dimensional vector* $\mathbb{S}$ *will contain* n^5p^2 *elements.*

Even though it will be sufficient to work with n^5p^2 elements in our VSS to get $\mathcal{O}(1)$ communication overhead, to get the same $\mathcal{O}(1)$ overhead in our MPC protocol, the parties need to evaluate the circuit $\mathtt{ckt}$ over $n^8(t+1)p^2 = \Theta(n^{11})$ inputs in parallel in an SIMD fashion. Hence each party P_i will have $n^8(t+1)p^2$ values corresponding to its input-wire x_i in $\mathtt{ckt}$. These inputs will be further grouped into n^3 groups, with each group consisting of $t+1$ sub-groups and where each sub-group will consist of n^5p^2 values. Hence, the inputs of each P_i will be arranged as $\{\mathbb{X}_i^{(\beta,\gamma)}\}_{\beta \in \{1,\ldots,n^3\}, \gamma \in \{1,\ldots,t+1\}}$ where each $|\mathbb{X}_i^{(\beta,\gamma)}| = n^5p^2$ and the parties will hold ${}^{A}\langle \mathbb{X}_i^{(\beta,\gamma)} \rangle_d$ as per the same structure as discussed in the previous paragraph. Each gate in $\mathtt{ckt}$ will be evaluated over $n^8(t+1)p^2$ inputs in parallel, producing $n^8(t+1)p^2$ outputs. Consequently, during the circuit-evaluation, with every wire in $\mathtt{ckt}$, there will be $n^8(t+1)p^2$ values which will remain ${}^{A}\langle \cdot \rangle_d$-shared.

The evaluation of each gate (over $n^8(t+1)p^2$ many inputs) will be divided into n^3 groups. Within each group, the gate will be evaluated $t+1$ times, over n^5p^2 inputs. In more detail, consider an arbitrary gate g in $\mathtt{ckt}$ with inputs wires x, y and output wire z. Let $\{\mathbb{X}^{(\beta,\gamma)}\}_{\beta \in \{1,\ldots,n^3\}, \gamma \in \{1,\ldots,t+1\}}$ and $\{\mathbb{Y}^{(\beta,\gamma)}\}_{\beta \in \{1,\ldots,n^3\}, \gamma \in \{1,\ldots,t+1\}}$ be the inputs, associated with wires x an y respectively and goal is to compute $\{\mathbb{Z}^{(\beta,\gamma)}\}_{\beta \in \{1,\ldots,n^3\}, \gamma \in \{1,\ldots,t+1\}}$, where $\mathbb{Z}^{(\beta,\gamma)}$ is the result of applying the gate function on the elements of $\mathbb{X}^{(\beta,\gamma)}$ and $\mathbb{Y}^{(\beta,\gamma)}$ component-wise. For example, if g is an addition gate then component-wise, $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} + \mathbb{Y}^{(\beta,\gamma)}$, while if g is a multiplication-gate then component-wise, $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$. Then during the shared-circuit evaluation, the parties will hold ${}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d$ and ${}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d$. For evaluating the gate g, the computation will be divided into n^3 groups, where within the group $\beta \in \{1, \ldots, n^3\}$, gate g will be evaluated $t+1$ times in parallel over the inputs $\{\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}\}_{\gamma \in \{1,\ldots,t+1\}}$. Looking ahead, evaluating a gate $t+1$ times within each group will ensure that we have "sufficiently many" ${}^{A}\langle \cdot \rangle_{\ell}$-shared inputs for degree-reduction when g is a multiplication-gate. On the other hand, having n^3 such groups of evaluations of gate g will ensure that we maintain $\mathcal{O}(1)$ communication overhead even if the instances of underlying degree-reduction fails and the parties need to run a fault-identification protocol to let the parties identify corrupt parties (refer to Section 2 for the overview of statistically-secure degree-reduction and Beaver's protocol).

For a pictorial illustration of evaluation of a gate in an SIMD fashion, see Fig 6. In summary, each of our vector $\mathbf{s}$ will be of size p, each $\mathbf{S}$ will consist of n^2p many $\mathbf{s}$ vectors. Then, each $\mathbb{S}$ will consist of n^3 such $\mathbf{S}$. We group $t+1$ many $\mathbb{S}$ together for degree-reduction and finally n^3 such groups will be operated together to ensure constant-overhead even when degree-reduction fails. Based on this discussion on the size of our SIMD circuit, we next present the preprocessing phase protocol followed by the circuit-evaluation protocol.

8 The Preprocessing Phase

In this section, we present our preprocessing protocol, which securely realizes the ideal functionality presented in Functionality 8.1. The functionality generates a *one-time* setup, which can be used across

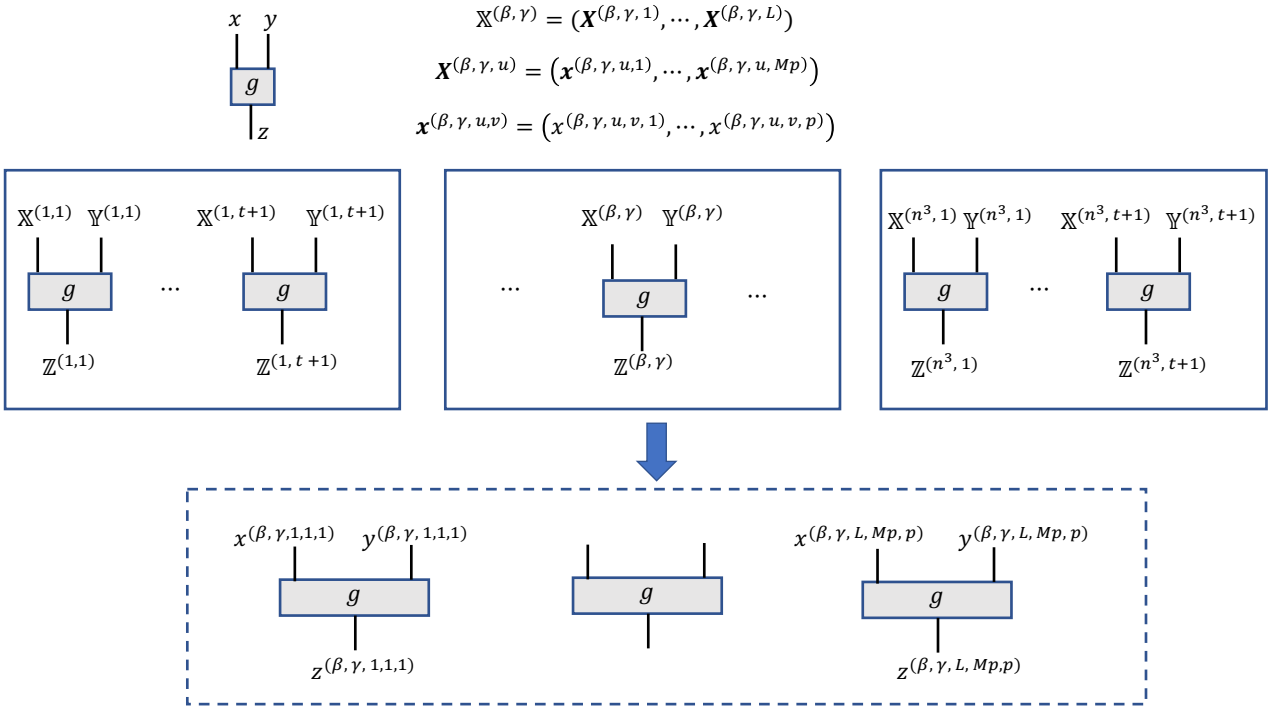

Fig. 6: Pictorial illustration of evaluation of a gate g in circuit `ckt` in SIMD fashion. The top-most figure shows gate g with input-wires x, y and output-wire z. The second figure shows the evaluation of gate g into n^3 groups, where within each group, it is evaluated $t+1$ times in parallel over LMp^2 inputs, where $L = n^3$ and $M = n^2$. Consider the βth group where $\beta \in \{1, \ldots, n^3\}$ and within the group, the γth evaluation of the gate g. As illustrated in the last figure, this internally consists of evaluating the gate LMp^2 times. During the shared circuit-evaluation, parties will hold inputs ${}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d$ and ${}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d$ and will compute ${}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d$. The top-most figure also shows the arrangement of elements of $\mathbb{X}^{(\beta,\gamma)}$. It consists of L collections of vectors. Each collection $\mathbf{X}^{(\beta,\gamma,u)}$ further consists of Mp vectors $\boldsymbol{x}^{(\beta,\gamma,u,v)}$, where each $\boldsymbol{x}^{(\beta,\gamma,u,v)}$ consists of p elements from $\mathbb{F}$. As part of ${}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d$, parties will hold ${}^{A}\langle \mathbf{X}^{(\beta,\gamma,1)} \rangle_d, \ldots, {}^{A}\langle \mathbf{X}^{(\beta,\gamma,L)} \rangle_d$

multiple instances of our circuit-evaluation protocol. The setup basically generates a twisted t-sharing for the $\boldsymbol{\mu}$ component of the MAC-key across every pair of parties (recall that for key-consistent sharings, every P_j has to use the same $\boldsymbol{\mu}_{ji}$ for all the MAC-keys that it selects for P_i). Namely, for every pair of parties (P_i, P_j), the functionality sets up a twisted t-sharing $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ with respect to P_i, where the twisted-share of P_i is set to 0. The sharing is generated as follows: if P_j is *honest*, then the functionality picks $\boldsymbol{\mu}_{ji}$ randomly and also generates $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ randomly. On the other hand, if P_j is *corrupt*, then the functionality receives $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ from the ideal-world adversary.

Next, the functionality generates two types of preprocessing data to be used later by our online-phase protocol. The first type of preprocessing data is ${}^{A}\langle \cdot \rangle_d$-sharing of random multiplication-triples. In more detail, for every multiplication gate in `ckt`, the functionality generates $n^3(t+1)$ key-consistent ${}^{A}\langle \cdot \rangle_d$-sharing of n^5p^2 random multiplication triples and $n^3(t+1)$ key-consistent ${}^{A}\langle \cdot \rangle_d$-sharing of n^5p^2 random triples (these need not be multiplication triples). The reason for associating so many multiplication-triples and triples with every multiplication-gate is as follows. Recall from Section 7 that each multiplication-gate will be evaluated over n^5p^2 pairs of inputs in parallel in SIMD fashion $n^3(t+1)$ times. To achieve $\mathcal{O}(1)$ communication overhead, this computation will be divided into n^3 batches, where in each batch the gate is evaluated over n^5p^2 pairs of inputs $t+1$ times. As discussed in Section 2.1.2, the protocol for evaluating multiplication gates will have a *non-robust* step where the evaluation may fail if corrupt parties cheat. However, in that case, the parties can run a "fault-localization" process which will enable the honest parties to locally identify a subset of corrupt parties. The fault-localization process will require

a vector of triples to be "sacrificed" in the sense that their privacy will be completely lost. After the fault-localization process, it would be guaranteed that the multiplication gate will be evaluated robustly.

The second type of pre-processing data is double-packed-secret-shared vectors of random values. In more detail, for every multiplication gate in `ckt`, the functionality generates $2n^3(t+1)$ key-consistent ${}^A\langle\cdot\rangle_d$-sharing as well as key-consistent ${}^A\langle\cdot\rangle_{d+p-1}$-sharing of n^5p^2 random values. The reason for associating an additional random pair with every multiplication gate is same as that for associating a sacrificing triple with every multiplication gate. Namely, evaluation of a multiplication gate may fail once and the fault localization for a successful completion will "sacrifice" a pair of randomness.

In $\mathcal{F}_{\mathsf{PREP}}$, the ideal adversary specifies all the "data" that the corrupt parties would like to hold as part of the various ${}^A\langle\cdot\rangle$-sharings generated by the functionality. Namely it specifies the shares and MAC-keys. The functionality then generates the ${}^A\langle\cdot\rangle$-sharings while keeping them consistent with the data specified by the ideal-world adversary.

Functionality 8.1: $\mathcal{F}_{\mathsf{PREP}}$ – The ideal functionality for one-time setup generation and pre-processing

The functionality interacts with the parties in $\mathcal{P}$ and an adversary $\mathcal{S}$ as follows. Let $\mathcal{C}$ be the set of corrupt parties.

- **Setup Generation**: On input SetUp from all the parties in $\mathcal{P} \setminus \mathcal{C}$, do the following.
  - On behalf of every $P_j \in \mathcal{P} \setminus \mathcal{C}$, do the following.
    - Select n random vectors $\{\boldsymbol{\mu}_{ji}\}_{i=1}^n$ where each $\boldsymbol{\mu}_{ji} \in \mathbb{F}^{n^2p}$ and where the ith value is designated to be used in the MAC-key for party P_i.
    - For every $\boldsymbol{\mu}_{ji}$, generate a random $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ where the twisted-share of P_i is 0. Then send to every P_k its shares corresponding to $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$.
  - On behalf of each $P_i \in \mathcal{C}$, do the following.
    - Receive from $\mathcal{S}$ the n vectors $\{\boldsymbol{\mu}_{ji}\}_{i=1}^n$ where each $\boldsymbol{\mu}_{ji} \in \mathbb{F}^{n^2p}$ and where the ith value is designated to be used in the MAC-key for P_i.
    - For every $\boldsymbol{\mu}_{ji}$, receive from $\mathcal{S}$ the sharing $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ such that the twisted-share of P_i is 0. Then send to every P_k its shares corresponding to $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$.
- **Random Multiplication-Triple Sharings**: On input $\mathsf{MultTriple}$ from all the parties in $\mathcal{P} \setminus \mathcal{C}$, generate $n^3(t+1)c_M$ number of ${}^A\langle\cdot\rangle_d$-sharing of random multiplication-triples in parallel, where c_M is the number of multiplication gates in `ckt`. To generate one such sharing $({}^A\langle\mathbb{A}\rangle_d, {}^A\langle\mathbb{B}\rangle_d, {}^A\langle\mathbb{C}\rangle_d)$, where $\mathbb{A}, \mathbb{B}, \mathbb{C}$ are three-dimensional vectors consisting of total n^5p^2 field elements, do the following.
  - For $u \in \{1, \ldots, n^3\}, v \in \{1, \ldots, n^2p\}$, randomly select vectors $\mathbf{a}^{(u,v)}$ and $\mathbf{b}^{(u,v)}$, where each $\mathbf{a}^{(u,v)}$ and $\mathbf{b}^{(u,v)}$ consists of p random elements from $\mathbb{F}$.
  - Compute $\mathbf{c}^{(u,v)} = \mathbf{a}^{(u,v)} \times \mathbf{b}^{(u,v)}$, the component-wise product of $\mathbf{a}^{(u,v)}$ and $\mathbf{b}^{(u,v)}$.
  - Let $\mathbb{A} = (\mathbf{A}^{(1)}, \ldots, \mathbf{A}^{(n^3)})$ where each $\mathbf{A}^{(u)} = (\mathbf{a}^{(u,1)}, \ldots, \mathbf{a}^{(u,n^2p)})$. Similarly, define $\mathbb{B}$ and $\mathbb{C}$ from the vectors $\{\mathbf{b}^{(u,v)}\}$ and $\{\mathbf{c}^{(u,v)}\}$ respectively. Run the steps "Single ${}^A\langle\cdot\rangle_d$-Sharing Generation" (see below) to generate ${}^A\langle\mathbb{A}\rangle_d$, ${}^A\langle\mathbb{B}\rangle_d$ and ${}^A\langle\mathbb{C}\rangle_d$.
- **Random Triple Sharings**: On input Triple from all the parties in $\mathcal{P} \setminus \mathcal{C}$, generate $n^3(t+1)c_M$ number of ${}^A\langle\cdot\rangle_d$-sharings, each consisting of n^5p^2 random triples, in parallel. To generate one such sharing $({}^A\langle\hat{\mathbb{A}}\rangle_d, {}^A\langle\hat{\mathbb{B}}\rangle_d, {}^A\langle\hat{\mathbb{C}}\rangle_d)$, do the following.
  - For $u \in \{1, \ldots, n^3\}, v \in \{1, \ldots, n^2\}$, randomly select vectors $\hat{\mathbf{a}}^{(u,v)}, \hat{\mathbf{b}}^{(u,v)}$ and $\hat{\mathbf{c}}^{(u,v)}$, where each $\hat{\mathbf{a}}^{(u,v)}, \hat{\mathbf{b}}^{(u,v)}$ and $\hat{\mathbf{c}}^{(u,v)}$ consists of p random elements from $\mathbb{F}$.
  - Let $\hat{\mathbb{A}} = (\hat{\mathbf{A}}^{(1)}, \ldots, \hat{\mathbf{A}}^{(n^3)})$ where each $\hat{\mathbf{A}}^{(u)} = (\hat{\mathbf{a}}^{(u,1)}, \ldots, \hat{\mathbf{a}}^{(u,n^2p)})$. Similarly, define $\hat{\mathbb{B}}$ and $\hat{\mathbb{C}}$ from the vectors $\{\hat{\mathbf{b}}^{(u,v)}\}$ and $\{\hat{\mathbf{c}}^{(u,v)}\}$ respectively. Run the steps "Single ${}^A\langle\cdot\rangle_d$-Sharing Generation" (see below) to generate ${}^A\langle\hat{\mathbb{A}}\rangle_d$, ${}^A\langle\hat{\mathbb{B}}\rangle_d$ and ${}^A\langle\hat{\mathbb{C}}\rangle_d$.
- **Random Double-Sharings**: On input Double from all the parties in $\mathcal{P} \setminus \mathcal{C}$, generate $2n^3(t+1)c_M$ number of ${}^A\langle\cdot\rangle_d$-sharing and ${}^A\langle\cdot\rangle_{d+p-1}$-sharing of n^5p^2 random values in parallel. To generate one such pair $({}^A\langle\mathbb{R}\rangle_d, {}^A\langle\mathbb{R}\rangle_{d+p-1})$, do the following.
  - For $u \in \{1, \ldots, n^3\}, v \in \{1, \ldots, n^2p\}$, randomly select vectors $\mathbf{r}^{(u,v)}$, each consisting of p random elements from $\mathbb{F}$.
  - Let $\mathbb{R} = (\mathbf{R}^{(1)}, \ldots, \mathbf{R}^{(n^3)})$ where each $\mathbf{R}^{(u)} = (\mathbf{r}^{(u,1)}, \ldots, \mathbf{r}^{(u,n^2p)})$. Run the steps "Single ${}^A\langle\cdot\rangle_d$-Sharing Generation" and "Single ${}^A\langle\cdot\rangle_{d+p-1}$-Sharing Generation" (see below) to generate ${}^A\langle\mathbb{R}\rangle_d$ and ${}^A\langle\mathbb{R}\rangle_{d+p-1}$ respectively.

Single $^{A}\langle\cdot\rangle_d$-Sharing Generation: The functionality does the following to generate $^{A}\langle\mathbb{S}\rangle_d$ for a given set of one-dimensional vectors $\{\mathbf{s}^{(u,v)}\}_{u\in\{1,\ldots,n^3\},v\in\{1,\ldots,n^2p\}}$, where each $\mathbf{s}^{(u,v)} \in \mathbb{F}^p$.

- Let $\mathbb{S} = (\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(n^3)})$ where each $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,n^2p)})$. Let $\mathbf{s}^{(u,v)} = (s^{(u,v,1)}, \ldots, s^{(u,v,p)})$.
- For each vector $\mathbf{s}^{(u,v)}$, on receiving the shares $\{s_i^{(u,v)}\}_{P_i\in\mathcal{C}}$ from $\mathcal{S}$ on behalf of the corrupt parties, compute $\langle\mathbf{s}^{(u,v)}\rangle_d$ as follows.
  - Select a d-degree polynomial $S^{(u,v)}(\cdot)$, which is otherwise a random polynomial such that:
    - $S^{(u,v)}(i) = s_i^{(u,v)}$, for every $P_i \in \mathcal{C}$;
    - $S^{(u,v)}(-1) = s^{(u,v,-1)}, \ldots, S^{(u,v)}(-p) = s^{(u,v,p)}$.
  - For every $P_i \notin \mathcal{C}$, compute $s_i^{(u,v)} = S^{(u,v)}(i)$.
- Corresponding to each $\mathbf{S}^{(u)}$, generate $^{A}\langle\mathbf{S}^{(u)}\rangle_d$ as follows which completes the generation of $^{A}\langle\mathbb{S}\rangle_d$.
  - On receiving $\{v_{ji}^{(u)}\}_{P_j\in\mathcal{C},P_i\notin\mathcal{C}}$ from $\mathcal{S}$, the second components of the MAC-key that a corrupt P_j will have for an honest P_i, set $\mathsf{K}_{ji}^{(u)} = (\boldsymbol{\mu}_{ji}, v_{ji}^{(u)})$, where $\boldsymbol{\mu}_{ji}$ was specified by $\mathcal{S}$ in "Setup Generation" stage.
  - For every $P_j \notin \mathcal{C}$ and every $P_i \in \mathcal{P}$, randomly pick $v_{ji}^{(u)} \in \mathbb{F}$ and set $\mathsf{K}_{ji}^{(u)} = (\boldsymbol{\mu}_{ji}, v_{ji}^{(u)})$, where $\boldsymbol{\mu}_{ji}$ is taken from the "Setup Generation" stage.
  - For every $P_i \in \mathcal{P}$, let $\mathbf{s}_i^{(u)}$ be the vector of shares of P_i corresponding to $\langle\mathbf{S}^{(u)}\rangle_d$. That is, $\mathbf{s}_i^{(u)} = \{s_i^{(u,v)}\}_{v\in\{1,\ldots,n^2p\}}$. For every $P_j \in \mathcal{P}$, compute the MAC-tag $\tau_{ij}^{(u)} = \mathsf{MAC}_{\mathsf{K}_{ji}^{(u)}}(\mathbf{s}_i^{(u)})$.
  - For each $P_i \in \mathcal{P}$, send $\{\mathbf{s}_i^{(u)}, \{\tau_{ij}^{(u)}\}_{j=1}^n, \{\mathsf{K}_{ij}^{(u)}\}_{j=1}^n\}$ to P_i.

Single $^{A}\langle\cdot\rangle_{d+p-1}$-Sharing Generation: The steps here are almost the same as for "**Single $^{A}\langle\cdot\rangle_d$-Sharing Generation**:" except that for each vector of values, functionality generate a $(d+p-1)$-packed-secret-sharing.

To securely realize the functionality $\mathcal{F}_{\mathsf{PREP}}$, we next present various subprotocols.

8.1 Functionality for Generating a One-Time Set Up for Consistent MAC-Keys

Functionality $\mathcal{F}_{\mathsf{MacKeys}}$ presented in Functionality 8.2 generates a *one-time* setup of twisted t-sharing for the $\boldsymbol{\mu}$ component of the MAC-key across every pair of parties (recall that for key-consistent sharings, every P_j has to use the same $\boldsymbol{\mu}_{ji}$ for all the MAC-keys that it selects for P_i). Namely, for every pair of parties (P_i, P_j), the functionality sets up a twisted t-sharing $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$ with respect to P_i, where the twisted-share of P_i is set to 0. The sharing is generated as follows: if P_j is *honest*, then the functionality picks $\boldsymbol{\mu}_{ji}$ randomly and also generates $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$ randomly. On the other hand, if P_j is *corrupt*, then the functionality receives $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$ from the ideal-world adversary.

Functionality 8.2: $\mathcal{F}_{\mathsf{MacKeys}}$ — The ideal functionality for generating a one-time set up for consistent MAC-keys

---

The functionality is parametrized by the set $\mathcal{C}$ of corrupt parties. The functionality interacts with the parties in $\mathcal{P}$ and an adversary $\mathcal{S}$ as follows.

- **Setup Generation**: On receiving the input SetUp from all the parties in $\mathcal{P}\setminus\mathcal{C}$, do the following.
  - On behalf of every $P_j \in \mathcal{P}\setminus\mathcal{C}$, do the following.
    - Select n random vectors $\{\boldsymbol{\mu}_{ji}\}_{i=1}^n$ where each $\boldsymbol{\mu}_{ji} \in \mathbb{F}^{n^2p}$ and where the ith value is designated to be used in the MAC-key for party P_i.
    - Corresponding to every $\boldsymbol{\mu}_{ji}$, generate a random $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$ where the twisted-share of P_i is 0. For this, randomly pick n^2p random t-degree polynomials which evaluate to the elements of $\boldsymbol{\mu}_{ji}$ at i and which evaluate to 0 at 0. Then send to every P_k its shares corresponding to $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$.
  - On behalf of each $P_i \in \mathcal{C}$, do the following.
    - Receive from $\mathcal{S}$ the n vectors $\{\boldsymbol{\mu}_{ji}\}_{i=1}^n$ where each $\boldsymbol{\mu}_{ji} \in \mathbb{F}^{n^2p}$ and where the ith value is designated to be used in the MAC-key for P_i.
    - For every $\boldsymbol{\mu}_{ji}$, receive from $\mathcal{S}$ the sharing $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$ such that the twisted-share of P_i is 0. Then send to every P_k its shares corresponding to $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$. If $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$ is not a valid twisted t-sharing with twisted-shares of P_i being not 0, then send 0s as the shares to every P_k.

To securely realize the functionality $\mathcal{F}_{\mathsf{MacKeys}}$, the parties can run the protocol Π_{MacKeys} (see Protocol 8.3). The high level idea behind the protocol is very simple. For every pair of parties (P_i, P_j), party P_j picks a random vector of values $\boldsymbol{\mu}_{ji} \in \mathbb{F}^{n^2p}$. It then generates twisted t-sharing of these values with respect to P_i, by embedding these values in random t-degree polynomials at i, such that P_i's twisted-shares are set to 0. The generation of twisted sharing is done verifiably by calling the functionality $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$. For the dealers who are identified to be corrupt (by sending incorrect polynomials in their respective $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ instances), the parties pick 0 as the default shares.

Protocol 8.3: Π_{MacKeys} — Protocol for securely realizing $\mathcal{F}_{\mathsf{MacKeys}}$ in the $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$-hybrid model

- Every party $P_j \in \mathcal{P}$ does the following for every $P_i \in \mathcal{P}$.
  - Randomly pick $\boldsymbol{\mu}_{ji} = (u_{ji}^{(1)}, \ldots, u_{ji}^{(n^2p)}) \in \mathbb{F}^{n^2p}$.
  - For every $k \in \{1, \ldots, n^2p\}$, pick a random t-degree polynomial $q_{ji}^{(k)}(\cdot)$, such that $q_{ji}^{(k)}(i) = u_{ji}^{(k)}$ and $q_{ji}^{(k)}(0) = 0$.
  - Send $\mathcal{T}_{ji} = \{P_i\}$ and $q_{ji}^{(1)}(\cdot), \ldots, q_{ji}^{(n^2p)}(\cdot)$ to $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$.
- Every party $P_k \in \mathcal{P}$ computes its output as follows.
  - Include party P_j in a set Cr, provided there exists any $i \in \{1, \ldots, n\}$, such that no output is obtained from the $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ instance where P_j is the dealer and P_i is the party with respect to whom P_j has invoked the $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ instance.
  - Corresponding to every $P_j \in \mathsf{Cr}$, set $\boldsymbol{\mu}_{ji}$ to the default value $(0, \ldots, 0$ (n^2p times)), for every $i \in \{1, \ldots, n\}$.
  - For every pair (P_i, P_j), output the shares corresponding to $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$ as follows.
    - If $P_j \in \mathsf{Cr}$, then set $(0, \ldots, 0$ (n^2p times)) as the share for $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$.
    - Else set the share as follows.
      - If $P_i = P_k$, then set $(0, \ldots, 0$ (n^2(p times))) as the share for $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$.
      - Else set $(q_{ji}^{(1)}(k), \ldots, q_{ji}^{(n^2p)}(k))$ as the share for $\lceil\boldsymbol{\mu}_{ji}\rfloor_t^i$, where $q_{ji}^{(1)}(k), \ldots, q_{ji}^{(n^2p)}(k)$ is received from the $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ instance invoked by P_j as the dealer, corresponding to P_i.

We show that protocol Π_{MacKeys} securely realizes the functionality $\mathcal{F}_{\mathsf{MacKeys}}$ in the $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$-hybrid model.

Lemma 8.4. *Protocol Π_{MacKeys} securely realizes $\mathcal{F}_{\mathsf{MacKeys}}$ in $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$-hybrid model with perfect security. The protocol calls $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ n^2 times, each with n^2p inputs.*

Proof. The number of calls to $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ is straightforward. The security of the protocol simply follows from the fact that all $\boldsymbol{\mu}$ vectors are twisted t-shared through calls to $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$. We formalize this by giving a simulator $\mathcal{S}_{\mathsf{MacKeys}}$ (Simulator 8.5) for the protocol Π_{MacKeys}.

Let $\mathcal{A}$ be an arbitrary real-world adversary, attacking protocol Π_{MacKeys} and let $\mathcal{Z}$ be an arbitrary environment. We show the existence of a simulator $\mathcal{S}_{\mathsf{MacKeys}}$, such that the output of all parties and the adversary in an execution of the real protocol Π_{MacKeys} with $\mathcal{A}$ is identical to the output in an execution with $\mathcal{S}_{\mathsf{MacKeys}}$ involving $\mathcal{F}_{\mathsf{MacKeys}}$ in the ideal model. This further implies that $\mathcal{Z}$ cannot distinguish between the two executions. The idea behind the simulator is extremely simple. The simulator handles all the interaction with $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ in the simulated execution. Namely, on behalf of every honest party, the simulator picks random $\boldsymbol{\mu}$ vectors and generates random twisted-sharings of those vectors as per the steps of $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$. On the other hand, for corrupt parties, the simulator records the polynomials that these parties want to send to $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$. From these polynomials, the simulator computes the underlying vectors of twisted-shares and forwards them to $\mathcal{F}_{\mathsf{MacKeys}}$.

Simulator 8.5: $\mathcal{S}_{\mathsf{MacKeys}}$: Simulator for the Protocol Π_{MacKeys}

$\mathcal{S}_{\mathsf{Prep}}$ constructs virtual real-world honest parties and invokes the real-world adversary $\mathcal{A}$ who controls at most t parties. The simulator simulates the view of $\mathcal{A}$, namely its communication with $\mathcal{Z}$, the messages sent by the honest parties and the interaction with $\mathcal{F}_{\mathsf{MacKeys}}$. In order to simulate $\mathcal{Z}$, the simulator forwards every message it receives from $\mathcal{Z}$ to $\mathcal{A}$ and vice-versa. The simulator then simulates the various steps of the protocol as follows.

– On behalf of every party P_j not under the control of $\mathcal{A}$ and every $P_i \in \mathcal{P}$, the simulator does the following.
  - Randomly pick $\boldsymbol{\mu}_{ji} = (u_{ji}^{(1)}, \ldots, u_{ji}^{(n^2p)}) \in \mathbb{F}^{n^2p}$.
  - For every $k \in \{1, \ldots, n^2p\}$, pick a random t-degree polynomial $q_{ji}^{(k)}(\cdot)$, such that $q_{ji}^{(k)}(i) = u_{ji}^{(k)}$ and $q_{ji}^{(k)}(0) = 0$.
  - For every party P_k under the control of $\mathcal{A}$, send the following to $\mathcal{A}$ on behalf of the instance of $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ called by P_j corresponding to P_i:
    * $(0, \ldots, 0$ (n^2p times $))$, if $P_i = P_k$;
    * $q_{ji}^{(1)}(k), \ldots, q_{ji}^{(n^2p)}(k)$, otherwise.

– On behalf of every P_j under the control of $\mathcal{A}$, the simulator does the following.
  - Let $q_{ji}^{(1)}(\cdot), \ldots, q_{ji}^{(n^2p)}(\cdot)$ be the polynomials which P_j wants to send to $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$, corresponding to P_i. For each $k \in \{1, \ldots, n^2p\}$, let $u_{ji}^{(k)} = q_{ji}^{(k)}(i)$ and let $\boldsymbol{\mu}_{ji} = (u_{ji}^{(1)}, \ldots, u_{ji}^{(n^2p)})$.
  - Send $\{\boldsymbol{\mu}_{ji}\}_{i=1}^n$ to $\mathcal{F}_{\mathsf{MacKeys}}$ on behalf of P_j.
  - For every $\boldsymbol{\mu}_{ji}$, send the vectors $\{q_{ji}^{(k)}(1), \ldots, q_{ji}^{(k)}(i-1), q_{ji}^{(k)}(0), q_{ji}^{(k)}(i+1), \ldots, q_{ji}^{(k)}(n)\}_{k \in \{1,\ldots,n^2p\}}$ to $\mathcal{F}_{\mathsf{MacKeys}}$ on behalf of P_j

---

We next prove a sequence of claims, which help us to show that the joint distribution of the parties are identical in both the real-world, as well as ideal-world. We first show that the view generated by $\mathcal{S}_{\mathsf{MacKeys}}$ for $\mathcal{A}$ is identically distributed as $\mathcal{A}$'s view during the real execution of Π_{MacKeys}.

Claim 8.6. *The view of $\mathcal{A}$ in the simulated execution with $\mathcal{S}_{\mathsf{MacKeys}}$ is identically distributed as the view of $\mathcal{A}$ in the real execution of Π_{MacKeys}.*

Proof. In the real execution of Π_{MacKeys}, for every honest party P_j, the adversary $\mathcal{A}$ receives up to t twisted-shares from $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ for a random $\boldsymbol{\mu}_{ji}$, corresponding to every $P_i \in \mathcal{P}$. On the other hand, during the simulated execution, the simulator provides t random shares for a random $\boldsymbol{\mu}_{ji}$ selected by the simulator to $\mathcal{A}$. It is easy to see that these twisted-shares are identically distributed, as they are randomly chosen both in the real execution as well as in the simulated execution. This proves the lemma as well. □

We next claim that conditioned on the view of $\mathcal{A}$, the output of the honest parties is identically distributed in both the real as well as simulated executions.

Claim 8.7. *Conditioned on the view of $\mathcal{A}$, the output of the honest parties is identically distributed in the real execution of Π_{MacKeys} involving $\mathcal{A}$, as well as in the ideal execution involving $\mathcal{S}_{\mathsf{MacKeys}}$ and $\mathcal{F}_{\mathsf{MacKeys}}$.*

Proof. Let $\mathcal{H}$ be the set of honest parties. During the real execution, the output of the honest parties consist of the following.

– Corresponding to every $P_j \in \mathcal{H}$, the twisted-shares for random $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$, lying on random degree-t sharing polynomials.
– Corresponding to every $P_j \notin \mathcal{H}$, the twisted-shares for $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$, provided P_j selected and sent degree-t sharing polynomials for them to $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$. Otherwise the twisted-shares are set to 0s, implying that $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$ are all set to 0s.

Now consider the simulated execution. It then follows that corresponding to every $P_j \in \mathcal{H}$, the honest parties receive twisted-shares for random $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$, lying on random degree-t sharing polynomials, selected by $\mathcal{F}_{\mathsf{MacKeys}}$. So they are identically distributed in both the executions. On the other hand, for every $P_j \notin \mathcal{H}$, the simulator will learn the entire underlying sharing-polynomials that P_j would like to send to instances of $\mathcal{F}_{\lceil\cdot\rfloor_t^i}$ for computing twisted-shares for $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$. Based on these polynomials, the simulator computes the underlying $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$ and the corresponding vectors of shares and forwards them to $\mathcal{F}_{\mathsf{MacKeys}}$. If these vectors of shares define twisted t-sharings of $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$, then $\mathcal{F}_{\mathsf{MacKeys}}$ will distrubute the shares and thus the output of the honest parties will be identically distributed in both the executions. On ther other hand, if the vectors of shares do not define twisted t-sharings of $\boldsymbol{\mu}_{j1}, \ldots, \boldsymbol{\mu}_{jn}$, then $\mathcal{F}_{\mathsf{MacKeys}}$ will distribute 0s as twisted-shares. So even in this case, the outputs of the honest parties are identically distributed in both the executions. □

The lemma now follows from the previous two claims. □

Recall that one call to $\mathcal{F}_{\lceil \cdot \rfloor_t^i}$ with N polynomials as inputs can be securely realized with statistical security in a constant expected number of rounds in the $\mathcal{F}_{\mathsf{Rand}}$-hybrid model, incurring a communication of $\mathcal{O}(N \cdot n\kappa + n^4 \log n)$ bits, apart from one call to $\mathcal{F}_{\mathsf{Rand}}$. In the protocol, there are n^2 calls to $\mathcal{F}_{\lceil \cdot \rfloor_t^i}$, each with $N = n^2(\frac{t}{4}) = \mathcal{O}(n^3)$ polynomials. Securely realizing these calls will incur a constant expected number of rounds, as all the calls are made in parallel and incur a total communication of $\mathcal{O}(n^6\kappa)$ bits. The number of calls to $\mathcal{F}_{\mathsf{Rand}}$ will be 1, since the same call be be synchronized and used in realizing all the n^2 instances of $\mathcal{F}_{\lceil \cdot \rfloor_t^i}$. Also recall that $\mathcal{F}_{\mathsf{Rand}}$ can be securely realized with a constant expected round protocol, incurring a communication of $\mathcal{O}(n^5 \log n)$ bits. We summarize the above discussion in the following Lemma.

Lemma 8.8. *If $t < \frac{n}{3}$, then there exists a protocol for securely realizing $\mathcal{F}_{\mathsf{MacKeys}}$ with statistical security in the plain model. The protocol requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^6\kappa)$ bits.*

8.2 Robust Degree Reduction Protocol with a Constant Overhead

Our next protocol is a a *robust* degree-reduction protocol with a constant communication overhead. We have already discussed in Section 2 the generic blueprint which can be used to design the protocol based on the properties of RS error-correction, given any VSS protocol and a robust reconstruction protocol. Protocol Π_{RoDegRed} instantiates that blueprint for ${}^{A}\langle\cdot\rangle$.-sharings for three-dimensional vectors of values. The protocol takes input $t+1$ number of ${}^{A}\langle\cdot\rangle_d$-shared inputs $\mathbb{S}_1, \ldots, \mathbb{S}_{t+1}$, each consisting of n^5p^2 elements from $\mathbb{F}$. The protocol outputs ${}^{A}\langle\cdot\rangle_{p-1}$-sharing of $\mathbb{S}_1, \ldots, \mathbb{S}_{t+1}$. Note that the privacy of $\mathbb{S}_1, \ldots, \mathbb{S}_{t+1}$ will not be guaranteed and adversary will completely know these values, however that is fine for our purpose.

The protocol is based on the "distributed king" approach of [30] and works as follows. Parties first locally "expand" the $t+1$ input ${}^{A}\langle\cdot\rangle_d$-sharings into n number of ${}^{A}\langle\cdot\rangle_d$-sharings using RS linear error-correcting codes that can error-correct up to t errors. Next, each party robustly reconstruct one of the designated expanded sharings, followed by distributing a ${}^{A}\langle\cdot\rangle_{p-1}$-sharing of the reconstructed values. Note that up to t corrupt parties may distribute *incorrect* ${}^{A}\langle\cdot\rangle_{p-1}$-shared values. However, by leveraging the linearity property of RS error-correction, parties can error-correct these errors (on ${}^{A}\langle\cdot\rangle_{p-1}$-shared values), thus obtaining ${}^{A}\langle\cdot\rangle_{p-1}$-sharing of $\mathbb{S}_1, \ldots, \mathbb{S}_{t+1}$. The protocol is presented in Protocol 8.9.

Protocol 8.9: Π_{RoDegRed} – Robust Degree-Reduction Protocol

Input: Parties hold inputs ${}^{A}\langle\mathbb{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_d$, where each $\mathbb{S}^{(\gamma)} \in n^5p^2$. The goal is to obtain ${}^{A}\langle\mathbb{S}^{(1)}\rangle_{p-1}, \ldots, {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_{p-1}$.

1. Parties define a vector of n^5p^2 degree-$\frac{t}{2}$ polynomials $\mathbb{S}(X) = \mathbb{S}^{(1)} + \mathbb{S}^{(2)} \cdot X + \ldots + \mathbb{S}^{(t+1)} \cdot X^t$.
2. For $i \in \{1, \ldots, n\}$, parties locally compute ${}^{A}\langle\mathbb{S}(i)\rangle_d = {}^{A}\langle\mathbb{S}^{(1)}\rangle_d + i \cdot {}^{A}\langle\mathbb{S}^{(2)}\rangle_d + \ldots + i^t \cdot {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_d$.
3. For $i \in \{1, \ldots, n\}$, parties reconstruct $\mathbb{S}(i)$ towards P_i.
4. For each $i \in \{1, \ldots, n\}$, party P_i generates ${}^{A}\langle\mathbb{S}(i)\rangle_{p-1}$ by acting as a dealer and invoking an instance of protocol $\Pi_{A\langle\cdot\rangle}$ (Protocol 6.1).
5. The parties in $\mathcal{P}$ apply the RS-decoding procedure on ${}^{A}\langle\mathbb{S}(1)\rangle_{p-1}, \ldots, {}^{A}\langle\mathbb{S}(n)\rangle_{p-1}$ to error-correct t errors among $(\mathbb{S}(1), \ldots, \mathbb{S}(n))$ and compute ${}^{A}\langle\mathbb{S}^{(1)}\rangle_{p-1}, \ldots, {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_{p-1}$.

Lemma 8.10. *Given inputs ${}^{A}\langle\mathbb{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_d$, where each $\mathbb{S}^{(\gamma)} \in n^5p^2$, protocol Π_{RoDegRed} outputs ${}^{A}\langle\mathbb{S}^{(1)}\rangle_{p-1}, \ldots, {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_{p-1}$, except with a negligible probability.*

Proof. Let $\mathbb{S}(X)$ be the vector of degree-$t$polynomials where $\mathbb{S}(X) = \mathbb{S}^{(1)} + \mathbb{S}^{(2)} \cdot X + \ldots + \mathbb{S}^{(t+1)} \cdot X^t$. Since the parties hold ${}^{A}\langle\mathbb{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_d$, from the linearity property of ${}^{A}\langle\cdot\rangle_d$-sharing it follows that at the end of step 2, parties will hold ${}^{A}\langle\mathbb{S}(1)\rangle_d, \ldots, {}^{A}\langle\mathbb{S}(n)\rangle_d$. During step 3, each P_i will reconstruct $\mathbb{S}(i)$, except with a negligible probability, which follows from the reconstruction protocol for reconstructing ${}^{A}\langle\cdot\rangle_d$-sharing. During step 4, each *honest* P_i will generate ${}^{A}\langle\mathbb{S}(i)\rangle_{p-1}$, except with a negligible probability, which follows from the correctness property of $\Pi_{A\langle\cdot\rangle}$ (Lemma 6.6). Note that a *corrupt* P_i may generate ${}^{A}\langle\cdot\rangle_d$-sharing of an *incorrect* $\mathbb{S}(i)$. However, the commitment property of $\Pi_{A\langle\cdot\rangle}$ guarantees that on behalf

of P_i, there will be some ${}^{A}\langle\cdot\rangle_d$-sharing generated at the end of step 4. Now $\mathbb{S}(1),\ldots,\mathbb{S}(n)$ supposedly constitute vectors of n distinct points on the vectors of degree-t polynomials $\mathbb{S}(X)$. So even if up to t elements among $\mathbb{S}(1),\ldots,\mathbb{S}(n)$ are incorrect, RS error-correction can error-correct those errors and output $\mathbb{S}(X)$ and hence its coefficients, which are $\mathbb{S}^{(1)},\ldots,\mathbb{S}^{(t+1)}$. Since RS error-correction involves performing linear operations on $\mathbb{S}(1),\ldots,\mathbb{S}(n)$, it follows from the linearity property of ${}^{A}\langle\cdot\rangle_d$-sharing that applying the same linear operations on ${}^{A}\langle\mathbb{S}(1)\rangle_{p-1},\ldots,{}^{A}\langle\mathbb{S}(n)\rangle_{p-1}$ will output ${}^{A}\langle\mathbb{S}^{(1)}\rangle_{p-1},\ldots,{}^{A}\langle\mathbb{S}^{(t+1)}\rangle_{p-1}$. □

Lemma 8.11. *Given inputs ${}^{A}\langle\mathbb{S}^{(1)}\rangle_d,\ldots,{}^{A}\langle\mathbb{S}^{(t+1)}\rangle_d$, where each $\mathbb{S}^{(\gamma)}\in n^5p^2$, protocol Π_{RoDegRed} requires a constant expected number of rounds. The protocol incurs a communication of $\mathcal{O}(n^8\kappa)$ bits and calls $\mathcal{F}_{\mathsf{Rand}}$ 15 times.*

If the protocol is executed L times in parallel, then it requires a constant expected number of rounds, incurring a communication of $\mathcal{O}(L\cdot n^8\kappa)$ bits and calls $\mathcal{F}_{\mathsf{Rand}}$ 15 times.

Proof. Step 1 and 2 requires the parties to perform local computation. In step 3, for every $i\in\{1,\ldots,n\}$, $\mathbb{S}(i)$ consisting of n^5p^2 elements from $\mathbb{F}$ are reconstructed towards P_i. This requires one round and incurs a *total* communication of $\mathcal{O}(n^6p^2)$ field elements and hence $\mathcal{O}(n^8\kappa)$ bits. In step 4, for every $i\in\{1,\ldots,n\}$, party P_i invokes an instance of $\Pi_{{}^{A}\langle\cdot\rangle}$ to secret-share n^5p^2 field elements. From Lemma 6.8, this requires a constant expected number of rounds and a *total* communication of $\mathcal{O}(n^6p^2)$ field elements and hence $\mathcal{O}(n^8\kappa)$ bits. The calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized across all the dealers and all the instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ invoked in the protocol and hence the maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ will be 15. Step 5 of the protocol requires only local computation. Hence the protocol requires a constant expected number of rounds and a communication of $\mathcal{O}(n^8\kappa)$ bits.

If the protocol is executed L times in parallel, then the calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized across all the dealers and all the instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ and for all the L instances. Consequently, the number of calls to $\mathcal{F}_{\mathsf{Rand}}$ remains at most 15 only. □

Communication Overhead of the Protocol Π_{RoDegRed}. If the protocol is executed L times in parallel where $L\geq 1$, then the output of the protocol consists of $L\cdot n^5p^2(t+1)$ field elements and hence $\Theta(L\cdot n^8\kappa)$ bits. Since the communication complexity of the protocol will be $\mathcal{O}(L\cdot n^8\kappa)$ bits, apart from 15 calls to $\mathcal{F}_{\mathsf{Rand}}$, it follows that for sufficiently large L, the asymptotic communication overhead of the protocol is $\mathcal{O}(1)$ bits.

Although we do not model the requirements of Π_{RoDegRed} through a functionality, we describe the simulation steps for Π_{RoDegRed} which will be used when the protocol is used as a primitive.

Simulator 8.12: Simulator for the Protocol Π_{RoDegRed}

- On behalf of each honest party P_i not under the control of $\mathcal{A}$, perform the local computations as an honest party would as per the protocol specifications and for each P_k under the control of $\mathcal{A}$, send the honest parties' shares of ${}^{A}\langle\mathbb{S}(k)\rangle_d$ to $\mathcal{A}$.
- Receive from the adversary the shares of corrupt parties corresponding to ${}^{A}\langle\mathbb{S}(i)\rangle_d$ on behalf of each honest party P_i not under the control of $\mathcal{A}$.
- On behalf of each honest P_i, simulate the steps of VSS for an honest dealer as per Simulator 6.10 with P_i as the dealer and simulate the steps of VSS for a corrupt dealer as per Simulator 6.10 for each P_k under the control of $\mathcal{A}$ as the dealer.
- Perform local operations on behalf of the honest parties as per the protocol specifications.

8.3 Generating Double-Secret-Shared Random Values with a Constant Overhead

Our next protocol is Π_{RandPair} (Protocol 8.13), which generates double-packed-secret-shared random values. That is, the protocol outputs $\{({}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_d,{}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_{d+p-1})\}_{l\in\{1,\ldots,t\},m\in\{1,\ldots,n-t\}}$, where each $\widetilde{\mathbb{R}}^{(l,m)}\in\mathbb{F}^{n^5p^2}$ and remains random for the adversary.

The idea behind the protocol is standard and straight forward. Every party P_i selects $t+1$ groups of random values $\mathbb{R}_i^{(1)},\ldots,\mathbb{R}_i^{(t+1)}$ and generates ${}^{A}\langle\cdot\rangle_d$ as well as ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharings of these values through instances of $\Pi_{{}^{A}\langle\cdot\rangle}$. A potentially *corrupt* P_i could generate ${}^{A}\langle\cdot\rangle_d$-sharings and ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharings of *different* values. To verify the same, we use the standard "sacrificing" trick and publicly verify a random

linear combination of the various ${}^{A}\langle\cdot\rangle_d$-sharings and the various ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharings generated by P_i and check if they are the same. The randomness for the linear combination is generated through $\mathcal{F}_{\mathsf{Rand}}$. Once it is confirmed that every party has secret-shared the same random pairs, the parties locally extract secret-shared random pairs by deploying the Vandermonde-matrix based randomness extraction [27,12] technique, which involves performing linear operations on the values secret-shared by the parties.

Protocol 8.13: Generating Secret-Shared Random Pairs – Π_{RandPair}

1. Each party $P_i \in \mathcal{P}$ randomly selects $\mathbb{R}_i^{(1)}, \ldots, \mathbb{R}_i^{(t+1)}$ where each $\mathbb{R}_i^{(l)} \in \mathbb{F}^{n^5p^2}$.
2. Each $P_i \in \mathcal{P}$ acts a dealer and invokes instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ (Protocol 6.1) to generate ${}^{A}\langle\mathbb{R}_i^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{R}_i^{(t+1)}\rangle_d$ and ${}^{A}\langle\mathbb{R}_i^{(1)}\rangle_{d+p-1}, \ldots, {}^{A}\langle\mathbb{R}_i^{(t+1)}\rangle_{d+p-1}$.
3. Every $P_i \in \mathcal{P}$ sends Rand to $\mathcal{F}_{\mathsf{Rand}}$ and receives r from $\mathcal{F}_{\mathsf{Rand}}$.
4. For each $P_i \in \mathcal{P}$, parties locally compute
   - ${}^{A}\langle\hat{\mathbb{R}}_i\rangle_d = {}^{A}\langle\mathbb{R}_i^{(1)}\rangle_d + r \cdot {}^{A}\langle\mathbb{R}_i^{(2)}\rangle_d + \ldots + r^t {}^{A}\langle\mathbb{R}_i^{(t+1)}\rangle_d$
   - ${}^{A}\langle\widetilde{\mathbb{R}}_i\rangle_{d+p-1} = {}^{A}\langle\mathbb{R}_i^{(1)}\rangle_{d+p-1} + r \cdot {}^{A}\langle\mathbb{R}_i^{(2)}\rangle_{d+p-1} + \ldots + r^t {}^{A}\langle\mathbb{R}_i^{(t+1)}\rangle_{d+p-1}$
5. Parties publicly reconstruct $\hat{\mathbb{R}}_i, \widetilde{\mathbb{R}}_i$ for each $P_i \in \mathcal{P}$.
6. For each P_i where $\hat{\mathbb{R}}_i \neq \widetilde{\mathbb{R}}_i$, parties set $\mathbb{R}_i^{(1)} = \ldots = \mathbb{R}_i^{(t+1)} = 0^{n^5p^2}$ and takes the default sharing of ${}^{A}\langle\mathbb{R}_i^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{R}_i^{(t+1)}\rangle_d$ and ${}^{A}\langle\mathbb{R}_i^{(1)}\rangle_{d+p-1}, \ldots, {}^{A}\langle\mathbb{R}_i^{(t+1)}\rangle_{d+p-1}$ on behalf of P_i.
7. For $l \in \{1, \ldots, t\}$, let $\mathbb{R}^{(l)}(X)$ be the vectors of n^5p^2 degree-$(n-1)$ polynomials passing through the vectors of points $(1, \mathbb{R}_1^{(l)}), \ldots, (n, \mathbb{R}_n^{(l)})$. Moreover, for $m \in \{1, \ldots, n-t\}$, let $\widetilde{\mathbb{R}}^{(l,m)} = \mathbb{R}^{(l)}(n+m)$, where $\widetilde{\mathbb{R}}^{(l,m)} = c_{m,1} \cdot \mathbb{R}_1^{(l)} + \ldots + c_{m,n} \cdot \mathbb{R}_n^{(l)}$. Here $c_{m,1}, \ldots, c_{m,n}$ are publicly-known Lagrange's interpolation coefficients from $\mathbb{F}$.
8. For each $l \in \{1, \ldots, t\}$ and $m \in \{1, \ldots, n-t\}$, parties locally compute and output the following
   - ${}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_d = c_{m,1} \cdot {}^{A}\langle\mathbb{R}_1^{(l)}\rangle_d + \ldots + c_{m,n} \cdot {}^{A}\langle\mathbb{R}_n^{(l)}\rangle_d$.
   - ${}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_{d+p-1} = c_{m,1} \cdot {}^{A}\langle\mathbb{R}_1^{(l)}\rangle_{d+p-1} + \ldots + c_{m,n} \cdot {}^{A}\langle\mathbb{R}_n^{(l)}\rangle_{d+p-1}$.

We next prove the properties of the protocol Π_{RandPair}.

Lemma 8.14. *Protocol Π_{RandPair} outputs $\{({}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_{d+p-1})\}$, where $l \in \{1, \ldots, t\}, m \in \{1, \ldots, n-t\}$ and where each $\widetilde{\mathbb{R}}^{(l,m)} \in \mathbb{F}^{n^5p^2}$, except with a negligible probability.*

Proof. From the correctness and commitment properties of $\Pi_{{}^{A}\langle\cdot\rangle}$ (Lemma 6.6 and Lemma 6.7), at the end of step 2, parties will hold $t+1$ number of ${}^{A}\langle\cdot\rangle_d$-sharings and $t+1$ number of ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharings on behalf of each party P_i. Moreover, except with a negligible probability, parties will robustly reconstruct $\hat{\mathbb{R}}_i$ and $\widetilde{\mathbb{R}}_i$ during step 5. We claim that if P_i is *corrupt* and generated ${}^{A}\langle\mathbb{R}_i^{(l)}\rangle_d$ and ${}^{A}\langle\mathbb{R}_i'^{(l)}\rangle_d$ for any $l \in \{1, \ldots, \frac{t}{2}+1\}$, then except with a negligible probability, $\hat{\mathbb{R}}_i \neq \widetilde{\mathbb{R}}_i$ will hold and hence parties will take default ${}^{A}\langle\cdot\rangle_d$ and ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharings of 0's on behalf of P_i. This simply follows from the fact that r will be random for P_i when it generates ${}^{A}\langle\mathbb{R}_i^{(l)}\rangle_d$ and ${}^{A}\langle\mathbb{R}_i'^{(l)}\rangle_d$. Hence it is guaranteed that at the end of step 6, for every party P_i, parties hold ${}^{A}\langle\cdot\rangle_d$-sharings and ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharings of same values. The lemma now simply follows from the protocol steps. □

Lemma 8.15. *In protocol Π_{RandPair}, the view of the adversary will be independent of $\{({}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_{d+p-1})\}$, where $l \in \{1, \ldots, t\}, m \in \{1, \ldots, n-t\}$ and where each $\widetilde{\mathbb{R}}^{(l,m)} \in \mathbb{F}^{n^5p^2}$.*

Proof. Let $\mathcal{H}$ be the set of honest parties in $\mathcal{P}$, where $|\mathcal{H}| \geq n-t$. For simplicity and without loss of generality, we consider the worst case when $|\mathcal{H}| = n-t$. From the privacy property of $\Pi_{{}^{A}\langle\cdot\rangle}$ (Lemma 6.4), corresponding to every $P_i \in \mathcal{H}$, the adversary's view remain independent of $\mathbb{R}_i^{(1)}, \ldots, \mathbb{R}_i^{(t+1)}$ at the end of step 2. During step 4, for every $P_i \in \mathcal{H}$, adversary learns a linear combination of $\mathbb{R}_i^{(1)}, \ldots, \mathbb{R}_i^{(t+1)}$. Since $\mathbb{R}_i^{(1)}, \ldots, \mathbb{R}_i^{(t+1)}$ are randomly chosen by P_i, it follows that even after learning this linear combination, the view of the adversary remains independent of $\mathbb{R}_i^{(1)}, \ldots, \mathbb{R}_i^{(t)}$.

Next consider an arbitrary $l \in \{1, \ldots, t\}$ and consider the vectors of degree-$(n-1)$ polynomials $\mathbb{R}^{(l)}(X)$, passing through the vectors of points $(1, \mathbb{R}_1^{(l)}), \ldots, (n, \mathbb{R}_n^{(l)})$. Among these points, the points $\{\mathbb{R}_i^{(l)}\}_{P_i \in \mathcal{H}}$ are random for the adversary and there are $n-t$ such vectors of points. Hence it follows that

the vectors of points $\mathbb{R}^{(l)}(n+1), \ldots, \mathbb{R}^{(l)}(n+n-t)$ will also remain random for the adversary. Namely, for every candidate values for $\{\mathbb{R}_i^{(l)}\}_{P_i \in \mathcal{H}}$, there exists vectors of unique degree-$(n-1)$ polynomials passing through those candidate points and the vectors of points $\{\mathbb{R}_i^{(l)}\}_{P_i \notin \mathcal{H}}$. Consequently, the vectors of points $\widetilde{\mathbb{R}}^{(l,1)}, \ldots, \widetilde{\mathbb{R}}^{(l,n-t)}$ will remain random for the adversary. □

Lemma 8.16. *Protocol* Π_{RandPair} *outputs* $(n-t)tn^5p^2$ *double-packed-secret-shared elements from* $\mathbb{F}$*, incurring a communication of* $\mathcal{O}(n^9\kappa)$ *bits and makes at most* 16 *calls to* $\mathcal{F}_{\mathsf{Rand}}$*.*

If the protocol is executed L *times in parallel, then it outputs* $L\cdot(n-t)tn^5p^2$ *double-packed-secret-shared values, incurring a communication of* $\mathcal{O}(L \cdot n^9\kappa)$ *bits and makes at most* 16 *calls to* $\mathcal{F}_{\mathsf{Rand}}$*.*

Proof. From Lemma 8.14, Π_{RandPair} outputs $\{({}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{R}}^{(l,m)}\rangle_{d+p-1})\}$, where $l \in \{1, \ldots, t\}, m \in \{1, \ldots, n-t\}$ and where each $\widetilde{\mathbb{R}}^{(l,m)} \in \mathbb{F}^{n^5p^2}$. Thus $(n-t)tn^5p^2$ field elements are finally double-shared.

In the protocol, each party has to invoke $2t+2$ instances of $\Pi_{A\langle\cdot\rangle}$ as a dealer during step 2. From Lemma 6.8, this incurs a total communication of $\mathcal{O}(n^9\kappa)$ bits. The calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized across all the dealers and all the instances of $\Pi_{A\langle\cdot\rangle}$. Hence the maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ due to instances of $\Pi_{A\langle\cdot\rangle}$ will be 15. In addition to this, the parties again call $\mathcal{F}_{\mathsf{Rand}}$ during step 3, so the maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ is 16.

If the protocol is executed L times in parallel, then the calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized across various stages for all the L instances. Consequently, the number of calls to $\mathcal{F}_{\mathsf{Rand}}$ remains at most 16. □

Communication Overhead of Protocol Π_{RandPair}. If the protocol Π_{RandPair} is executed L times in parallel where $L \geq 1$, then the output of the protocol consists of packed-double-secret-sharing of $L \cdot (n-t)tn^5p^2 = \Theta(L \cdot n^9)$ field elements and hence $\Theta(L \cdot n^9\kappa)$ bits. Since the communication complexity of the protocol will be $\mathcal{O}(L \cdot n^9\kappa)$ bits, apart from 16 calls to $\mathcal{F}_{\mathsf{Rand}}$, it follows that for sufficiently large L, the asymptotic communication overhead of the protocol is $\mathcal{O}(1)$ bits.

We describe the simulation steps for Π_{RandPair} which will be used when the protocol is used as a primitive.

Simulator 8.17: Simulator for the Protocol Π_{RandPair}

- On behalf of every party P_j not under the control of $\mathcal{A}$, randomly select $\mathbb{R}_j^{(1)}, \ldots, \mathbb{R}_j^{(t+1)}$ where each $\mathbb{R}_j^{(l)} \in \mathbb{F}^{n^5p^2}$ and simulate the steps for VSS for an honest dealer as per Simulator 6.10.
- On behalf of every P_j under the control of $\mathcal{A}$, simulate the steps for VSS for a corrupt dealer as per Simulator 6.10.
- For each P_k under the control of $\mathcal{A}$, receive Rand from $\mathcal{A}$, select a random r and send r to $\mathcal{A}$ for every P_k.
- On behalf of every party P_j not under the control of $\mathcal{A}$, for every $P_i \in \mathcal{P}$, compute P_j's share of ${}^{A}\langle\hat{\mathbb{R}}_i\rangle_d$ and ${}^{A}\langle\widetilde{\mathbb{R}}_i\rangle_{d+p-1}$ and send it to every party P_k under the control of $\mathcal{A}$ for reconstruction. Receive shares of ${}^{A}\langle\hat{\mathbb{R}}_i\rangle_d$ and ${}^{A}\langle\widetilde{\mathbb{R}}_i\rangle_{d+p-1}$ from $\mathcal{A}$ corresponding to each P_k under its control.
- For each P_k under the control of $\mathcal{A}$, if $\hat{\mathbb{R}}_i \neq \widetilde{\mathbb{R}}_i$, assume $\mathbb{R}_i^{(1)} = \ldots = \mathbb{R}_i^{(t+1)} = 0^{n^5p^2}$ and take its default sharing on behalf of each party P_j that is not under the control of $\mathcal{A}$.

8.4 Generating Secret-Shared Random Triples with a Constant Overhead

Our next protocol is $\Pi_{\mathsf{RandTriples}}$ (Protocol 8.18), which generates packed-secret-shared triples that are random for the adversary. That is, the protocol outputs $\{({}^{A}\langle\widetilde{\mathbb{A}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{B}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{C}}^{(m)}\rangle_d)\}_{m\in\{1,\ldots,n-t\}}$, where each $\widetilde{\mathbb{A}}^{(m)}, \widetilde{\mathbb{B}}^{(m)}, \widetilde{\mathbb{C}}^{(m)} \in \mathbb{F}^{n^5p^2}$ and are random for the adversary. We stress that the triples *need not* be multiplication-triples. That is, it is *not* guaranteed that $\widetilde{\mathbb{C}} = \widetilde{\mathbb{A}} \times \widetilde{\mathbb{B}}$ component-wise.

The idea behind the protocol is straight-forward and similar to the protocol Π_{RandPair}, where every party secret-shares sufficiently many vectors of random triples and then the parties extract secret-sharing of vectors of random triples from them based on Vandermonde's matrix. However, the protocol is relatively simpler because now we *do not* need any verification mechanism to check the equality of the underlying values shared by a party.

Protocol 8.18: Generating Secret-Shared Random Triples – $\Pi_{\mathsf{RandTriples}}$

1. Each party $P_i \in \mathcal{P}$ randomly selects $(\mathbb{A}_i, \mathbb{B}_i, \mathbb{C}_i)$ where $\mathbb{A}_i, \mathbb{B}_i, \mathbb{C}_i \in \mathbb{F}^{n^5p^2}$.
2. Each $P_i \in \mathcal{P}$ acts a dealer and invokes instances of $\Pi_{A\langle\cdot\rangle}$ (Protocol 6.1) to generate ${}^{A}\langle\mathbb{A}_i\rangle_d$, ${}^{A}\langle\mathbb{B}_i\rangle_d$, ${}^{A}\langle\mathbb{C}_i\rangle_d$.
3. Let $\mathbb{A}(X), \mathbb{B}(X), \mathbb{C}(X)$ be the vectors of n^5p^2 degree-$(n-1)$ polynomials passing through the vectors of points $\{(1, \mathbb{A}_1), \ldots, (n, \mathbb{A}_n)\}$, $\{(1, \mathbb{B}_1), \ldots, (n, \mathbb{B}_n)\}$ and $\{(1, \mathbb{C}_1), \ldots, (n, \mathbb{C}_n)\}$ respectively.
4. For $m \in \{1, \ldots, n-t\}$, let
   - $\widetilde{\mathbb{A}}^{(m)} = \mathbb{A}(n+m)$, where $\widetilde{\mathbb{A}}^{(m)} = c_{m,1} \cdot \mathbb{A}_1 + \ldots + c_{m,n} \cdot \mathbb{A}_n$.
   - $\widetilde{\mathbb{B}}^{(m)} = \mathbb{B}(n+m)$, where $\widetilde{\mathbb{B}}^{(m)} = c_{m,1} \cdot \mathbb{B}_1 + \ldots + c_{m,n} \cdot \mathbb{B}_n$.
   - $\widetilde{\mathbb{C}}^{(m)} = \mathbb{C}(n+m)$, where $\widetilde{\mathbb{C}}^{(m)} = c_{m,1} \cdot \mathbb{C}_1 + \ldots + c_{m,n} \cdot \mathbb{C}_n$.

   Here $c_{m,1}, \ldots, c_{m,n}$ are publicly-known Lagrange's interpolation coefficients from $\mathbb{F}$.
5. For each $m \in \{1, \ldots, n-t\}$, parties locally compute and output the following
   - ${}^{A}\langle\widetilde{\mathbb{A}}^{(m)}\rangle_d = c_{m,1} \cdot {}^{A}\langle\mathbb{A}_1\rangle_d + \ldots + c_{m,n} \cdot {}^{A}\langle\mathbb{A}_n\rangle_d$.
   - ${}^{A}\langle\widetilde{\mathbb{B}}^{(m)}\rangle_d = c_{m,1} \cdot {}^{A}\langle\mathbb{B}_1\rangle_d + \ldots + c_{m,n} \cdot {}^{A}\langle\mathbb{B}_n\rangle_d$.
   - ${}^{A}\langle\widetilde{\mathbb{C}}^{(m)}\rangle_d = c_{m,1} \cdot {}^{A}\langle\mathbb{C}_1\rangle_d + \ldots + c_{m,n} \cdot {}^{A}\langle\mathbb{C}_n\rangle_d$.

We now prove the properties of the protocol $\Pi_{\mathsf{RandTriples}}$.

Lemma 8.19. *Protocol $\Pi_{\mathsf{RandTriples}}$ outputs $\{({}^{A}\langle\widetilde{\mathbb{A}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{B}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{C}}^{(m)}\rangle_d)\}$, where $m \in \{1, \ldots, n-t\}$ and where each $\widetilde{\mathbb{A}}^{(m)}, \widetilde{\mathbb{B}}^{(m)}, \widetilde{\mathbb{C}}^{(m)} \in \mathbb{F}^{n^5p^2}$, except with a negligible probability.*

Proof. From the correctness and commitment properties of $\Pi_{A\langle\cdot\rangle}$ (Lemma 6.6 and Lemma 6.7), at the end of step 2, except with negligible probability, parties will hold ${}^{A}\langle\cdot\rangle_d$-sharing of one triple on behalf of each party P_i. The lemma now simply follows from the protocol steps. □

Lemma 8.20. *In protocol $\Pi_{\mathsf{RandTriples}}$, the view of the adversary will be independent of $\{({}^{A}\langle\widetilde{\mathbb{A}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{B}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{C}}^{(m)}\rangle_d)\}$, where $m \in \{1, \ldots, n-t\}$ and where each $\widetilde{\mathbb{A}}^{(m)}, \widetilde{\mathbb{B}}^{(m)}, \widetilde{\mathbb{C}}^{(m)} \in \mathbb{F}^{n^5p^2}$.*

Proof. Let $\mathcal{H}$ be the set of honest parties in $\mathcal{P}$, where $|\mathcal{H}| \geq n-t$. For simplicity and without loss of generality, we consider the worst case when $|\mathcal{H}| = n-t$. From the privacy property of $\Pi_{A\langle\cdot\rangle}$ (Lemma 6.4), corresponding to every $P_i \in \mathcal{H}$, the adversary's view remain independent of $\mathbb{A}_i, \mathbb{B}_i, \mathbb{C}_i$ at the end of step 2.

Next consider the vector of degree-$(n-1)$ polynomials $\mathbb{A}(X)$, passing through the vectors of points $(1, \mathbb{A}_1), \ldots, (n, \mathbb{A}_n)$. Among these points, the points $\{\mathbb{A}_i\}_{P_i \in \mathcal{H}}$ are random for the adversary and there are $n-t$ such vectors of points. Hence it follows that the vectors of points $\mathbb{A}(n+1), \ldots, \mathbb{A}(n+n-t)$ will also remain random for the adversary. Specifically, for every candidate value for $\{\mathbb{A}_i\}_{P_i \in \mathcal{H}}$, there exists a vector of unique degree-$(n-1)$ polynomials passing through those candidate points and the vectors of points $\{\mathbb{A}_i\}_{P_i \notin \mathcal{H}}$. Consequently, the vectors of points $\widetilde{\mathbb{A}}^{(1)}, \ldots, \widetilde{\mathbb{A}}^{(n-t)}$ will remain random for the adversary. The same argument holds for $\widetilde{\mathbb{B}}^{(1)}, \ldots, \widetilde{\mathbb{B}}^{(n-t)}$ and $\widetilde{\mathbb{C}}^{(1)}, \ldots, \widetilde{\mathbb{C}}^{(n-t)}$. □

Lemma 8.21. *Protocol $\Pi_{\mathsf{RandTriples}}$ generates $(n-t)n^5p^2$ packed-secret-shared triples from $\mathbb{F}$, incurring a communication of $\mathcal{O}(n^8\kappa)$ bits and makes at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$.*

If the protocol is executed L times in parallel, then it generates $L \cdot (n-t)n^5p^2$ packed-secret-shared triples from $\mathbb{F}$, incurring a communication of $\mathcal{O}(L \cdot n^8)$ bits and makes at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$.

Proof. From Lemma 8.19, $\Pi_{\mathsf{RandTriples}}$ outputs $\{({}^{A}\langle\widetilde{\mathbb{A}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{B}}^{(m)}\rangle_d, {}^{A}\langle\widetilde{\mathbb{C}}^{(m)}\rangle_d)\}$, where $m \in \{1, \ldots, n-t\}$ and where each $\widetilde{\mathbb{A}}^{(m)}, \widetilde{\mathbb{B}}^{(m)}, \widetilde{\mathbb{C}}^{(m)} \in \mathbb{F}^{n^5p^2}$. Thus $(n-t)n^5p^2$ triplets of field elements are finally secret-shared.

In the protocol, each party has to invoke a single instance of $\Pi_{A\langle\cdot\rangle}$ as a dealer during step 2. From Lemma 6.8, this incurs a total communication of $\mathcal{O}(n^8\kappa)$ bits. The calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized across all the dealers and all the instances of $\Pi_{A\langle\cdot\rangle}$. Hence the maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ due to instances of $\Pi_{A\langle\cdot\rangle}$ will be 15. □

Communication Overhead of the Protocol $\Pi_{\mathsf{RandTriples}}$. If the protocol $\Pi_{\mathsf{RandTriples}}$ is executed L times in parallel where $L \geq 1$, then the output of the protocol consists of packed-double-secret-sharing of $L \cdot (n-t)n^5p^2 = \Theta(L \cdot n^8)$ field elements and hence $\Theta(L \cdot n^8\kappa)$ bits. Since the communication complexity

of the protocol will be $\mathcal{O}(L \cdot n^8 \kappa)$ bits, apart from 15 calls to $\mathcal{F}_{\mathsf{Rand}}$, it follows that for sufficiently large L, the asymptotic communication overhead of the protocol is $\mathcal{O}(1)$ bits.

We describe the steps for $\Pi_{\mathsf{RandTriples}}$ that will be used in simulation where $\Pi_{\mathsf{RandTriples}}$ protocol is used as a primitive.

Simulator 8.22: Simulator for the Protocol $\Pi_{\mathsf{RandTriples}}$

- On behalf of every party P_j not under the control of $\mathcal{A}$, randomly select $\mathbb{A}_j, \mathbb{B}_j, \mathbb{C}_j \in \mathbb{F}^{n^5p^2}$ and simulate the steps for VSS for an honest dealer as per Simulator 6.10.
- On behalf of every P_j under the control of $\mathcal{A}$, simulate the steps for VSS for a corrupt dealer as per Simulator 6.10.

8.5 Beaver Multiplication with a Constant Overhead

In this section, we present our multiplication protocol Π_{Beaver} (Protocol 8.23) for securely multiplying vectors of ${}^{A}\langle\cdot\rangle_d$-shared values. We have already explained in detail the high-level overview of the protocol for multiplying one-dimensional vectors of elements. In protocol Π_{Beaver}, we extend that idea for three-dimensional vectors of elements to achieve $\mathcal{O}(1)$ communication overhead.

Suppose parties have inputs ${}^{A}\langle\mathbb{X}^{(\gamma)}\rangle_d$ and ${}^{A}\langle\mathbb{Y}^{(\gamma)}\rangle_d$ and the goal is to securely compute ${}^{A}\langle\mathbb{Z}^{(\gamma)}\rangle_d$, where $\gamma \in \{1, \ldots, t+1\}$ and where component-wise $\mathbb{Z}^{(\gamma)} = \mathbb{X}^{(\gamma)} \times \mathbb{Y}^{(\gamma)}$. Here $\mathbb{X}^{(\gamma)}, \mathbb{Y}^{(\gamma)} \in \mathbb{F}^{n^5p^2}$ and hence we are trying to compute products of $(t+1)n^5p^2$ values in parallel. For this, the parties would be provided with secret-shared random multiplication-triples $({}^{A}\langle\mathbb{A}^{(\gamma)}\rangle_d, {}^{A}\langle\mathbb{B}^{(\gamma)}\rangle_d, {}^{A}\langle\mathbb{C}^{(\gamma)}\rangle_d)$ and double-packed-secret-shared random values $({}^{A}\langle\mathbb{R}^{(\gamma)}\rangle_d, {}^{A}\langle\mathbb{R}^{(\beta)}\rangle_{d+p-1})$ as auxiliary information. Additionally, the parties will be also provided with secret-shared random triples $({}^{A}\langle\hat{\mathbb{A}}^{(\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{B}}^{(\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{C}}^{(\gamma)}\rangle_d)$ and double-packed-secret-shared random values $({}^{A}\langle\hat{\mathbb{R}}^{(\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{R}}^{(\beta)}\rangle_{d+p-1})$ as "sacrificing randomness".

The parties first locally compute ${}^{A}\langle\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}\rangle_d$ and ${}^{A}\langle\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}\rangle_d$. The parties next perform the "degree-reduction" of these sharings. Namely, the parties run the protocol Π_{RoDegRed} on the sharings ${}^{A}\langle\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}\rangle_d$ and ${}^{A}\langle\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}\rangle_d$ and obtain the sharings ${}^{A}\langle\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}\rangle_{p-1}$ and ${}^{A}\langle\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}\rangle_{p-1}$, where the degree of sharing is now $p-1$ instead of d. Note that this will reveal $\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}$ and $\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}$ to the adversary, as adversary will get up to t shares for degree-$(p-1)$ sharing-polynomials. However, the privacy of $\mathbb{X}^{(\gamma)}$ and $\mathbb{Y}^{(\gamma)}$ is still preserved, since $\mathbb{A}^{(\gamma)}$ and $\mathbb{B}^{(\gamma)}$ are random for the adversary. Let $\mathbb{Z}^{(\gamma)} = \mathbb{X}^{(\gamma)} \times \mathbb{Y}^{(\gamma)}$ component-wise. It then follows that the following holds:

$$\mathbb{Z}^{(\gamma)} = (\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}) \times (\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}) - (\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}) \times \mathbb{B}^{(\gamma)} - (\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}) \times \mathbb{A}^{(\gamma)} + \mathbb{C}^{(\gamma)}.$$

It then follows that the parties can locally compute the following over un-authenticated sharings:

$$\langle\mathbb{Z}^{(\gamma)}\rangle_{d+p-1} = \langle\mathbb{X}^{(\gamma)}+\mathbb{A}^{(\gamma)}\rangle_{p-1} \times \langle\mathbb{Y}^{(\gamma)}+\mathbb{B}^{(\gamma)}\rangle_{p-1} - \langle\mathbb{X}^{(\gamma)}+\mathbb{A}^{(\gamma)}\rangle_{p-1} \times \langle\mathbb{B}^{(\gamma)}\rangle_{d} - \langle\mathbb{Y}^{(\gamma)}+\mathbb{B}^{(\gamma)}\rangle_{p-1} \times \langle\mathbb{A}^{(\gamma)}\rangle_{d} + \langle\mathbb{C}^{(\gamma)}\rangle_{d}.$$

Note that the parties *cannot* locally compute ${}^{A}\langle\mathbb{Z}^{(\gamma)}\rangle_{d+p-1}$, as parties cannot compute their respective MAC-tags and MAC-keys locally for ${}^{A}\langle\mathbb{Z}^{(\gamma)}\rangle_{d+p-1}$. Let $\mathbb{S}^{(\gamma)} = \mathbb{Z}^{(\gamma)} + \mathbb{R}^{(\gamma)}$. The parties then locally compute the un-authenticated sharings $\langle\mathbb{S}^{(\gamma)}\rangle_{d+p-1}$ from $\langle\mathbb{Z}^{(\gamma)}\rangle_{d+p-1}$ and $\langle\mathbb{R}^{(\gamma)}\rangle_{d+p-1}$, where the latter can be simply derived from ${}^{A}\langle\mathbb{R}^{(\gamma)}\rangle_{d+p-1}$ by ignoring the MAC-tags and MAC-keys. Next the parties do the degree-reduction and compute the authenticated-sharings ${}^{A}\langle\mathbb{S}^{(\gamma)}\rangle_{p-1}$ from the un-authenticated sharings $\langle\mathbb{S}^{(\gamma)}\rangle_{d+p-1}$, followed by setting ${}^{A}\langle\mathbb{Z}^{(\gamma)}\rangle_d = {}^{A}\langle\mathbb{S}^{(\gamma)}\rangle_{p-1} - {}^{A}\langle\mathbb{R}^{(\gamma)}\rangle_d$. The steps of the degree-reduction will be similar to the protocol Π_{RoDegRed}, except that we now have the challenge to reconstruct degree-$d+p-1$ sharings in the *absence* of MAC-tags.

Let the elements of $\mathbb{S}^{(1)}, \ldots, \mathbb{S}^{(t+1)}$ constitute coefficients of vectors of degree-t polynomials $\mathbb{S}(X)$. The parties then locally compute $\langle\mathbb{S}(1)\rangle_{d+p-1}, \ldots, \langle\mathbb{S}(n)\rangle_{d+p-1}$ and reconstruct $\langle\mathbb{S}(i)\rangle_{d+p-1}$ towards P_i by sending their shares of $\langle\mathbb{S}(i)\rangle_{d+p-1}$ to P_i. As $p = \frac{t}{4} + 1$, we have $d + p - 1 = t + \frac{t}{2}$. Hence P_i tries to error-correct $\frac{t}{2}$ errors among the received shares. If there are at most $\frac{t}{2}$ errors among the received shares, P_i reconstructs $\mathbb{S}(i)$, otherwise P_i detects the presence of more than $\frac{t}{2}$ errors (as the minimum distance of the underlying RS code will be $t + \frac{t}{2} + 1$) and complaints publicly. If there is no complaint from any party, then every P_i can redistribute $\mathbb{S}(i)$ by generating ${}^{A}\langle\mathbb{S}(i)\rangle_{p-1}$. As up to t corrupt parties

may redistribute incorrect values, the parties apply RS error-correction on the redistributed ${}^{A}\langle \mathbb{S}(i)\rangle_{p-1}$-sharings to error-correct up to t errors and output ${}^{A}\langle \mathbb{S}^{(1)}\rangle_{p-1}, \ldots, {}^{A}\langle \mathbb{S}^{(t+1)}\rangle_{p-1}$.

On the other hand, if any P_i complaints about detecting more that $\frac{t}{2}$ errors while reconstructing $\langle \mathbb{S}(i)\rangle_{d+p-1}$, then the parties start a fault-localization process to let the complaining parties identify more than $\frac{t}{2}$ corrupt parties as follows. Let $\hat{\mathbb{S}}^{(\gamma)} = \hat{\mathbb{Z}}^{(\gamma)} + \hat{\mathbb{R}}^{(\gamma)}$, where

$$\hat{\mathbb{Z}}^{(\gamma)} = (\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}) \times (\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}) - (\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}) \times \hat{\mathbb{B}}^{(\gamma)} - (\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}) \times \hat{\mathbb{A}}^{(\gamma)} + \hat{\mathbb{C}}^{(\gamma)}.$$

It then follows that the parties can locally compute $\langle \hat{\mathbb{S}}^{(\gamma)}\rangle_{d+p-1}$. Basically, $\hat{\mathbb{S}}^{(\gamma)}$ is computed similarly as $\mathbb{S}^{(\gamma)}$, except that we now use $(\hat{\mathbb{A}}^{(\gamma)}, \hat{\mathbb{B}}^{(\gamma)}, \hat{\mathbb{C}}^{(\gamma)})$ and $\hat{\mathbb{R}}^{(\gamma)}$ as the randomness. Next, the parties run the steps of degree-reduction for $\langle \hat{\mathbb{S}}^{(\gamma)}\rangle_{d+p-1}$ by letting elements of $\hat{\mathbb{S}}^{(1)}, \ldots, \hat{\mathbb{S}}^{(t+1)}$ to be the coefficients of vectors of degree-t polynomials $\hat{\mathbb{S}}(X)$ and letting each party P_i try to reconstruct $\langle \hat{\mathbb{S}}(i)\rangle_{d+p-1}$ (by performing simultaneous error-correction and detection). This is followed by the parties generating a random value R. The parties then locally compute the authenticated ${}^{A}\langle\cdot\rangle_d$-sharings of $\mathbb{X}^{(\gamma)} + \mathbb{A}^{(\gamma)}$, $\mathbb{Y}^{(\gamma)} + \mathbb{B}^{(\gamma)}$, $\mathbb{A}^{(\gamma)} + R \cdot \hat{\mathbb{A}}^{(\gamma)}$, $\mathbb{B}^{(\gamma)} + R \cdot \hat{\mathbb{B}}^{(\gamma)}$, $\mathbb{C}^{(\gamma)} + R \cdot \hat{\mathbb{C}}^{(\gamma)}$ and $\mathbb{R}^{(\gamma)} + R \cdot \hat{\mathbb{R}}^{(\gamma)}$, for $\gamma \in \{1, \ldots, t+1\}$, followed by reconstructing these authenticated sharings towards the parties P_i who earlier complained about not being able to reconstruct $\mathbb{S}(i)$. Once the above sharings are robustly reconstructed by P_i, it can recompute $\langle \mathbb{S}(i) + R \cdot \hat{\mathbb{S}}(i)\rangle_{d+p-1}$ and identify all the corrupt parties who earlier sent incorrect shares for $\langle \mathbb{S}(i)\rangle_{d+p-1}$ (and possibly for $\langle \hat{\mathbb{S}}(i)\rangle_{d+p-1}$). Then by ignoring the shares of those corrupt parties (which will be more than $\frac{t}{2}$), party P_i error-corrects the remaining errors among the received shares of $\langle \mathbb{S}(i)\rangle_{d+p-1}$, reconstructs $\mathbb{S}(i)$ and redistrubutes ${}^{A}\langle \mathbb{S}(i)\rangle_{p-1}$.

The approach will have a communication overhead of $\mathcal{O}(n)$. This is because if $\Theta(n)$ parties complaint, then we need to reveal $\Theta(n)$ number of ${}^{A}\langle\cdot\rangle$.-sharings to $\Theta(n)$ parties.[20] To achieve $\mathcal{O}(1)$ communication overhead, we run the protocol for "sufficiently many" batches, where in each batch we have $t+1$ products to be computed. If any party complaints for not being able to reconstruct its designated vectors of points for any batch (as part of degree-reduction), then instead of considering *all* the batches, we consider a *single* batch and reveal the appropriate sharings to that complainant party, using which it can reconstruct its designated vectors of points for all the batches. It turns out that taking n^3 batches will suffice to achieve $\mathcal{O}(1)$ communication overhead.

Protocol 8.23: Beaver Multiplication Protocol – Π_{Beaver}

Initialize:

- Parties initialize a set $\mathsf{C} = \emptyset$.
- Each P_i initializes a set $\mathsf{Dp}_i = \emptyset$.

Input:

- Parties hold $({}^{A}\langle \mathbb{X}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{Y}^{(\beta,\gamma)}\rangle_d)$ for each $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{1, \ldots, t+1\}$.
- Parties hold the (actual randomness) multiplication triples $({}^{A}\langle \mathbb{A}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{B}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{C}^{(\beta,\gamma)}\rangle_d)$ and the random pairs $({}^{A}\langle \mathbb{R}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{R}^{(\beta,\gamma)}\rangle_{d+p-1})$ for each $\beta \in \{1, \ldots, n^3\}, \gamma \in \{1, \ldots, t+1\}$.
- Parties hold the (sacrificing randomness) triples $({}^{A}\langle \hat{\mathbb{A}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \hat{\mathbb{B}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \hat{\mathbb{C}}^{(\beta,\gamma)}\rangle_d)$ and the random pairs $({}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)}\rangle_{d+p-1})$ for each $\beta \in \{1, \ldots, n^3\}, \gamma \in \{1, \ldots, t+1\}$.

The protocol:

1. Parties locally compute for each $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{1, \ldots, t+1\}$:
   - ${}^{A}\langle \mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)}\rangle_d = {}^{A}\langle \mathbb{X}^{(\beta,\gamma)}\rangle_d + {}^{A}\langle \mathbb{A}^{(\beta,\gamma)}\rangle_d$.
   - ${}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)}\rangle_d = {}^{A}\langle \mathbb{Y}^{(\beta,\gamma)}\rangle_d + {}^{A}\langle \mathbb{B}^{(\beta,\gamma)}\rangle_d$.
2. For each $\beta \in \{1, \ldots, n^3\}$, invoke the degree reduction protocol Π_{RoDegRed} (Protocol 8.9)
   (a) with inputs $({}^{A}\langle \mathbb{X}^{(\beta,1)} + \mathbb{A}^{(\beta,1)}\rangle_d, \ldots, {}^{A}\langle \mathbb{X}^{(\beta,t+1)} + \mathbb{A}^{(\beta,t+1)}\rangle_d)$ to obtain $({}^{A}\langle \mathbb{X}^{(\beta,1)} + \mathbb{A}^{(\beta,1)}\rangle_{p-1}, \ldots, {}^{A}\langle \mathbb{X}^{(\beta,t+1)} + \mathbb{A}^{(\beta,t+1)}\rangle_{p-1})$.
   (b) with inputs $({}^{A}\langle \mathbb{Y}^{(\beta,1)} + \mathbb{B}^{(\beta,1)}\rangle_d, \ldots, {}^{A}\langle \mathbb{Y}^{(\beta,t+1)} + \mathbb{B}^{(\beta,t+1)}\rangle_d)$ to obtain $({}^{A}\langle \mathbb{Y}^{(\beta,1)} + \mathbb{B}^{(\beta,1)}\rangle_{p-1}, \ldots, {}^{A}\langle \mathbb{Y}^{(\beta,t+1)} + \mathbb{B}^{(\beta,t+1)}\rangle_{p-1})$.
3. Parties locally compute for each $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{1, \ldots, t+1\}$:

[20] Recall that revealing one ${}^{A}\langle\cdot\rangle$.-sharing to a single party has $\mathcal{O}(1)$ communication overhead.

– $\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_{d+p-1} = \langle \mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)} \rangle_{p-1} \times \langle \mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)} \rangle_{p-1} - \langle \mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)} \rangle_{p-1} \times \langle \mathbb{B}^{(\beta,\gamma)} \rangle_d - \langle \mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)} \rangle_{p-1} \times \langle \mathbb{A}^{(\beta,\gamma)} \rangle_d + \langle \mathbb{C}^{(\beta,\gamma)} \rangle_d$.
– $\langle \mathbb{Z}^{(\beta,\gamma)} + \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1} = \langle \mathbb{Z}^{(\beta,\gamma)} \rangle_{d+p-1} + \langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1}$ and let $\mathbb{S}^{(\beta,\gamma)} = \mathbb{Z}^{(\beta,\gamma)} + \mathbb{R}^{(\beta,\gamma)}$.

(Non-robust Degree Reduction - Attempt)

4. For each $\beta \in \{1, \ldots, n^3\}$, parties define a vector of n^5p^2 degree-t polynomials $\mathbb{S}^{(\beta)}(X) = \mathbb{S}^{(\beta,1)} + \mathbb{S}^{(\beta,2)} \cdot X + \ldots + \mathbb{S}^{(\beta,t+1)} \cdot X^t$.
5. For each $\beta \in \{1, \ldots, n^3\}$ and $i \in \{1, \ldots, n\}$, parties locally compute $\langle \mathbb{S}^{(\beta)}(i) \rangle_{d+p-1} = \langle \mathbb{S}^{(\beta,1)} \rangle_{d+p-1} + i \cdot \langle \mathbb{S}^{(\beta,2)} \rangle_{d+p-1} + \ldots + i^t \cdot \langle \mathbb{S}^{(\beta,t+1)} \rangle_{d+p-1}$.
6. For each $\beta \in \{1, \ldots, n^3\}$ and $i \in \{1, \ldots, n\}$, parties reconstruct $\mathbb{S}^{(\beta)}(i)$ towards P_i by sending their respective shares of $\langle \mathbb{S}^{(\beta)}(i) \rangle_{d+p-1}$ to P_i. Party P_i applies the error-correction algorithm of RS codes on the shares received from each P_j and try to error-correct $\frac{t}{2}$ errors. If P_i does not reconstruct degree-$(d+p-1)$ polynomials using the above process, then it sends $\mathsf{complain}(i, B_i)$ to $\mathcal{F}_{\mathsf{BC}}$ where $B_i = \{\beta |$ reconstruction failed for $\mathbb{S}^{(\beta)}(i)\}$.
7. Add party P_i to the set C if $\mathsf{complain}(i, B_i)$ is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender.
8. Let $B = \cup_i B_i$ for each $P_i \in \mathsf{C}$. For each $\beta \in \{1, \ldots, n^3\}$, parties define $\mathsf{C}^{(\beta)} = \{P_i \in \mathsf{C} \mid \beta \in B_i\}$.
9. If $|\mathsf{C}^{(\beta)}| = 0$ for each $\beta \in \{1, \ldots, n^3\}$, then proceed to Step 14a. Otherwise, let β_i be the smallest value of β for which P_i failed the reconstruction and go to the next step.

(Complaint Resolution and Fault Identification)

10. For every $P_i \in \mathsf{C}$, parties do the following
    – for each $\gamma \in \{1, \ldots, t+1\}$ locally compute:
      • $\langle \hat{\mathbb{Z}}^{(\beta_i,\gamma)} \rangle_{d+p-1} = \langle \mathbb{X}^{(\beta_i,\gamma)} + \mathbb{A}^{(\beta_i,\gamma)} \rangle_{p-1} \times \langle \mathbb{Y}^{(\beta_i,\gamma)} + \mathbb{B}^{(\beta_i,\gamma)} \rangle_{p-1} - \langle \mathbb{X}^{(\beta_i,\gamma)} + \mathbb{A}^{(\beta_i,\gamma)} \rangle_{p-1} \times \langle \hat{\mathbb{B}}^{(\beta_i,\gamma)} \rangle_d - \langle \mathbb{Y}^{(\beta_i,\gamma)} + \mathbb{B}^{(\beta_i,\gamma)} \rangle_{p-1} \times \langle \hat{\mathbb{A}}^{(\beta_i,\gamma)} \rangle_d + \langle \hat{\mathbb{C}}^{(\beta_i,\gamma)} \rangle_d$ and
      • $\langle \hat{\mathbb{Z}}^{(\beta_i,\gamma)} + \hat{\mathbb{R}}^{(\beta_i,\gamma)} \rangle_{d+p-1} = \langle \hat{\mathbb{Z}}^{(\beta_i,\gamma)} \rangle_{d+p-1} + \langle \hat{\mathbb{R}}^{(\beta_i,\gamma)} \rangle_{d+p-1}$ and let $\hat{\mathbb{S}}^{(\beta_i,\gamma)} = \hat{\mathbb{Z}}^{(\beta_i,\gamma)} + \hat{\mathbb{R}}^{(\beta_i,\gamma)}$.
    – define a vector of n^5p^2 degree-t polynomials $\hat{\mathbb{S}}^{(\beta_i)}(X) = \hat{\mathbb{S}}^{(\beta_i,1)} + \hat{\mathbb{S}}^{(\beta_i,2)} \cdot X + \ldots + \hat{\mathbb{S}}^{(\beta_i,t+1)} \cdot X^t$.
    – locally compute $\langle \hat{\mathbb{S}}^{(\beta_i)}(i) \rangle_{d+p-1} = \langle \hat{\mathbb{S}}^{(\beta_i,1)} \rangle_{d+p-1} + i \cdot \langle \hat{\mathbb{S}}^{(\beta_i,2)} \rangle_{d+p-1} + \ldots + i^t \cdot \langle \hat{\mathbb{S}}^{(\beta_i,t+1)} \rangle_{d+p-1}$.
    – send their respective shares of $\langle \hat{\mathbb{S}}^{(\beta_i)}(i) \rangle_{d+p-1}$ to P_i.
11. Every party sends Rand to $\mathcal{F}_{\mathsf{Rand}}$ and receives R from it.
12. For each $P_i \in \mathsf{C}$, parties robustly reconstruct the following towards only P_i, for every $\gamma \in \{1, \ldots, t+1\}$:
    – ${}^A\langle \mathbb{X}^{(\beta_i,\gamma)} + \mathbb{A}^{(\beta_i,\gamma)} \rangle_d$, ${}^A\langle \mathbb{Y}^{(\beta_i,\gamma)} + \mathbb{B}^{(\beta_i,\gamma)} \rangle_d$
    – ${}^A\langle \mathbb{A}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{A}}^{(\beta_i,\gamma)} \rangle_d$, ${}^A\langle \mathbb{B}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{B}}^{(\beta_i,\gamma)} \rangle_d$, ${}^A\langle \mathbb{C}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{C}}^{(\beta_i,\gamma)} \rangle_d$
    – ${}^A\langle \mathbb{R}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{R}}^{(\beta_i,\gamma)} \rangle_d$
13. Every $P_i \in \mathsf{C}$ does the following.
    – For $\gamma \in \{1, \ldots, t+1\}$,
      (a) recompute $\langle \mathbb{X}^{(\beta_i,\gamma)} + \mathbb{A}^{(\beta_i,\gamma)} \rangle_d$, $\langle \mathbb{Y}^{(\beta_i,\gamma)} + \mathbb{B}^{(\beta_i,\gamma)} \rangle_d$, $\langle \mathbb{A}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{A}}^{(\beta_i,\gamma)} \rangle_d$, $\langle \mathbb{B}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{B}}^{(\beta_i,\gamma)} \rangle_d$, $\langle \mathbb{C}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{C}}^{(\beta_i,\gamma)} \rangle_d$ and $\langle \mathbb{R}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{R}}^{(\beta_i,\gamma)} \rangle_d$.[21]
      (b) compute $\langle \mathbb{S}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{S}}^{(\beta_i,\gamma)} \rangle_{d+p-1} = \langle \mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)} \rangle_{p-1} \times \langle \mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)} \rangle_{p-1} + R \cdot \langle \mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)} \rangle_{p-1} \times \langle \mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)} \rangle_{p-1} - \langle \mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)} \rangle_{p-1} \times \langle \mathbb{B}^{(\beta,\gamma)} + R \cdot \hat{\mathbb{B}}^{(\beta,\gamma)} \rangle_d - \langle \mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)} \rangle_{p-1} \times \langle \mathbb{A}^{(\beta,\gamma)} + R \cdot \hat{\mathbb{A}}^{(\beta,\gamma)} \rangle_d + \langle \hat{\mathbb{C}}^{(\beta,\gamma)} + R \cdot \hat{\mathbb{C}}^{(\beta,\gamma)} \rangle_d + \langle \mathbb{R}^{(\beta,\gamma)} + R \cdot \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_d$.
      (c) Compute $\langle \mathbb{S}^{(\beta)}(i) + R \cdot \hat{\mathbb{S}}^{(\beta)}(i) \rangle_{d+p-1} = \langle (\mathbb{S}^{(\beta,1)} + R \cdot \hat{\mathbb{S}}^{(\beta,1)}) \rangle_{d+p-1} + i \cdot \langle (\mathbb{S}^{(\beta,2)} + R \cdot \hat{\mathbb{S}}^{(\beta,2)}) \rangle_{d+p-1} + \ldots + i^t \cdot \langle (\mathbb{S}^{(\beta,t+1)} + R \cdot \hat{\mathbb{S}}^{(\beta,t+1)}) \rangle_{d+p-1}$.
    – For every P_j, use the shares sent by P_j during Step 6 and Step 10 to recompute the share of P_j for $\langle \mathbb{S}^{(\beta)}(i) + R \cdot \hat{\mathbb{S}}^{(\beta)}(i) \rangle_{d+p-1}$ and compare it with the shares of P_j in $\langle \mathbb{S}^{(\beta)}(i) + \hat{\mathbb{S}}^{(\beta)}(i) \rangle_{d+p-1}$ computed in the previous step. If the shares are different then $\mathsf{Dp}_i = \mathsf{Dp}_i \cup \{P_j\}$.
    – Recover $\mathbb{S}^{(\beta)}(i)$ from the shares of $\langle \mathbb{S}^{(\beta)}(i) \rangle_{d+p-1}$ received from the parties outside Dp_i during Step 6 by error correcting at most $t - |\mathsf{Dp}_i|$ corruptions. %Dp_i will contain more $t/2$ error for an honest P_i. So $t - |\mathsf{Dp}_i|$ will be less than $\frac{t}{2}$.

(Output Computation)

[21] See Section 4.3 for how P_i can recompute $\langle \mathbb{S} \rangle_d$ corresponding to ${}^A\langle \mathbb{S} \rangle_d$ after robustly reconstructing $\mathbb{S}$.

14. For every $\beta \in \{1, \ldots, n^3\}$,
    (a) each P_i generates ${}^{A}\langle \mathbb{S}^{(\beta)}(i) \rangle_{p-1}$ by acting as a dealer and invoking an instance of the protocol $\Pi_{{}^{A}\langle\cdot\rangle}$ (Protocol 6.1) with input $\mathbb{S}^{(\beta)}(i)$.
    (b) parties apply the RS-decoding procedure on $\{{}^{A}\langle \mathbb{S}^{(\beta)}(i) \rangle_{p-1}\}_{P_i \in \mathcal{P}}$ to correct t errors and compute $({}^{A}\langle \mathbb{S}^{(\beta,1)} \rangle_{p-1}, \ldots, {}^{A}\langle \mathbb{S}^{(\beta,t+1)} \rangle_{p-1}) = ({}^{A}\langle \mathbb{Z}^{(\beta,1)} + \mathbb{R}^{(\beta,1)} \rangle_{p-1}, \ldots, {}^{A}\langle \mathbb{Z}^{(\beta,t+1)} + \mathbb{R}^{(\beta,t+1)} \rangle_{p-1})$.
    (c) for each $\gamma \in \{1, \ldots, t+1\}$, parties output ${}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_{d} = {}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} + \mathbb{R}^{(\beta,\gamma)} \rangle_{p-1} - {}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d}$.

Claim 8.24. *In protocol Π_{Beaver} if P_i is honest, and if P_i is included in C, then at least $\frac{t}{2}+1$ corrupt parties are included by P_i in Dp_i at the end of Complaint Resolution and Fault Identification, except with a negligible probability.*

Proof. Let P_i be an arbitrary honest party and let $P_i \in \mathsf{C}$. This implies that P_i broadcasted $\mathsf{complain}(i, B_i)$ during Step 6. This further implies that there exists some batch $\beta_i \in B_i$ with β_i being the smallest index in B_i, such that party P_i failed to robustly reconstruct $\mathbb{S}^{(\beta)}(i)$. This happened because at least $\frac{t}{2}+1$ corrupt parties sent incorrect shares of $\langle \mathbb{S}^{(\beta_i)}(i) \rangle_{d+p-1}$ while reconstructing $\mathbb{S}^{(\beta_i)}(i)$ towards P_i. If this is not true then P_i would have error-correct all the incorrect shares of $\langle \mathbb{S}^{(\beta_i)}(i) \rangle_{d+p-1}$ and robustly reconstruct $\mathbb{S}^{(\beta_i)}(i)$, implying $\beta_i \notin B_i$, which is a contradiction. This also means that $\beta_i \in B_i$ is chosen as the smallest index for which the Complaint Resolution is performed corresponding to party P_i. During step 12, party P_i robustly reconstructs $\mathbb{X}^{(\beta_i,\gamma)} + \mathbb{A}^{(\beta_i,\gamma)}$, $\mathbb{Y}^{(\beta_i,\gamma)} + \mathbb{B}^{(\beta_i,\gamma)}$, $\mathbb{A}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{A}}^{(\beta_i,\gamma)}$, $\mathbb{B}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{B}}^{(\beta_i,\gamma)}$, $\mathbb{C}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{C}}^{(\beta_i,\gamma)}$ and $\mathbb{R}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{R}}^{(\beta_i,\gamma)}$, where $\gamma \in \{1, \ldots, t+1\}$, except with a negligible probability, from the ${}^{A}\langle\cdot\rangle_{d}$-sharings of these values. This is because P_i will be able to identify the correct shares due to the presence of MAC-tags. Moreover, P_i will be able to recompute the entire ${}^{A}\langle\cdot\rangle_{d}$-sharing of each of these values. Consequently, P_i will be able to compute the entire sharing $\langle \mathbb{S}^{(\beta_i)} + R \cdot \hat{\mathbb{S}}^{(\beta_i)} \rangle_{d+p-1}$ during step 13. This is because $\langle \mathbb{S}^{(\beta_i)} + R \cdot \hat{\mathbb{S}}^{(\beta_i)} \rangle_{d+p-1}$ is computed deterministically from the ${}^{A}\langle\cdot\rangle_{d}$-sharings recomputed by P_i. Then by comparing the shares of each party in the recomputed $\langle \mathbb{S}^{(\beta_i)} + R \cdot \hat{\mathbb{S}}^{(\beta_i)} \rangle_{d+p-1}$ with the shares received from various parties during step 6 and step 10, party P_i will be able to identify all corrupt parties who sent incorrect shares to P_i in step 6 during the reconstruction of $\mathbb{S}^{(\beta_i)}$ and include them in Dp_i. Since more than $\frac{t}{2}+1$ corrupt parties sent incorrect shares to P_i during step 6, it follows that $|\mathsf{Dp}_i| \geq \frac{t}{2}+1$. □

Based on Claim 8.24, we next show that for any *honest* party that complains, its complaints are resolved and it will reconstruct $\mathbb{S}^{(\beta)}(i)$ for each $\beta \in \{1, \ldots, n^3\}$.

Claim 8.25. *In protocol Π_{Beaver} if P_i is honest, and $P_i \in \mathsf{C}$ then by the end of the Complaint Resolution and Fault Identification phase, P_i will reconstruct $\mathbb{S}^{(\beta)}(i)$ for each $\beta \in \{1, \ldots, n^3\}$, except with a negligible probability.*

Proof. Let P_i be an arbitrary honest party such that $P_i \in \mathsf{C}$. Then from Claim 8.24, except with a negligible probability, party P_i includes at least $\frac{t}{2}+1$ corrupt parties in the set Dp_i during Step 13.

To see that P_i will be able to robustly reconstruct each $\mathbb{S}^{(\beta)}(i)$, consider any arbitrary vector $\hat{\mathbf{s}} \in \mathbb{S}^{(\beta)}(i)$ where $\hat{\mathbf{s}} \in \mathbb{F}^p$. Note that as part of $\langle \mathbb{S}^{(\beta)}(i) \rangle_{d+p-1}$, the vector $\hat{\mathbf{s}}$ is degree-$(d+p-1)$-packed-secret-shared and the parties hold $\langle \hat{\mathbf{s}} \rangle_{d+p-1}$. During step 6, while reconstructing $\mathbb{S}^{(\beta)}(i)$ towards P_i, every party will send their respective share of $\langle \hat{\mathbf{s}} \rangle_{d+p-1}$ to P_i. Note that $|\mathsf{Dp}_i| \geq \frac{t}{2}+1$ and hence P_i will ignore the shares of at least $\frac{t}{2}+1$ corrupt parties. Then among the remaining shares, there could be at most $\frac{t}{2}-1$ unknown corrupt shares. Moreover, the underlying sharing-polynomial of $\langle \hat{\mathbf{s}} \rangle_{d+p-1}$ is of degree-$(t+\frac{t}{2})$. Since $n = 3t+1$, it follows that P_i can easily error-correct the remaining $\frac{t}{2}-1$ incorrect shares by using RS error-correction and robustly reconstruct the vector $\hat{\mathbf{s}}$. □

Now based on all the previous results, we show that protocol Π_{Beaver} will generate the desired outcome and terminate.

Lemma 8.26. *Given inputs $\{{}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_{d}, {}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_{d}\}$ and auxiliary inputs $\{{}^{A}\langle \mathbb{A}^{(\beta,\gamma)} \rangle_{d}, {}^{A}\langle \mathbb{B}^{(\beta,\gamma)} \rangle_{d}, {}^{A}\langle \mathbb{C}^{(\beta,\gamma)} \rangle_{d}\}$, $\{{}^{A}\langle \hat{\mathbb{A}}^{(\beta,\gamma)} \rangle_{d}, {}^{A}\langle \hat{\mathbb{B}}^{(\beta,\gamma)} \rangle_{d}, {}^{A}\langle \hat{\mathbb{C}}^{(\beta,\gamma)} \rangle_{d}\}$, $\{{}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d}, {}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1}\}$ and $\{{}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_{d}, {}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_{d+p-1}\}$. Then except with a negligible probability, Π_{Beaver} generates $\{{}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_{d}\}$, where $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$ component-wise, if and only if $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ constitute multiplication-triples component-wise. Here $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{1, \ldots, t+1\}$.*

Proof. From Claim 8.25, we have that each honest party reconstructs its $\mathbb{S}^{(\beta)}(i)$ for each $\beta \in \{1,\ldots,n^3\}$ either during Step 6 or after Complaint Resolution and Fault Identification during Step 13. Consequently, it must hold that all parties execute step 14a of the protocol, where every P_i acts as dealer and supposedly generates ${}^{A}\langle \mathbb{S}^{(\beta)}(i)\rangle_{p-1}$ through $\Pi_{{}^{A}\langle\cdot\rangle}$ by acting as a dealer. Note that if P_i is corrupt then it might generate ${}^{A}\langle\cdot\rangle_{p-1}$-sharings of incorrect $\mathbb{S}^{(\beta)}(i)$ and there could be up to t such corrupt parties for each $\beta \in \{1,\ldots,n^3\}$. Notice that for each $\beta \in \{1,\ldots,n^3\}$, $\{\mathbb{S}^{(\beta)}(i)\}_{P_i\in\mathcal{P}}$ supposedly constitute $3t+1$ distinct points on vectors of degree-t polynomials $\mathbb{S}^{(\beta)}(X)$. So even if up to t elements in $\{\mathbb{S}^{(\beta)}(i)\}_{P_i\in\mathcal{P}}$ are incorrect, they can be error-corrected using RS error-correction on $\{\mathbb{S}^{(\beta)}(i)\}_{P_i\in\mathcal{P}}$, which will output polynomials $\mathbb{S}^{(\beta)}(X)$ and hence its coefficients $\mathbb{S}^{(\beta,1)},\ldots,\mathbb{S}^{(\beta,t+1)}$. Since parties hold $\{{}^{A}\langle \mathbb{S}^{(\beta)}(i)\rangle_{p-1}\}_{P_i\in\mathcal{P}}$ instead of $\{\mathbb{S}^{(\beta)}(i)\}_{P_i\in\mathcal{P}}$ and since the RS error-correction process involves performing linear operations on $\{\mathbb{S}^{(\beta)}(i)\}_{P_i\in\mathcal{P}}$, it follows that by applying the same linear operations on $\{{}^{A}\langle \mathbb{S}^{(\beta)}(i)\rangle_{p-1}\}_{P_i\in\mathcal{P}}$ during step 14b, parties will compute ${}^{A}\langle \mathbb{S}^{(\beta,1)}\rangle_{p-1},\ldots,{}^{A}\langle \mathbb{S}^{(\beta,t+1)}\rangle_{p-1}$, followed by outputting ${}^{A}\langle \mathbb{Z}^{(\beta,1)}\rangle_{p-1},\ldots,{}^{A}\langle \mathbb{Z}^{(\beta,t+1)}\rangle_{p-1}$ during step 14c.

Now we have that the output of the protocol will be $\{{}^{A}\langle \mathbb{Z}^{(\beta,\gamma)}\rangle_{d}\}$, where $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$. To complete the proof, we just need to show that component-wise $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$ holds, if and only if $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ constitute multiplication-triples component-wise; i.e. $\mathbb{C}^{(\beta,\gamma)} = \mathbb{A}^{(\beta,\gamma)} \times \mathbb{B}^{(\beta,\gamma)}$ holds. However, this simply follows from the fact that $\mathbb{Z}^{(\beta,\gamma)} = (\mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)}) \times (\mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)}) - (\mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)}) \times \mathbb{B}^{(\beta,\gamma)} - (\mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)}) \times \mathbb{A}^{(\beta,\gamma)} + \mathbb{C}^{(\beta,\gamma)}$. Hence if $\mathbb{C}^{(\beta,\gamma)} = \mathbb{A}^{(\beta,\gamma)} \times \mathbb{B}^{(\beta,\gamma)}$, then $\mathbb{Z}^{(\beta,\gamma)} = (\mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)} - \mathbb{A}^{(\beta,\gamma)}) \times (\mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)} - \mathbb{B}^{(\beta,\gamma)})$, which further implies that $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$. On the other hand, if $\mathbb{C}^{(\beta,\gamma)} \neq \mathbb{A}^{(\beta,\gamma)} \times \mathbb{B}^{(\beta,\gamma)}$ component-wise then clearly $\mathbb{Z}^{(\beta,\gamma)} \neq \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$.

□

We next show the privacy of the protocol Π_{Beaver}.

Lemma 8.27. *In protocol Π_{Beaver}, the view of the adversary remains independent of $\{\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}\}$, provided $\{\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)}, \hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)}, \mathbb{R}^{(\beta,\gamma)}, \hat{\mathbb{R}}^{(\beta,\gamma)}\}$ are random for the adversary. Here $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$.*

Proof. During step 2, adversary completely learns $\mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)}$ and $\mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)}$. Since $\mathbb{A}^{(\beta,\gamma)}$ and $\mathbb{B}^{(\beta,\gamma)}$ are random for the adversary, it follows that $\mathbb{X}^{(\beta,\gamma)}$ and $\mathbb{Y}^{(\beta,\gamma)}$ still remain random for the adversary.

During step 6 and step 14c, adversary may learn $t+1$ points on the vector of polynomials $\mathbb{S}^{(\beta)}(X)$. However, even learning the whole vector of polynomials $\mathbb{S}^{(\beta)}(X)$ does not reveal additional information to the adversary. This is because, from $\mathbb{S}^{(\beta)}(X)$, the adversary may learn each $\mathbb{S}^{(\beta,\gamma)}$. However, $\mathbb{S}^{(\beta,\gamma)} = \mathbb{Z}^{(\beta,\gamma)} + \mathbb{R}^{(\beta,\gamma)}$. Since $\mathbb{R}^{(\beta,\gamma)}$ is random for the adversary, it follows that $\mathbb{Z}^{(\beta,\gamma)}$ remains random for the adversary.

Further, for each corrupt party P_i, for some specific index β_i, the adversary may additionally learn $\hat{\mathbb{S}}^{(\beta_i)}(X)$ and hence each $\hat{\mathbb{S}}^{(\beta_i,\gamma)}$. However, $\hat{\mathbb{S}}^{(\beta_i,\gamma)} = \hat{\mathbb{Z}}^{(\beta_i,\gamma)} + \hat{\mathbb{R}}^{(\beta,\gamma)}$ and since $\hat{\mathbb{R}}^{(\beta,\gamma)}$ is random for the adversary, it follows that $\hat{\mathbb{Z}}^{(\beta_i,\gamma)}$ remains random from the adversary.

Following this, the adversary may learn $\mathbb{X}^{(\beta_i,\gamma)} + \mathbb{A}^{(\beta_i,\gamma)}$, $\mathbb{Y}^{(\beta_i,\gamma)} + \mathbb{B}^{(\beta_i,\gamma)}$, $\mathbb{A}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{A}}^{(\beta_i,\gamma)}$, $\mathbb{B}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{B}}^{(\beta_i,\gamma)}$, $\mathbb{C}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{C}}^{(\beta_i,\gamma)}$, and $\mathbb{R}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{R}}^{(\beta_i,\gamma)}$ in Step 13. In this case, the adversary can learn $\mathbb{Z}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{Z}}^{(\beta_i,\gamma)} = (\mathbb{S}^{(\beta_i,\gamma)} + R \cdot \hat{\mathbb{S}}^{(\beta_i,\gamma)}) - (\mathbb{R}^{(\beta_i,\gamma,\delta_r)} + R \cdot \hat{\mathbb{R}}^{(\beta_i,\gamma,\delta_r)})$. However, $\mathbb{Z}^{(\beta_i,\gamma)}$ remains random for the adversary since $\mathbb{A}^{(\beta_i,\gamma)}, \mathbb{B}^{(\beta_i,\gamma)}$ is random and unknown to the adversary.

□

We next do the complexity analysis of the protocol Π_{Beaver}.

Lemma 8.28. *Given inputs ${}^{A}\langle \mathbb{X}^{(\beta,\gamma)}\rangle_{d}, {}^{A}\langle \mathbb{Y}^{(\beta,\gamma)}\rangle_{d}$ where $\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$, $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$, protocol Π_{Beaver} incurs a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^{11}\kappa)$ bits to generate outputs ${}^{A}\langle \mathbb{Z}^{(\beta,\gamma)}\rangle_{d}$, apart from at most 31 number of calls to $\mathcal{F}_{\mathsf{Rand}}$.*

If the protocol is executed L times in parallel where $L \geq 1$, then it computes the product of $L \cdot n^3(t+1)n^5p^2$ pairs of elements over $\mathbb{F}$, incurring a communication of $\mathcal{O}(L \cdot n^{11}\kappa)$ bits and makes at most 31 calls to $\mathcal{F}_{\mathsf{Rand}}$.

Proof. We analyze the round and communication complexity of each stage individually.

Consider the first stage (namely steps 1-3). In step 1, the parties perform only local computation. In step 2, the parties execute $2n^3$ instances of the protocol Π_{RoDegRed}, each with $t+1$ number of ${}^{A}\langle\cdot\rangle_{d}$-sharings of n^5p^2 elements from $\mathbb{F}$. From Lemma 8.11, this incurs a constant expected number of rounds

and a communication of $\mathcal{O}(n^{11}\kappa)$ bits, apart from at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$. Note that the instances of Π_{RoDegRed} for $\mathbb{X}^{(\beta,\gamma)} + \mathbb{A}^{(\beta,\gamma)}$ and $\mathbb{Y}^{(\beta,\gamma)} + \mathbb{B}^{(\beta,\gamma)}$ can be executed in parallel and so the number of calls to $\mathcal{F}_{\mathsf{Rand}}$ across both the steps remain at most 15. Finally step 3 requires only local computation. Hence the first stage will incur a constant expected number of rounds and a *total* communication of $\mathcal{O}(n^{11}\kappa)$ bits, apart from at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$.

Next consider the second stage (namely steps 4-9). Step 4-5 involves only local computation. In step 6, each party P_i need to reconstruct n^3 number of $\langle\cdot\rangle_{d+p-1}$-sharings of n^5p^2 values. For *one* P_i this requires a single round and a communication of $\mathcal{O}(n^{10}\kappa)$ bits and hence for *all* the parties in $\mathcal{P}$ this requires a total communication of $\mathcal{O}(n^{11}\kappa)$ bits. Additionally, each P_i may broadcast a set B_i, which can be encoded as a Boolean vector of size n^3. This will require a communication of $n \times \mathcal{BC}(n^3)$ bits. Finally steps 7-9 does not require any interaction. Hence the second stage of the protocol will incur 1 round and a *total* communication of $\mathcal{O}(n^{11}\kappa)$ bits and $n \times \mathcal{BC}(n^3)$ bits.

Next consider the third stage (namely steps 10-13). In step 10, each P_i may need to reconstruct *exactly one* $\langle\cdot\rangle_{d+p-1}$-sharing of n^5p^2 elements from $\mathbb{F}$. This requires one round and a communication of $\mathcal{O}(n^8\kappa)$ bits. Step 11 involves a single invocation of $\mathcal{F}_{\mathsf{Rand}}$. In Step 12, each P_i may need to reconstruct *exactly* $6 \cdot (t+1)$ number of ${}^{A}\langle\cdot\rangle_d$-sharings of n^5p^2 elements from $\mathbb{F}$. This will require a single round and a communication of $\mathcal{O}(n^8\kappa)$ bits for P_i and hence for all the parties this step will require one round and a communication of $\mathcal{O}(n^9\kappa)$ bits. Finally, step 13 does not require any communication. So this stage of the protocol will require 2 rounds and a total communication of $\mathcal{O}(n^9\kappa)$ bits, apart from a single call to $\mathcal{F}_{\mathsf{Rand}}$.

Finally consider the last stage (steps 14a-14c) where interaction happens only during step 14a. Here each P_i may need to generate n^3 number of ${}^{A}\langle\cdot\rangle_{p-1}$-sharings of n^5p^2 values. For this, P_i needs to run n^3 instances of $\Pi_{{}^{A}\langle\cdot\rangle}$. From Lemma 6.8, this requires a constant expected number of rounds and a communication of $\mathcal{O}(n^{10}\kappa)$ bits, apart from at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$ for P_i. Since every party can run its designated instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ in parallel and since this stage is executed exactly once, this will require a constant expected number of rounds and a total communication of $\mathcal{O}(n^{11}\kappa)$ bits. The number of calls to $\mathcal{F}_{\mathsf{Rand}}$ remains 15 for this stage, as the calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized for various dealers.

By securely realizing the calls to $\mathcal{F}_{\mathsf{BC}}$ using the reliable broadcast protocol of [10], protocol Π_{Beaver} requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^{11}\kappa)$ bits. Moreover, there will be at most 31 calls to $\mathcal{F}_{\mathsf{Rand}}$. □

Communication Overhead of Protocol Π_{Beaver}. If the protocol Π_{Beaver} is executed L times in parallel where $L \geq 1$, then the output of the protocol consists of the product of $L \cdot n^3(t+1)n^5p^2$ pairs of elements over $\mathbb{F}$ and hence $\Theta(L \cdot n^{11}\kappa)$ bits. Since the communication complexity of the protocol will be $\mathcal{O}(L \cdot n^{11}\kappa)$ bits, apart from at most 31 calls to $\mathcal{F}_{\mathsf{Rand}}$, it follows that for sufficiently large L, the asymptotic communication overhead of the protocol is $\mathcal{O}(1)$ bits.

Although we do not model the requirements of Π_{Beaver} via a functionality, we describe the simulation steps for Π_{Beaver} which will be used when the protocol is used as a primitive.

Simulator 8.29: Simulator for the Protocol Π_{Beaver}

- On behalf of each honest party P_i, the initialize $\mathsf{C} = \emptyset$ and $\mathsf{Dp}_i = \phi$.
- On behalf of each honest party, perform the local computations on the shares as an honest party would and simulate Π_{RoDegRed} as in Simulation 8.12 using the shares already chosen on behalf of the honest parties.
- For $\beta \in \{1, \ldots, n^3\}$, send shares of $\langle \mathbb{S}^{(\beta)}(k) \rangle_{d+p-1}$ to each P_k under the control of $\mathcal{A}$ to allow it to reconstruct $\mathbb{S}^{(\beta)}(k)$ towards P_i. Receive from $\mathcal{A}$, the shares corresponding to $\langle \mathbb{S}^{(\beta)}(i) \rangle_{d+p-1}$ for each honest party P_i not under the control of $\mathcal{A}$.
- If more than $\frac{t}{2}$ erroneous shares are received from $\mathcal{A}$ corresponding to some honest party P_i, then emulate $\mathcal{F}_{\mathsf{BC}}$, and send $\mathsf{complain}(i, B_i)$ to each P_k under the control of $\mathcal{A}$. Additionally, if $\mathsf{complain}(k, B_k)$ is received from $\mathcal{A}$ corresponding to some corrupt party P_k, then emulate $\mathcal{F}_{\mathsf{BC}}$ and send it to all.
- Compute the steps locally on behalf of the honest parties as per the protocol specification.
- Receives Rand from $\mathcal{A}$ and emulate $\mathcal{F}_{\mathsf{Rand}}$ by sending R to all.
- For each $P_k \in \mathsf{C}$ such that P_k is under the control of $\mathcal{A}$, send the honest parties' shares corresponding to the following to $\mathcal{A}$:
  - ${}^{A}\langle \mathbb{X}^{(\beta_k,\gamma)} + \mathbb{A}^{(\beta_k,\gamma)} \rangle_d$, ${}^{A}\langle \mathbb{Y}^{(\beta_k,\gamma)} + \mathbb{B}^{(\beta_k,\gamma)} \rangle_d$

- $^{A}\langle \mathbb{A}^{(\beta_k,\gamma)} + R \cdot \hat{\mathbb{A}}^{(\beta_k,\gamma)} \rangle_d$, $^{A}\langle \mathbb{B}^{(\beta_k,\gamma)} + R \cdot \hat{\mathbb{B}}^{(\beta_k,\gamma)} \rangle_d$, $^{A}\langle \mathbb{C}^{(\beta_k,\gamma)} + R \cdot \hat{\mathbb{C}}^{(\beta_k,\gamma)} \rangle_d$
- $^{A}\langle \mathbb{R}^{(\beta_k,\gamma)} + R \cdot \hat{\mathbb{R}}^{(\beta_k,\gamma)} \rangle_d$

Also receive the shares corresponding to each honest $P_i \in \mathsf{C}$ from $\mathcal{A}$.

– Compute the steps on behalf of the honest parties as per the protocol specification.
– On behalf of each honest P_i, simulate the steps of VSS for an honest dealer as in Simulator 6.10 and for each P_k under the control of $\mathcal{A}$, simulate the steps of VSS for a corrupt dealer as in Simulator 6.10.

8.6 Triple Sharing with a Constant Overhead

Our next protocol Π_{TripSh} (Protocol 8.30) allows a dealer D to verifiably secret-share random multiplication-triples. The verifiability here guarantees that even if D is *corrupt*, it has secret-shared multiplication-triples, except with a negligible probability. Moreover, if the dealer is *honest*, then the privacy of dealer's triples in maintained. The high-level overview of the protocol has already been discussed in Section 2.1.2 for one-dimensional vectors of elements. In protocol Π_{TripSh}, we extend the idea for three-dimensional vectors of elements.

In more detail, D generates $^{A}\langle\cdot\rangle_d$-sharings of vectors of multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$, where $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,2t+1\}$. To prove that these vectors of triples are indeed multiplication-triples, the parties transform them into vectors of triples $\{(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})\}$, such that the following holds.

– There exist vectors of degree-t, t and degree-$2t$ polynomials $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ respectively, passing through the elements in $\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}$ and $\mathbb{Z}^{(\beta,\gamma)}$ respectively, where $\gamma \in \{1,\ldots,2t+1\}$.
– Component-wise, $\mathbb{X}^{(\beta)}(\cdot) \times \mathbb{Y}^{(\beta)}(\cdot) = \mathbb{Z}^{(\beta)}(\cdot)$ if and only if the triples shared by the dealer are multiplication-triples.

Next to verify the multiplicativity relationship among $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$, the parties publicly check the relationship at a random point r generated through $\mathcal{F}_{\mathsf{Rand}}$. If the verification is successful, then the parties extract vectors of secret-shared random multiplication-triples on behalf of the dealer, by extending the vectors of polynomials $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ at new points. Note that the output triples are well-defined and known to the dealer, since they are completely determined by the triples secret-shared by the dealer.

Protocol 8.30: Packed Triple Sharing – Π_{TripSh}

(Sharing Uncorrelated Triples)

1. For $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,2t+1\}$, dealer D selects vectors of random elements $\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)}$ from $\mathbb{F}$ where each $\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$ and where component-wise, $\mathbb{C}^{(\beta,\gamma)} = \mathbb{A}^{(\beta,\gamma)} \times \mathbb{B}^{(\beta,\gamma)}$. Dealer D generates $^{A}\langle \mathbb{A}^{(\beta,\gamma)} \rangle_d$, $^{A}\langle \mathbb{B}^{(\beta,\gamma)} \rangle_d$ and $^{A}\langle \mathbb{C}^{(\beta,\gamma)} \rangle_d$ by acting as a dealer and invoking instances of $\Pi_{A\langle\cdot\rangle}$ (protocol 6.1).
2. For $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{t+2,\ldots,2t+1\}$, dealer D selects vectors of random elements $\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)}$ from $\mathbb{F}^{n^5p^2}$. Dealer D generates $^{A}\langle \hat{\mathbb{A}}^{(\beta,\gamma)} \rangle_d$, $^{A}\langle \hat{\mathbb{B}}^{(\beta,\gamma)} \rangle_d$ and $^{A}\langle \hat{\mathbb{C}}^{(\beta,\gamma)} \rangle_d$ by acting as a dealer and invoking instances of $\Pi_{A\langle\cdot\rangle}$ (protocol 6.1).

(Generating Secret-Shared Random Pairs)

3. The parties invoke instances of Π_{RandPair} (Protocol 8.13) to generate $2 \cdot t \cdot n^3$ number of vectors of secret-shared random double pairs, denoted by $(^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_d, ^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1})$ and $(^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_d, ^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_{d+p-1})$, where $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{t+2,\ldots,2t+1\}$ and where $\mathbb{R}^{(\beta,\gamma)}, \hat{\mathbb{R}}^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$.

(Transformation to Correlated Triples)

4. For each $(\beta,\gamma) \in \{1,\ldots,n^3\} \times \{1,\ldots,t+1\}$, parties locally set
   – $^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d = ^{A}\langle \mathbb{A}^{(\beta,\gamma)} \rangle_d$,
   – $^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d = ^{A}\langle \mathbb{B}^{(\beta,\gamma)} \rangle_d$, and

– ${}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d = {}^{A}\langle \mathbb{C}^{(\beta,\gamma)} \rangle_d$.

5. For each $\beta \in \{1,\ldots,n^3\}$, let $\mathbb{X}^{(\beta)}(\cdot)$ and $\mathbb{Y}^{(\beta)}(\cdot)$ be the vectors of n^5p^2 degree-t polynomials, passing through the vectors of $t+1$ points $(1,\mathbb{X}^{(\beta,1)}),\ldots,(t+1,\mathbb{X}^{(\beta,t+1)})$ and $(1,\mathbb{Y}^{(\beta,1)}),\ldots,(t+1,\mathbb{Y}^{(\beta,t+1)})$ respectively.
6. For each $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{t+2,\ldots,2t+1\}$, parties locally compute ${}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d = {}^{A}\langle \mathbb{X}^{(\beta)}(\gamma) \rangle_d$ from $\{{}^{A}\langle \mathbb{X}^{(\beta,1)} \rangle_d,\ldots,{}^{A}\langle \mathbb{X}^{(\beta,t+1)} \rangle_d\}$.
7. For each $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{t+2,\ldots,2t+1\}$, parties locally compute ${}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d = {}^{A}\langle \mathbb{Y}^{(\beta)}(\gamma) \rangle_d$ from $\{{}^{A}\langle \mathbb{Y}^{(\beta,1)} \rangle_d,\ldots,{}^{A}\langle \mathbb{Y}^{(\beta,t+1)} \rangle_d\}$.
8. For each $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{t+2,\ldots,2t+1\}$, the parties invoke instances of Π_{Beaver} with inputs $\{{}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d\}$ and auxiliary inputs $\{{}^{A}\langle \mathbb{A}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{B}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{C}^{(\beta,\gamma)} \rangle_d\}$, $\{{}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1}\}$, $\{{}^{A}\langle \hat{\mathbb{A}}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \hat{\mathbb{B}}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \hat{\mathbb{C}}^{(\beta,\gamma)} \rangle_d\}$ and $\{{}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_{d+p-1}\}$ to compute $\{{}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d\}$ where $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$.
9. For each $\beta \in \{1,\ldots,n^3\}$, let $\mathbb{Z}^{(\beta)}(\cdot)$ be the vectors of n^5p^2 degree-$2t$ polynomials passing through the vectors of $2t+1$ points $(1,\mathbb{Z}^{(\beta,1)}),\ldots,(2t+1,\mathbb{Z}^{(\beta,2t+1)})$.

(Verification of Triples and Output Computation)

10. Parties call $\mathcal{F}_{\mathsf{Rand}}$ and receive a random value $r \in \mathbb{F}$.
11. For every $\beta \in \{1,\ldots,n^3\}$, parties locally compute ${}^{A}\langle \mathbb{X}^{(\beta)}(r) \rangle_d$, ${}^{A}\langle \mathbb{Y}^{(\beta)}(r) \rangle_d$ and ${}^{A}\langle \mathbb{Z}^{(\beta)}(r) \rangle_d$ from $\{{}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d\}$, $\{{}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d\}$ and $\{{}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d\}$ respectively, where $\gamma \in \{1,\ldots,2t+1\}$.
12. For every $\beta \in \{1,\ldots,n^3\}$, parties publicly reconstruct $\mathbb{X}^{(\beta)}(r), \mathbb{Y}^{(\beta)}(r)$ and $\mathbb{Z}^{(\beta)}(r)$ and check if component-wise $\mathbb{Z}^{(\beta)}(r) = \mathbb{X}^{(\beta)}(r) \times \mathbb{Y}^{(\beta)}(r)$ holds.
13. If $\mathbb{Z}^{(\beta)}(r) \neq \mathbb{X}^{(\beta)}(r) \times \mathbb{Y}^{(\beta)}(r)$ then the parties discard D and output default ${}^{A}\langle \cdot \rangle_d$-sharings of 0's on behalf of D.
14. Else let $\eta_1,\ldots,\eta_t$ be distinct elements from $\mathbb{F}\backslash\{1,\ldots,n,r\}$. For $i \in \{1,\ldots,t\}$, let $\mathbb{X}_i^{(\beta)} = \mathbb{X}^{(\beta)}(\eta_i)$, $\mathbb{Y}_i^{(\beta)} = \mathbb{Y}^{(\beta)}(\eta_i)$ and $\mathbb{Z}_i^{(\beta)} = \mathbb{Z}^{(\beta)}(\eta_i)$. Parties locally compute ${}^{A}\langle \mathbb{X}_i^{(\beta)} \rangle_d$, ${}^{A}\langle \mathbb{Y}_i^{(\beta)} \rangle_d$ and ${}^{A}\langle \mathbb{Z}_i^{(\beta)} \rangle_d$ from $\{{}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d\}$, $\{{}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d\}$ and $\{{}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d\}$ respectively, where $\gamma \in \{1,\ldots,2t+1\}$ and output ${}^{A}\langle \mathbb{X}_i^{(\beta)} \rangle_d$, ${}^{A}\langle \mathbb{Y}_i^{(\beta)} \rangle_d$ and ${}^{A}\langle \mathbb{Z}_i^{(\beta)} \rangle_d$, where $\beta \in \{1,\ldots,n^3\}$ and $i \in \{1,\ldots,t\}$.

We next prove the properties of the protocol Π_{TripSh} for an *honest* dealer.

Lemma 8.31. *If D is honest, then except with a negligible probability, parties output $\{{}^{A}\langle \mathbb{X}_i^{(\beta)} \rangle_d, {}^{A}\langle \mathbb{Y}_i^{(\beta)} \rangle_d, {}^{A}\langle \mathbb{Z}_i^{(\beta)} \rangle_d\}$, where $\beta \in \{1,\ldots,n^3\}$ and $i \in \{1,\ldots,t\}$, such that $\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}, \mathbb{Z}_i^{(\beta)} \in \mathbb{F}^{n^5p^2}$ and component-wise $\mathbb{Z}_i^{(\beta)} = \mathbb{X}_i^{(\beta)} \times \mathbb{Y}_i^{(\beta)}$. Moreover, the view of the adversary will be independent of $\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}$ and $\mathbb{Z}_i^{(\beta)}$.*

Proof. If D is *honest*, then it will select vectors of random multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ during step 1 and vectors of random multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ during step 2. From the correctness of $\Pi_{A\langle\cdot\rangle}$ (Lemma 6.6), parties will hold ${}^{A}\langle \cdot \rangle_d$-sharing of these values, except with a negligible probability. Moreover, parties will hold ${}^{A}\langle \cdot \rangle_d$-sharings of the vectors of triples $(\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)})$. From the correctness of Π_{RandPair} (Lemma 8.14), at the end of step 3, parties will hold $({}^{A}\langle \mathbb{R}^{(\beta,\gamma,)} \rangle_d, {}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1})$ and $({}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_{d+p-1})$, except with a negligible probability. From the protocol steps, it follows that for each $(\beta,\gamma) \in \{1,\ldots,n^3\} \times \{1,\ldots,t+1\}$, the vectors of triples $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$ constitute multiplication-triples. The correctness property of Π_{Beaver} (Lemma 8.26) guarantees that the same hold even for $(\beta,\gamma) \in \{1,\ldots,n^3\} \times \{t+2,\ldots,2t+1\}$, except with a negligible probability. Consequently, the vectors of polynomials $(\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot), \mathbb{Z}^{(\beta)}(\cdot))$ will satisfy the multiplicative relation and hence component-wise, $\mathbb{Z}^{(\beta)}(r) = \mathbb{X}^{(\beta)}(r) \times \mathbb{Y}^{(\beta)}(r)$ will hold for any $r \in \mathbb{F}$. Since the parties will robustly reconstruct $\mathbb{X}^{(\beta)}(r), \mathbb{Y}^{(\beta)}(r)$ and $\mathbb{Z}^{(\beta)}(r)$ except with a negligible probability, it follows that D will not be discarded. From the protocol steps, it now follows that parties will output $\{{}^{A}\langle \mathbb{X}_i^{(\beta)} \rangle_d, {}^{A}\langle \mathbb{Y}_i^{(\beta)} \rangle_d, {}^{A}\langle \mathbb{Z}_i^{(\beta)} \rangle_d\}$ during step 14. Moreover, the vectors of triples $(\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}, \mathbb{Z}_i^{(\beta)})$ will be multiplication-triples, as they constitute vectors of points on the vectors of polynomials $(\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot), \mathbb{Z}^{(\beta)}(\cdot))$ which satisfy the multiplicative relation.

To argue the privacy of the output multiplication-triples $(\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}, \mathbb{Z}_i^{(\beta)})$, we need to show that throughout the protocol, adversary will learn at most one vector of points on the vectors of polynomials $(\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}, \mathbb{Z}_i^{(\beta)})$. Since the polynomials in $\mathbb{X}^{(\beta)}(\cdot)$ and $\mathbb{Y}^{(\beta)}(\cdot)$ are of degree-t, the privacy follows.

From the privacy of $\Pi_{A\langle\cdot\rangle}$ (Lemma 6.4), the vectors of multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$, the vectors of multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ and the vectors of triples $(\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)})$ will remain random for the adversary, at the end of step 1 and 2 respectively. From the privacy of Π_{RandPair} (Lemma 8.15), the vectors of elements $\mathbb{R}^{(\beta,\gamma)}$ and $\hat{\mathbb{R}}^{(\beta,\gamma)}$ will remain random for the adversary. From the protocol steps and from the privacy of protocol Π_{Beaver} (Lemma 8.27), it follows that the view of the adversary will be independent of the multiplication-triples $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}.\mathbb{Z}^{(\beta,\gamma)})$, for every $(\beta,\gamma) \in \{1,\ldots,n^3\} \times \{1,\ldots,2t+1\}$. Hence till the end of step 9, the view of the adversary will be independent of the vectors of polynomials $(\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot), \mathbb{Z}^{(\beta)}(\cdot))$, except that they satisfy the multiplicative relationship. Finally during step 12, adversary may learn the vectors of points $\mathbb{X}^{(\beta)}(r), \mathbb{Y}^{(\beta)}(r)$ and $\mathbb{Z}^{(\beta)}(r)$, as they are publicly reconstructed. This completes the proof of the lemma. □

We next prove the properties of the protocol Π_{TripSh} for a *corrupt* dealer.

Lemma 8.32. *If D is corrupt, then except with a negligible probability, parties output $\{{}^{A}\langle \mathbb{X}_i^{(\beta)} \rangle_d, {}^{A}\langle \mathbb{Y}_i^{(\beta)} \rangle_d, {}^{A}\langle \mathbb{Z}_i^{(\beta)} \rangle_d\}$, where $\beta \in \{1,\ldots,n^3\}$ and $i \in \{1,\ldots,t\}$, such that $\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}, \mathbb{Z}_i^{(\beta)} \in \mathbb{F}^{n^5p^2}$ and component-wise $\mathbb{Z}_i^{(\beta)} = \mathbb{X}_i^{(\beta)} \times \mathbb{Y}_i^{(\beta)}$.*

Proof. If D is discarded during step 13 then the lemma holds trivially, since in this case $\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}$ and $\mathbb{Z}_i^{(\beta)}$ are all 0's. So we consider the case when D is not discarded. In this case, we show that except with a negligible probability, the vectors of polynomials $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ defined by the end of step 9, satisfy the multiplicative relationship. On contrary, if this is not the case, then component-wise $\mathbb{Z}^{(\beta)}(r) \neq \mathbb{X}^{(\beta)}(r) \times \mathbb{Y}^{(\beta)}(r)$ will hold, except with a negligible probability. This is because r is selected randomly from $\mathbb{F}$ and not known to the adversary at the end of step 9, when the polynomials are determined. Moreover, the vectors of polynomials $\mathbb{Z}^{(\beta)}(\cdot) - \mathbb{X}^{(\beta)}(\cdot) \times \mathbb{Y}^{(\beta)}(\cdot)$ can have at most $2t$ roots, since each polynomial here is a degree-$2t$ polynomial.

To show that $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ satisfy the multiplicative relationship, we need to show that for every $\gamma \in \{1,\ldots,2t+1\}$, the vectors of triples $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$ computed in the protocol are multiplication-triples. Note that $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$ is the same as $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ shared by D, provided $\gamma \in \{1,\ldots,t+1\}$. On the other hand, if $\gamma \in \{t+2,\ldots,2t+1\}$, then $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$ depends upon the triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ shared by D.

Consider an arbitrary $\gamma \in \{1,\ldots,t+1\}$. If $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ do not constitute multiplication-triples then clearly $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$ will also be a non-multiplication-triple. On the other hand, consider an arbitrary $\gamma \in \{t+2,\ldots,2t+1\}$. In this case, $\mathbb{Z}^{(\beta,\gamma)}$ is computed from the inputs $\mathbb{X}^{(\beta,\gamma)}$ and $\mathbb{Y}^{(\beta,\gamma)}$ through protocol Π_{Beaver} using the triple $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ shared by D. It follows from the correctness of Π_{Beaver} (Lemma 8.26) that if $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ is not a multiplication-triple, then $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$ will also be a non-multiplication-triple. □

We finally do the complexity analysis of the protocol.

Lemma 8.33. *Protocol Π_{TripSh} requires a constant expected number of rounds and makes at most 47 number of calls to $\mathcal{F}_{\mathsf{Rand}}$. The protocol generates $n^3tn^5p^2$ packed-secret-shared multiplication-triples over $\mathbb{F}$, incurring a communication of $\mathcal{O}(n^{11}\kappa)$ bits.*

If the protocol is executed L times in parallel, then it outputs $L{\cdot}n^3tn^5p^2$ packed-secret-shared multiplication-triples over $\mathbb{F}$, incurring a communication of $\mathcal{O}(L \cdot n^{11}\kappa)$ bits and calling $\mathcal{F}_{\mathsf{Rand}}$ at most 47 times.

Proof. During steps 1-2, D needs to generate ${}^{A}\langle\cdot\rangle_d$-sharings of $\mathcal{O}(n^4)$ number of vectors of n^5p^2 elements from $\mathbb{F}$ through instances of $\Pi_{A\langle\cdot\rangle}$. From Lemma 6.8, this will incur a constant expected number of rounds and a communication of $\mathcal{O}(n^{11})$ field elements, apart from at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$. Note that all the instances of $\Pi_{A\langle\cdot\rangle}$ can be executed in parallel and so the calls to $\mathcal{F}_{\mathsf{Rand}}$ inside various instances of $\Pi_{A\langle\cdot\rangle}$ can be synchronized across various stages of the instances of $\Pi_{A\langle\cdot\rangle}$. Hence, the instances of $\Pi_{A\langle\cdot\rangle}$ will require at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$.

During step 3, the parties need to run $\mathcal{O}(n^2)$ number of instances of the protocol Π_{RandPair} to generate the required number of double-secret-shared random values. From Lemma 8.16, this will incur a constant expected number of rounds and incur a communication of $\mathcal{O}(n^{11})$ field elements, apart from 16 calls to $\mathcal{F}_{\mathsf{Rand}}$ since all the instances can be executed in parallel. We also note that steps 1-3 can be executed in parallel and so all the instances of $\Pi_{A\langle\cdot\rangle}$, required as part of steps 1-2 and the instances of $\Pi_{A\langle\cdot\rangle}$ inside Π_{RandPair} can be executed in parallel. Consequently, the calls to $\mathcal{F}_{\mathsf{Rand}}$ during various stages of various

instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ can be synchronized. Hence the maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ across steps 1-3 is 16.

To transform the dealer's secret-shared triples, parties need to run two instances of Π_{Beaver} (in parallel). From Lemma 8.28, this requires a constant expected number of rounds, communication of $\mathcal{O}(n^{11})$ field elements and at most 31 number of calls to $\mathcal{F}_{\mathsf{Rand}}$. Finally during step 12, the parties need to publicly reconstruct $\mathcal{O}(n^3)$ number of ${}^{A}\langle\cdot\rangle_d$-shared vectors of values, which costs one round and incurs a communication of $\mathcal{O}(n^{11})$ field elements.

From the above analysis, the protocol requires a constant expected number of rounds, incurs a communication of $\mathcal{O}(n^{11})$ field elements and requires at most 47 number of calls to $\mathcal{F}_{\mathsf{Rand}}$.

It is easy to see that if the protocol is executed L times in parallel, then the calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized across all the L instances and the number of calls to $\mathcal{F}_{\mathsf{Rand}}$ remains at most 47. Moreover, the total number of multiplication-triples generated will be $L \cdot n^3tn^5p^2$. □

Communication Overhead of the Protocol Π_{TripSh}. If the protocol Π_{TripSh} is executed L times in parallel where $L \geq 1$, then the output of the protocol consists of $L \cdot n^3tn^5p^2$ multiplication-triples over $\mathbb{F}$ and hence $\Theta(n^{11}\kappa)$ bits. Since the communication complexity of the protocol will be $\mathcal{O}(L \cdot n^{11}\kappa)$ bits, apart from at most 47 calls to $\mathcal{F}_{\mathsf{Rand}}$, it follows that for sufficiently large L, the asymptotic communication overhead of the protocol is $\mathcal{O}(1)$ bits.

Although we do not model the requirements of Π_{TripSh} via a functionality, we describe the simulation steps for Π_{TripSh} which will be used when the protocol is used as a primitive.

Simulator 8.34: Simulator for the Protocol Π_{TripSh}

– **Case 1: Honest Dealer**

- On behalf of the dealer, for each $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{1, \ldots, 2t+1\}$, select vectors of random elements $\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)}$ from $\mathbb{F}$ where each $\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$ and where component-wise, $\mathbb{C}^{(\beta,\gamma)} = \mathbb{A}^{(\beta,\gamma)} \times \mathbb{B}^{(\beta,\gamma)}$.
- Simulate the steps of VSS for an honest dealer as per Simulator 6.10 to generate ${}^{A}\langle\mathbb{A}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{B}^{(\beta,\gamma)}\rangle_d$ and ${}^{A}\langle\mathbb{C}^{(\beta,\gamma)}\rangle_d$.
- On behalf of the dealer, for each $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{t+2, \ldots, 2t+1\}$, select vectors of random elements $\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)}$ from $\mathbb{F}^{n^5p^2}$.
- Simulate the steps of VSS for an honest dealer as per Simulator 6.10 to generate ${}^{A}\langle\hat{\mathbb{A}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{B}}^{(\beta,\gamma)}\rangle_d$ and ${}^{A}\langle\hat{\mathbb{C}}^{(\beta,\gamma)}\rangle_d$.
- Simulate the steps of Π_{RandPair} as in Simulation 8.17.
- Perform the local computations as in the protocol steps on behalf of the honest parties and invoke the simulation steps of Π_{Beaver} as in Simulation 8.29.
- Select a random r to emulate $\mathcal{F}_{\mathsf{Rand}}$ and send r to each P_k under the control of $\mathcal{A}$.
- Send the honest parties shares corresponding to ${}^{A}\langle\mathbb{X}^{(\beta)}(r)\rangle_d, {}^{A}\langle\mathbb{Y}^{(\beta)}(r)\rangle_d$ and ${}^{A}\langle\mathbb{Z}^{(\beta)}(r)\rangle_d$ for each $\beta \in \{1, \ldots, n^3\}$.

– **Case 2: Corrupt Dealer**

- Simulate the steps of VSS for a corrupt dealer as per Simulator 6.10 to obtain ${}^{A}\langle\mathbb{A}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{B}^{(\beta,\gamma)}\rangle_d$ and ${}^{A}\langle\mathbb{C}^{(\beta,\gamma)}\rangle_d$ for each $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{1, \ldots, 2t+1\}$ and ${}^{A}\langle\hat{\mathbb{A}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{B}}^{(\beta,\gamma)}\rangle_d$ and ${}^{A}\langle\hat{\mathbb{C}}^{(\beta,\gamma)}\rangle_d$ for each $\beta \in \{1, \ldots, n^3\}$ and $\gamma \in \{t+2, \ldots, 2t+1\}$.
- Simulate the steps of Π_{RandPair} as in Simulation 8.17.
- Perform the local computations as in the protocol steps on behalf of the honest parties and invoke the simulation steps of Π_{Beaver} as in Simulation 8.29.
- Select a random r to emulate $\mathcal{F}_{\mathsf{Rand}}$ and send r to each P_k under the control of $\mathcal{A}$.
- Send the honest parties shares corresponding to ${}^{A}\langle\mathbb{X}^{(\beta)}(r)\rangle_d, {}^{A}\langle\mathbb{Y}^{(\beta)}(r)\rangle_d$ and ${}^{A}\langle\mathbb{Z}^{(\beta)}(r)\rangle_d$ for each $\beta \in \{1, \ldots, n^3\}$.
- If $\mathbb{Z}^{(\beta)}(r) \neq \mathbb{X}^{(\beta)}(r) \times \mathbb{Y}^{(\beta)}(r)$ then discard the dealer on behalf of the honest parties and assume a default sharing of 0's corresponding to the corrupt dealer.

8.7 Triple-Extraction with Constant Overhead

Our final component is a triple extraction protocol Π_{TripExt} (Protocol 8.35). The protocol takes input vectors of multiplication-triples shared by individual parties where it will be guaranteed that the multiplication-triples shared by the honest parties are truly random and unknown to the adversary. The protocol outputs vectors of random multiplication-triples unknown to the adversary. The high-level idea of the protocol for one-dimensional vectors of elements has been already discussed in Section 2; here we extend the idea for three-dimensional vectors of elements.

In more detail, for $\gamma \in \{1, \ldots, \frac{3t}{2}+1\}$, the input of the party P_γ will be ${}^{A}\langle\cdot\rangle_d$-shared vectors of multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$, where $\beta \in \{1, \ldots, n^3\}$. On the other hand, for $\gamma \in \{\frac{3t}{2}+2, \ldots, n\}$, party P_γ will have input ${}^{A}\langle\cdot\rangle_d$-shared multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ and ${}^{A}\langle\cdot\rangle_d$-shared triples $(\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)})$. It would be ensured that if P_γ is *honest* then adversary's view remains independent of P_γ's inputs. The idea is to transform the multiplication-triples in $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ into multiplication-triples $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$, such that the elements in $\mathbb{X}^{(\beta,\gamma)}$, $\mathbb{Y}^{(\beta,\gamma)}$ and $\mathbb{Z}^{(\beta,\gamma)}$ define vectors of degree-$\frac{3t}{2}$, $\frac{3t}{2}$ and degree-$3t$ polynomials $\mathbb{X}^{(\beta)}(\cdot)$, $\mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ respectively, with adversary learning at most t vectors of points on these polynomials. This is done using a similar idea as used in the protocol Π_{TripSh}. Note that the polynomials will satisfy the multiplicative relationship as the input triples of all the parties will be guaranteed to be multiplication-triples. Once the polynomials $\mathbb{X}^{(\beta)}(\cdot)$, $\mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ are set, the parties extend these polynomials at $\frac{t}{2}+1$ new points (different from $1, \ldots, n$) and output ${}^{A}\langle\cdot\rangle_d$-sharings of these vectors of elements.

Protocol 8.35: Packed Triple Extraction – Π_{TripExt}

Input:

- For each $(\beta,\gamma) \in \{1, \ldots, n^3\} \times \{1, \ldots, \frac{3t}{2}+1\}$, party P_γ has the vectors of multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ and parties hold ${}^{A}\langle\mathbb{A}^{(\beta,\gamma)}\rangle_d$, ${}^{A}\langle\mathbb{B}^{(\beta,\gamma)}\rangle_d$, ${}^{A}\langle\mathbb{C}^{(\beta,\gamma)}\rangle_d$.
- For each $(\beta,\gamma) \in \{1, \ldots, n^3\} \times \{\frac{3t}{2}+2, \ldots, n\}$, party P_γ has the vectors of multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ and vectors of triples $(\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)})$ and parties hold ${}^{A}\langle\mathbb{A}^{(\beta,\gamma)}\rangle_d$, ${}^{A}\langle\mathbb{B}^{(\beta,\gamma)}\rangle_d$, ${}^{A}\langle\mathbb{C}^{(\beta,\gamma)}\rangle_d$ and ${}^{A}\langle\hat{\mathbb{A}}^{(\beta,\gamma)}\rangle_d$, ${}^{A}\langle\hat{\mathbb{B}}^{(\beta,\gamma)}\rangle_d$, ${}^{A}\langle\hat{\mathbb{C}}^{(\beta,\gamma)}\rangle_d$.

(Generating Double-Packed-Secret-Shared Random Values)

1. For each $(\beta,\gamma) \in \{1, \ldots, n^3\} \times \{\frac{3t}{2}+2, \ldots, n\}$, parties generate $({}^{A}\langle\mathbb{R}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{R}^{(\beta,\gamma)}\rangle_{d+p-1})$ and $({}^{A}\langle\hat{\mathbb{R}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{R}}^{(\beta,\gamma)}\rangle_{d+p-1})$ through instances of Π_{RandPair}.

(Transformation to Correlated Triples)

2. For each $(\beta,\gamma) \in \{1, \ldots, n^3\} \times \{1, \ldots, \frac{3t}{2}+1\}$, parties locally set
   - ${}^{A}\langle\mathbb{X}^{(\beta,\gamma)}\rangle_d = {}^{A}\langle\mathbb{A}^{(\beta,\gamma)}\rangle_d$,
   - ${}^{A}\langle\mathbb{Y}^{(\beta,\gamma)}\rangle_d = {}^{A}\langle\mathbb{B}^{(\beta,\gamma)}\rangle_d$, and
   - ${}^{A}\langle\mathbb{Z}^{(\beta,\gamma)}\rangle_d = {}^{A}\langle\mathbb{C}^{(\beta,\gamma)}\rangle_d$.
3. For each $\beta \in \{1, \ldots, n^3\}$, let $\mathbb{X}^{(\beta)}(\cdot)$ and $\mathbb{Y}^{(\beta)}(\cdot)$ be the vectors of n^5p^2 degree-$\frac{3t}{2}$ polynomials, passing through the vectors of points from $\{(\gamma, \mathbb{X}^{(\beta,\gamma)})\}$ and $\{(\gamma, \mathbb{Y}^{(\beta,\gamma)})\}$ respectively, where $\gamma \in \{1, \ldots, \frac{3t}{2}+1\}$.
4. For each $\gamma \in \{\frac{3t}{2}+2, \ldots, n\}$, parties locally compute
   - ${}^{A}\langle\mathbb{X}^{(\beta,\gamma)}\rangle_d = {}^{A}\langle\mathbb{X}^{(\beta)}(\gamma)\rangle_d$ from ${}^{A}\langle\mathbb{X}^{(\beta,1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{X}^{(\beta,\frac{3t}{2}+1)}\rangle_d$, and
   - ${}^{A}\langle\mathbb{Y}^{(\beta,\gamma)}\rangle_d = {}^{A}\langle\mathbb{Y}^{(\beta)}(\gamma)\rangle_d$ from ${}^{A}\langle\mathbb{Y}^{(\beta,1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{Y}^{(\beta,\frac{3t}{2}+1)}\rangle_d$.
5. For each $(\beta,\gamma) \in \{1, \ldots, n^3\} \times \{\frac{3t}{2}+2, \ldots, n\}$, the parties invoke instances of Π_{Beaver} (two times as described below) with inputs ${}^{A}\langle\mathbb{X}^{(\beta,\gamma)}\rangle_d$, ${}^{A}\langle\mathbb{Y}^{(\beta,\gamma)}\rangle_d$ and auxiliary inputs $\{{}^{A}\langle\mathbb{A}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{B}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{C}^{(\beta,\gamma)}\rangle_d\}$, $\{{}^{A}\langle\mathbb{R}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{R}^{(\beta,\gamma)}\rangle_{d+p-1}\}$, $\{{}^{A}\langle\hat{\mathbb{A}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{B}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{C}}^{(\beta,\gamma)}\rangle_d\}$ and $\{{}^{A}\langle\hat{\mathbb{R}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{R}}^{(\beta,\gamma)}\rangle_{d+p-1}\}$ to compute ${}^{A}\langle\mathbb{Z}^{(\beta,\gamma)}\rangle_d$, where $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$ component-wise.
   The first and second instance of Π_{Beaver} will take inputs indexed from $\gamma \in \{\frac{3t}{2}+2, \frac{5t}{2}+2\}$ and $\gamma \in \{\frac{5t}{2}+3, \ldots, n\}$ respectively.
6. For each $\beta \in \{1, \ldots, n^3\}$, let $\mathbb{Z}^{(\beta)}(\cdot)$ be the vectors of n^5p^2 degree-$(n-1)$ polynomials defined by the vectors of points $(1, \mathbb{Z}^{(\beta,1)}), \ldots, (n, \mathbb{Z}^{(\beta,n)})$.

(Output Computation)

7. For each $(\beta, i) \in \{1, \ldots, n^3\} \times \{1, \frac{t}{2}+1\}$, let $\mathbb{A}_i^{(\beta)} = \mathbb{X}^{(\beta)}(n+i), \mathbb{B}_i^{(\beta)} = \mathbb{Y}^{(\beta)}(n+i)$ and $\mathbb{C}_i^{(\beta)} = \mathbb{Z}^{(\beta)}(n+i)$. Parties locally compute ${}^A\langle \mathbb{A}_i^{(\beta)} \rangle_d$, ${}^A\langle \mathbb{B}_i^{(\beta)} \rangle_d$ and ${}^A\langle \mathbb{C}_i^{(\beta)} \rangle_d$ from $\{{}^A\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d\}, \{{}^A\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d\}$ and $\{{}^A\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d\}$ respectively, where $\gamma \in \{1, \ldots, n\}$ and output ${}^A\langle \mathbb{A}_i^{(\beta)} \rangle_d$, ${}^A\langle \mathbb{B}_i^{(\beta)} \rangle_d$ and ${}^A\langle \mathbb{C}_i^{(\beta)} \rangle_d$.

We next prove the properties of the protocol Π_{TripExt}.

Lemma 8.36. *For each $(\beta, \gamma) \in \{1, \ldots, n^3\} \times \{1, \ldots, \frac{3t}{2}+1\}$, let $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ be multiplication-triples which are ${}^A\langle \cdot \rangle_d$-shared among the parties. Moreover, for each $(\beta, \gamma) \in \{1, \ldots, n^2\} \times \{\frac{3t}{2}+2, \ldots, n\}$, let $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ be multiplication-triples which are ${}^A\langle \cdot \rangle_d$-shared among the parties. Then except with a negligible probability, for each $(\beta, i) \in \{1, \ldots, n^3\} \times \{1, \ldots, \frac{t}{2}+1\}$, protocol Π_{TripExt} outputs ${}^A\langle \cdot \rangle_d$-sharing of multiplication-triples $(\mathbb{A}_i^{(\beta)}, \mathbb{B}_i^{(\beta)}, \mathbb{C}_i^{(\beta)})$.*

Proof. To prove the lemma, we show that for each $\beta \in \{1, \ldots, n^3\}$, the vectors of polynomials $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ satisfy the multiplicative relationship; i.e. $\mathbb{Z}^{(\beta)}(\cdot) = \mathbb{X}^{(\beta)}(\cdot) \times \mathbb{Y}^{(\beta)}(\cdot)$ holds.

From the protocol steps, $\mathbb{Z}^{(\beta)}(\gamma) = \mathbb{X}^{(\beta)}(\gamma) \times \mathbb{Y}^{(\beta)}(\gamma)$ holds component-wise, for each $\gamma \in \{1, \ldots, \frac{3t}{2}+1\}$. This is because for each $\gamma \in \{1, \ldots, \frac{3t}{2}+1\}$, we have $(\mathbb{X}^{(\beta)}(\gamma), \mathbb{Y}^{(\beta)}(\gamma), \mathbb{Z}^{(\beta)}(\gamma)) = (\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)}) = (\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ and it is given that $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ is a multiplication-triple. On the other hand, for each $\gamma \in \{\frac{3t}{2}+2, \ldots, n\}$, we have $(\mathbb{X}^{(\beta)}(\gamma), \mathbb{Y}^{(\beta)}(\gamma), \mathbb{Z}^{(\beta)}(\gamma)) = (\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$, where $\mathbb{Z}^{(\beta,\gamma)}$ is computed from inputs $\mathbb{X}^{(\beta,\gamma)}$ and $\mathbb{Y}^{(\beta,\gamma)}$ using protocol Π_{Beaver}. During the instance of Π_{Beaver}, the triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ are used as auxiliary triples, which are given to be multiplication-triples. From the correctness of Π_{Beaver}, it follows that except with a negligible probability, $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$ will hold. Consequently, $(\mathbb{X}^{(\beta)}(\gamma), \mathbb{Y}^{(\beta)}(\gamma), \mathbb{Z}^{(\beta)}(\gamma))$ will constitute multiplication-triples, even for $\gamma \in \{\frac{3t}{2}+2, \ldots, n\}$. Since $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ are vectors of degree-$\frac{3t}{2}$, $\frac{3t}{2}$ and degree-$3t$ polynomials, it follows that $\mathbb{Z}^{(\beta)}(\cdot) = \mathbb{X}^{(\beta)}(\cdot) \times \mathbb{Y}^{(\beta)}(\cdot)$ holds, thus proving the lemma. □

Lemma 8.37. *For each $(\beta, \gamma) \in \{1, \ldots, n^3\} \times \{1, \ldots, \frac{3t}{2}+1\}$, let $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ be multiplication-triples held by party P_γ, which are ${}^A\langle \cdot \rangle_d$-shared among the parties. Moreover, for each $(\beta, \gamma) \in \{1, \ldots, n^3\} \times \{\frac{3t}{2}+2, \ldots, n\}$, let $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ be the multiplication-triples and let $(\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)})$ be the triples held by party P_γ, which are ${}^A\langle \cdot \rangle_d$-shared among the parties. Furthermore, if P_γ is honest then let the triples and multiplication-triples held by P_γ be random for the adversary. Then for each $(\beta, i) \in \{1, \ldots, n^3\} \times \{1, \ldots, \frac{t}{2}+1\}$, the view of the adversary in the protocol Π_{TripExt} will be independent of the output multiplication-triples $(\mathbb{A}_i^{(\beta)}, \mathbb{B}_i^{(\beta)}, \mathbb{C}_i^{(\beta)})$.*

Proof. From the proof of Lemma 8.36, for every $\beta \in \{1, \ldots, n^3\}$, the vectors of polynomials $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot)$ and $\mathbb{Z}^{(\beta)}(\cdot)$ satisfy the multiplicative relationship, where these polynomials have degree-$\frac{3t}{2}$, $\frac{3t}{2}$ and degree-$3t$ respectively. Also, from the protocol steps, for every $\gamma \in \{1, \ldots, n\}$, the conditions $\mathbb{X}^{(\beta)}(\gamma) = \mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta)}(\gamma) = \mathbb{Y}^{(\beta,\gamma)}$ and $\mathbb{Z}^{(\beta)}(\gamma) = \mathbb{Z}^{(\beta,\gamma)}$ hold. To prove the lemma, we show that throughout the protocol, adversary learns at most t points on these vectors of polynomials. For this consider any arbitrary *honest* party P_γ. We claim that throughout the protocol, the view of the adversary remains independent of the vectors of multiplication-triples $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$. This will automatically imply that adversary may learn the vectors of multiplication-triples $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$ only if P_γ is corrupt. Since there can be at most t corrupt parties, it follows that adversary may learn at most t points on the vectors of polynomials.

To prove the above claim, we consider two cases. If $\gamma \in \{1, \ldots, \frac{3t}{2}+1\}$, then it is obvious that the view of the adversary remains independent of $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)})$. This is because from the protocol steps, we have $(\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}, \mathbb{Z}^{(\beta,\gamma)}) = (\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$, where the elements of $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ are given to be random for the adversary as per the lemma conditions. The other possibility is when $\gamma \in \{\frac{3t}{2}+2, \ldots, n\}$. In this case, $\mathbb{Z}^{(\beta,\gamma)}$ is computed from $\mathbb{X}^{(\beta,\gamma)}$ and $\mathbb{Y}^{(\beta,\gamma)}$ through an instance of Π_{Beaver}. During the instance of Π_{Beaver}, parties use the multiplication-triples $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ and the triples $(\hat{\mathbb{A}}^{(\beta,\gamma)}, \hat{\mathbb{B}}^{(\beta,\gamma)}, \hat{\mathbb{C}}^{(\beta,\gamma)})$ as auxiliary information. From the lemma conditions, these are given to be random for the adversary. Apart from the above auxiliary information, the parties also use vectors of double-packed-secret-shared elements $\mathbb{R}^{(\beta,\gamma)}$ and $\hat{\mathbb{R}}^{(\beta,\gamma)}$, which are generated through instances of

Π_{RandPair}. From the privacy property of Π_{RandPair} (Lemma 8.15), these elements will be random for the adversary. It now follows from the privacy property of Π_{Beaver} (Lemma 8.27) that during the instance of Π_{Beaver}, the view of the adversary will remain independent of $\mathbb{X}^{(\beta,\gamma)}, \mathbb{Y}^{(\beta,\gamma)}$ and $\mathbb{Z}^{(\beta,\gamma)}$, thus completing the proof. □

We finally do the complexity analysis of the protocol.

Lemma 8.38. *Given inputs* $\{{}^{A}\langle \mathbb{A}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{B}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{C}^{(\beta,\gamma)}\rangle_d\}$ *where* $(\beta,\gamma) \in \{1,\ldots,n^2\}\times\{1,\ldots,\frac{3t}{2}+1\}$ *and inputs* $\{{}^{A}\langle \mathbb{A}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{B}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \mathbb{C}^{(\beta,\gamma)}\rangle_d\}$ *and* $\{{}^{A}\langle \hat{\mathbb{A}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \hat{\mathbb{B}}^{(\beta,\gamma)}\rangle_d, {}^{A}\langle \hat{\mathbb{C}}^{(\beta,\gamma)}\rangle_d\}$, *where* $(\beta,\gamma) \in \{1,\ldots,n^3\}\times\{\frac{3t}{2}+2,\ldots,n\}$, *protocol* Π_{TripExt} *outputs* ${}^{A}\langle\cdot\rangle_d$*-shared vectors of multiplication-triples* $(\mathbb{A}_i^{(\beta)}, \mathbb{B}_i^{(\beta)}, \mathbb{C}_i^{(\beta)})$, *where* $\beta \in \{1,\ldots,n^3\}$, $i \in \{1,\ldots,\frac{t}{2}+1\}$ *and where each* $\mathbb{A}_i^{(\beta)}, \mathbb{B}_i^{(\beta)}, \mathbb{C}_i^{(\beta)} \in \mathbb{F}^{n^5p^2}$. *The protocol requires a constant expected number of rounds and incurs a communication of* $\mathcal{O}(n^{11}\kappa)$ *bits, apart from* 47 *number of calls to* $\mathcal{F}_{\mathsf{Rand}}$.

If the protocol is executed L *times in parallel where* $L \geq 1$, *then the protocol outputs* $L \cdot n^3(t+1)n^5p^2$ *secret-shared multiplication-triples over* $\mathbb{F}$, *incurring a communication of* $\mathcal{O}(L \cdot n^{11}\kappa)$ *bits, apart from* 47 *number of calls to* $\mathcal{F}_{\mathsf{Rand}}$.

Proof. In the protocol, the parties interact only during the instances of Π_{RandPair} and Π_{Beaver}, both of which requires a constant expected number of rounds. The parties need to run $\Theta(n^2)$ instances of Π_{RandPair}, since they need to generate $\Theta(n^4)$ number of double-packed-secret-shared vectors of random elements and each instance can generate $\Theta(n^2)$ such vectors. Note that all these instances can be executed in parallel and the calls to $\mathcal{F}_{\mathsf{Rand}}$ can be synchronized and reused across various stages of different instances of Π_{RandPair}. Hence, from Lemma 8.16, the instances of Π_{RandPair} in the protocol will incur a communication of $\mathcal{O}(n^{11})$ field elements, apart from at most 16 calls to $\mathcal{F}_{\mathsf{Rand}}$. Next, there will be 2 instances of Π_{Beaver}, both of which can be executed in parallel. From Lemma 8.28, this will require a constant expected number of rounds, incur a communication of $\mathcal{O}(n^{10})$ field elements and requires at most 31 number of calls to $\mathcal{F}_{\mathsf{Rand}}$. From the above analysis, the protocol incurs a communication of $\mathcal{O}(n^{11}\kappa)$ bits and requires a constant expected number of rounds, apart from at most 47 number of calls to $\mathcal{F}_{\mathsf{Rand}}$.

If the protocol is executed L times in parallel, then it outputs $L \cdot n^3(t+1)n^5p^2$ secret-shared multiplication-triples over $\mathbb{F}$. The maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ remains 47, as they can be synchronized across all the L instances of the protocol. □

Communication Overhead of the Protocol Π_{TripExt}. If the protocol Π_{TripExt} is executed L times in parallel where $L \geq 1$, then the output of the protocol consists of $L \cdot n^3(t+1)n^5p^2$ secret-shared multiplication-triples over $\mathbb{F}$ and hence $\Theta(n^{11}\kappa)$ bits. Since the communication complexity of the protocol will be $\mathcal{O}(L \cdot n^{11}\kappa)$ bits, apart from at most 47 calls to $\mathcal{F}_{\mathsf{Rand}}$, it follows that for sufficiently large L, the asymptotic communication overhead of the protocol is $\mathcal{O}(1)$ bits.

We describe the simulation steps for Π_{TripExt} which will be used when the protocol is used as a primitive.

Simulator 8.39: Simulator for the Protocol Π_{TripExt}

- Simulate the steps of Π_{RandPair} as in Simulation 8.17.
- On behalf of each honest party P_i not under the control of $\mathcal{A}$, perform the local computations as an honest party would as per the protocol specifications.
- Simulate the steps of Π_{Beaver} as in Simulation 8.29 and perform local computations as per the protocol specifications on behalf of each honest party P_i not under the control of $\mathcal{A}$.

8.8 The Complete Preprocessing Phase

We now give a protocol for the complete preprocessing phase, as a composition of the building blocks defined so far. The preprocessing phase protocol Π_{Prep} is given in Protocol 8.40. It realizes $\mathcal{F}_{\mathsf{PREP}}$ and is described in the $\mathcal{F}_{\mathsf{MacKeys}}$-hybrid model. The high-level idea of the protocol is simple and as follows. Parties begin by invoking the $\mathcal{F}_{\mathsf{MacKeys}}$ functionality for setup generation, that is, establishing the MAC key required for further computation. The setup is used across all the instances of $\Pi_{A\langle\cdot\rangle}$ used inside various sub-protocols and thus guarantees that all the ${}^{A}\langle\cdot\rangle_d$-sharings generated are key-consistent.[22]

[22] Recall that protocol $\Pi_{A\langle\cdot\rangle}$ requires this setup to be already available which is used inside the protocol $\Pi_{A\langle\cdot\rangle}$.

Subsequently, the goal is to generate shares of ${}^{A}\langle\cdot\rangle_d$-shared random multiplication triples, random triples and double-packed-secret-shared vectros of random elements, which, looking ahead, will be used for Beaver multiplication in the online phase. Recall that $\mathcal{F}_{\mathsf{PREP}}$ generates $c_M n^3(t+1)$ vectors of random multiplication-triples and vectors of random triples, each of size n^5p^2. Along with this, the functionality also generates $2c_M n^3(t+1)$ vectors of double-packed-secret-shared random values, each of size n^5p^2.

The generation of multiplication-triples in Π_{Prep} happens by applying the framework of [24], extending it for authenticated packed-secret-sharing, as discussed in Section 2. Namely, each party generates ${}^{A}\langle\cdot\rangle_d$-sharing of "sufficiently many" random multiplication-triples by acting as a dealer through instances of Π_{TripSh}. The multiplication-triples shared by honest parties will be random for the adversary. However, the identity of honest parties will not be known and consequently, parties run instances of Π_{TripExt} on the multiplication-triples shared by different parties and extract ${}^{A}\langle\cdot\rangle_d$-sharing of truly random multiplication-triples, unknown to the adversary. To calculate the number of vectors of multiplication-triples that each party should provide, let us first compute the number of instances of Π_{TripExt} that the parties need to run to generate the desired number of multiplication-triples.

Recall that one instance of Π_{TripExt} generates $n^3(t+1)n^5p^2$-secret-shared random multiplication-triples. Consequently, c_M instances of Π_{TripExt} will suffice to generate $c_M n^3(t+1)n^5p^2$-secret-shared random multiplication-triples. To run one instance of Π_{TripExt}, we need n^3 number of ${}^{A}\langle\cdot\rangle_d$-sharing of vectors of n^5p^2 multiplication-triples and triples from each party.[23] Now based on this calculation, let us calculate the number of instances of Π_{TripSh} that each party has to run as a dealer. Recall that one instance of Π_{TripSh} generates $n^3tn^5p^2$ number of secret-shared multiplication-triples on behalf of the designated dealer. Consequently, each party has to invoke $\frac{c_M}{t}$ instances of Π_{TripSh} as a dealer, so that $c_M n^3 n^5 p^2$ secret-shared multiplication-triples are available on behalf of that party. To generate $c_M n^3 n^5 p^2$ secret-shared triples on its behalf, each designated party can act as a dealer and run $3c_M n^3$ instances of $\Pi_{A\langle\cdot\rangle}$ as a dealer, as each instance generates secret-sharing of n^5p^2 elements.

To generate $c_M n^3(t+1)$ vectors of random secret-shared triples, the parties run $\frac{c_M}{n-t}n^3(t+1)$ instances of the protocol $\Pi_{\mathsf{RandTriples}}$. This is because one instance of $\Pi_{\mathsf{RandTriples}}$ generates $n-t$ vectors of secret-shared random triples. Finally, to generate $2c_M n^3(t+1)$ vectors of double-packed-secret-shared vectors of random values, the parties need to run the protocol Π_{RandPair} $\frac{2c_M}{t(n-t)}n^3(t+1)$ times, as one instance of Π_{RandPair} generates $t(n-t)$ such vectors.

Protocol 8.40: The Preprocessing Phase Protocol – Π_{Prep}

1. **Mack-Key Setup Generation:** Every $P_i \in \mathcal{P}$ interacts with $\mathcal{F}_{\mathsf{MacKeys}}$ (Functionality 8.2) with input SetUp and receives from it the following.
   - Vectors $\{\boldsymbol{\mu}_{ij}\}_{j=1}^{n}$ where each $\boldsymbol{\mu}_{ij} \in \mathbb{F}^{n^2p}$ and where $\boldsymbol{\mu}_{ij}$ is designated to be used in the MAC-key for party P_j.
   - Corresponding to every pair of parties (P_j, P_k), the twisted-shares of P_i corresponding to $[\boldsymbol{\mu}_{jk}]_t^k$.

   For the rest of the protocol, the parties use the above setup in all the instances of $\Pi_{A\langle\cdot\rangle}$ involved to generate key-consistent ${}^{A}\langle\cdot\rangle_d$-sharings and key-consistent ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharings.
2. **Generating double-packed-secret-shared Random Values**: The parties execute $\frac{2c_M}{t(n-t)}n^3(t+1)$ instances of Π_{RandPair} (Protocol 8.13) to generate packed-double-secret-sharing of $2c_M n^3(t+1)n^5p^2$ random elements from $\mathbb{F}$.
3. **Generating Packed-Secret-Shared Random Triples**: The parties execute $\frac{c_M}{(n-t)}n^3(t+1)$ instances of $\Pi_{\mathsf{RandTriples}}$ (Protocol 8.18) to generate packed-secret-sharing of $c_M n^3(t+1)n^5p^2$ random triples from $\mathbb{F}$.
4. **Generating Packed-Secret-Shared Random Multiplication-Triples**:
   - Each $P_i \in \mathcal{P}$ acts as a dealer and invokes $\frac{2c_M}{t}$ instances of Π_{TripSh} (Protocol 8.30) and generates packed-secret-sharing of $2c_M n^8p^2$ random multiplication-triples over $\mathbb{F}$.
   - Each $P_i \in \mathcal{P}$ acts as a dealer and invokes $6c_M n^3$ instances of $\Pi_{A\langle\cdot\rangle}$ (Protocol 6.1) and generates packed-secret-sharing of $2c_M n^8p^2$ random triples over $\mathbb{F}$.

[23] Technically, we need the first $\frac{3t}{2}+1$ parties to provide only n^3 number of vectors of secret-shared multiplication-triples and the remaining $n-(\frac{3t}{2}+1)$ parties to provide n^3 number of vectors of secret-shared multiplication-triples and triples. However, for simplicity and to maintain uniformity, we ask for n^3 number of vectors of secret-shared multiplication-triples and triples from each party. Asymptotically, the communication cost still remains the same.

– The multiplication-triples and the triples shared by different parties are grouped into $2c_M$ disjoint groups, where in each group, there are $n^3n^5p^2$ number of packed-secret-shared multiplication-triples and triples from each party. The parties execute $2c_M$ instances of Π_{TripExt} (Protocol 8.35), one on behalf of each group and generate packed-secret-sharing of $c_Mn^3(t+1)n^5p^2$ random multiplication-triples over $\mathbb{F}$ [24].

We next prove the properties of the protocol Π_{Prep}.

Lemma 8.41. *Protocol Π_{Prep} generates the following, except with a negligible probability:*

- $c_Mn^3(t+1)n^5p^2$ ${}^{A}\langle\cdot\rangle_d$*-shared multiplication-triples over* $\mathbb{F}$.
- $c_Mn^3(t+1)n^5p^2$ ${}^{A}\langle\cdot\rangle_d$*-shared triples over* $\mathbb{F}$.
- $2c_Mn^3(t+1)n^5p^2$ *double-packed-secret-shared elements over* $\mathbb{F}$.

Proof. In the protocol, the parties execute $\frac{2c_M}{t(n-t)}n^3(t+1)$ instances of Π_{RandPair}. From Lemma 8.14, except with a negligible probability, each instance outputs packed-double-secret-sharing of $t(n-t)n^5p^2$ elements over $\mathbb{F}$. So $\frac{2c_M}{t(n-t)}n^3(t+1)$ instances will output $2c_Mn^3(t+1)n^5p^2$ such elements.

In the protocol, the parties execute $\frac{c_M}{(n-t)}n^3(t+1)$ instances of $\Pi_{\mathsf{RandTriples}}$. From Lemma 8.19, except with a negligible probability, each instance outputs $(n-t)n^5p^2$ ${}^{A}\langle\cdot\rangle_d$-shared triples over $\mathbb{F}$. So $\frac{c_M}{(n-t)}n^3(t+1)$ instances will output $c_Mn^3(t+1)n^5p^2$ such triples.

Finally, in the protocol, the parties execute $2c_M$ instances of Π_{TripExt} on the multiplication-triples and triples secret-shared by various parties. From Lemma 8.36, except with a negligible probability, each instance outputs $n^3(t+1)n^5p^2$ ${}^{A}\langle\cdot\rangle_d$-secret-shared multiplication-triples over $\mathbb{F}$. So $2c_M$ instances will output $c_Mn^3(t+1)n^5p^2$ such multiplication-triples [25]. □

Lemma 8.42. *In protocol Π_{Prep}, the view of the adversary remains independent of the output secret-shared triples, secret-shared multiplication-triples and double-secret-shared elements.*

Proof. This simply follows from the privacy of Π_{RandPair} (Lemma 8.15), privacy of $\Pi_{\mathsf{RandTriples}}$ (Lemma 8.20), privacy of Π_{TripSh} (Lemma 8.31), privacy of Π_{TripExt} (Lemma 8.37) and privacy of $\Pi_{{}^{A}\langle\cdot\rangle}$ (Lemma 6.4). □

Lemma 8.43. *Protocol Π_{Prep} requires a constant expected number of rounds, makes at most* 94 *calls to $\mathcal{F}_{\mathsf{Rand}}$ and incurs a communication of $\mathcal{O}(c_Mn^{11}\kappa)$ bits.*

Proof. As each building block used in the protocol requires a constant expected number of rounds, overall the protocol requires a constant expected number of rounds.

The number of calls to $\mathcal{F}_{\mathsf{Rand}}$ is computed as follows: we first note that step 1, 2, 3, 4 can be executed in parallel. Consequently, the number of calls to $\mathcal{F}_{\mathsf{Rand}}$ is dominated by step 4, where the parties generate packed-secret-shared multiplication-triples. The instances of Π_{TripSh} require at most 47 calls to $\mathcal{F}_{\mathsf{Rand}}$ for all the dealers (follow from Lemma 8.33). On the other hand, the instances of Π_{TripExt} will call $\mathcal{F}_{\mathsf{Rand}}$ at most 47 times. So the maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ is 94.

During step 2, the parties run $\frac{2c_M}{t(n-t)}n^3(t+1)$ instances of Π_{RandPair}. From Lemma 8.16, by substituting $L=\frac{2c_M}{t(n-t)}n^3(t+1)$, this incurs a communication cost of $\mathcal{O}(c_Mn^{11}\kappa)$ bits, as $t=\Theta(n)$.

During step 3, the parties run $\frac{c_M}{(n-t)}n^3(t+1)$ instances of $\Pi_{\mathsf{RandTriples}}$. From Lemma 8.21, by substituting $L=\frac{c_M}{(n-t)}n^3(t+1)$, this incurs a communication cost of $\mathcal{O}(c_Mn^{11}\kappa)$ bits.

During step 4, there are total $\frac{2c_Mn}{t}$ instances of Π_{TripSh} executed by all n parties. From Lemma 8.33, by substituting $L=\frac{2c_Mn}{t}$, this incurs a communication cost of $\mathcal{O}(c_Mn^{11}\kappa)$ bits. Moreover, there will be total $6c_Mn^4$ instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ executed by all n parties. From Lemma 6.8, this incurs a communication cost of $\mathcal{O}(c_Mn^{11}\kappa)$ bits. Furthermore, there will be $2c_M$ instances of Π_{TripExt}. From Lemma 8.38, by substituting $L=2c_M$, this incurs a communication cost of $\mathcal{O}(c_Mn^{11}\kappa)$ bits. □

[24] Using these parameters, parties actually generate packed-secret-sharing of $c_Mn^3(t+2)n^5p^2$ random multiplication-triples, however, we use only $c_Mn^3(t+1)n^5p^2$ as per our notation for simplicity.

[25] As mentioned earlier, this actually generates $c_Mn^3(t+2)n^5p^2$ random multiplication-triples, however, we use only $c_Mn^3(t+1)n^5p^2$ for simplicity.

As discussed in Section 4, each call to $\mathcal{F}_{\mathsf{Rand}}$ can be emulated by a constant expected rounds protocol, incurring a communication of $\mathcal{O}(n^5 \log n)$ bits. By using this protocol to securely realize the calls to $\mathcal{F}_{\mathsf{Rand}}$ in Π_{Prep}, the communication complexity of the protocol turns out to be $\mathcal{O}(c_M n^{11}\kappa)$ bits. The call to $\mathcal{F}_{\mathsf{MacKeys}}$ during step 1 can be securely realized by using the protocol $\mathit{\Pi}_{\mathsf{MacKeys}}$, which will incur a communication of $\mathcal{O}(n^6\kappa)$ bits. Hence the overall cost of the protocol in the plain model will be $\mathcal{O}(c_M n^{11}\kappa)$ bits.

From Lemma 8.43 and 8.41, the output of protocol Π_{Prep} consists of $\Theta(c_M n^{11})$ field elements and hence $\Theta(c_M n^{11}\kappa)$ bits. Since the total communication cost of the protocol is $\mathcal{O}(c_M n^{11}\kappa)$ bits, it follows that the protocol Π_{Prep} has $\mathcal{O}(1)$ communication overhead. We summarize the above discussion as the following theorem.

Theorem 8.44. *Protocol Π_{Prep} generates the following, except with a negligible probability:*

- $c_M n^3(t+1)n^5p^2$ ${}^{A}\langle\cdot\rangle_d$*-shared multiplication-triples over* $\mathbb{F}$.
- $c_M n^3(t+1)n^5p^2$ ${}^{A}\langle\cdot\rangle_d$*-shared triples over* $\mathbb{F}$.
- $2c_M n^3(t+1)n^5p^2$ *double-packed-secret-shared elements over* $\mathbb{F}$.

The view of the adversary remains independent of the output secret-shared triples, secret-shared multiplication-triples and double-secret-shared elements.

The protocol requires a constant expected number of rounds, incurs a communication of $\mathcal{O}(c_M n^{11}\kappa)$ bits and has a communication overhead of $\mathcal{O}(1)$ bits.

Lemma 8.45. *Protocol Π_{Prep} securely realizes $\mathcal{F}_{\mathsf{PREP}}$ in $\mathcal{F}_{\mathsf{MacKeys}}$-hybrid model with perfect security. The protocol calls $\mathcal{F}_{\mathsf{MacKeys}}$ once.*

Proof. The number of calls to $\mathcal{F}_{\mathsf{MacKeys}}$ is straight-forward. We now give the simulator $\mathcal{S}_{\mathsf{Prep}}$ for Π_{Prep}.

Let $\mathcal{A}$ be an arbitrary real-world adversary, attacking protocol Π_{Prep} and let $\mathcal{Z}$ be an arbitrary environment. We show the existence of a simulator $\mathcal{S}_{\mathsf{Prep}}$, such that the output of all parties and the adversary in an execution of the real protocol Π_{Prep} with $\mathcal{A}$ is identical to the output in an execution with $\mathcal{S}_{\mathsf{Prep}}$ involving $\mathcal{F}_{\mathsf{PREP}}$ in the ideal model. This further implies that $\mathcal{Z}$ cannot distinguish between the two executions.

The high-level idea behind the simulator $\mathcal{S}_{\mathsf{Prep}}$ is standard. During the simulated preprocessing, the simulator itself performs the role of the functionality $\mathcal{F}_{\mathsf{MacKeys}}$. This allows the simulator to learn the setup data that is supposed to be generated by the corrupt parties during $\mathcal{F}_{\mathsf{MacKeys}}$. The simulator then invokes the functionality $\mathcal{F}_{\mathsf{PREP}}$ with this setup data and receives. During simulation of Π_{RandPair} and $\Pi_{\mathsf{RandTriples}}$, whenever adversary invokes instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ on behalf of a corrupt party, the simulator plays the roles of honest parties as per the steps of $\Pi_{{}^{A}\langle\cdot\rangle}$ and learns the entire ${}^{A}\langle\cdot\rangle_d$-sharings of the corrupt party. Moreover, the simulator ensures consistency of the setup information corresponding to the honest parties with that of the corrupt parties fixed in the prior stage. On the other hand, for the honest parties, the simulator picks arbitrary values as their inputs and simulates the steps of the honest parties as per the protocol.

Note that due to the privacy guarantee of each primitive with the corrupt parties learning shares of random values unknown to them in each primitive in the preprocesing phase, their view remains independent of the underlying values corresponding to the random pairs, random triples and the multiplication triples. Hence, the simulator can ensure the indistinguishability of the adversary's view from that in the real world.

Simulator 8.46: $\mathcal{S}_{\mathsf{Prep}}$: Simulator for the Protocol Π_{Prep}

$\mathcal{S}_{\mathsf{Prep}}$ constructs virtual real-world honest parties and invokes the real-world adversary $\mathcal{A}$ who controls at most t parties. The simulator simulates the view of $\mathcal{A}$, namely its communication with $\mathcal{Z}$, the messages sent by the honest parties and the interaction with $\mathcal{F}_{\mathsf{MacKeys}}$. In order to simulate $\mathcal{Z}$, the simulator forwards every message it receives from $\mathcal{Z}$ to $\mathcal{A}$ and vice-versa. The simulator then simulates the various steps of the protocol as follows.

(Simulating $\mathcal{F}_{\mathsf{MacKeys}}$): Emulate the functionality $\mathcal{F}_{\mathsf{MacKeys}}$ as follows:

- On behalf of every P_j under the control of $\mathcal{A}$:

- Receive from $\mathcal{A}$ the n vectors $\{\boldsymbol{\mu}_{ji}\}_{i=1}^{n}$ where each $\boldsymbol{\mu}_{ji} \in \mathbb{F}^{n^2p}$.
- For every $\boldsymbol{\mu}_{ji}$, receive from $\mathcal{A}$ the sharing $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ such that the twisted-share of P_i is 0. If $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ is not a valid twisted t-sharing with twisted-shares of P_i being not 0, then send 0s as the input to the functionality.
- Invoke $\mathcal{F}_{\mathsf{PREP}}$ with the above values, receive from it the shares of every P_k under the control of $\mathcal{A}$ corresponding to $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ and send it to $\mathcal{A}$.

– On behalf of every party P_j not under the control of $\mathcal{A}$:
- For every $P_i \in \mathcal{P}$, receive from $\mathcal{F}_{\mathsf{PREP}}$ the shares of $\lceil \boldsymbol{\mu}_{ji} \rfloor_t^i$ corresponding to each P_k controlled by $\mathcal{A}$ and send it to $\mathcal{A}$.
- Randomly pick t-degree polynomials $q_{ji}^{(1)}(\cdot), \ldots, q_{ji}^{(n^2p)}$, such that they are consistent with the shares of each P_k under the control of $\mathcal{A}$ received from $\mathcal{F}_{\mathsf{PREP}}$ and $q_{ji}^{(\ell)}(0) = 0$ for all $\ell \in \{1, \ldots, n^2p\}$. Let $u_{ji}^{(\ell)} = q_{ji}^{(\ell)}(i)$.

In each of the following cases, extract the shares of the corrupt parties and forward them to $\mathcal{F}_{\mathsf{PREP}}$. Also, choose the second component of the MAC key on behalf of each honest party randomly and simulate the following while ensuring consistency with the entire setup.
(Simulating Π_{RandPair}): Simulate the steps of Π_{RandPair} as in Simulation 8.17.
(Simulating $\Pi_{\mathsf{RandTriples}}$): Simulate the steps of $\Pi_{\mathsf{RandTriples}}$ as in Simulation 8.22.
(Simulating Packed-Secret-Shared Random Multiplication-Triples Generation)

– **(Simulating Π_{TripSh}):** Simulate the steps of Π_{TripSh} as in Simulation 8.34.
– **(Simulating Random Triple Generation):** Simulate the steps of $\Pi_{^A\langle\cdot\rangle}$ as in Simulation 6.10 for an honest dealer for each party not under the control of $\mathcal{A}$ and for a corrupt dealer for each party P_k under the control of $\mathcal{A}$.
– **(Simulating Π_{TripExt}):** Simulate the steps of Π_{TripExt} as in Simulation 8.39.

□

We next prove a sequence of claims, which help us to show that the joint distribution of the parties are identical in both the real-world, as well as ideal-world, except with a negligible probability. We first show that the view generated by $\mathcal{S}_{\mathsf{Prep}}$ for $\mathcal{A}$ is identically distributed as $\mathcal{A}$'s view during the real execution of Π_{Prep}.

Claim 8.47. *The view of $\mathcal{A}$ in the simulated execution with $\mathcal{S}_{\mathsf{Prep}}$ is identically distributed as the view of $\mathcal{A}$ in the real execution of Π_{Prep}.*

Proof. It is easy to see that the view of $\mathcal{A}$ during the emulation of $\mathcal{F}_{\mathsf{MacKeys}}$ is identically distributed in both the executions. This is because in both the executions, $\mathcal{A}$ receives no messages from the honest parties and the steps of $\mathcal{F}_{\mathsf{MacKeys}}$ are executed by the simulator itself in the simulated execution. Namely, in both the executions, $\mathcal{A}$'s view consists of the MAC-key components of the parties in Cr and the twisted-shares of the MAC-key components of the honest parties. Let us fix all this data. Now conditioned on this data, for Π_{RandPair} and $\Pi_{\mathsf{RandTriples}}$, $\mathcal{A}$ learns its supposed data corresponding to the sharings $\{^A\langle\mathbb{R}\rangle_d, ^A\langle\mathbb{R}\rangle_{d+p-1}\}$ and $\{^A\langle\hat{\mathbb{A}}\rangle_d, ^A\langle\hat{\mathbb{B}}\rangle_d, ^A\langle\hat{\mathbb{C}}\rangle_d\}$ during the real execution, while in the simulated execution it learns its supposed data corresponding to the sharings $\{^A\langle\widetilde{\mathbb{R}}\rangle_d, ^A\langle\widetilde{\mathbb{R}}\rangle_{d+p-1}\}$ and $\{^A\langle\tilde{\hat{\mathbb{A}}}\rangle_d, ^A\langle\tilde{\hat{\mathbb{B}}}\rangle_d, ^A\langle\tilde{\hat{\mathbb{C}}}\rangle_d\}$, where the elements of $\widetilde{\mathbb{R}}, \tilde{\hat{\mathbb{A}}}, \tilde{\hat{\mathbb{B}}}, \tilde{\hat{\mathbb{C}}}$ are randomly chosen by the simulator. It is easy to see that the distributions of the data held by $\mathcal{A}$ in both the real and simulated executions are identical, as simulator honestly plays the role of honest parties in the underlying $\Pi_{^A\langle\cdot\rangle}$ instances involving honest dealers. So let us fix the data held by $\mathcal{A}$ for both these protocols.

Similarly, during the instances of Π_{TripSh}, $\mathcal{A}$ learns its supposed data corresponding to the sharings $\{^A\langle\mathbb{X}\rangle_d, ^A\langle\mathbb{Y}\rangle_d, ^A\langle\mathbb{Z}\rangle_d\}$ during the real execution, while in the simulated execution it learns its supposed data corresponding to the sharings $\{^A\langle\hat{\mathbb{X}}\rangle_d, ^A\langle\hat{\mathbb{Y}}\rangle_d, ^A\langle\hat{\mathbb{Z}}\rangle_d\}$, where the elements of $\hat{\mathbb{X}}, \hat{\mathbb{Y}}, \hat{\mathbb{Z}}$ are randomly chosen by the simulator.

As in the prior case, during random triple generation as well, the difference between the real and simulated execution corresponds to the simulator choosing the random triples in the latter case in contrast to the honest party choosing them in the real execution. It is thus easy to see that the distribution of

data held by $\mathcal{A}$ is identical in both the cases, since the simulator follows the protocol steps honestly on behalf of the honest parties.

Finally, during triple extraction, the simulator uses the random triples it chose on behalf of the honest parties as the input, while ensuring that the shares of the corrupt parties are as obtained from the functionality $\mathcal{F}_{\mathsf{PREP}}$. Since the simulator performs the role of the honest parties as in the real-world, the distribution in both the worlds is identically distributed. □

We next claim that conditioned on the view of $\mathcal{A}$, the output of the honest parties is identically distributed in both worlds, except with a negligible probability.

Claim 8.48. *Conditioned on the view of $\mathcal{A}$, the output of the honest parties is identically distributed in the real execution of Π_{Prep} involving $\mathcal{A}$, as well as in the ideal execution involving $\mathcal{S}_{\mathsf{Prep}}$ and $\mathcal{F}_{\mathsf{PREP}}$.*

Proof. Let view be an arbitrary view of $\mathcal{A}$. According to view, for every gate in each of the SIMD circuits, there exist double-packed secret-shared random values. Specifically, there exist $2c_M n^3(t+1)$ vectors of random values $\{\mathbb{R}\}$ where $\mathbb{R} \in \mathbb{F}^{n^5p^2}$, such that the parties hold $({}^A\langle\mathbb{R}\rangle_d, {}^A\langle\mathbb{R}\rangle_{d+p-1})$. Further, there exist $c_M n^3(t+1)$ vectors of random triples of the form $\{\hat{\mathbb{A}}, \hat{\mathbb{B}}, \hat{\mathbb{C}}\}$ where $\hat{\mathbb{A}}, \hat{\mathbb{B}}, \hat{\mathbb{C}} \in \mathbb{F}^{n^5p^2}$, such that the parties hold $({}^A\langle\hat{\mathbb{A}}\rangle_d, {}^A\langle\hat{\mathbb{B}}\rangle_d, {}^A\langle\hat{\mathbb{C}}\rangle_d)$. Finally, there exist $c_M n^3(t+1)$ vectors of multiplication triples of the form $\{\mathbb{A}, \mathbb{B}, \mathbb{C}\}$ where $\mathbb{A}, \mathbb{B}, \mathbb{C} \in \mathbb{F}^{n^5p^2}$, such that the parties hold $({}^A\langle\mathbb{A}\rangle_d, {}^A\langle\mathbb{B}\rangle_d, {}^A\langle\mathbb{C}\rangle_d)$. From Claim 9.5, we have that the values $\mathbb{R}$, $(\hat{\mathbb{A}}, \hat{\mathbb{B}}, \hat{\mathbb{C}})$ and $(\mathbb{A}, \mathbb{B}, \mathbb{C})$ are uniformly distributed conditioned on the data for the corresponding sharings that is available to $\mathcal{A}$ as determined by the view.

It is easy to see that the output of the honest parties in the ideal-world correspond to shares of random double-packed pairs, random triples and random multiplication triples. We now show that the output of the honest parties in the real world is the same as that in the ideal world except with negligible probability. This follows in a straightforward manner from the correctness of Π_{Prep} given in Lemma 8.41, which in turn relies on the correctness of Π_{RandPair} (Lemma 8.14), $\Pi_{\mathsf{RandTriples}}$ (Lemma 8.19), Π_{TripSh} (Lemmas 8.31, 8.32), Π_{TripExt} (Lemma 8.36). □

9 The Circuit-Evaluation Protocol

In this section, we present our circuit-evaluation protocol, which securely realizes the functionality $\mathcal{F}_{\mathsf{CktEval}}$ presented in Functionality 9.1. The functionality evaluates the circuit ckt in SIMD fashion over $n^8(t+1)p^2$ inputs in parallel. The functionality receives $n^8(t+1)p^2$ values for the input x_i from every party P_i. As discussed in Section 7, these inputs are arranged into n^3 groups, where within each group there will be further $(t+1)$ subgroups, with each subgroup consisting of n^5p^2 values. The functionality then evaluates each gate of ckt in topological order over $n^8(t+1)p^2$ inputs in parallel. Once all the gates are evaluated, the functionality sends the $n^8(t+1)p^2$ outputs to all.

Functionality 9.1: $\mathcal{F}_{\mathsf{CktEval}}$ — The ideal functionality for evaluating the circuit ckt in SIMD fashion

The functionality is parametrized by a publicly-known function $f : \mathbb{F}^n \to \mathbb{F}$, where each party $P_i \in \mathcal{P}$ has an input x_i for f and every party in $\mathcal{P}$ is supposed to receive the function output $o = f(x_1, \ldots, x_n)$. The function f is represented by a publicly-known circuit ckt over $\mathbb{F}$. The functionality is also parametrized by the set Cr of corrupt parties. The functionality interacts with the parties in $\mathcal{P}$ and an adversary $\mathcal{S}$ as follows.

- For every $P_i \notin$ Cr, upon receiving $n^8(t+1)p^2$ values from $\mathbb{F}$ for the input x_i, arrange them as $\{\mathbb{X}_i^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ and associate them with wire x_i, where $|\mathbb{X}_i^{(\beta,\gamma)}| = n^5p^2$.
- For every $P_i \in$ Cr, upon receiving $n^8(t+1)p^2$ values from $\mathbb{F}$ for the input x_i from $\mathcal{S}$, arrange them in the same way as in the previous step.
- For every gate $g \in$ ckt in topological order, on having the inputs $\{\mathbb{X}^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ and $\{\mathbb{Y}^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ associated with x and y respectively, do the following.
  - **Addition Gate**: If g is an addition gate then component-wise compute

$$\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} + \mathbb{Y}^{(\beta,\gamma)}.$$

  Associate $\{\mathbb{Z}^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ with wire z.

- **Multiplication Gate**: If g is a multiplication-gate, then component-wise compute

$$\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}.$$

  Associate $\{\mathbb{Z}^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ with wire z.

– On having the values $\{\mathbb{O}^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ associated with the output-wire o of $\mathtt{ckt}$, send them to $\mathcal{S}$ and all the parties $P_i \notin \mathsf{Cr}$. .

The circuit-evaluation protocol Π_{ckteval} for securely realizing the functionality $\mathcal{F}_{\mathsf{CktEval}}$ is present in Protocol 9.2. The protocol is present in $\mathcal{F}_{\mathsf{PREP}}$-hybrid model. The high-level idea of the protocol is very simple and standard. The parties call the functionality $\mathcal{F}_{\mathsf{PREP}}$ that generates the setup for MAC-keys and all the "raw data" for securely evaluating the multiplication-gates in $\mathtt{ckt}$. The parties then start evaluating each gate in ${}^{A}\langle\cdot\rangle_d$-shared fashion, where $d = t+p-1$. To begin with, the parties secret-share their inputs for $\mathtt{ckt}$ using instances of our VSS protocol $\Pi_{{}^{A}\langle\cdot\rangle}$. The addition gates are evaluated non-interactively by deploying the linearity property of ${}^{A}\langle\cdot\rangle_d$-sharings. The multiplication-gates are evaluated using instances of the protocol Π_{Beaver}. Finally, the parties publicly reconstruct the ${}^{A}\langle\cdot\rangle_d$-shared circuit outputs.

Protocol 9.2: Π_{ckteval}: The circuit-evaluation Protocol

Every party $P_i \in \mathcal{P}$ interacts with $\mathcal{F}_{\mathsf{PREP}}$ with input $\mathsf{SetUp}, \mathsf{Multiplication-Triples}, \mathsf{Triples}$ and Double and receives the following.

– Vectors $\{\boldsymbol{\mu}_{ij}\}_{j=1}^{n}$ where each $\boldsymbol{\mu}_{ij} \in \mathbb{F}^{n^2p}$ and where $\boldsymbol{\mu}_{ij}$ is designated to be used in the MAC-key for party P_j.

– Corresponding to every pair of parties (P_j, P_k), the twisted-shares of P_i corresponding to $[\boldsymbol{\mu}_{jk}]_t^k$.

– For $\alpha \in \{1,\ldots,c_M\}, \beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$, its information corresponding to $({}^{A}\langle\mathbb{A}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{B}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{C}^{(\alpha,\beta,\gamma)}\rangle_d)$ and $({}^{A}\langle\hat{\mathbb{A}}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{B}}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{C}}^{(\alpha,\beta,\gamma)}\rangle_d)$. Here each $\mathbb{A}^{(\alpha,\beta,\gamma)}$, $\mathbb{B}^{(\alpha,\beta,\gamma)}, \mathbb{C}^{(\alpha,\beta,\gamma)}, \hat{\mathbb{A}}^{(\alpha,\beta,\gamma)}, \hat{\mathbb{B}}^{(\alpha,\beta,\gamma)}, \hat{\mathbb{C}}^{(\alpha,\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$ and component-wise $\mathbb{C}^{(\alpha,\beta,\gamma)} = \mathbb{A}^{(\alpha,\beta,\gamma)} \times \mathbb{B}^{(\alpha,\beta,\gamma)}$.
  % Recall that $\mathcal{F}_{\mathsf{PREP}}$ generates $c_M n^3(t+1)$ number of ${}^{A}\langle\cdot\rangle_d$-sharing of n^5p^2 random multiplication-triples and random triples each.

– For $\alpha \in \{1,\ldots,c_M\}, \beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$, its information corresponding to $({}^{A}\langle\mathbb{R}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{R}^{(\alpha,\beta,\gamma)}\rangle_{d+p-1})$ and $({}^{A}\langle\hat{\mathbb{R}}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{R}}^{(\alpha,\beta,\gamma)}\rangle_{d+p-1})$. Here each $\mathbb{R}^{(\alpha,\beta,\gamma)}, \hat{\mathbb{R}}^{(\alpha,\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$.
  % Recall that $\mathcal{F}_{\mathsf{PREP}}$ generates $2c_M n^3(t+1)$ number of ${}^{A}\langle\cdot\rangle_d$-sharing and ${}^{A}\langle\cdot\rangle_{d+p-1}$-sharing of n^5p^2 random values.

For $\alpha \in \{1,\ldots,c_M\}$, the parties associate the secret-sharings $({}^{A}\langle\mathbb{A}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{B}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{C}^{(\alpha,\beta,\gamma)}\rangle_d)$, $({}^{A}\langle\hat{\mathbb{A}}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{B}}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\hat{\mathbb{C}}^{(\alpha,\beta,\gamma)}\rangle_d)$ and the secret-sharings $({}^{A}\langle\mathbb{R}^{(\alpha,\beta,\gamma)}\rangle_d, {}^{A}\langle\mathbb{R}^{(\alpha,\beta,\gamma)}\rangle_{d+p-1})$, $({}^{A}\langle\hat{\mathbb{R}}^{(\alpha,\beta,\gamma)}\rangle_d$, ${}^{A}\langle\hat{\mathbb{R}}^{(\alpha,\beta,\gamma)}\rangle_{d+p-1})$ with the αth multiplication gate in $\mathtt{ckt}$, where $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$.

% Each multiplication gate will be evaluated over $n^3(t+1)$ number of ${}^{A}\langle\cdot\rangle_d$-shared vectors of inputs of size n^5p^2 using instances of Π_{Beaver}.

The parties then evaluate each gate in $\mathtt{ckt}$ in topological order as follows.

– **Input Gate**: Every $P_i \in \mathcal{P}$ on having $n^8(t+1)p^2$ different values for input-wire x_i of $\mathtt{ckt}$ does the following.
  - Divide the inputs into $n^3(t+1)$ groups, each consisting of n^5p^2 values. For $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$, let these groups be denoted by $\mathbb{X}_i^{(\beta,\gamma)}$, where $\mathbb{X}^{(\beta,\gamma)} = (\mathbf{X}^{(\beta,\gamma,1)},\ldots,\mathbf{X}^{(\beta,\gamma,n^3)})$. Moreover, each $\mathbf{X}^{(\beta,\gamma,u)} = (\boldsymbol{x}^{(\beta,\gamma,u,1)},\ldots,\boldsymbol{x}^{(\beta,\gamma,u,n^2p)})$. Furthermore, each $\boldsymbol{x}^{(\beta,\gamma,u,v)} \in \mathbb{F}^p$.
  - For $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$, generate ${}^{A}\langle\mathbb{X}_i^{(\beta,\gamma)}\rangle_d$ by acting as a dealer with input $\mathbb{X}_i^{(\beta,\gamma)}$ and invoking an instance of protocol $\Pi_{{}^{A}\langle\cdot\rangle}$ (Protocol 6.1).

  For every $P_i \in \mathcal{P}$, the parties in $\mathcal{P}$ associate the sharings $\{{}^{A}\langle\mathbb{X}_i^{(\beta,\gamma)}\rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ with wire x_i.

– **Addition Gate**: For every addition gate in $\mathtt{ckt}$ with input wires x, y and output wire z, the parties in $\mathcal{P}$ do the following.

- For $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$, on having the inputs ${}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d$ and ${}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d$ associated with wires x and y respectively, locally compute

$$ {}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d = {}^{A}\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d + {}^{A}\langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d.$$

- Associate the sharings $\{{}^{A}\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ with wire z.

– **Multiplication Gate**: If this is the αth multiplication gate in `ckt` with input wires $x^{(\alpha)}, y^{(\alpha)}$ and output wire $z^{(\alpha)}$ where $\alpha \in \{1,\ldots,c_M\}$, then the parties with inputs $\{{}^{A}\langle \mathbb{X}^{(\alpha,\beta,\gamma)} \rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ and $\{{}^{A}\langle \mathbb{Y}^{(\alpha,\beta,\gamma)} \rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ associated with wires $x^{(\alpha)}$ and $y^{(\alpha)}$ respectively where $\mathbb{X}^{(\alpha,\beta,\gamma)}$, $\mathbb{Y}^{(\alpha,\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$, do the following.
  - For each $k \in \{1,\ldots,n\}$, run an instance of Π_{Beaver} (Protocol 8.23) with inputs $\{{}^{A}\langle \mathbb{X}^{(\alpha,\beta,\gamma)} \rangle_d$, ${}^{A}\langle \mathbb{Y}^{(\alpha,\beta,\gamma)} \rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ and auxiliary inputs $({}^{A}\langle \mathbb{A}^{(\alpha,\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{B}^{(\alpha,\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{C}^{(\alpha,\beta,\gamma)} \rangle_d)$, $({}^{A}\langle \hat{\mathbb{A}}^{(\alpha,\beta,\gamma)} \rangle_d, {}^{A}\langle \hat{\mathbb{B}}^{(\alpha,\beta,\gamma)} \rangle_d, {}^{A}\langle \hat{\mathbb{C}}^{(\alpha,\beta,\gamma)} \rangle_d)$, $({}^{A}\langle \mathbb{R}^{(\alpha,\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{R}^{(\alpha,\beta,\gamma)} \rangle_{d+p-1})$ and $({}^{A}\langle \hat{\mathbb{R}}^{(\alpha,\beta,\gamma)} \rangle_d, {}^{A}\langle \hat{\mathbb{R}}^{(\alpha,\beta,\gamma)} \rangle_{d+p-1})$ associated with the αth multiplication-gate where $\beta \in \{(k-1)n^3+1,\ldots,kn^3\}$ and $\gamma \in \{1,\ldots,t+1\}$.[26] Let $\{{}^{A}\langle \mathbb{Z}^{(\alpha,\beta,\gamma)} \rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ be the output from the instance of Π_{Beaver}.
  - The parties associate the sharings ${}^{A}\langle \mathbb{Z}^{(\alpha,\beta,\gamma)} \rangle_d$ with the output wite $z^{(\alpha)}$, where $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$.

– **Output Gate**: Let o be the output-wire of `ckt`. For $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$, parties on having the input ${}^{A}\langle \mathbb{O}^{(\beta,\gamma)} \rangle_d$ associated with wire o does the following.
  - Publicly reconstruct $\mathbb{O}^{(\beta,\gamma)}$, output $\mathbb{O}^{(\beta,\gamma)}$ and terminate.

Lemma 9.3. *Protocol Π_{ckteval} requires $\mathcal{O}(D_M)$ expected number of rounds and incurs a communication of $\mathcal{O}(c_M n^{11}\kappa + n^{12}\kappa)$ bits. Here c_M is the number of multiplication gates in `ckt` and D_M is the multiplicative depth of `ckt`.*

Proof. During the input sharing phase, each party $P_i \in \mathcal{P}$ invokes $\Pi_{{}^{A}\langle\cdot\rangle}$ $n^3(t+1)$ times. Note that all these instances can be executed in parallel for all the n parties. Hence from Lemma 6.8, this requires a total communication of $\mathcal{O}(n^{12}\kappa)$ bits, a constant expected number of rounds and at most 15 calls to $\mathcal{F}_{\mathsf{Rand}}$. Addition gates are computed locally and hence do not incur any communication cost. Further, for each multiplication gate $\alpha \in \{1,\ldots,c_M\}$, parties invoke n instances of Π_{Beaver}. From Lemma 8.28, this requires a communication of $\mathcal{O}(n^{11}\kappa)$ bits, a constant expected number of rounds and at most 31 calls to $\mathcal{F}_{\mathsf{Rand}}$. Note that the multiplication gates at the same multiplicative depth can be evaluated in parallel by executing the corresponding instances of Π_{Beaver} in parallel. The total communication cost for evaluating all the multiplication gates will be $\mathcal{O}(c_M n^{11}\kappa)$ bits. On the other hand, the expected number of rounds required for evaluating all the multiplication gates will be $\mathcal{O}(D_M)$. The maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ for evaluating all the multiplication gates could be $31c_M$. The calls to $\mathcal{F}_{\mathsf{Rand}}$ can be securely realized using a constant expected round protocol, which will incur a total communication of $\mathcal{O}(c_M n^5\kappa)$ bits. Finally, the output gate values are publicly reconstructed which incurs a communication of $\mathcal{O}(n^{12}\kappa)$ and one round as described in Section 4.3. □

The security of the protocol Π_{ckteval} is standard and directly follows from the security of the protocols $\Pi_{{}^{A}\langle\cdot\rangle}$ and Π_{Beaver} and from the properties of the preprocessing data generated by $\mathcal{F}_{\mathsf{PREP}}$. Namely, the privacy and correctness of $\Pi_{{}^{A}\langle\cdot\rangle}$ (Lemma 6.4 and Lemma 6.6 respectively) guarantees that the inputs of every honest party remain private and are ${}^{A}\langle\cdot\rangle_d$-shared. On the other hand, the commitment of $\Pi_{{}^{A}\langle\cdot\rangle}$ (Lemma 6.7) guarantees that even the inputs of the corrupt parties are ${}^{A}\langle\cdot\rangle_d$-shared. The linearity of ${}^{A}\langle\cdot\rangle_d$-sharing guarantees that the addition gates are evaluated correctly. On the other hand, the security of Π_{Beaver} guarantees that even the multiplication gates are evaluated securely. Finally, the honest parties correctly reconstruct the circuit outputs. We formalize the above intuition by giving a simulator $\mathcal{S}_{\mathsf{ckteval}}$ (Simulator 9.4) for the protocol Π_{ckteval}.

More formally, let $\mathcal{A}$ be an arbitrary real-world adversary, attacking protocol Π_{ckteval} and let $\mathcal{Z}$ be an arbitrary environment. We show the existence of a simulator $\mathcal{S}_{\mathsf{ckteval}}$, such that for any set of corrupted parties Cr with $|\mathsf{Cr}| \leq t$ and for all inputs, the output of all parties and the adversary in an execution of

[26] Recall that a single instance of Π_{Beaver} can compute product of $n^2(t+1)n^5p^2$ secret-shared inputs. Since we want to compute product of $n^3(t+1)n^5p^2$ secret-shared inputs, we have to run n such instances.

the real protocol with $\mathcal{A}$ is identical to the output in an execution with $\mathcal{S}_{\mathsf{ckteval}}$ involving $\mathcal{F}_{\mathsf{CktEval}}$ in the ideal model, except with a negligible probability. This further implies that $\mathcal{Z}$ cannot distinguish between the two executions.

The high-level idea behind the simulator $\mathcal{S}_{\mathsf{ckteval}}$ is standard. During the simulated circuit-evaluation, the simulator itself performs the role of the functionality $\mathcal{F}_{\mathsf{PREP}}$. This allows the simulator to learn the complete data supposed to be generated by $\mathcal{F}_{\mathsf{PREP}}$. For the input gates, whenever adversary invokes instances of $\Pi_{A\langle\cdot\rangle}$ on behalf of a corrupt party, the simulator plays the roles of honest parties as per the steps of $\Pi_{A\langle\cdot\rangle}$ and learns the entire ${}^{A}\langle\cdot\rangle_d$-sharings of the circuit-inputs of the corrupt party. On the other hand, for the honest parties, the simulator picks arbitrary values as their circuit-inputs and simulates the steps of the honest parties as per the steps of the protocol $\Pi_{A\langle\cdot\rangle}$. The simulator then sends the circuit-inputs of the corrupt parties to $\mathcal{F}_{\mathsf{CktEval}}$ and learns the circuit-outputs. The simulator then simulates the steps of the honest parties for the evaluation of various gates as per the protocol. Finally, for the output gate, the simulator arbitrarily computes ${}^{A}\langle\cdot\rangle_d$-sharings of the circuit-outputs received from $\mathcal{F}_{\mathsf{CktEval}}$, which are consistent with the data which corrupt parties hold for output-gate sharings during the simulated execution. Note that this is possible for the simulator.

In more detail, let $\{{}^{A}\langle\widetilde{\mathbb{O}}^{(\beta,\gamma)}\rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ be the sharings, corresponding to the output gate, available with $\mathcal{S}_{\mathsf{ckteval}}$ during the simulated circuit-evaluation, where each $\widetilde{\mathbb{O}}^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$ and let $\mathbb{O}^{(\beta,\gamma)}$ be the outputs received by $\mathcal{S}_{\mathsf{ckteval}}$ from $\mathcal{F}_{\mathsf{PREP}}$ where each $\mathbb{O}^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$. The goal of the simulator is to compute the sharings ${}^{A}\langle\mathbb{O}^{(\beta,\gamma)}\rangle_d$ where the data held by the corrupt parties are identical for ${}^{A}\langle\mathbb{O}^{(\beta,\gamma)}\rangle_d$ and ${}^{A}\langle\widetilde{\mathbb{O}}^{(\beta,\gamma)}\rangle_d$

Let $\widetilde{\mathbb{O}}^{(\beta,\gamma)} = \widetilde{\mathbf{O}}^{(\beta,\gamma,1)},\ldots,\widetilde{\mathbf{O}}^{(\beta,\gamma,n^3)}$ where each $\widetilde{\mathbf{O}}^{(\beta,\gamma,u)} = (\widetilde{\mathbf{o}}^{(\beta,\gamma,u,1)},\ldots,\widetilde{\mathbf{o}}^{(\beta,\gamma,u,n^2p)})$ and where each $\widetilde{\mathbf{o}}^{(\beta,\gamma,u,v)} \in \mathbb{F}^p$. Then as part of ${}^{A}\langle\widetilde{\mathbb{O}}^{(\beta,\gamma)}\rangle_d$, the parties hold ${}^{A}\langle\widetilde{\mathbf{O}}^{(\beta,\gamma,1)}\rangle_d,\ldots,{}^{A}\langle\widetilde{\mathbf{O}}^{(\beta,\gamma,n^3)}\rangle_d$, where each vector $\widetilde{\mathbf{o}}^{(\beta,\gamma,u,v,w)}$ is degree-(d)-packed-secret-shared. Let $\mathbb{O}^{(\beta,\gamma)} = \mathbf{O}^{(\beta,\gamma,1)},\ldots,\mathbf{O}^{(\beta,\gamma,n^3)}$ where each $\mathbf{O}^{(\beta,\gamma,u)} = (\mathbf{o}^{(\beta,\gamma,u,1)},\ldots,\mathbf{o}^{(\beta,\gamma,u,n^2p)})$ and where each $\mathbf{o}^{(\beta,\gamma,u,v)} \in \mathbb{F}^p$. Then for each vector $\mathbf{o}^{(\beta,\gamma,u,v)}$, the simulator computes a degree-(d)-packed-secret-sharing, where the shares of the corrupt parties are identical as in the degree-(d)-packed-secret-sharing of $\widetilde{\mathbf{o}}^{(\beta,\gamma,u,v)}$. This is possible since the shares of t corrupt parties and the p elements of $\mathbf{o}^{(\beta,\gamma,u,v)}$ (resp. $\widetilde{\mathbf{o}}^{(\beta,\gamma,u,v)}$) uniquely define the underlying degree-d sharing-polynomials. Next, the simulator computes ${}^{A}\langle\mathbf{O}^{(\beta,\gamma,1)}\rangle_d,\ldots,{}^{A}\langle\mathbf{O}^{(\beta,\gamma,n^3)}\rangle_d$, where the MAC-tags of the parties $P_j \notin \mathsf{Cr}$ are consistent with the MAC-keys held by the parties in Cr as part of ${}^{A}\langle\widetilde{\mathbf{O}}^{(\beta,\gamma,1)}\rangle_d,\ldots,{}^{A}\langle\widetilde{\mathbf{O}}^{(\beta,\gamma,n^3)}\rangle_d$. For this, the simulator just need to compute the MAC-tags of the parties $P_j \notin \mathsf{Cr}$ on their shares of the vectors $\mathbf{o}^{(\beta,\gamma,u,v)}$ under the MAC-keys of the parties in Cr which are already known to the simulator (as part of ${}^{A}\langle\widetilde{\mathbb{O}}^{(\beta,\gamma)}\rangle_d$).

Once simulator computes the sharings ${}^{A}\langle\mathbb{O}^{(\beta,\gamma)}\rangle_d$ as above, on behalf of the honest parties, the simulator sends their shares and corresponding MAC-tags of ${}^{A}\langle\mathbb{O}^{(\beta,\gamma)}\rangle_d$ to the adversary. This ensures that in the simulated execution, Adv learns circuit-outputs $\mathbb{O}^{(\beta,\gamma)}$.

Simulator 9.4: $\mathcal{S}_{\mathsf{ckteval}}$: Simulator for the circuit-evaluation Protocol Π_{ckteval}

$\mathcal{S}_{\mathsf{ckteval}}$ constructs virtual real-world honest parties and invokes the real-world adversary $\mathcal{A}$ who controls the set of parties Cr. The simulator simulates the view of $\mathcal{A}$, namely its communication with $\mathcal{Z}$, the messages sent by the honest parties and the interaction with $\mathcal{F}_{\mathsf{PREP}}$. In order to simulate $\mathcal{Z}$, the simulator forwards every message it receives from $\mathcal{Z}$ to $\mathcal{A}$ and vice-versa. The simulator then simulates the various steps of the protocol as follows.

Interaction with $\mathcal{F}_{\mathsf{PREP}}$: The simulator itself playing the role of $\mathcal{F}_{\mathsf{PREP}}$. Namely, the simulator receives from $\mathcal{A}$ all the MAC-key components and their twisted t-sharings that the parties in Cr would like to have. Similarly, the simulator receives from $\mathcal{A}$ the shares of the parties in Cr for all multiplication-triples and double-sharings. The simulator then executes the steps of $\mathcal{F}_{\mathsf{PREP}}$ and generates key-consistent secret-sharings of random multiplication-triples and random double-sharings, which are consistent with the shares corresponding to the parties in Cr, as provided by $\mathcal{A}$. At the end of simulation of this step, the simulator will know the entire vectors of MAC-key components, their twisted t-sharings, all secret-shared multiplication-triples and all secret-shared random double-sharings.

Input Gates: The simulator simulates the evaluation of input gates as follows.

– For every $P_i \notin \mathsf{Cr}$, the simulator picks $n^8(t+1)p^2$ random inputs from $\mathbb{F}$, groups them as $\{\widetilde{\mathbb{X}}_i^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ where each $\widetilde{\mathbb{X}}_i^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$. For every $\mathbb{X}_i^{(\beta,\gamma)}$, the simulator then run an instance of $\Pi_{A\langle\cdot\rangle}$ on behalf of P_i by acting as a dealer and interacts with $\mathcal{A}$ to generate ${}^A\langle\widetilde{\mathbb{X}}_i^{(\beta,\gamma)}\rangle_d$.
– For every $P_i \in \mathsf{Cr}$, the simulator $\mathcal{S}_{\mathsf{ckteval}}$ plays the role of the honest parties as per the steps of $\Pi_{A\langle\cdot\rangle}$ and interacts with $\mathcal{A}$ on behalf of the honest parties in the instances of $\Pi_{A\langle\cdot\rangle}$ where P_i plays the role of the dealer. Let $\{\widetilde{\mathbb{X}}_i^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ be the inputs whose ${}^A\langle\cdot\rangle_d$-sharings are generated during these instances where each $\widetilde{\mathbb{X}}_i^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$. Note that $\mathcal{S}_{\mathsf{ckteval}}$ will know the complete ${}^A\langle\widetilde{\mathbb{X}}_i^{(\beta,\gamma)}\rangle_d$.

Interaction with $\mathcal{F}_{\mathsf{CktEval}}$: Once $\mathcal{S}_{\mathsf{ckteval}}$ learns $\{\widetilde{\mathbb{X}}_i^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ for every $P_i \in \mathsf{Cr}$, it sends to $\mathcal{F}_{\mathsf{CktEval}}$ these inputs on behalf of the parties $P_i \in \mathsf{Cr}$ and receives the outputs $\{\mathbb{O}^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$, where each $\mathbb{O} \in \mathbb{F}^{n^5p^2}$.

Circuit-Evaluation: The simulator simulates the evaluation of each gate in `ckt` in topological order as follows.

– **Addition Gate**: Since this step involves local computation, the simulator does not have to simulate any messages on the behalf of the honest parties. The simulator locally adds ${}^A\langle\cdot\rangle_d$-sharings corresponding to the gate-input and obtain the ${}^A\langle\cdot\rangle_d$-sharings corresponding to the gate-outputs.
– **Multiplication Gate**: The simulator takes the multiplication-triples and the double-pairs associated with the multiplication-gate (generated as part of interaction with $\mathcal{F}_{\mathsf{PREP}}$), along with the ${}^A\langle\cdot\rangle_d$-sharings corresponding to the gate-inputs and computes the messages of the honest parties as per the steps of the protocol Π_{Beaver}. The simulator then sends these messages to $\mathcal{A}$ on behalf of the honest parties. Once the simulation of the gate-evaluation is done, the simulator will know the complete ${}^A\langle\cdot\rangle_d$-sharings corresponding to the gate-outputs.

Output Gate: Let $\{{}^A\langle\widetilde{\mathbb{O}}^{(\beta,\gamma)}\rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ be the sharings, corresponding to the output gate, available with $\mathcal{S}_{\mathsf{ckteval}}$ during the simulated circuit-evaluation. The simulator computes the sharings $\{{}^A\langle\mathbb{O}^{(\beta,\gamma)}\rangle_d\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$, such that the shares, MAC-keys and MAC-tags of the parties in Cr for ${}^A\langle\mathbb{O}^{(\beta,\gamma)}\rangle_d$ are identical as in ${}^A\langle\widetilde{\mathbb{O}}^{(\beta,\gamma)}\rangle_d$. On behalf of each $P_i \notin \mathsf{Cr}$, the simulator sends to $\mathcal{A}$ the shares of ${}^A\langle\widetilde{\mathbb{O}}^{(\beta,\gamma)}\rangle_d$ held by P_i and the MAC-tags on these shares corresponding to each $P_j \in \mathsf{Cr}$.

We next prove a sequence of claims, which help us to show that the joint distribution of the parties are identical in both the real-world, as well as ideal-world, except with a negligible probability. We first show that the view generated by $\mathcal{S}_{\mathsf{ckteval}}$ for $\mathcal{A}$ is identically distributed as $\mathcal{A}$'s view during the real execution of Π_{ckteval}.

Claim 9.5. *The view of $\mathcal{A}$ in the simulated execution with $\mathcal{S}_{\mathsf{ckteval}}$ is identically distributed as the view of $\mathcal{A}$ in the real execution of Π_{ckteval}.*

Proof. It is easy to see that the view of $\mathcal{A}$ during the preprocessing phase is identically distributed in both the executions. This is because in both the executions, $\mathcal{A}$ receives no messages from the honest parties and the steps of $\mathcal{F}_{\mathsf{PREP}}$ are executed by the simulator itself in the simulated execution. Namely, in both the executions, $\mathcal{A}$'s view consists of information of the parties in Cr for key-consistent secret-shared random multiplication-triples and random double-pairs. Along with this adversary's view also consists of the MAC-key components of the parties in Cr and the twisted-shares of the MAC-key components of the honest parties. Let us fix all this data. Now conditioned on this data, for the input gate, $\mathcal{A}$ learns its supposed data corresponding to the sharings $\{{}^A\langle\mathbb{X}_i^{(\beta,\gamma)}\rangle_d\}_{P_i\notin\mathsf{Cr}}$ during the real execution, while in the simulated execution it learns its supposed data corresponding to the sharings $\{{}^A\langle\widetilde{\mathbb{X}}_i^{(\beta,\gamma)}\rangle_d\}_{P_i\notin\mathsf{Cr}}$, where the elements of $\widetilde{\mathbb{X}}^{(\beta,\gamma)}$ are randomly chosen by the simulator. It is easy to see that the distributions of the data held by $\mathcal{A}$ in both the real and simulated executions are identical, as simulator honestly plays the role of honest parties in the underlying $\Pi_{A\langle\cdot\rangle}$ instances involving honest dealers. So let us fix the data held by $\mathcal{A}$ for input gates.

During the evaluation of addition gates, no communication is involved. During the evaluation of multiplication gates, in the simulated execution, the simulator will know the complete ${}^A\langle\cdot\rangle_d$-sharings associated with gate-inputs. It will also known the complete ${}^A\langle\cdot\rangle_d$-sharings of the associated multiplication-triples

and also $({}^{A}\langle\cdot\rangle_d, {}^{A}\langle\cdot\rangle_{d+p-1})$-sharings of the associated double-pairs. Hence the simulator can correctly simulate the messages sent by the honest parties during the execution of the corresponding instance of Π_{Beaver}. This ensures that the $\mathcal{A}$'s view during the evaluation of multiplications gates is identically distributed, both in the real execution and the simulated execution.

For the output gate, the data received by $\mathcal{A}$ in the real execution from the honest parties correspond to ${}^{A}\langle\cdot\rangle_d$-sharings of the circuit-outputs $\mathbb{O}^{(\beta,\gamma)}$. From the steps of $\mathcal{S}_{\mathsf{ckteval}}$ (as discussed elaborately during the informal discussion), it is easy to see that the same holds even in the simulated execution. This is because the simulator computes ${}^{A}\langle\cdot\rangle_d$-sharings of $\mathbb{O}^{(\beta,\gamma)}$, where the data to be supposedly held by the parties in Cr is identical as in the simulated execution. Hence $\mathcal{A}$'s view is identically distributed in both executions during the evaluation of the output gate as well. □

We next claim that conditioned on the view of $\mathcal{A}$, the output of the honest parties is identically distributed in both worlds, except with a negligible probability.

Claim 9.6. *Conditioned on the view of $\mathcal{A}$, the output of the honest parties is identically distributed in the real execution of Π_{ckteval} involving $\mathcal{A}$, as well as in the ideal execution involving $\mathcal{S}_{\mathsf{ckteval}}$ and $\mathcal{F}_{\mathsf{CktEval}}$.*

Proof. Let view be an arbitrary view of $\mathcal{A}$. According to view, for every $P_i \in \mathcal{P}$, there exist inputs $\{\mathbb{X}_i^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$ where $\mathbb{X}_i^{(\beta,\gamma)} \in \mathbb{F}^{n^5p^2}$, such that the parties hold ${}^{A}\langle\mathbb{X}_i^{(\beta,\gamma)}\rangle_d$. Furthermore, from Claim 9.5, if $P_i \in \mathsf{Cr}$ then the corresponding ${}^{A}\langle\mathbb{X}_i^{(\beta,\gamma)}\rangle_d$ is included in view while for $P_i \notin \mathsf{Cr}$, the corresponding $\mathbb{X}_i^{(\beta,\gamma)}$ are uniformly distributed, conditioned on the data for ${}^{A}\langle\mathbb{X}_i^{(\beta,\gamma)}\rangle_d$ available with $\mathcal{A}$ as determined by view. Let us fix the $\mathbb{X}_i^{(\beta,\gamma)}$ values corresponding to the parties in $\mathcal{P}$ and consider the vector of values $(\mathbb{X}_1^{(\beta,\gamma)},\ldots,\mathbb{X}_n^{(\beta,\gamma)})$, where $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$.

It is easy to see that in the ideal-world, the output of the honest parties is $\{\mathbb{O}^{(\beta,\gamma)}\}_{\beta\in\{1,\ldots,n^3\},\gamma\in\{1,\ldots,t+1\}}$, where $\mathbb{O}^{(\beta,\gamma)}$ is the output of the circuit-evaluation over input $(\mathbb{X}_1^{(\beta,\gamma)},\ldots,\mathbb{X}_n^{(\beta,\gamma)})$. We now show that the honest parties output $\mathbb{O}^{(\beta,\gamma)}$ even in the real-world, except with a negligible probability. For this, we argue that all the values during the circuit-evaluation are correctly ${}^{A}\langle\cdot\rangle_d$-shared, except with a negligible probability.

Since the evaluation of addition gates needs only local computation, it follows that the output of addition gates will be correctly ${}^{A}\langle\cdot\rangle_d$-shared. Since multiplication-gates are evaluated using instances of Π_{Beaver}, it follows from the correctness of Π_{Beaver} (Lemma 8.26) that the output of multiplication-gates will be correctly ${}^{A}\langle\cdot\rangle_d$-shared, except with a negligible probability. Since all the gates are evaluated correctly except with a negligible probability, it follows that for the output-gates, the parties will correctly hold ${}^{A}\langle\mathbb{O}^{(\beta,\gamma)}\rangle_d$ which the honest parties reconstruct correctly, except with a negligible probability. □

Finally from the previous two claims we conclude that:

$$\left\{\mathrm{HYBRID}^{\mathcal{F}_{\mathsf{PREP}}}_{\Pi_{\mathsf{ckteval}},\mathcal{A}(z)}(\mathbb{X}_1^{(\beta,\gamma)},\ldots,\mathbb{X}_n^{(\beta,\gamma)})\right\}_{z\in\{0,1\}^\star\}} \stackrel{s}{\equiv} \left\{\mathsf{IDEAL}_{\mathcal{F}_{\mathsf{CktEval}},\mathcal{S}_{\mathsf{ckteval}}(z)}(\mathbb{X}_1^{(\beta,\gamma)},\ldots,\mathbb{X}_n^{(\beta,\gamma)})\right\}_{z\in\{0,1\}^\star},$$

where $\beta \in \{1,\ldots,n^3\}$ and $\gamma \in \{1,\ldots,t+1\}$. Based on the above discussion, we conclude the following theorem. The $\mathcal{O}(1)$ communication overhead is computed as follows: *each* multiplication gate is evaluated over $n^3(t+1)n^5p^2 = \Theta(n^{11})$ field elements, represented by $\Theta(n^{11}\kappa)$ bits. The total cost for evaluating the multiplication gates is $\mathcal{O}(c_M n^{11}\kappa)$ bits and so the communication overhead per multiplication gate is $\mathcal{O}(1)$ bits. Each party has $n^3(t+1)n^5p^2 = \Theta(n^{11})$ field elements as inputs and so there are total $\Theta(n^{12})$ field elements as inputs for $\mathtt{ckt}$, represented by $\Theta(n^{12}\kappa)$ bits. Since the total cost of secret-sharing of all inputs is $\mathcal{O}(n^{12}\kappa)$ bits, the communication overhead per input gate is $\mathcal{O}(1)$ bits. Finally, there are $n^3(t+1)n^5p^2 = \Theta(n^{11})$ field elements as outputs, which needs to be learnt by *all* the n parties. So in essence, $\mathtt{ckt}$ consists of $\Theta(n^{12})$ field elements as output, represented by $\Theta(n^{12}\kappa)$ bits. Since the total cost of reconstructing all output values is $\mathcal{O}(n^{12}\kappa)$ bits, the communication overhead for the output gate is $\mathcal{O}(1)$ bits.

Theorem 9.7. *Protocol Π_{ckteval} UC-securely realizes the functionality $\mathcal{F}_{\mathsf{CktEval}}$ with statistical security in the $\mathcal{F}_{\mathsf{PREP}}$-hybrid model, in the presence of a static malicious adversary, corrupting at most $t < \frac{n}{3}$ parties. The protocol requires $\mathcal{O}(D_M)$ expected number of rounds and incurs a communication of $\mathcal{O}(c_M n^{11}\kappa + n^{12}\kappa)$ bits. The communication overhead of the protocol is $\mathcal{O}(1)$ bits.*

10 Statistically Secure MPC in the Synchronous Network: The Generic Case

In this section, we discuss the more generic case of $n = 2t + s$, where $s = \Theta(n)$. We will provide instantiations of the core building blocks depicted in Fig. 1 with statistical security and $\mathcal{O}(1)$ overhead by describing modifications to the protocols designed for the case of $n = 3t + 1$. We set the packing parameter to be $p = \frac{s}{4} + 1$, resulting in $d = t + \frac{s}{4}$ and $d + p - 1 = t + \frac{s}{2}$ respectively. We will describe the changes required to the protocols described in the prior sections to make them work for the threshold of $n = 2t + s$. Note that the choice of p and the fact that $s = \Theta(n)$ guarantees that p, d and $d + p - 1$ are all $\Theta(n)$.

10.1 VSS for Generating $^{A}\langle\cdot\rangle$-Sharing

Although our VSS protocol is defined for $p = \frac{t}{4} + 1$, it seamlessly adapts to the case when $p = \frac{s}{4} + 1$. In fact, it is easy to see that the Protocol 6.1 as described immediately works for the case of $n = 2t + s$ since it is already given in terms of the number of corruptions t and the packing parameter p, without relying on the number of parties being $3t+1$. Moreover, the number of reruns required will also be $\mathcal{O}(1)$. For instance, if the dealer is not able to identify a Core set such that $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + p$, then some constant fraction of s corrupt parties will be included in Dp. And this can happen for a constant number of iterations, after which all the t corrupt parties will be identified (since $s = \Theta(t)$) and included in Dp. Once this happens, the condition $|\mathsf{Core} \cup \mathsf{Dp}| \geq 2t + p$ will be satisfied. A similar argument holds for the case where the rerun occurs in the tag generation phase.

10.2 Robust Degree Reduction Protocol

The robust degree reduction protocol proceeds very similarly to that described in Protocol 8.9, with the difference being the number of vectors of values handled simultaneously. Specifically, instead of taking $t+1$ vectors $^{A}\langle\mathbb{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{S}^{(t+1)}\rangle_d$ as input, the protocol takes inputs $^{A}\langle\mathbb{S}^{(1)}\rangle_d, \ldots, {}^{A}\langle\mathbb{S}^{(s-1)}\rangle_d$ and gives as output $^{A}\langle\mathbb{S}^{(1)}\rangle_{p-1}, \ldots, {}^{A}\langle\mathbb{S}^{(s-1)}\rangle_{p-1}$. Thus, during the protocol, the vectors of polynomials in $\mathbb{S}(X)$ will be of degree s instead of t. This ensures that even if up to t corrupt parties out of $2t + s$ parties redistribute incorrect vectors of points on the polynomials in $\mathbb{S}(X)$, they can be error-corrected using the RS-decoding procedure. Note that the number of inputs handled by the protocol will be s which is still $\Theta(n)$. The rest of the protocol and its analysis closely follows the protocol described for $n = 3t + 1$ and hence we avoid repetition.

10.3 Generating Double-Secret-Shared Random Values and Secret-Shared Random Triples

The protocol for these two primitives is identical to that described for the case of $n = 3t + 1$. Since the number of random values or triples output by the protocol is $n - t$, the exact number will vary based on the specific value of n. However, since $s = \Theta(t)$, we always have $n - t = \Theta(n)$. Hence, the analysis of the protocol also follows immediately from the prior case, and we do not repeat it here.

10.4 Beaver Multiplication

The Beaver multiplication protocol is the primitive which requires additional modifications when being adapted to the case of $n = 2t + s$. Observe that the protocol operates over $t + 1$ packed secret sharings (denoted by $\gamma \in \{1, \ldots, t+1\}$) within each batch (where batches are denoted by $\beta \in \{1, \ldots, n^3\}$), which in turn defines a vector of degree-t polynomials $\mathbb{S}^{(\beta)}(X)$ for each β. Crucially, $n = 3t + 1$, allows the RS-decoding procedure to be applied to correct up to t errors in this scenario during output computation in Step 14c. Since we now work with $n = 2t + s$, as in the case of degree reduction, we consider s packed secret sharings in each batch β instead of $t + 1$, to allow correction of up to t errors.

Another change in the protocol due to the number of parties is in Step 6 where parties attempt to reconstruct $\langle\cdot\rangle$-sharing of degree-$(d + p - 1 = t + \frac{s}{2})$. Note that we do not have tags on this sharing, and parties attempt to reconstruct this via the RS-decoding procedure. With $n = 3t + 1$ parties, a party

Iteration	N	T	Distance	Correct	Identify
1	$n = 2t + s$	t	$t + \frac{s}{2} - 1$	$\frac{s}{2} - 1$	$\frac{s}{2}$
2	$n = 2t + \frac{s}{2}$	$t - \frac{s}{2}$	$t - 1$	$\frac{s}{2} - 1$	$\frac{s}{2}$
3	$n = 2t$	$t - s$	$t - \frac{s}{2} - 1$	$\frac{s}{2} - 1$	$\frac{s}{2}$
4	$n = 2t - \frac{s}{2}$	$t - \frac{3s}{2}$	$t - s - 1$	$\frac{s}{2} - 1$	$\frac{s}{2}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Table 2: Number of errors to be corrected in each rerun for a party. Here N denotes the number of parties whose shares will be considered for reconstruction and T denotes the number of corrupt parties not yet identified to be corrupt. 'Distance' corresponds to the distance of the codewords when the codeword is a polynomial of degree-$(d + p - 1)$.

could attempt to correct $\frac{t}{2}$ errors, and in case of a successful decoding, it was ensured that the decoding corresponds to the correct polynomial owing to Theorem 4.1. However, a failed reconstruction would immediately allow the party to include at least $\frac{t}{2} + 1$ parties in its dispute set later during Complaint Resolution and Fault Identification during Step 13. This crucially helps that party reconstruct degree-$(d + p - 1)$ shared values by ignoring the shares of parties in its dispute set, since it now has to correct the remaining (at most) $\frac{t}{2} - 1$ errors.

In contrast, in the case of $n = 2t+s$, firstly, correcting $\frac{t}{2}$ errors may result in a successful reconstruction that *does not* correspond to the correct polynomial since for certain values of s, it may not adhere to the requirements of Theorem 4.1. To account for this, instead of $\frac{t}{2}$, Step 6 is modified where each party tries to correct $\frac{s}{2} - 1$ errors. Now, we get the same guarantee as in the prior case that if the decoding succeeds, then the reconstructed polynomial is guaranteed to be the correct one. However, if the decoding fails, a party can now include (at least) $\frac{s}{2}$ parties in its dispute set. Consequently, there may still be $T = t - \frac{s}{2}$ corrupt parties among the rest. Depending on the exact values of s, there may be instances where the RS-decoding algorithm cannot correct these $t - \frac{s}{2}$ errors even after ignoring the shares of $\frac{s}{2}$. That is the reconstruction may fail while considering the shares of only $N = n - \frac{s}{2}$ parties, of which up to $T = t - \frac{s}{2}$ may be erroneous. For this reason, a change that is required in the Beaver protocol is that even if a single party cannot reconstruct at this step, the protocol is rerun from the first step. In each iteration that fails, some party will identify (at least) $\frac{s}{2}$ errors. Note that in the worst case, after $\frac{2t}{s} = \mathcal{O}(1)$ iterations, a party would have identified all t corrupt parties, after which its reconstruction is guaranteed to succeed. Moreover, beyond $s = 2t + 4$, instead of $\frac{s}{2}$, parties try to correct t errors since that is the maximum number of errors that can be introduced by the adversary. In summary, parties correct $\min(\frac{s}{2}, t)$ errors during Step 6. Table 2 summarizes the number of shares N considered during reconstruction, and the number of errors T that may exist for a party in each iteration that it fails to reconstruct.

Observe that the protocol may require $\mathcal{O}(1)$ reruns for each party separately. Hence, in total, it may require $\mathcal{O}(n)$ reruns for all parties to obtain their respective values in the protocol. In order to ensure that a constant overhead is maintained even in this case, we can use the standard technique of circuit segmentation where we can divide the circuit into n segments of size $\frac{C}{n}$ each [43]. With this, even if a certain segment requires $\mathcal{O}(n)$ reruns, the amortized cost of circuit evaluation is constant.

10.5 Triple Sharing

The triple sharing protocol will also follow Protocol 8.30 described for the case of $n = 3t + 1$ very closely. The only difference will be in the number of invocations of the Beaver multiplication in Step 8. Since each Beaver instance now handles multiplication of s vectors simultaneously, the t multiplications corresponding to $\gamma \in \{t+2, \ldots, 2t+1\}$ will now require $\lceil \frac{t}{s} \rceil = \mathcal{O}(1)$ number of invocations of the Beaver protocol. The remaining protocol and its analysis easily follow from that of Protocol 8.30.

10.6 Triple Extraction

One of the primary modifications in triple extraction due to the change in the number of parties from $n = 3t + 1$ to $n = 2t + s$ arises in the degree of the polynomials $\mathbb{X}(\cdot), \mathbb{Y}(\cdot), \mathbb{Z}(\cdot)$. To ensure that $\mathbb{Z}(\cdot)$ is

completely defined, the degree of $\mathbb{X}(\cdot), \mathbb{Y}(\cdot)$ are set to $t + \frac{(s-1)}{2}$. This in turns results in $\mathbb{Z}(\cdot)$ of degree-$2t + (s-1)$, which is completely defined by the triples shared by $2t + s$ parties. The other change is in the number of instances of the Beaver multiplication protocol called within triple extraction. Since it handles multiplication of s vectors simultaneously, parties have to invoke approximately $\frac{t+s}{s} = \mathcal{O}(1)$ instances of the Beaver multiplication protocol. Finally, the number of triples extracted from each batch is now $\frac{(s-1)}{2} + 1 = \Theta(t)$ since that is the degree of freedom we can afford in each of the $\mathbb{X}(\cdot), \mathbb{Y}(\cdot)$ (and hence $\mathbb{Z}(\cdot)$) polynomials. The analysis of the protocol follows that of Protocol 8.35 closely and we do not provide it to avoid repetition.

10.7 The Complete Preprocessing Phase

The structure of the protocol is the same as that for $n = 3t+1$, with a few minor modifications. Instead of evaluating each gate over n^5p^2 pairs of inputs in a SIMD manner $n^3(t+1)$ times, to accommodate the Beaver protocol in this setting, parties now evaluate each gate over n^5p^2 pairs of inputs in a SIMD manner but n^3s times. For this, parties begin by invoking the $\mathcal{F}_{\mathsf{MacKeys}}$ functionality for setup generation, that is, establishing the MAC key required for further computation as before. Now, the modified $\mathcal{F}_{\mathsf{PREP}}$ has to generate $c_M n^3 s$ vectors of random multiplication-triples and vectors of random triples, each of size n^5p^2. Along with this, the functionality also generates $2c_M n^3 s$ vectors of double-packed-secret-shared random values, each of size n^5p^2. As earlier, the generation of multiplication-triples in Π_{Prep} happens by applying the framework of [24] extended for authenticated packed-secret-sharing.

Recall that one instance of Π_{TripExt} generates $n^3(\frac{(s-1)}{2}+1)n^5p^2$-secret-shared random multiplication-triples. Consequently, $2c_M$ instances of Π_{TripExt} will suffice to generate $c_M n^3 s n^5 p^2$-secret-shared random multiplication-triples. To run one instance of Π_{TripExt}, we need n^3 number of ${}^{A}\langle\cdot\rangle_d$-sharing of vectors of n^5p^2 multiplication-triples and triples from each party.[27]

Consequently, each party has to invoke $\frac{c_M}{t}$ instances of Π_{TripSh} as a dealer, so that $c_M n^3 n^5 p^2$ secret-shared multiplication-triples are available on behalf of that party. To generate $c_M n^3 n^5 p^2$ secret-shared triples on its behalf, each designated party can act as a dealer and run $3c_M n^3$ instances of $\Pi_{{}^{A}\langle\cdot\rangle}$ as a dealer, as each instance generates secret-sharing of n^5p^2 elements.

To generate $c_M n^3 s$ vectors of random secret-shared triples, the parties run $\frac{c_M}{n-t} n^3 s$ instances of the protocol $\Pi_{\mathsf{RandTriples}}$. This is because one instance of $\Pi_{\mathsf{RandTriples}}$ generates $n-t$ vectors of secret-shared random triples. Finally, to generate $2c_M n^3 s$ vectors of double-packed-secret-shared vectors of random values, the parties need to run the protocol Π_{RandPair} $\frac{2c_M}{t(n-t)} n^3 s$ times, as one instance of Π_{RandPair} generates $t(n-t)$ such vectors.

Lemma 10.1. *Let $n = 2t + s$. There exists a protocol Π_{Prep} that generates the following, except with a negligible probability:*

- $c_M n^3 s n^5 p^2$ ${}^{A}\langle\cdot\rangle_d$*-shared multiplication-triples over* $\mathbb{F}$.
- $c_M n^3 s n^5 p^2$ ${}^{A}\langle\cdot\rangle_d$*-shared triples over* $\mathbb{F}$.
- $2c_M n^3 s n^5 p^2$ *double-packed-secret-shared elements over* $\mathbb{F}$.

10.8 The Circuit Evaluation Protocol

This protocol follows easily from the case of $n = 3t + 1$, with the major modification being that parties evaluate the SIMD circuit over $n^8 s p^2$ inputs in parallel.

[27] Technically, we need the first $t + \frac{(s-1)}{2} + 1$ parties to provide only n^3 number of vectors of secret-shared multiplication-triples and the remaining parties to provide n^3 number of vectors of secret-shared multiplication-triples and triples. However, for simplicity and to maintain uniformity, we ask for n^3 number of vectors of secret-shared multiplication-triples and triples from each party. Asymptotically, the communication cost still remains the same.

Part II

Asynchronous Setting

11 Preliminaries

Similar to the case of synchronous network, we assume a set of parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ connected by pairwise private and authentic channels. We set the packing parameter $p = \frac{s}{4} + 1$ and assume that the number of parties is $n = 3t + s$, where t is the maximum number of parties, which can be under the control of a computationally-unbounded malicious (Byzantine) adversary $\mathcal{A}$. We assume an *asynchronous* communication network in which messages sent on a channel can be delayed by an arbitrary but a finite time; with the guarantee that every sent message is delivered eventually. In general, the order in which messages are received might be different from the order in which they were sent. This is modeled by a scheduler which decides on the sequence of message deliveries, where the scheduler is assumed to be controlled by the adversary. Yet, to simplify notation and improve readability, we assume that the messages that a party receives from a channel are guaranteed to be delivered in the order they were sent. This can be achieved using standard techniques, such as counters, and acknowledgments, and hence we make this simplification assumption. As earlier, the communication complexity of any protocol is defined to be the total number of bits sent by the *honest* parties, namely the parties which are not under adversary's control.

The remaining details such as the security parameter, size of the field, and the notation used follow similarly to the synchronous setting. We now briefly discuss two well-known asynchronous primitives which we use in our asynchronous protocols.

Agreement on a Core Set or Asynchronous Common Subset (ACS). The ACS primitive [18] allows parties to agree on a common subset of at least $n - t$ parties $\mathsf{Core} \in \mathcal{P}$, such that each party in Core satisfies some predefined property prop which has the following features:

1. Every honest party eventually satisfies prop.
2. If some honest P_i sees that a party P_j satisfies prop, then eventually all the honest parties see that P_j satisfies prop.

Informally, one can design ACS by running n instances of *asynchronous Byzantine agreement* (ABA), one on behalf of every party P_j to agree upon if P_j satisfies the property prop. Looking ahead, we use the ACS primitive to identify a common set of parties with the property that an asynchronous VSS instance initiated by a party in this set must terminate eventually for all the honest parties.

A-Cast. Bracha's reliable asynchronous broadcast (A-Cast) protocol [16] allows a designated sender to send a message m identically to all the parties at a cost of $\mathcal{O}(n^2\ell)$ bits where ℓ is the length of the message in bits. If the sender is honest, then all the honest parties eventually terminate with output m. If the sender is corrupt, and some honest party terminates with the output m', then all the honest parties eventually terminate with the same output m'. The cost of A-Cast was further improved to $\mathcal{O}(n\ell + n^2 \log n)$ bits [20,34,2]. Looking ahead, while adapting the synchronous protocols to the asynchronous setting, the invocations of $\mathcal{F}_{\mathsf{BC}}$ will be replaced by calls to the A-Cast primitive, where we will use the following terminologies.

- "P_i broadcasts a message m" or "P_i sends m to $\mathcal{F}_{\mathsf{BC}}$" will represent an instance of A-Cast with P_i as the sender.
- "P_j receives a message m from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender" will represent that P_j outputs m in the instance of A-Cast with P_i as the sender.

12 AVSS for Generating $\langle\cdot\rangle$-Secret Sharing with $\mathcal{O}(1)$ Overhead

In this section, we present a statistically-secure *asynchronous* VSS (AVSS) protocol, $\Pi_{\mathsf{A\text{-}VMBPSS}}$, which achieves an amortized sharing cost of $\mathcal{O}(1)$ per secret for the case of an asynchronous network with a non-optimal threshold of $n = 3t + s$. The protocol allows a designated dealer $D \in \mathcal{P}$ to verifiably generate degree-ℓ PSS of its vector of inputs where $\ell \in \{p - 1, d, d + p - 1\}$. This protocol follows in a straightforward manner from our VSS protocol in the synchronous network. However, the protocol is relatively simpler because it *need not* generate an authenticated PSS and hence there is *no* tag-generation phase. The reason that we *do not* need authenticated PSS is that with $n = 3t + s$, one can error-correct up to t errors and robustly reconstruct even a degree-$(d+p-1)$ PSS using *online error-correction* (OEC) [17]. This is because $d + p - 1 = t + \frac{s}{2}$ for $p = \frac{s}{4} + 1$.

Similar to the synchronous protocol, we require an information-checking protocol, which follows directly from Protocol 5.2. Moreover, the broadcast primitive is replaced by reliable broadcast [16].

ICP in the asynchronous network. As described earlier, our synchronous variant of the ICP was adapted from the asynchronous ICP from [48]. In Protocol 5.2, parties were synchronized by a global clock and proceeded as per the rounds in the protocol. However, in the asynchronous network, there is no notion of rounds. Hence, the protocol steps are executed upon receiving the respective messages. We avoid giving the formal details, as the protocol steps are identical to that of synchronous ICP, except that we do not have the notion of communication rounds now. The linearity property of the IC-signatures and the terminologies associated with the invocations of ICP carry over to asynchronous setting as well.

The AVSS Protocol. We present protocol $\Pi_{\mathsf{A\text{-}VMBPSS}}$ for the case when the degree of sharing $\ell = d = t + p - 1$. The steps for the case when $\ell \in \{p-1, d+p-1\}$ follows easily. Unlike the synchronous VSS protocol where D has to make public a linear combination of its bivariate polynomials 4 times, for the asynchronous case linear combinations of the committed bivariate polynomials may have to be made public n^2+1 times. This is because once a candidate Core set is identified, to verify the rows and column-polynomials of the parties in Core, the dealer has to make public a linear combination of those rows and columns. And later, every time a party outside Core asynchronously receives a supposedly common point on its rows and columns from a party inside Core, the dealer has to make public a linear combination of the row and column polynomials of the parties in Core, to enable the receiver party verify those points. Accordingly, D has to select so many masking-vectors.

Note that *unlike* the synchronous case, (an honest) D will eventually get a Core of desired size and hence the parties *need not* have to restart the protocol. This is because there are at least $2t+s$ honest parties and the desired size of Core is $\ell + t + 1 \leq 2t + s$, even if we consider the maximum degree $\ell = d + p - 1$.

In contrast to the synchronous network setting, where the reconstruction of row and column polynomials by parties outside the Core is a single-shot process, the asynchronous setting introduces potential delays that necessitate multiple reconstruction attempts. Specifically, a party outside Core may need to repeatedly attempt to reconstruct its row and column as and when it receives the necessary shares from parties within Core. This procedure closely follows the principles of online error correction. To reconstruct degree-d row (or column) polynomial, a party waits to receive $d+1$ shares from parties in Core and verifies them against the committed bivariate polynomial using a procedure analogous to that employed during the verification phase. If (at least) $d+1$ of these shares pass the verification check, the party can successfully reconstruct its corresponding row (or column) polynomial. However, in the asynchronous setting, it is possible that the party initially receives only up to $d+1-t$ "correct" shares, in which case the reconstruction fails. In this case, it can expect to receive more shares from slow honest parties. Hence, it repeats the reconstruction attempt with $d+2$, $d+3$, and so on shares, as they become available. The reconstruction is guaranteed to succeed once the party receives at least $d+1+t$ shares from the parties in Core, which ensures the presence of at least $d+1$ correct shares that pass verification. Using these points, the party outside Core can retrieve its degree-d row (or column) polynomial.

Note that each party outside Core may perform such a verification process up to t times, with each instance requiring the dealer to reveal a random linear combination of the committed bivariate polynomials. This clarifies the necessity of n^2 masking polynomials in the protocol. Moreover, the expected size of the Core, namely $|\mathsf{Core}| \geq d+t+1$, is consistent with the requirements of the reconstruction procedure. A similar argument holds for degree-$(d+p-1, d+p-1)$ bivariate polynomials. We now describe our protocol considering degree-(d,d) bivariate polynomials.

Protocol 12.1: VSS for Generating $\langle\cdot\rangle_d$-sharing – $\Pi_{\mathsf{A\text{-}VMBPSS}}$

Input: D has LMp^2 elements from $\mathbb{F}$ which are arranged as L batches of vectors $\mathbf{S}^{(1)}, \ldots, \mathbf{S}^{(L)}$, with each batch consisting of Mp^2 elements. For $u = 1, \ldots, L$, batch $\mathbf{S}^{(u)}$ consists of Mp vectors, each consisting of p elements. That is, $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,Mp)})$, where each vector $\mathbf{s}^{(u,v)} \in \mathbb{F}^p$, for $v = 1, \ldots, Mp$. The vector $\mathbf{s}^{(u,v)} = (s^{(u,v,1)}, \ldots, s^{(u,v,p)})$. Let $\mathbb{S} = \{s^{(u,v,w)}\}_{u\in\{1,\ldots,L\},v\in\{1,\ldots,Mp\},w\in\{1,\ldots,p\}}$.

(Sharing Phase)

1. D selects n^2+1 batches of random *masking-vectors* $\mathbf{S}^{(L+1)}, \ldots, \mathbf{S}^{(L+n^2+1)}$, each consisting of Mp^2 uniformly random elements from $\mathbb{F}$. For $u = L+1, \ldots, L+n^2+1$, batch $\mathbf{S}^{(u)} = (\mathbf{s}^{(u,1)}, \ldots, \mathbf{s}^{(u,Mp)})$, where each vector $\mathbf{s}^{(u,v)} \in \mathbb{F}^p$, for $v = 1, \ldots, Mp$. The vector $\mathbf{s}^{(u,v)} = (s^{(u,v,1)}, \ldots, s^{(u,v,p)})$.

2. D picks $(L+n^2+1)M$ bivariate polynomials $F^{(u,v)}(X,Y)$ over $\mathbb{F}$ for $u=1,\ldots,L+n^2+1$ and $v=1,\ldots,M$, where each of these polynomials is otherwise a random (d,d)-degree bivariate polynomial over $\mathbb{F}$ with the following constraints:
   - Polynomial $F^{(u,v)}(X,Y)$ embeds the vectors $\mathbf{s}^{(u,(v-1)\cdot p+1)},\ldots,\mathbf{s}^{(u,(v-1)\cdot p+p)}$ at $F^{(u,v)}(X,-1),\ldots,F^{(u,v)}(X,-p)$ respectively. Namely, for $w=1,\ldots,p$, the condition $F^{(u,v)}(-1,-w)=s^{(u,(v-1)p+w,1)},\ldots,F^{(u,v)}(-p,-w)=s^{(u,(v-1)p+w,p)}$ holds.
3. Corresponding to every $P_i\in\mathcal{P}$, let $f_i^{(u,v)}(X)\stackrel{\text{def}}{=}F^{(u,v)}(X,i)$ and $g_i^{(u,v)}(Y)\stackrel{\text{def}}{=}F^{(u,v)}(i,Y)$. Dealer D sends the polynomials $\{f_i^{(u,v)}(X),g_i^{(u,v)}(Y)\}_{u=1,\ldots,L+n^2+1,v=1,\ldots,M}$ to P_i.
4. Every $P_i\in\mathcal{P}$, upon receiving the polynomials $\{f_i^{(u,v)}(X),g_i^{(u,v)}(Y)\}_{u=1,\ldots,L+n^2+1,v=1,\ldots,M}$ from D does the following if each of these polynomials is a d-degree polynomial.
   - Send OK_i to $\mathcal{F}_{\mathsf{BC}}$.
   - For $u=1,\ldots,L+n^2+1$, give $\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(u,1)},\ldots,\mathbf{g}_{\star,i}^{(u,M)})$ to D, *where the parties follow the linearity property while generating IC-signatures* (see Notation 5.7). Here for $v=1,\ldots,M$, $\mathbf{g}_{\star,i}^{(u,v)}\stackrel{\text{def}}{=}(g_i^{(u,v)}(1),\ldots,g_i^{(u,v)}(n))$. %. Here P_i is invoking $L+n^2+1$ instances of ICP, where in each instance, it is signing M vectors for D, each consisting of n values.
5. For $u=1,\ldots,L+n^2+1$, upon receiving $\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(u,1)},\ldots,\mathbf{g}_{\star,i}^{(u,M)})$ from $P_i\in\mathcal{P}$, dealer D includes P_i to Core (initialized to $\emptyset$), provided $\mathbf{g}_{\star,i}^{(u,v)}=(g_i^{(u,v)}(1),\ldots,g_i^{(u,v)}(n))$ holds, for $v=1,\ldots,M$ and OK_i is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender.
   - If $|\mathsf{Core}|\geq d+t+1$, then D sends Core to $\mathcal{F}_{\mathsf{BC}}$.
6. The parties in $\mathcal{P}$ do the following.
   - If Core is received from $\mathcal{F}_{\mathsf{BC}}$ where D is the sender such that $|\mathsf{Core}|\geq d+t+1$, and OK_i is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender corresponding to each $P_i\in\mathsf{Core}$, then proceed to the next phase.
   - Else continue to listen to receive Core and the corresponding OK messages and repeat the above check.

(Verification Phase)

1. Every $P_i\in\mathcal{P}$ sends Rand to $\mathcal{F}_{\mathsf{Rand}}$ and receives r from $\mathcal{F}_{\mathsf{Rand}}$.
2. For $v=1,\ldots,M$, dealer D computes the following.
   - Polynomial $\mathsf{F}^{(v)}(X,Y)\stackrel{\text{def}}{=}F^{(1,v)}(X,Y)+r_1\cdot F^{(2,v)}(X,Y)+\ldots+r_1^{L+n^2}\cdot F^{(L+n^2+1,v)}(X,Y)$.
   - Corresponding to every $P_i\in\mathsf{Core}$, the polynomial $\mathsf{g}_i^{(v)}(y)\stackrel{\text{def}}{=}F^{(1,v)}(i,Y)+r_1\cdot F^{(2,v)}(i,Y)+\ldots+r_1^{L+n^2}\cdot F^{(L+n^2+1,v)}(i,Y)$. % If D is honest then $\mathsf{g}_i^{(v)}(y)=\mathsf{F}^{(v)}(i,Y)$ holds.
   - Corresponding to every $P_i\in\mathsf{Core}$, let $\mathbf{g}_{\star,i}^{(v)}\stackrel{\text{def}}{=}(\mathsf{g}_i^{(v)}(1),\ldots,\mathsf{g}_i^{(v)}(n))$. Then D locally computes the signature $\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(1)},\ldots,\mathbf{g}_{\star,i}^{(M)})$ from $\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(1,1)},\ldots,\mathbf{g}_{\star,i}^{(1,M)})$, $\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(2,1)},\ldots,\mathbf{g}_{\star,i}^{(2,M)}),\ldots,\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(L+n^2+1,1)},\ldots,\mathbf{g}_{\star,i}^{(L+n^2+1,M)})$.

   D sends $(\mathsf{F}^{(1)}(X,Y),\ldots,\mathsf{F}^{(M)}(X,Y))$ to $\mathcal{F}_{\mathsf{BC}}$. Moreover, corresponding to every $P_i\in\mathsf{Core}$, dealer D publicly reveals $\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(1)},\ldots,\mathbf{g}_{\star,i}^{(M)})$.
3. Every party $P_k\in\mathcal{P}$ discards D if any of the following holds, otherwise it proceeds to the next phase.
   - $(\mathsf{F}^{(1)}(X,Y),\ldots,\mathsf{F}^{(M)}(X,Y)))$ is received from $\mathcal{F}_{\mathsf{BC}}$ where D is the sender and there exists some $v\in\{1,\ldots,M\}$, such that the polynomial $\mathsf{F}^{(v)}(X,Y)$ is not a (d,d)-degree bivariate polynomial.
   - There exists some $P_i\in\mathsf{Core}$, such that P_k does not accept the signature $\mathsf{Sig}(P_i\rightarrow D,\mathbf{g}_{\star,i}^{(1)},\ldots,\mathbf{g}_{\star,i}^{(M)})$.
   - There exists some $P_i\in\mathsf{Core}$ and some $v\in\{1,\ldots,M\}$, such that the values in $\mathbf{g}_{\star,i}^{(v)}$ do not lie on the polynomial $\mathsf{F}^{(v)}(i,Y)$.

(Non-Core Row and Column Reconstruction Phase)

1. For each pair (P_i,P_j) where $P_i\in\mathsf{Core}$ and $P_j\in\mathcal{P}\setminus\mathsf{Core}$:
   (a) P_i sends $\{(f_i^{(u,v)}(j),g_i^{(u,v)}(j))\}_{u=1,\ldots,L+n^2+1,v=1,\ldots,M}$ to P_j.
   (b) Upon receiving $\{(f_i^{(u,v)}(j),g_i^{(u,v)}(j))\}_{u=1,\ldots,L+n^2+1,v=1,\ldots,M}$ from P_i, P_j includes P_i to a set $\mathcal{S}_{j,z}$ (initialized to $\emptyset$).
2. For each $P_j\in\mathcal{P}\setminus\mathsf{Core}$, parties do the following for $z=0,\ldots,t$:

(a) Upon receiving the first $d+1+z$ points from parties in Core, that is, when $|\mathcal{S}_j| \geq d+1+z$, $P_j \notin \mathsf{Core}$ chooses a random value $r_{j,z}$ and sends it to $\mathcal{F}_{\mathsf{BC}}$.

(b) For each $v = 1, \ldots, M$, upon receiving $r_{j,z}$ from $\mathcal{F}_{\mathsf{BC}}$ where P_j is the sender, the dealer D computes the following.

- Polynomial $\mathsf{F}^{(j,z,v)}(X,Y) \stackrel{\mathsf{def}}{=} F^{(1,v)}(X,Y) + r_{j,z} \cdot F^{(2,v)}(X,Y) + \ldots + r_{j,z}^{L+n^2} \cdot F^{(L+n^2+1,v)}(X,Y)$.
- Corresponding to every $P_i \in \mathsf{Core}$, the polynomial $\mathsf{g}_i^{(j,z,v)}(y) \stackrel{\mathsf{def}}{=} F^{(1,v)}(i,Y) + r_{j,z} \cdot F^{(2,v)}(i,Y) + \ldots + r_{j,z}^{L+n^2} \cdot F^{(L+n^2+1,v)}(i,Y)$. % If D is honest then $\mathsf{g}_i^{(j,z,v)}(y) = \mathsf{F}^{(j,z,v)}(i,Y)$ holds.
- Corresponding to every $P_i \in \mathsf{Core}$, let $\mathsf{g}_{\star,i}^{(j,z,v)} \stackrel{\mathsf{def}}{=} (\mathsf{g}_i^{(j,z,v)}(1), \ldots, \mathsf{g}_i^{(j,z,v)}(n))$. Then D locally computes the signature $\mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(j,z,1)}, \ldots, \mathsf{g}_{\star,i}^{(j,z,M)})$ from $\mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(1,1)}, \ldots, \mathsf{g}_{\star,i}^{(1,M)})$, $\mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(2,1)}, \ldots, \mathsf{g}_{\star,i}^{(2,M)}), \ldots, \mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(L+n^2+1,1)}, \ldots, \mathsf{g}_{\star,i}^{(L+n^2+1,M)})$

D sends $(\mathsf{F}^{(j,z,1)}(X,Y), \ldots, \mathsf{F}^{(j,z,M)}(X,Y))$ to $\mathcal{F}_{\mathsf{BC}}$. Moreover, corresponding to every $P_i \in \mathsf{Core}$, dealer D publicly reveals $\mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(j,z,1)}, \ldots, \mathsf{g}_{\star,i}^{(j,z,M)})$.

(c) Every party $P_k \in \mathcal{P}$ discards D and terminate if any of the following holds.

- $(\mathsf{F}^{(j,z,1)}(X,Y), \ldots, \mathsf{F}^{(j,z,M)}(X,Y))$ is received from $\mathcal{F}_{\mathsf{BC}}$ where D is the sender and there exists some $v \in \{1, \ldots, M\}$, such that the polynomial $\mathsf{F}^{(j,z,v)}(X,Y)$ is not a (d,d)-degree bivariate polynomial.
- There exists some $P_i \in \mathsf{Core}$, such that P_k does not accept the signature $\mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(j,z,1)}, \ldots, \mathsf{g}_{\star,i}^{(j,z,L)})$.
- There exists some $P_i \in \mathsf{Core}$ and some $v \in \{1, \ldots, M\}$, such that the values in $\mathsf{g}_{\star,i}^{(j,z,v)}$ do not lie on the polynomial $\mathsf{F}^{(j,z,v)}(i,Y)$.

(d) Party P_j does the following if the dealer is not discarded.

- Include $P_i \in \mathcal{S}_j$ to a set $\mathcal{S}_{j,z}$ (initialized to $\emptyset$), provided both the following hold for every $v \in \{1, \ldots, M\}$:

$$\mathsf{F}^{(j,z,v)}(i,j) = g_i^{(1,v)}(j) + r_{j,z} \cdot g_i^{(2,v)}(j) + \ldots + r_{j,z}^{L+n^2} \cdot g_i^{(L+n^2+1,v)}(j),$$

and

$$\mathsf{F}^{(j,z,v)}(j,i) = f_i^{(1,v)}(j) + r_{j,z} \cdot f_i^{(2,v)}(j) + \ldots + r_{j,z}^{L+n^2} \cdot f_i^{(L+n^2+1,v)}(j).$$

If $|\mathcal{S}_{j,z}| < d+1$, then proceed to the next iteration. Otherwise it computes the following and terminates.

- For every $u \in \{1, \ldots, L+n^2+1\}$ and $v \in \{1, \ldots, M\}$, interpolate the points $\{(i, f_i^{(u,v)}(j))\}_{P_i \in \mathcal{S}_{j,z}}$ and recompute the polynomial $g_i^{(u,v)}(Y)$.
- For every $u \in \{1, \ldots, L+n^2+1\}$ and $v \in \{1, \ldots, M\}$, interpolate the points $\{(i, g_i^{(u,v)}(j))\}_{P_i \in \mathcal{S}_{j,z}}$ and recompute the polynomial $f_i^{(u,v)}(X)$.

We determine the round and communication complexity of the protocol $\Pi_{\mathsf{A\text{-}VMBPSS}}$.

Lemma 12.2. *Protocol $\Pi_{\mathsf{A\text{-}VMBPSS}}$ requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^5\kappa)$ bits, apart from 1 call to $\mathcal{F}_{\mathsf{Rand}}$.*

Proof. Similar to the synchronous case, during the sharing phase, D distributes $\mathcal{O}(LM + n^2M)$ rows and column polynomials to each party, where each polynomial is a d-degree polynomial and hence consists of $\mathcal{O}(n)$ field elements as coefficients. This requires one round and incurs a total communication of $\mathcal{O}(LMn^2 + n^4M)$ field elements. Every party upon receiving its rows and column polynomials may broadcast an OK message, so this step requires a communication of $n \times \mathcal{BC}(1)$ bits. Additionally each P_i may sign its column polynomials for D, for which $\mathcal{O}(L+n^2)$ instances of Gen and Auth protocols are executed, where in each instances, P_i as a signer has M vectors for signing, each consisting of n field elements. From Lemma 5.7, by substituting $\ell = n$, this step requires a total communication of $\mathcal{O}(LMn^2 + n^4M)$ field elements plus $n \times \mathcal{BC}(Ln + n^3)$ field elements.[28] Finally, D may broadcast the identity of a Core set, which requires a communication of $\mathcal{BC}(n)$ bits. Now securely realizing the calls to

[28] Since in all the $(L+n^2)$ instances of Gen and Auth invoked by P_i as a signer, the dealer D is playing the role of I, it can broadcast all the data across all these ICP instances through a single call to $\mathcal{F}_{\mathsf{BC}}$.

$\mathcal{F}_{\mathsf{BC}}$ through the A-Cast protocol of [2], the sharing phase will require a constant runtime and incurs a communication of $\mathcal{O}(LMn^2\kappa+n^4M\kappa+n^3L\kappa+n^5\kappa+n^3\log n)$ bits, since each field element is represented by κ bits.

During verification phase, the parties call $\mathcal{F}_{\mathsf{Rand}}$ to generate the random value r. The dealer D broadcasts M bivariate polynomials, each of degree-(d,d). This requires a communication of $\mathcal{BC}(Mn^2)$ field elements. Additionally, D may invoke n instances of Rev to publicly reveal IC-signatures on M vectors, each consisting of n field elements. Form Lemma 5.7, by substituting $\ell = n$, this step requires constant runtime and incurs a communication of $\mathcal{BC}(Mn^2)$ field elements.[29] By securely realizing the calls to $\mathcal{F}_{\mathsf{BC}}$ through the broadcast protocol of [2], the verification phase will require a constant number of rounds and incurs a communication of $\mathcal{O}(Mn^2\kappa + n^3\log n)$ bits.

Next consider the non-Core row and column reconstruction phase. Here every party in Core may send the supposedly common points on their row and column polynomials to the parties outside Core. This requires one round and a total communication of $\mathcal{O}(LMn^2)$ field elements. This is followed by similar steps as in the verification phase, where D may broadcast M bivariate polynomials and publicly reveals IC-signatures of up to n parties on M vectors, each consisting of n field elements. Moreover, this may be repeated up to $\mathcal{O}(n)$ times for up to $\mathcal{O}(n)$ parties that are outside Core. So overall this phase will requires $\mathcal{O}(n^2)$ runtime and incurs a communication of $\mathcal{O}(LMn^2\kappa + Mn^4\kappa + n^5\log n)$ bits.

So the overall cost of the protocol is $\mathcal{O}(LMn^2\kappa + Mn^4\kappa + Ln^3\kappa + n^5\log n)$ bits.

Recall that in the protocol $\Pi_{\mathsf{A\text{-}VMBPSS}}$, dealer D has LMp^2 elements from $\mathbb{F}$ (and hence $\Theta(LMp^2\kappa)$ bits) to secret-share. If we set $L = n^2$ and $M = n$, then D's input consists of $\Theta(n^5)$ field elements, the protocol incurs a communication of $\mathcal{O}(n^5)$ field elements, apart from at most 1 call to $\mathcal{F}_{\mathsf{Rand}}$. Ignoring the latter, the communication overhead of the protocol will be a constant.

If D has multiple groups of n^3p^2 elements to be shared with a constant overhead, then it can run an instance of $\Pi_{\mathsf{A\text{-}VMBPSS}}$ for each group, and the call to $\mathcal{F}_{\mathsf{Rand}}$ can be unified across the verification phase of all the instances of $\Pi_{\mathsf{A\text{-}VMBPSS}}$. This will ensure that a single call of $\mathcal{F}_{\mathsf{Rand}}$ is sufficient, irrespective of the number of instances of $\Pi_{\mathcal{A}\langle\cdot\rangle}$ run by D. We can thus state the following theorem for $\Pi_{\mathsf{A\text{-}VMBPSS}}$.

Theorem 12.3. *Let $n = 3t + s$. Let $D \in \mathcal{P}$ be a designated party with inputs $\mathbb{S}^{(1)}, \ldots, \mathbb{S}^{(N)} \in \mathbb{F}^{n^3p^2}$, where $N \geq 1$ and known publicly. Then there exists a protocol in the $\mathcal{F}_{\mathsf{Rand}}$-hybrid model, which generates $\langle\mathbb{S}^{(1)}\rangle_d, \ldots, \langle\mathbb{S}^{(N)}\rangle_d$, such that the view of the adversary remains independent of $\mathbb{S}^{(1)}, \ldots, \mathbb{S}^{(N)}$ if D is honest. Moreover, even if D is corrupt, there exist some $\mathbb{S}^{(1)}, \ldots, \mathbb{S}^{(N)} \in \mathbb{F}^{n^3p^2}$ known to D, such that the protocol generates $\langle\mathbb{S}^{(1)}\rangle_d, \ldots, \langle\mathbb{S}^{(N)}\rangle_d$. Here $p = \frac{s}{4} + 1$ and $d = t + p - 1$.*

The protocol requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(n^5N\kappa)$ bits, apart from at most one call to $\mathcal{F}_{\mathsf{Rand}}$. Asymptotically, for sufficiently large N, the communication overhead of the protocol is $\mathcal{O}(1)$.

□

Private reconstruction. Here, we describe the private reconstruction of a degree-d polynomial towards a specified party, say P_R. For this, all the parties send their shares to P_R, who tries to recover the secret as follows. P_R waits for $d + t + 1 = 2t + \frac{s}{4} + 1$ shares, all of which lie on the same degree-d polynomial. This requires P_R to apply the Reed Solomon (RS) error correction repeatedly in an "online" manner, also known as online error correction (OEC) [18]. If P_R obtains such a polynomial, it is guaranteed to be the correct degree-d polynomial since it agrees with the shares of at least $d+1$ honest parties. Reconstruction towards all parties can be achieved by running the private reconstruction protocol n times, once towards each party. Moreover, the same procedure also works for reconstructing a degree-$(d + p - 1)$ polynomial towards a party, where the party now waits for receive $d + p + t = 2t + \frac{s}{2} + 1$ shares, all of which lie on the same degree-$d + p - 1$ polynomial. Note that with $n = 3t + s$, an honest party will definitely receive $2t + s$ shares lying on the same polynomial eventually.

13 The Asynchronous Preprocessing Phase

In this section, we describe our asynchronous preprocessing phase protocol which follows with minor modifications to subprotocols its synchronous counterparts.

[29] Since D is playing the role of I in all the Rev, it can broadcast all the data across these Rev instances through a single call to $\mathcal{F}_{\mathsf{BC}}$.

First, we observe that we do not require the MAC-based authentication in this setting and hence do not require the corresponding one-time setup of MAC keys. Reconstruction of values shared via degree-d and well as degree-$(d+p-1)$ polynomials can be done robustly using OEC as described above. Similar to the synchronous case, during the preprocessing phase, parties are required to generate two types of preprocessing data, the packed-double-secret-shared vectors of random values and the packed-shared multiplication triples. Both will be obtained by following a procedure very similar to that of the synchronous network. Hence, below, we discuss the major changes to the synchronous protocols while avoiding repetition.

13.1 Robust Degree Reduction Protocol with $\mathcal{O}(1)$ Communication Overhead

The robust degree reduction protocol (Protocol 8.9) translates very easily to the asynchronous setting with a few changes. First, because of the asynchronous VSS (Protocol 12.1), parties hold $\langle\cdot\rangle_d$-sharing of values instead of ${}^A\langle\cdot\rangle_d$. However, as mentioned earlier, in this setting, values shared via $\langle\cdot\rangle_d$ polynomials can be robustly reconstructed using OEC [18] even without the authentication information. Moreover, instead of reducing the degree for $t+1$ vectors of values simultaneously, the protocol now works with s vectors. That is, it takes as input $\langle\mathbb{S}^{(1)}\rangle_\ell, \ldots, \langle\mathbb{S}^{(s)}\rangle_\ell$ for $\ell \in \{d, d+p-1\}$ and gives $\langle\mathbb{S}^{(1)}\rangle_{p-1}, \ldots, \langle\mathbb{S}^{(s)}\rangle_{p-1}$ as output. The reason for this will be clear subsequently. Further, to account for delays due to asynchrony and ensure that all parties are consistent with each other, after Step 4 (of the synchronous protocol), parties run an instance of ACS to agree on a set of (at least) $n-t \geq 2t+s$ parties that re-shared their value $\langle\mathbb{S}(i)\rangle_{p-1}$ and then proceed to apply the RS-decoding algorithm to obtain the output. Note that the ACS instance can be common across many instances of degree reduction and can be invoked using the protocol from [5] which incurs a cost of $\mathcal{O}(n^5 \log n)$ bits in expected constant time. Since we may have exactly $2t+s$ shares $\mathbb{S}(i)$ on the vector of polynomials $\mathbb{S}(X)$, which may include up to t erroneous shares, setting the degree of $\mathbb{S}(X)$ to $s-1$ allows us to correct these via RS-decoding procedure. Note that in this case, parties can also robustly reduce the degree for values that are shared via $\langle\cdot\rangle_{d+p-1}$ polynomials. This can be attributed to the higher number of honest parties in this setting combined with OEC which allows robust reconstruction of degree-$(d+p-1)$ polynomials.

Lemma 13.1. *Let $n = 3t+s$ in an asynchronous network. Given inputs $\langle\mathbb{S}^{(1)}\rangle_d, \ldots, \langle\mathbb{S}^{(s)}\rangle_d$ (or $\langle\mathbb{S}^{(1)}\rangle_{d+p-1}, \ldots, \langle\mathbb{S}^{(s)}\rangle_{d+p-1}$ where each $\mathbb{S}^{(\gamma)} \in LMp^2$, there exists a protocol that outputs $\langle\mathbb{S}^{(1)}\rangle_{p-1}, \ldots, \langle\mathbb{S}^{(s)}\rangle_{p-1}$, except with a negligible probability. The protocol has a communication of $\mathcal{O}(LMnp^2\kappa)$ bits. It has one invocation of ACS and calls $\mathcal{F}_{\mathsf{Rand}}$ n times (once for each dealer)[30].*

13.2 Generating Packed-Double-Secret-Shared Random Values with $\mathcal{O}(1)$ Communication Overhead

In this case as well, the protocol in the asynchronous network follows its synchronous counterpart, with a minor change to account for asynchrony. First, parties invoke the asynchronous VSS protocol (Protocol 12.1) in the second step of Protocol 8.13 to share their vectors of random values. Further, before proceeding to invoke $\mathcal{F}_{\mathsf{Rand}}$, in order to ensure that sufficient parties have committed to their values by completing the sharing phase, parties run an instance of the ACS protocol and identify a subset of (at least) $n-t$ parties who indeed shared their random values. Following this, the protocol proceeds exactly as in the synchronous setting, while only considering the values shared by the parties identified by ACS. This implies that the polynomial passing through values shared by parties is now of degree $n-t-1$ instead of $n-1$ from the synchronous case. Similarly, since t out of $n-t$ packed-double-secret-shared vectors may be contributed by corrupt parties, parties compute the output by evaluating the polynomials on $n-2t$ distinct points instead of $n-t$ points considered in the synchronous setting.

Lemma 13.2. *Let $n = 3t+s$ in an asynchronous network. There exists a protocol that outputs $\{(\langle\widetilde{\mathbb{R}}^{(\ell,m)}\rangle_d, \langle\widetilde{\mathbb{R}}^{(\ell,m)}\rangle_{d+p-1})\}$, where $\ell \in \{1, \ldots, t\}, m \in \{1, \ldots, n-2t\}$ and where each $\widetilde{\mathbb{R}}^{(\ell,m)} \in \mathbb{F}^{LMp^2}$, except with a negligible probability. The protocol has a communication of $\mathcal{O}(LMn^2p^2\kappa)$ bits and makes at most $n+1$ calls to $\mathcal{F}_{\mathsf{Rand}}$ apart from one invocation of ACS.*

[30] Due to asynchrony of the network, we cannot unify the calls to $\mathcal{F}_{\mathsf{Rand}}$ across different dealers.

13.3 Batch Beaver Multiplication with $\mathcal{O}(1)$ Communication Overhead

Given the robust degree reduction protocol for values shared through degree-d as well as degree-$(d+p-1)$ polynomials, the protocol for batch Beaver multiplication is significantly simplified. In fact, it no longer requires the sacrificing randomness to allow for detection of disputes. Moreover, to accommodate the asychronous degree reduction protocol seamlessly, it works with $\gamma \in \{1, \ldots, s\}$. The protocol follows the high-level outline of the multiplication protocol from [30], or equivalently, is a simplification of our batch Beaver protocol from the synchronous setting. Specifically, parties will proceed as in Π_{Beaver} (Protocol 8.23) till Step 4. Following this, parties will invoke the robust degree reduction protocol to obtain $\langle \mathbb{S}^{(\beta,1)} \rangle_{p-1}, \ldots, \langle \mathbb{S}^{(\beta,s)} \rangle_{p-1}$ from $\langle \mathbb{S}^{(\beta,1)} \rangle_{d+p-1}, \ldots, \langle \mathbb{S}^{(\beta,s)} \rangle_{d+p-1}$ and directly compute their output as described in Step 14c. This is equivalent to parties applying OEC instead of simultaneous error correction and detection at Step 6 which guarantees that each P_i will always obtain its $\mathbb{S}^{(\beta)}(i)$ from $\langle \mathbb{S}^{(\beta)}(i) \rangle_{d+p-1}$. Consequently, parties do not require to complain or identify disputes, and the protocol directly proceed to Step 14a in the output computation phase with the set $\mathsf{C}^{(\beta)} = \phi$ for each $\beta \in \{1, \ldots, n\}$. Following this, parties apply ACS to identify the set of parties who reshared their values and proceed with the remaining computation as in Steps 14b-14c of Protocol 8.23.

Lemma 13.3. *Let $n = 3t+s$ in an asynchronous network. Given inputs $\{\langle \mathbb{X}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{Y}^{(\beta,\gamma)} \rangle_d\}$ and auxiliary inputs $\{\langle \mathbb{A}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{B}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{C}^{(\beta,\gamma)} \rangle_d\}$, and $\{\langle \mathbb{R}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1}\}$. Then except with a negligible probability, Π_{Beaver} generates $\{\langle \mathbb{Z}^{(\beta,\gamma)} \rangle_d\}$ in a constant expected number of rounds and incurs a communication of $\mathcal{O}(LMn^2p^2\kappa)$ bits, apart from at most $2n$ number of calls to $\mathcal{F}_{\mathsf{Rand}}$ and two invocations of ACS. Moreover, $\mathbb{Z}^{(\beta,\gamma)} = \mathbb{X}^{(\beta,\gamma)} \times \mathbb{Y}^{(\beta,\gamma)}$ component-wise, if and only if $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$ constitute multiplication-triples component-wise. Here $\beta \in \{1, \ldots, n\}$ and $\gamma \in \{1, \ldots, s\}$.*

13.4 Packed Multiplication Triple Sharing with $\mathcal{O}(1)$ Communication Overhead

We now describe the packed triple sharing protocol, which allows a dealer to verifiably share random multiplication triples. The protocol is obtained by tweaking the synchronous protocol for triple sharing in a straightforward manner. Note that in this setting, parties have $\langle \cdot \rangle_d$-sharing instead of ${}^A\langle \cdot \rangle_d$, and correspondingly, each reconstruction mentioned in the protocol refers to the reconstruction of $\langle \cdot \rangle_d$-shared values using OEC as described earlier. Moreover, it invokes the asynchronous versions of the protocols for Π_{RandPair} and Π_{Beaver} described above. Apart from this, the primary change in the protocol is due to the batch Beaver protocol. First, it operates over batches of s instead of batches of $t+1$ considered in the synchronous network. Hence, we require $\frac{t}{s} = \mathcal{O}(1)$ parallel invocations of Π_{Beaver}, with the inputs corresponding to $\gamma \in \{t+2, \ldots, 2t+1\}$ split into $\{t+2, \ldots, t+s+1\}$ for the first instance, $\{t+s+2, \ldots, t+2s+1\}$ in the second instance and so on. Further, since the Beaver protocol in the asynchronous network is robust without dispute identification, parties do not require sacrificing randomness, that is the shared random triples, in the Beaver protocol within triple sharing and need only one set of shared randomness. Specifically, the dealer only requires to generate $(\langle \mathbb{A}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{B}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{C}^{(\beta,\gamma)} \rangle_d)$ and $({}^A\langle \mathbb{R}^{(\beta,\gamma)} \rangle_d, {}^A\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1})$ for $\beta \in \{1, \ldots, n\}$ and $\gamma \in \{1, \ldots, 2t+1\}$. Whereas, the sacrificing randomness $(\langle \hat{\mathbb{A}}^{(\beta,\gamma)} \rangle_d, \langle \hat{\mathbb{B}}^{(\beta,\gamma)} \rangle_d, \langle \hat{\mathbb{C}}^{(\beta,\gamma)} \rangle_d)$ and $({}^A\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_d, {}^A\langle \hat{\mathbb{R}}^{(\beta,\gamma)} \rangle_{d+p-1})$ for $\beta \in \{1, \ldots, n\}$ and $\gamma \in \{t+2, \ldots, 2t+1\}$ is not required due to the guarantees of the Beaver protocol. We thus get the following result.

Lemma 13.4. *Let $n = 3t+s$ in an asynchronous network. There exists a protocol such that except with a negligible probability, parties output $\{\langle \mathbb{X}_i^{(\beta)} \rangle_d, \langle \mathbb{Y}_i^{(\beta)} \rangle_d, \langle \mathbb{Z}_i^{(\beta)} \rangle_d\}$, where $\beta \in \{1, \ldots, n\}$ and $i \in \{1, \ldots, t\}$, such that $\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}, \mathbb{Z}_i^{(\beta)} \in \mathbb{F}^{LMp^2}$ and component-wise $\mathbb{Z}_i^{(\beta)} = \mathbb{X}_i^{(\beta)} \times \mathbb{Y}_i^{(\beta)}$. Moreover, if D is honest, the view of the adversary will be independent of $\mathbb{X}_i^{(\beta)}, \mathbb{Y}_i^{(\beta)}$ and $\mathbb{Z}_i^{(\beta)}$. It requires a constant expected number of rounds and makes at most $2n+2$ number of calls to $\mathcal{F}_{\mathsf{Rand}}$ apart from three invocations of ACS. The protocol generates $ntLMp^2$ packed-secret-shared multiplication-triples over $\mathbb{F}$, incurring a communication of $\mathcal{O}(LMn^2p^2\kappa)$ bits.*

13.5 Packed Multiplication Triple Extraction with $\mathcal{O}(1)$ Communication Overhead

Similar to triple sharing, one of the primary changes in the asynchronous version of triple extraction is due to the batch Beaver protocol invoked within it. Since parties now invoke the asynchronous batch Beaver

protocol, this immediately implies that we do not require sacrificing randomness in the Beaver protocol. Another difference is in the input to the triple extraction protocol. Since the network is asynchronous, and we cannot distinguish between corrupt parties that do not provide an input and honest parties that are slow, the protocol takes as input triples shared by (at least) $n - t \geq 2t + s$ parties. Without loss of generality, let us assume that the first $2t+s$ parties, that is, $\{P_1, \ldots, P_{2t+s}\}$ shared their triples and these triples will be considered during triple extraction. Thus, in the protocol $\gamma \in \{1, \ldots, 2t + s\}$. Moreover, the polynomials defined over these triples $\mathbb{X}^{(\beta)}(\cdot), \mathbb{Y}^{(\beta)}(\cdot), \mathbb{Z}^{(\beta)}(\cdot)$ are now of degree $t + \frac{(s-1)}{2}, t + \frac{(s-1)}{2}$ and $2t + (s - 1)$ respectively. The choice of degrees is to ensure that the polynomials are completely defined by the vectors of triples shared by parties. That is, the vector $\mathbb{Z}^{(\beta)}(\cdot)$ corresponds to polynomials of degree $2t + (s - 1)$ which require at least $2t + s$ points to be well-defined. In the worst case, this is the exact number of parties who may have shared their triples and thus ensure the required condition holds.

Hence, in the protocol, parties hold $(\langle \mathbb{A}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{B}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{C}^{(\beta,\gamma)} \rangle_d)$ for $\gamma \in \{1, \ldots, 2t+s\}$, where w.l.o.g P_γ holds the triple $(\mathbb{A}^{(\beta,\gamma)}, \mathbb{B}^{(\beta,\gamma)}, \mathbb{C}^{(\beta,\gamma)})$. During the protocol, parties generate $({}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_d, {}^{A}\langle \mathbb{R}^{(\beta,\gamma)} \rangle_{d+p-1})$ for $\beta \in \{1, \ldots, n\}$ and $\gamma \in \{t + \frac{(s-1)}{2} + 2, \ldots, 2t + s\}$. Moreover, since the Beaver protocol operates with batches of s, parties invoke approximately $\frac{t+s}{s} = \mathcal{O}(1)$ parallel instances of batch Beaver with the inputs corresponding to $\gamma \in \{t + \frac{(s-1)}{2} + 2, \ldots, 2t + s\}$ split into $\{t + \frac{(s-1)}{2} + 2, \ldots, t + \frac{(3s-1)}{2} + 1\}$ in the first instance, $\{t + \frac{(3s-1)}{2} + 2, \ldots, t + \frac{(5s-1)}{2} + 1$ in the second instance and so on. Finally, parties output triples as the evaluation of each polynomial at $\frac{(s-1)}{2} + 1$ points owing to the degree of each polynomial. We thus get the following result.

Lemma 13.5. *Given inputs the multiplication triples $\{\langle \mathbb{A}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{B}^{(\beta,\gamma)} \rangle_d, \langle \mathbb{C}^{(\beta,\gamma)} \rangle_d\}$ where $(\beta, \gamma) \in \{1, \ldots, n\} \times \{1, \ldots, 2t+s\}$, there exists a protocol that outputs $\langle \cdot \rangle_d$-shared vectors of multiplication-triples $(\mathbb{A}_i^{(\beta)}, \mathbb{B}_i^{(\beta)}, \mathbb{C}_i^{(\beta)})$, where $\beta \in \{1, \ldots, n\}$, $i \in \{1, \ldots, \frac{(s-1)}{2} + 1\}$ and where each $\mathbb{A}_i^{(\beta)}, \mathbb{B}_i^{(\beta)}, \mathbb{C}_i^{(\beta)} \in \mathbb{F}^{LMp^2}$. The protocol requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(LMn^2p^2\kappa)$ bits, apart from $3n + 1$ number of calls to $\mathcal{F}_{\mathsf{Rand}}$ and three invocations of ACS.*

13.6 The Complete Preprocessing Phase

The high-level structure of the preprocessing phase is similar to that in the synchronous network, wherein the goal is to generate two types of preprocessing data: the packed-double-secret-shared random values and the packed-secret-shared random triples. Unlike the synchronous setting, parties do not require the MAC-key setup generation and directly proceed to generating the packed-double-secret-shared random values. In this setting parties require to run only $\frac{c_M}{t(n-2t)}ns$ instances of the shared-random-pair generation protocol. Parties do not require to execute the random triple generation protocol since sacrificing randomness is not required in this setting. Following this, for random multiplication triple generation, each party acts as a dealer and executes $\frac{2c_M}{t}$ instances of the triple sharing protocol. Finally, parties invoke the triple extraction protocol $2c_M$ times to obtain the required number packed-secret-shared random multiplication triples.

14 The Asynchronous MPC Protocol

We have already described the complete preprocessing phase primitives for the asynchronous network in the previous section. What remains to complete the MPC protocol are the primitives for the online phase. Note that we already have an asynchronous VSS protocol (Protocol 12.1) for parties to share their inputs. To account for asynchrony while avoiding indefinite wait, parties run an instance of the ACS protocol to determine the set of (at least) $n - t$ parties whose inputs will be considered for evaluation of the circuit. For the remaining parties, a default value is taken as their input. Further, the addition gates are computed locally, whereas the multiplication gates can be computed using the batch Beaver protocol already described in the preprocessing phase. Finally, the output computation requires a simple reconstruction of $\langle \cdot \rangle_d$-shared values, which is performed using OEC as described earlier. The online phase follows closely to that in [30], which is designed for $n = 4t + 1$, although for a stronger guarantee of perfect security. With this, the result below follows.

Theorem 14.1. *Let $n = 3t + s$ in an asynchronous network. There exists a protocol that evaluates any function $f : \{\{0,1\}^*\}^n \to \{\{0,1\}^*\}^n$ securely and requires $\mathcal{O}(D_M)$ expected number of rounds and incurs*

a communication of $\mathcal{O}(c_M n^7\kappa + n^8\kappa)$ *bits. Here* c_M *is the number of multiplication gates in* `ckt` *and* D_M *is the multiplicative depth of* `ckt`.

Proof. During the preprocessing phase, parties execute $\frac{c_M ns}{t(n-2t)}$ instances of shared-random-pair generation protocol, which incurs a communication of $\mathcal{O}(n^7\kappa)$ bits in constant expected number of rounds apart from $2n$ calls to $\mathcal{F}_{\mathsf{Rand}}$. It also requires one invocation of the ACS protocol. Further, parties invoke the triple sharing protocol $\frac{2c_M}{t}$ times which incurs a communication of $\mathcal{O}(n^7\kappa)$ in constant expected rounds, apart from $2n+2$ calls to $\mathcal{F}_{\mathsf{Rand}}$ and three invocations of the ACS protocol. Finally, parties execute the triple sharing protocol which incurs a communication of $\mathcal{O}(n^7\kappa)$ in addition to three invocations of the ACS protocol and $3n+1$ calls to $\mathcal{F}_{\mathsf{Rand}}$ in constant expected rounds.

During the input sharing phase, each party $P_i \in \mathcal{P}$ invokes $\Pi_{\mathsf{A\text{-}VMBPSS}}$ ns times. Note that all these instances can be executed in parallel for all n parties. Hence from Lemma 6.8, this requires a total communication of $\mathcal{O}(n^8\kappa)$ bits, a constant expected number of rounds and at most n calls to $\mathcal{F}_{\mathsf{Rand}}$. Addition gates are computed locally and hence do not incur any communication cost. Further, for each multiplication gate $\alpha \in \{1,\ldots,c_M\}$, parties invoke Π_{Beaver}. From Lemma 13.3, this requires a communication of $\mathcal{O}(n^7\kappa)$ bits, a constant expected number of rounds and at most $2n$ calls to $\mathcal{F}_{\mathsf{Rand}}$ in addition to two invocations of the ACS protocol. Note that the multiplication gates at the same multiplicative depth can be evaluated in parallel by executing the corresponding instances of Π_{Beaver} in parallel. The total communication cost for evaluating all the multiplication gates will be $\mathcal{O}(c_M n^7\kappa)$ bits. On the other hand, the expected number of rounds required for evaluating all the multiplication gates will be $\mathcal{O}(D_M)$. The maximum number of calls to $\mathcal{F}_{\mathsf{Rand}}$ for evaluating all the multiplication gates could be $2c_M n$. The calls to $\mathcal{F}_{\mathsf{Rand}}$ can be securely realized using a constant expected round protocol, which will incur a total communication of $\mathcal{O}(c_M n^6\kappa)$ bits. Similarly, the ACS invocations incur a total communication of $\mathcal{O}(c_M n^5 \log n)$ bits in constant expected rounds using the protocol from [5]. Finally, the output gate values are publicly reconstructed which incurs a communication of $\mathcal{O}(n^8\kappa)$ and one round as described in Section 4.3. □

References

1. Abraham, I., Asharov, G., Patil, S., Patra, A.: Detect, Pack and Batch: Perfectly-Secure MPC with Linear Communication and Constant Expected Time. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14005, pp. 251–281. Springer (2023)
2. Abraham, I., Asharov, G., Chandramouli, A.: Simple is cool: Graded dispersal and its applications for byzantine fault tolerance. In: 16th Innovations in Theoretical Computer Science Conference (ITCS 2025). pp. 1–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik (2025)
3. Abraham, I., Asharov, G., Patil, S., Patra, A.: Asymptotically free broadcast in constant expected time via packed vss. In: Theory of Cryptography: 20th International Conference, TCC 2022, Chicago, IL, USA, November 7–10, 2022, Proceedings, Part I. pp. 384–414. Springer (2023)
4. Abraham, I., Asharov, G., Patil, S., Patra, A.: Perfect asynchronous mpc with linear communication overhead. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 280–309. Springer (2024)
5. Abraham, I., Asharov, G., Patra, A., Stern, G.: Perfectly secure asynchronous agreement on a core set in constant expected time. IACR Cryptol. ePrint Arch. p. 1130 (2023), https://eprint.iacr.org/2023/1130
6. Agarwal, A., Bienstock, A., Damgård, I., Escudero, D.: Honest Majority GOD MPC with $\mathcal{O}(\text{depth}(C))$ Rounds and Low Online Communication. In: ASIACRYPT. Lecture Notes in Computer Science, vol. 15489, pp. 234–265. Springer (2024)
7. Applebaum, B., Kachlon, E., Patra, A.: The resiliency of MPC with low interaction: The benefit of making errors (extended abstract). In: Pass, R., Pietrzak, K. (eds.) Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12551, pp. 562–594. Springer (2020)
8. Applebaum, B., Kachlon, E., Patra, A.: The round complexity of perfect MPC with active security and optimal resiliency. In: Irani, S. (ed.) 61st IEEE Annual Symposium on Foundations of Computer Science, FOCS 2020, Durham, NC, USA, November 16-19, 2020. pp. 1277–1284. IEEE (2020)
9. Asharov, G., Chandramouli, A.: Perfect (Parallel) Broadcast in Constant Expected Rounds via Statistical VSS. In: Joye, M., Leander, G. (eds.) Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May

26-30, 2024, Proceedings, Part V. Lecture Notes in Computer Science, vol. 14655, pp. 310–339. Springer (2024)
10. Asharov, G., Cohen, R., Shochat, O.: Static vs. adaptive security in perfect MPC: A separation and the adaptive security of BGW. In: 3rd Conference on Information-Theoretic Cryptography, ITC 2022 (2022)
11. Beaver, D.: Efficient multiparty protocols using circuit randomization. In: Annual International Cryptology Conference (1991)
12. Beerliová-Trubíniová, Z., Hirt, M.: Perfectly-secure mpc with linear communication complexity. In: Proceedings of the 5th Conference on Theory of Cryptography. p. 213–230. TCC'08, Springer-Verlag, Berlin, Heidelberg (2008)
13. Ben-Or, M., Goldwasser, S., Wigderson, A.: Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In: Annual ACM Symposium on Theory of Computing (1988)
14. Ben-Sasson, E., Fehr, S., Ostrovsky, R.: Near-linear unconditionally-secure multiparty computation with a dishonest minority. In: Advances in Cryptology–CRYPTO 2012: 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings. pp. 663–680. Springer (2012)
15. Blum, E., Boyle, E., Cohen, R., Liu-Zhang, C.: Communication lower bounds for cryptographic broadcast protocols. In: Oshman, R. (ed.) 37th International Symposium on Distributed Computing, DISC 2023, October 10-12, 2023, L'Aquila, Italy. LIPIcs, vol. 281, pp. 10:1–10:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023)
16. Bracha, G.: An asynchronous [(n-1)/3]-resilient consensus protocol. In: PODC. pp. 154–162 (1984)
17. Canetti, R.: Asynchronous secure computation. Technion - Computer Science Department - Technical Report **CS0755** (1993)
18. Canetti, R.: Studies in secure multiparty computation and applications. Ph.D. thesis, Citeseer (1996)
19. Chaum, D., Crépeau, C., Damgård, I.: Multiparty unconditionally secure protocols (extended abstract). In: 20th Annual ACM Symposium on Theory of Computing (1988)
20. Chen, J.: Optimal error-free multi-valued byzantine agreement. In: 35th International Symposium on Distributed Computing (DISC 2021). pp. 17–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik (2021)
21. Chor, B., Goldwasser, S., Micali, S., Awerbuch, B.: Verifiable secret sharing and achieving simultaneity in the presence of faults (extended abstract). In: 26th Annual Symposium on Foundations of Computer Science (1985)
22. Chor, B., Kushilevitz, E.: A communication-privacy tradeoff for modular addition. Inf. Process. Lett. **45**(4), 205–210 (1993)
23. Choudhury, A., Patra, A.: Optimally resilient asynchronous mpc with linear communication complexity. In: Proceedings of the 16th International Conference on Distributed Computing and Networking. ICDCN '15, Association for Computing Machinery, New York, NY, USA (2015). https://doi.org/10.1145/2684464.2684470, https://doi.org/10.1145/2684464.2684470
24. Choudhury, A., Patra, A.: An efficient framework for unconditionally secure multiparty computation. IEEE Transactions on Information Theory (2016)
25. Damgård, I., Ishai, Y., Krøigaard, M.: Perfectly secure multiparty computation and the computational overhead of cryptography. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 445–465. Springer (2010)
26. Damgård, I., Larsen, K.G., Nielsen, J.B.: Communication lower bounds for statistically secure mpc, with or without preprocessing. In: Annual International Cryptology Conference. pp. 61–84. Springer (2019)
27. Damgård, I., Nielsen, J.B.: Scalable and unconditionally secure multiparty computation. In: Annual International Cryptology Conference (2007)
28. Damgård, I., Nielsen, J.B., Ostrovsky, R., Rosén, A.: Unconditionally secure computation with reduced interaction. In: Fischlin, M., Coron, J. (eds.) Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9666, pp. 420–447. Springer (2016)
29. Damgård, I., Nielsen, J.B., Polychroniadou, A., Raskin, M.A.: On the communication required for unconditionally secure multiplication. In: Robshaw, M., Katz, J. (eds.) Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9815, pp. 459–488. Springer (2016)
30. Damgård, I., Patil, S., Patra, A., Roy, L.: New upper and lower bounds for perfectly secure mpc. IACR Cryptol. ePrint Arch. p. XXXX (2025), https://eprint.iacr.org/2025/XXXX
31. Damgård, I., Patil, S., Patra, A., Roy, L.: New upper and lower bounds for perfectly secure MPC. IACR Cryptol. ePrint Arch. p. 1206 (2025), https://eprint.iacr.org/2025/1206
32. Damgård, I., Schwartzbach, N.I.: Communication lower bounds for perfect maliciously secure mpc. Cryptology ePrint Archive (2020)
33. Damgård, I.B., Li, B., Schwartzbach, N.I.: More communication lower bounds for information-theoretic MPC. In: Tessaro, S. (ed.) 2nd Conference on Information-Theoretic Cryptography, ITC 2021, July 23-26, 2021, Virtual Conference. LIPIcs, vol. 199, pp. 2:1–2:18

34. Das, S., Xiang, Z., Ren, L.: Near-optimal balanced reliable broadcast and asynchronous verifiable information dispersal. Cryptology ePrint Archive (2022)
35. Data, D., Prabhakaran, V.M., Prabhakaran, M.M.: Communication and randomness lower bounds for secure computation. IEEE Trans. Inf. Theory **62**(7), 3901–3929 (2016)
36. Dolev, D., Reischuk, R.: Bounds on information exchange for byzantine agreement. Journal of the ACM (JACM) (1985)
37. Escudero, D., Goyal, V., Polychroniadou, A., Song, Y.: Turbopack: honest majority mpc with constant online communication. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 951–964 (2022)
38. Feige, U., Kilian, J., Naor, M.: A minimal model for secure computation (extended abstract). In: Leighton, F.T., Goodrich, M.T. (eds.) Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing, 23-25 May 1994, Montréal, Québec, Canada. pp. 554–563. ACM (1994)
39. Franklin, M.K., Yung, M.: Communication complexity of secure computation (extended abstract). In: Kosaraju, S.R., Fellows, M., Wigderson, A., Ellis, J.A. (eds.) Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada. pp. 699–710. ACM (1992)
40. Goyal, V., Liu, Y., Song, Y.: Communication-efficient unconditional mpc with guaranteed output delivery. In: CRYPTO (2019)
41. Goyal, V., Liu, Y., Song, Y.: Communication-efficient unconditional MPC with guaranteed output delivery. In: Boldyreva, A., Micciancio, D. (eds.) Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II. Lecture Notes in Computer Science, vol. 11693, pp. 85–114. Springer (2019)
42. Goyal, V., Liu-Zhang, C., Song, Y.: Towards achieving asynchronous MPC with linear communication and optimal resilience. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VIII. Lecture Notes in Computer Science, vol. 14927, pp. 170–206. Springer (2024)
43. Goyal, V., Song, Y., Zhu, C.: Guaranteed output delivery comes free in honest majority MPC. In: Micciancio, D., Ristenpart, T. (eds.) Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II. Lecture Notes in Computer Science, vol. 12171, pp. 618–646. Springer (2020)
44. Ji, X., Li, J., Song, Y.: Linear-communication asynchronous complete secret sharing with optimal resilience. In: Annual International Cryptology Conference. pp. 418–453. Springer (2024)
45. Kushilevitz, E.: Privacy and communication complexity. In: 30th Annual Symposium on Foundations of Computer Science, Research Triangle Park, North Carolina, USA, 30 October - 1 November 1989. pp. 416–421. IEEE Computer Society (1989). https://doi.org/10.1109/SFCS.1989.63512, https://doi.org/10.1109/SFCS.1989.63512
46. Kushilevitz, E.: Privacy and communication complexity. SIAM J. Discret. Math. **5**(2), 273–284 (1992). https://doi.org/10.1137/0405021, https://doi.org/10.1137/0405021
47. MacWilliams, F.J., Sloane, N.J.A.: The theory of error correcting codes, vol. 16. Elsevier (1977)
48. Patra, A., Choudhury, A., Rangan, C.P.: Asynchronous Byzantine Agreement with Optimal Resilience. Distributed Comput. **27**(2), 111–146 (2014)
49. Patra, A., Choudhary, A., Rangan, C.P.: Communication efficient perfectly secure VSS and MPC in asynchronous networks with optimal resilience. In: Progress in Cryptology - AFRICACRYPT 2010. vol. 6055, pp. 184–202. Springer (2010)
50. Rabin, T., Ben-Or, M.: Verifiable Secret Sharing and Multiparty Protocols with Honest Majority (Extended Abstract). In: ACM Symposium on Theory of Computing (1989)
51. Shamir, A.: How to share a secret. Commun. ACM **22**(11), 612–613 (1979)
52. Song, Y., Ye, X.: Perfectly-secure MPC with constant online communication complexity. In: Boyle, E., Mahmoody, M. (eds.) Theory of Cryptography - 22nd International Conference, TCC 2024, Milan, Italy, December 2-6, 2024, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 15367, pp. 329–361. Springer (2024)
53. Song, Y., Ye, X.: Perfectly-secure mpc with constant online communication complexity. Cryptology ePrint Archive (2024)

A Properties of the Preprocessing phase

Lemma A.1. *If D is honest then adversary's view remains independent of D's input $\mathbb{S}$.*

Proof. For *honest* D we consider the view of the adversary in each phase.

– **Sharing Phase**: The adversary controls t parties. In this phase, for any particular bivariate polynomial $F^{(u,v)}(X,Y)$, the adversary has access to t rows and t columns of the bivariate polynomial, each of which are degree-$t+\frac{t}{4}$ univariate polynomials. The inputs are embedded in $(\frac{t}{4}+1)$ columns and the rows and columns of each party in $\mathsf{Dp}_D \cup \mathsf{Cr}$ is publicly known. Since D is *honest*, the members of $|\mathsf{Dp}_D \cup \mathsf{Cr}|$ must be all *corrupt* and $|\mathsf{Dp}_D \cup \mathsf{Cr}| \leq t$. Thus, for each $t+\frac{t}{4}$-degree univariate polynomial the adversary will know at most t points, so its view will be independent of the input in this phase as there will still be $(\frac{t}{4}+1)^2$ "degrees of freedom" in the polynomial. This will be the case even during any rerun as $|\mathsf{Dp}_D \cup \mathsf{Cr}| \leq t$ will be invariant in all reruns.
– **Verification Phase**: In this phase, D makes random linear combinations $\mathsf{F}^{(1,v)}(X,Y)$ public for $v \in \{1,\ldots,n^2\}$. For each of the n^2 batches of polynomials, there will be four additional masking polynomials chosen randomly in the previous phase by the *honest* D. Thus, $\mathsf{F}^{(1,v)}(X,Y)$ will be a $(t+\frac{t}{4}, t+\frac{t}{4})$-degree polynomial with the rows and columns corresponding to all the parties in $\mathsf{Dp}_D \cup \mathsf{Cr}$ being 0. The adversary will again have access to t rows and columns of the linear combinations of each bach of polynomials. Since there are 4 random masking polynomials added, revealing the linear combination will still make the view of the adversary independent of the inputs. Moreover, according to 5.4, from the IC-signatures the view of the adversary remains independent of the inputs.
– **Non-Core Row and Column Reconstruction Phase**: Again, in this phase, D makes random linear combinations $\mathsf{F}^{(2,v)}(X,Y)$ public for $v \in \{1,\ldots,n^2\}$. As established in 5.4, from the IC-signatures the view of the adversary remains independent of the inputs. The adversary has access to t rows and columns of each of the $(t+\frac{t}{4}, t+\frac{t}{4})$-degree polynomials and since there are four masking polynomials added, with only one reveal done till now, revealing one more linear combination for each batch will still keep the view of the adversary independent of the inputs.
– **Pairwise Tag Generation**: In the setup stage of the pairwise tag generation phase, each party P_j selects $(t+\frac{t}{4}+1)$ number of random t-degree polynomials. On each *honest* party running $\mathcal{F}_{\lceil \cdot \rfloor_t^i}$, the adversary receives at most t shares on each t-degree polynomial and thus its view is independent of the random values chosen by the *honest* parties. D also selects random masks $\mathsf{mask}_{ji}^{(\ell)}$ and $\mathtt{mask}_{ji}^{(\ell)}$ using t-degree polynomials and again since the adversary has access to at most t points on the polynomials, its view is independent of masks chosen by D. In the tag computation phase, each party gets $(2t+\frac{t}{4})$-degree sharing of each MAC-tag. With the adversary having access to t shares, its view is independent of the MAC-tag. In the tag share verification phase, D makes random linear combinations $\mathsf{F}^{(3,v)}(X,Y)$ public for $v \in \{1,\ldots,n^2\}$ along with the corresponding IC-signatures. According to 5.4, from the IC-signatures the view of the adversary remains independent of the inputs. The adversary has access to t rows and columns of each of the $(t+\frac{t}{4}, t+\frac{t}{4})$-degree polynomials and since there are four masking polynomials added, with only two reveals done till now, revealing one more linear combination for each batch will still keep the view of the adversary independent of the inputs. In the tag share complaint resolution stage, once again D makes random linear combinations $\mathsf{F}^{(4,v)}(X,Y)$ public for $v \in \{1,\ldots,n^2\}$ along with the corresponding IC-signatures. By 5.4, from the IC-signatures the view of the adversary remains independent of the inputs. The adversary has access to t rows and columns of each of the $(t+\frac{t}{4}, t+\frac{t}{4})$-degree polynomials and since there are four masking polynomials added, with only three reveals done till now, revealing one more linear combination for each batch will still keep the view of the adversary independent of the inputs.

Thus, in each phase of the protocol $\Pi_{A\langle\cdot\rangle}$, the view of the adversary is independent of the inputs if D is *honest*. □

Lemma A.2. *In protocol $\Pi_{A\langle\cdot\rangle}$ if D is honest and participates with input $\mathbb{S} \in \mathbb{F}^{n^5(\frac{t}{4}+1)^2}$, then except with probability $2^{-\Omega(\kappa)}$, all (honest) parties output their respective information corresponding to ${}^A\langle\mathbb{S}\rangle_{t+\frac{t}{4}}$.*

Proof. Let D starts with an input $\mathbb{S} \in \mathbb{F}^{n^5(\frac{t}{4}+1)^2}$. In the Sharing Phase, an *honest* D distributes the correct shares $\{f_i^{(u,v)}(X), g_i^{(u,v)}(Y)\}_{u=1,\ldots,n^3+4, v=1,\ldots,n^2}$ to all the parties P_i. If more than $2t+\frac{t}{4}+1$ parties send IC-signatures to D then D defines a set Core of size more than $2t+\frac{t}{4}+1$. If not, the process so far is repeated by including the parties who did not send IC-signatures into the set Dp_D and setting their shares to 0. Thus by the end of the Sharing Phase, D can identify a set of at least $2t+\frac{t}{4}+1$ parties $\mathsf{Core} \cup \mathsf{Dp}_D$ to whom it has shared consistent shares of each $\mathbf{S}^{(1)},\ldots,\mathbf{S}^{(n^3)}$.

Since D is *honest*, it shares consistent shares of $\mathbf{S}^{(1)},\ldots,\mathbf{S}^{(n^3)}$ with all the parties in the Sharing Phase. In the Verification Phase, when D publicly reveals $\mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(1,1)},\ldots,\mathsf{g}_{\star,i}^{(1,n^2)})$ for each party

$P_i \in \mathsf{Core}$, by the non-repudiation property of ICP described in 5.5, except with probability $2^{-\Omega(\kappa)}$ the signatures accepted by D will be correct signatures of the shares of the parties.

Since D is *honest*, and all *honest* parties P_i send $\mathsf{Sig}(P_i \to D, \mathbf{g}_{\star,i}^{(u,1)}, \ldots, \mathbf{g}_{\star,i}^{(u,n^2)})$ to D in the Sharing Phase, all *honest* parties get added to Core. Since parties P_i belonging to Core have received their shares $\{f_i^{(u,v)}(X), g_i^{(u,v)}(Y)\}_{u=1,\ldots,n^3+4, v=1,\ldots,n^2}$ from D, all honest parties will have correct shares of their rows and columns by the end of the Non-Core Row and Column Reconstruction Phase. Moreover, D will not be discarded in this phase except with probability $2^{-\Omega(\kappa)}$ because an *honest* D will reveal the correct linear combination polynomials and the IC-signatures will be accepted except with a probability of $2^{-\Omega(\kappa)}$ according to 5.5.

In the Pairwise Tag Generation Phase, since D is *honest* all *honest* parties will receive correct shares of $\mathsf{mask}_{ji}^{(\ell)}$ and $\mathtt{mask}_{ji}^{(\ell)}$ from $\mathcal{F}_{\lceil \cdot \rfloor_t^i}$ by the end of the Setup stage. Moreover, each *honest* party will have shares of $[v_{ji}^{(u)}]_{2t+\frac{t}{4}}^i$, some of which may be incorrect. Each *honest* party computes their shares of $\tau_{ij}^{(u)}]_{2t+\frac{t}{4}}^i$ and send it to P_i for each party P_j and each party $P_i \notin \mathsf{Cr}$. If P_ℓ is honest, then $\tau_{ij,\ell}^{(u)} = \mathbf{s}_\ell^{(u)} \cdot \boldsymbol{\mu}_{ji,\ell} + v_{ji,\ell}^{(u)}$, where $\boldsymbol{\mu}_{ji,\ell}$ denotes the twisted-share of $\boldsymbol{\mu}_{ji}$ for party P_ℓ and $\mathbf{s}_i^{(\ell)} \stackrel{\mathsf{def}}{=} (g_\ell^{(u,1)}(0), g_\ell^{(u,1)}(-1), \ldots, g_\ell^{(u,1)}(-\frac{t}{4}), \ldots, g_\ell^{(u,n^2)}(0), g_\ell^{(u,n^2)}(-1), \ldots, g_\ell^{(u,n^2)}(-\frac{t}{4})) \in \mathbb{F}^{n^2 \cdot (\frac{t}{4}+1)}$.

To verify if the tag shares reconstructed by P_i are correct or not D once again reveals a random linear combination for each batch of polynomials, along with the corresponding linear combinations. D will not be discarded in this phase except with probability $2^{-\Omega(\kappa)}$ because an *honest* D will reveal the correct linear combination polynomials and the IC-signatures will be accepted except with a probability of $2^{-\Omega(\kappa)}$ according to 5.5.

If every *honest* P_i sends Tag_i to $\mathcal{F}_{\mathsf{BC}}$ in the tag share verification phase, P_i has received shares from at least $(2t + \frac{t}{4} + 1)$ parties P_j for which $[\beta_{ij,\ell}]_t^i$ is the same as $\tau_{ij,\ell}^{(1)} + r_3 \cdot \tau_{ij,\ell}^{(2)} + \ldots + r_3^{n^3+3} \cdot \tau_{ij,\ell}^{(n^3+4)}$. Then according to A.3, the *honest* P_i can reconstruct their tag with correct shares except with probability $2^{-\Omega(\kappa)}$.

If $(\mathsf{NOK}, \mathsf{Dp}_i)$ is received from $\mathcal{F}_{\mathsf{BC}}$ where P_i is the sender, then the *honest* D will make public random linear combinations $(\mathsf{F}^{(4,1)}(X,Y), \ldots, \mathsf{F}^{(4,n^2)}(X,Y))$ along with the corresponding IC-signatures. According to 5.5, D will not be discarded in this phase except with probability $2^{-\Omega(\kappa)}$ because an *honest* D will reveal the correct linear combination polynomials and the IC-signatures will be accepted except with a probability of $2^{-\Omega(\kappa)}$. Then each *honest* P_i will be able to interpolate the shares of the MAC-tags $\tau_{ij}^{(u)}$ from all the parties in the $\mathcal{R}_{ij}$, and finally each *honest* party will output respective information corresponding to ${}^A\langle \mathbb{S} \rangle_{t+\frac{t}{4}}$.

□

Lemma A.3. *For two sets of degree-$(t + \frac{t}{4}, t + \frac{t}{4})$ bivariate polynomials $\mathcal{F}^{(u)}(X,Y)$ and $\mathcal{F}'^{(u)}(X,Y)$ with $u \in \{1, \ldots, n^3 + 4\}$, and for a random element $r \in \mathbb{F}$, define*

$$\mathsf{F}(X,Y) \stackrel{\mathsf{def}}{=} \mathcal{F}^{(1)}(X,Y) + r \cdot \mathcal{F}^{(2)}(X,Y) + \ldots + r^{n^3+3} \cdot \mathcal{F}^{(n^3+4)}(X,Y).$$

Let $\mathsf{g}_i'(y) \stackrel{\mathsf{def}}{=} \mathcal{F}'^{(1)}(i,Y) + r \cdot \mathcal{F}'^{(2)}(i,Y) + \ldots + r^{n^3+3} \cdot \mathcal{F}'^{(n^3+4)}(i,Y)$. If for more than $2t + \frac{t}{4} + 1$ values of i, $\mathsf{g}_i'(y) = \mathsf{F}(i,Y)$ then except with probability $2^{-\Omega(\kappa)}$, $\mathcal{F}^{(u)}(X,Y) = \mathcal{F}'^{(u)}(X,Y)$ for all $u \in \{1, \ldots, n^3 + 4\}$.

Lemma A.4. *In protocol $\Pi_{A\langle\cdot\rangle}$ if D is corrupt, then except with probability $2^{-\Omega(\kappa)}$, there exists some $\mathbb{S} \in \mathbb{F}^{n^5(\frac{t}{4}+1)^2}$ known to D, such that at the end of the protocol, all (honest) parties output their respective information corresponding to ${}^A\langle \mathbb{S} \rangle_{t+\frac{t}{4}}$.*

Proof. Let us consider the case where D is *corrupt* and no fixed $\mathbb{S}$ exists for which all *honest* parties output their respective information corresponding to ${}^A\langle \mathbb{S} \rangle_{t+\frac{t}{4}}$. This can occur only in the following ways:

- D distributes inconsistent shares to the parties: In the Verification Phase, the parties verify whether the shares received from D lie on the same bivariate polynomial or not. This is done by the parties choosing a random r_1 and D making public a random linear combination of the bivariates based on the r_1. So, the verification will pass with inconsistent shares only if for all $P_i \in \mathsf{Core}$ and for all $v \in \{1, \ldots, n^2\}$ the values in $\mathsf{g}_{\star,i}^{(1,v)}$ lie on the polynomial $\mathsf{F}^{(1,v)}(i,Y)$ and $\mathsf{Sig}(P_i \to D, \mathsf{g}_{\star,i}^{(1,1)}, \ldots, \mathsf{g}_{\star,i}^{(1,n^2)})$

is valid. According to Lemma 5.5, the probability of D using an incorrect linear combination and pass the verification of the IC-signatures is $2^{-\Omega(\kappa)}$.
If D shares inconsistent shares but use the correct IC-signatures for these shares, then D can pass the verification only with a probability of $2^{-\Omega(\kappa)}$ as described in Lemma A.3.
Thus, if the D shares inconsistent shares, then except with a probability of $2^{-\Omega(\kappa)}$, D will get discarded and all *honest* parties output information regarding the default $\mathbb{S}$.
Thus, if D is not discarded by the end of the verification phase, then a set $\mathsf{Core} \cup \mathsf{Dp}_D$ of at least $(2t + \frac{t}{4} + 1)$ parties have been identified whose shares define unique bivariate polynomials.

- *Honest* parties outside $\mathsf{Core} \cup \mathsf{Dp}_D$ reconstruct incorrect shares: In the Non-Core row and column reconstruction phase, the parties in Core send the common points to all parties outside $\mathsf{Core} \cup \mathsf{Dp}_D \cup \mathsf{Cr}$. D reveals a random linear combination for each batch of polynomials along with the corresponding IC-signatures. The parties check whether the rows and columns corresponding to Dp_D of these bivariate polynomials are $\mathbf{0}$ and if not, then discards D and output information about the default values. If the IC-signatures are valid then except with probability $2^{-\Omega(\kappa)}$ D has made the correct linear combination public according to Lemma 5.5. Then according to Lemma A.3, D has made the correct bivariate polynomial public except with probability $2^{-\Omega(\kappa)}$. Thus, the *honest* parties outside $\mathsf{Core} \cup \mathsf{Dp}_D$ will be able to reconstruct correct shares.
- Incorrect shares of a tag get verified by an *honest* party in the Tag Share Verification stage: Again, in the tag share verification stage, D makes a random linear combination of its batch of polynomials public, along with the corresponding IC-signatures. If D is discarded, the parties output the default value. If D is not discarded at this stage then the IC-signatures correspond to the correct shares except with probability $2^{-\Omega(\kappa)}$ as described in Lemma 5.5. Thus, if at this stage an *honest* party P_i finds that for some P_j $[\beta_{ij,\ell}]_t^i$ is the same as $\tau_{ij,\ell}^{(1)} + r_3 \cdot \tau_{ij,\ell}^{(2)} + \ldots + r_3^{n^3+3} \cdot \tau_{ij,\ell}^{(n^3+4)}$ for more than $(2t + \frac{t}{4} + 1)$ parties P_ℓ then according to Lemma A.3 the tag shares accepted by P_i are correct except with probability $2^{-\Omega(\kappa)}$.
- Incorrect tag gets reconstructed by an *honest* party in the Tag Share Complaint Resolution stage: In the tag share complaint resolution stage, again D makes a random linear combination of its batch of polynomials public, along with the corresponding IC-signatures. If D is discarded, the parties output the default value. If D is not discarded at this stage then the IC-signatures correspond to the correct shares except with probability $2^{-\Omega(\kappa)}$ as described in Lemma 5.5. Thus, according to Lemma A.3 if an *honest* party outputs the tags in this phase they will be those corresponding to $\mathbb{S}$ except with probability $2^{-\Omega(\kappa)}$.

□

B Perfectly-secure MPC protocols for SIMD Circuits

In this section, we discuss results for the perfect security in the non-optimal threshold setting. We first consider the synchronous setting, followed by the asynchronous setting.

B.1 Perfectly-secure MPC in the Synchronous Network

We consider the case when $n = 3t + s$ where $s = \Theta(n)$ and discuss how to instantiate the core building blocks with perfect security for instantiating the generic blueprint presented in Fig 1, so that we achieve $\mathcal{O}(1)$ communication overhead. Following the statistical setting, we set our packing parameter $p = \frac{s}{4} + 1$, implying that $d = t + p - 1 = t + \frac{s}{4}$ and $d + p - 1 = t + \frac{s}{2}$. Since $s = \Theta(n)$, it implies that $p = \Theta(n)$ and d as well as $d + p - 1$ are $\Theta(n)$ as well. We will discuss the building blocks in Fig 1 in a bottom-up fashion and use the protocol names mentioned in that figure. We will not be providing formal details, as these building blocks are obtained by extending existing perfectly-secure gadgets to operate with PSS.

Perfectly-secure VSS with a Constant Overhead (Protocol Π_{VSS}).

The work of [1] provided a packed secret sharing (PSS) protocol with perfect security for the *optimal* threshold of $n \geq 3t + 1$, with $\mathcal{O}(1)$ communication overhead. In their protocol, there exists a dealer D with L vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)} \in \mathbb{F}^p$. The protocol can verifiably generate $\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(L)} \rangle_\ell$ for any given $\ell \in \{p - 1, d, d + p - 1\}$. The verifiability ensures that even if D is corrupt, it has generated

$\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(L)} \rangle_\ell$ for vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)} \in \mathbb{F}^p$ held by D. The protocol requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(Lp \log |\mathbb{F}| + n^4 \log |\mathbb{F}|)$ bits. Consequently, if $L = n^3$, then the communication overhead of the protocol with be a constant, as $p = \Theta(n)$. If $\ell \in \{d, d+p-1\}$, then the view of the adversary in the protocol remains independent of D's inputs for an *honest* D.

In the protocol, D essentially selects "several" random bivariate polynomials of different degrees in x and y, depending upon the value of given ℓ and embeds the elements of its input vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)}$ in these polynomials and then verifiably distributes the rows and column polynomials of these bivariate polynomials, where the verifiability guarantees that even a potentially *corrupt* D has distributed rows and column polynomials lying on consistent bivariate polynomials of appropriate degrees. Once this is done, the parties output shares lying on their row/column polynomials. We refer the readers to [1] for the complete formal details.

Perfect-Secure Designated Reconstruction with a Constant Overhead (Protocol Π_{RecPriv}).

Given $\langle \mathbf{s} \rangle_\ell$ where $\ell \in \{p-1, d, d+p-1\}$ and a designated party $P_R \in \mathcal{P}$, one can use the following reconstruction protocol to let P_R reconstruct $\mathbf{s}$. Every party sends its share of $\langle \mathbf{s} \rangle_\ell$ to P_R, who applies the RS error-correction on the received shares to error-correct up to t errors, obtain the underlying sharing-polynomial and outputs $\mathbf{s}$. Since $n = 3t + s$ and ℓ could be at most $t + \frac{s}{2}$, it follows that P_R will always be able to error-correct up to t errors. Note that we *do not* want any MACs here to identify the incorrect shares, as there will be sufficient redundancy to error-correct up to t errors. The protocol will require a single round and a communication of $\mathcal{O}(n \log |\mathbb{F}|)$ bits. As $|\mathbf{s}| = p = \Theta(n)$, the communication overhead of the protocol is a constant.

Perfectly-Secure Degree Reduction with a Constant Overhead (Protocol Π_{DegRed}).

This can be achieved by exactly following the blueprint discussed in Section 2 and outlined in Fig 1. Moreover, *unlike* the statistical case where we faced subtle issues while trying to reduce the degree of $\langle \cdot \rangle_{d+p-1}$-sharings (due to non-linearity of MACs), we will *not* face any issue in the perfect setting. This is because parties *can* now robustly reconstruct degree-$d+p-1$ sharings.

In more detail, given inputs $\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(t+s)} \rangle_\ell$, where $\ell \in \{d, d+p-1\}$, the goal is to generate $\langle \mathbf{s}^{(1)} \rangle_{p-1}, \ldots, \langle \mathbf{s}^{(t+s)} \rangle_{p-1}$ with a constant communication overhead. For this, the parties first define a vector of degree-$(t+s-1)$ polynomials $\mathbf{S}(X) = \mathbf{s}^{(1)} + \mathbf{s}^{(2)} \cdot X + \ldots + \mathbf{s}^{(t+s)} \cdot X^{t+s-1}$. The parties then locally compute $\langle \mathbf{S}(i) \rangle_\ell$ and robustly reconstruct $\mathbf{S}(i)$ towards P_i. Party P_i then generates $\langle \mathbf{S}(i) \rangle_{p-1}$ using the perfect VSS protocol. This is followed by the parties error-correcting up to t incorrect vectors of points on the polynomials in $\mathbf{S}(X)$ and computing $\langle \mathbf{s}^{(1)} \rangle_{p-1}, \ldots, \langle \mathbf{s}^{(t+s)} \rangle_{p-1}$.

To achieve $\mathcal{O}(1)$ communication overhead, we need to ensure that each P_i has at least n^3 vectors of size p elements each for redistribution through VSS.[31]. Hence, we need to run the steps of above degree-reduction protocol for n^3 batches, where in each batch there are $t+s$ vectors. Thus the protocol will take inputs $\{\langle \mathbf{s}^{(L,M)} \rangle_\ell\}$ and output $\{\langle \mathbf{s}^{(L,M)} \rangle_{p-1}\}$, where $L \in \{1, \ldots, n^3\}$ denotes the batch number and $M \in \{1, \ldots, t+s\}$ denotes the input within a specific batch. Hence the input will consists of $n^3(t+s)p = \Theta(n^5)$ elements from $\mathbb{F}$. The protocol will involve nL instances of designated private reconstruction and n instances of VSS, each for sharing L vectors. The total communication cost of the protocol will be then $\mathcal{O}(n^2 L \log |\mathbb{F}| + nLp \log |\mathbb{F}| + n^5 \log |\mathbb{F}|) = \mathcal{O}(n^5 \log |\mathbb{F}|)$ bits. The protocol will require a constant expected number of rounds.

Verifiably Generating Double-Packed-Secret-Sharing with Perfect Security.

We will need a perfectly-secure protocol Π_{Double}, which enables a designated dealer $D \in \mathcal{P}$ to verifiably generate double-packed-secret-sharing (DPSS) of its vectors of inputs, which remain private if D is *honest*. This protocol will be then used to generate DPSS of random vectors with perfect security based on random-extraction techniques. Note that instantiating Π_{Double} with statistical security can be done easily by publicly checking a random linear combination of D's input vectors. To achieve the same with perfect security, we simply extend the idea of Hyper-invertible matrices [12] for vectors of values.

To begin with let D has input vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(t+s)} \in \mathbb{F}^p$. It generates $\langle \mathbf{s}^{(i)} \rangle_d$ as well as $\langle \mathbf{s}^{(i)} \rangle_{d+p-1}$ for $i \in \{1, \ldots, t+s\}$ and the parties want to verify whether D has secret-shared the same pair of vectors. Let parties hold $\{\langle \mathbf{s}^{(i)} \rangle_d\}$ and $\{\langle \mathbf{s}'^{(i)} \rangle_{d+p-1}\}$, where $\mathbf{s}^{(i)} = \mathbf{s}'^{(i)}$ for an honest D. To verify that the same

[31] Recall that perfect VSS achieves $\mathcal{O}(1)$ overhead only if the designated dealer has at least n^3 vectors for packed-secret-sharing.

holds even for a *corrupt* D, let $\mathbf{S}(X)$ and $\mathbf{S}'(X)$ be degree-$(t+s-1)$ polynomials defined by the vectors of points $\{(i, \mathbf{s}^{(i)})\}$ and $\{(i, \mathbf{s}'^{(i)})\}$ respectively, where $i \in \{1, \ldots, t+s\}$. The goal will be then to check if $\mathbf{S}(X) = \mathbf{S}'(X)$. For this, the parties locally compute $\langle \mathbf{S}(n+i) \rangle_d$ and $\langle \mathbf{S}'(n+i) \rangle_{d+p-1}$ for $i \in \{1, \ldots, n\}$ and reconstruct $\langle \mathbf{S}(n+i) \rangle_d$ and $\langle \mathbf{S}'(n+i) \rangle_{d+p-1}$ towards P_i. Party P_i upon reconstructing $\mathbf{S}(n+i)$ and $\mathbf{S}'(n+i)$ verifies if they are the same vector and publicly confirms the same. If at least $n - t = 2t + s$ confirm positively, then it implies that at least $n - 2t = t + s$ honest parties have verified that the polynomials in $\mathbf{S}(X)$ and $\mathbf{S}'(X)$ have $t + s$ common points, implying that $\mathbf{S}(X) = \mathbf{S}'(X)$ and hence $\mathbf{s}^{(i)} = \mathbf{s}'^{(i)}$. During the verification process, the adversary may learn t points on the polynomials in $\mathbf{S}(X)$ and $\mathbf{S}'(X)$ for an *honest* D. Consequently, the parties extend the polynomials in $\mathbf{S}(X)$ and $\mathbf{S}'(X)$ at $t+s-t = s$ new points and output degree-d and degree-$d+p-1$ PSS of these vectors of points. That is, the parties output $\{\langle \mathbf{S}(2n+i) \rangle_d\}$ and $\{\langle \mathbf{S}'(2n+i) \rangle_{d+p-1}\}$, for $i \in \{1, \ldots, s\}$. So even though D started with PSS of $t+s$ pairs of vectors of inputs, up to t pairs are sacrificed for verification and parties output PSS of s pairs of vectors on behalf of D, if the verification is successful. If the verification is unsuccessful and less than $n - t$ parties confirm positively, then clearly D is corrupt. In this case, the parties discard D and output default DPSS of 0^p on behalf of D.

To maintain a constant overhead in the above protocol, we need to ensure that D has at least n^3 vectors for secret-sharing so that it can use the perfect VSS for sharing them. We also need to ensure that we run the above steps for several batches of input vectors of D in parallel, so that the cost of parties publicly acknowledging the positive confirmation through broadcast is amortized. Consequently, we run the steps of the above protocol for n^2 batches, where in each batch D will have $(t + s)$ pairs of vectors each consisting of p elements. There will be $\Theta(n^3)$ instances of private reconstruction in the protocol, incurring a communication of $\mathcal{O}(n^4 \log |\mathbb{F}|)$ bits. Additionally the parties will broadcast $\mathcal{O}(n)$ bits. Note that each party needs to perform its designated verification of vectors for all the n^2 batches and then broadcast once. The protocol will output DPSS of $n^2 s$ pairs of vectors on behalf of D, which will be random if D is honest. Consequently, the communication overhead of the protocol will be $\mathcal{O}(1)$, as output consists of $\Theta(n^4)$ field elements. The protocol will require a constant expected number of rounds.

Random Double-Packed-Secret-Sharing with Perfect Security (Protocol Π_{RandPair}).

Given protocol Π_{Double}, we generate DPSS of random vectors of elements easily using the same method as used in the statistical world. Namely, each party P_i acts as a dealer and invokes an instance of Π_{Double} for generating DPSS of $n^2 s$ pairs of random vectors of elements. The DPSS generated by the parties are then grouped into $n^2 s$ batches, where the j^{th} batch consists of the j^{th} DPSS generated by all the n parties. In this batch, there are at least $n - t$ DPSS generated by the honest parties, which are random for the adversary. Consequently, by using randomness-extraction techniques based on polynomial interpolation, the parties can locally extract DPSS of $n - t$ random vectors from the j^{th} batch.

The protocol will involve n instances of Π_{Double} which will cost $\mathcal{O}(n^5 \log |\mathbb{F}|)$ bits. The protocol will output DPSS of $n^2 s \cdot (n - t) = \Theta(n^4)$ random vectors, each consisting of p elements from $\mathbb{F}$. Hence the communication overhead of the protocol will be a constant. The protocol will require a constant expected number of rounds.

Perfectly-Secure Beaver Multiplication with a Constant Overhead (Protocol Π_{Beaver}).

Unlike the statistical case, performing Beaver's multiplication over PSS will be very simple in the perfect setting. This is because we can now perform degree-reduction, both for degree-d as well as degree-$(d + p - 1)$ PSS. Hence we simply perform the blueprint discussed in Section 2. To achieve $\mathcal{O}(1)$ communication overhead, we ensure that we have "sufficiently" many pairs of vectors to be multiplied, so that we can satisfy the constraints on the minimum number of inputs required in the instances of degree-reduction to achieve a constant overhead.

Namely, the protocol will take degree-d PSS of $n^3(t + s)$ pairs of vectors as input and will securely compute degree-d PSS of product of these vectors (component-wise). There will be two instances of degree-reduction, one for reducing the degree of sharing from d to $p - 1$ and the second for reducing the degree of sharing from $d + p - 1$ to $p - 1$. Both these instances will incur a communication cost of $\mathcal{O}(n^5 \log |\mathbb{F}|)$ bits. We will also need to generate DPSS of $n^3(t + s) = \Theta(n^4)$ vectors of random elements. This will cost $\mathcal{O}(n^5 \log |\mathbb{F}|)$ bits of communication. So the communication overhead of the protocol will be a constant. This is because the input size (namely the number of field elements whose product is computed) is $\Theta(n^5)$. The protocol will require a constant expected number of rounds.

Perfectly-Secure Polynomial Verification with a Constant Overhead (Protocol Π_{PolyVer}).

Let $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ be vectors of p polynomials of degree-ℓ, ℓ and 2ℓ respectively, where $\ell \geq t$ and $2\ell + t + 1 \leq n$, such that the parties hold $\langle \boldsymbol{X}(i) \rangle_d, \langle \boldsymbol{Y}(i) \rangle_d$ and $\langle \boldsymbol{Z}(i) \rangle_d$ for $i \in \{1, \ldots, 2\ell + t + 1\}$. It turns out that we can set $\ell = t + \frac{(s-1)}{2}$ and $2\ell = 2t + (s-1)$ respectively. The parties now want to verify if $\boldsymbol{Z}(\cdot) = \boldsymbol{X}(\cdot)\boldsymbol{Y}(\cdot)$ holds, such that adversary learns at most t points on the polynomials during the process and nothing additional. This can be achieved by verifying if $\boldsymbol{Z}(i) = \boldsymbol{X}(i) \times \boldsymbol{Y}(i)$ holds (component-wise), for at least $2\ell+1$ distinct is. With this idea, we let each party $P_i \in \{P_1, \ldots, P_{2\ell+t+1}\}$ to robustly reconstruct the vectors of points $\boldsymbol{X}(i), \boldsymbol{Y}(i)$ and $\boldsymbol{Z}(i)$, verify if $\boldsymbol{Z}(i) = \boldsymbol{X}(i) \times \boldsymbol{Y}(i)$ (component-wise) and publicly announce the result. If P_i complaints that $\boldsymbol{Z}(i) \neq \boldsymbol{X}(i) \times \boldsymbol{Y}(i)$, then the parties publicly reconstruct the vectors $\boldsymbol{X}(i), \boldsymbol{Y}(i)$ and $\boldsymbol{Z}(i)$ and check if indeed P_i's complaint is genuine. If it is not then the complaint is discarded, else the parties conclude that $\boldsymbol{Z}(\cdot) \neq \boldsymbol{X}(\cdot)\boldsymbol{Y}(\cdot)$. If there are no genuine complaints, then it implies that $\boldsymbol{Z}(i) = \boldsymbol{X}(i) \times \boldsymbol{Y}(i)$ for at least $2\ell + t + 1 - t = 2\ell + 1$ honest P_i's, implying that $\boldsymbol{Z}(\cdot) = \boldsymbol{X}(\cdot) \times \boldsymbol{Y}(\cdot)$. During the protocol, the adversary may learn at most t points on the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$.

The protocol as described above will *not* achieve a constant communication overhead. This is because publicly reconstructing the vector of points $\boldsymbol{X}(i), \boldsymbol{Y}(i)$ and $\boldsymbol{Z}(i)$ corresponding to any complaining party P_i will incur a communication of $\mathcal{O}(n^2 \log |\mathbb{F}|)$ bits. There could be up to $\mathcal{O}(n)$ such complaints, which will make the communication complexity $\mathcal{O}(n^3 \log |\mathbb{F}|)$, while to achieve a constant overhead, the total communication complexity should be $\mathcal{O}(n^2 \log |\mathbb{F}|)$ bits, as the input consists of $\ell = \Theta(n)$ vectors of points, each consisting of $p = \Theta(n)$ field elements. A simple trick to fix this problem is to run the above protocol for "multiple" batches of $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ polynomials. If P_i complaints, then instead of publicly opening the vector of points corresponding to P_i for all batches for which it complained, the parties publicly open the vector of points for *exactly one* batch of polynomials. This will be sufficient to judge whether P_i has complained genuinely for all the batches for which it complained. It turns out that running the protocol in parallel for n batches of vectors of polynomials will suffice to achieve $\mathcal{O}(1)$ communication overhead.

In more detail, the input of protocol will be vectors of polynomials $\{\boldsymbol{X}^{(j)}, \boldsymbol{Y}^{(j)}, \boldsymbol{Z}^{(j)}\}_{j \in \{1,\ldots,n\}}$, such that the parties will hold degree-d PSS of the vector of points $\boldsymbol{X}^{(j)}(i), \boldsymbol{Y}^{(j)}(i), \boldsymbol{Z}^{(j)}(i)$, where $i \in \{1, \ldots, 2\ell + t + 1\}$. The parties run the steps of the protocol outlined earlier for each batch of polynomials $(\boldsymbol{X}^{(j)}, \boldsymbol{Y}^{(j)}, \boldsymbol{Z}^{(j)})$. If party P_i complaints during the verification process, then parties consider the least-indexed batch $j \in \{1, \ldots, n\}$, for which it complained and publicly open the vector of points $\boldsymbol{X}^{(j)}(i), \boldsymbol{Y}^{(j)}(i), \boldsymbol{Z}^{(j)}(i)$ to check if P_i's complaint is genuine or not. The idea here is that if P_i is honest, then it would have complained genuinely, which parties would identify after public opening. Otherwise, parties will identify that P_i is corrupt and its complaints for all the batches are discarded.

The above protocol will incur a constant expected number of rounds. There will be n instances of private reconstruction, where in each instance, $\Theta(n)$ vectors of points are reconstructed towards a designated party. This will cost overall $\mathcal{O}(n^3 \log |\mathbb{F}|)$ bits of communication. For every complaining party, there will be an instance of public reconstruction of a triplet of PSS, which will costs $\mathcal{O}(n^2 \log |\mathbb{F}|)$ bits. As there could be $\mathcal{O}(n)$ such complaints, resolving all the complaints will cost $\mathcal{O}(n^3 \log |\mathbb{F}|)$ bits of communication. So the total communication cost will be $\mathcal{O}(n^3 \log |\mathbb{F}|)$ bits. Since the input consists of $\Theta(n^3)$ field elements (there are n batches, where in each batch there are $\Theta(\ell) = \Theta(n)$ PSS of vectors of size $p = \Theta(n)$ elements), the communication overhead of the protocol is now a constant.

Perfectly-Secure Verifiable Triple-Sharing with a Constant Overhead (Protocol Π_{TripSh}).

The design of the verifiable multiplication-triple-sharing protocol follows exactly the same blueprint as discussed in Section 2, except that we slightly modify the number of triples generated, to ensure that we achieve a constant overhead using the building-blocks discussed till now. To begin with $D \in \mathcal{P}$ picks vectors of multiplication-triples $(\mathbf{u}^{(i)}, \mathbf{v}^{(i)}, \mathbf{w}^{(i)})$ and verifiably generates degree-d PSS of these vectors using the perfect VSS protocol, where $i \in \{1, \ldots, 2t + s\}$. To verify that triples shared by D are indeed multiplication-triples, PSS of these triples are transformed into degree-d PSS of correlated triples $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}, \boldsymbol{z}^{(i)})$, such that there exist vectors of polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot), \boldsymbol{Z}(\cdot)$ of degree $t + \frac{(s-1)}{2}, t + \frac{(s-1)}{2}$ and $2t+(s-1)$ respectively, such that $\boldsymbol{X}(i) = \boldsymbol{x}^{(i)}, \boldsymbol{Y}(i) = \boldsymbol{y}^{(i)}$ and $\boldsymbol{Z}(i) = \boldsymbol{z}^{(i)}$ holds for $i \in \{1, \ldots, 2t+s\}$. Moreover, $\boldsymbol{Z}(\cdot) = \boldsymbol{X}(\cdot)\boldsymbol{Y}(\cdot)$ holds if and only if all the triples shared by D are multiplication-triples. The

transformation of the triples is done by running instances of perfectly-secure Beaver's multiplication protocol. The parties then locally compute degree-d PSS of $\boldsymbol{X}(i), \boldsymbol{Y}(i)$ and $\boldsymbol{Z}(i)$, for $i \in \{2t+s+1, \ldots, n\}$ and then verify the multiplicative relationship among the polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ using the perfectly-secure polynomial-verification protocol. If the relationship is not confirmed then D is corrupt and discarded and the parties output default degree-d PSS of 0'S on behalf of D. Else, adversary may learn at most t vectors of points on the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$, leaving $\frac{(s-1)}{2}+1$ "degree-of-freedom" in these polynomials. Consequently, in the latter case, the parties extend the polynomials in $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ at $\frac{(s-1)}{2}+1$ new points $n+1, \ldots, n+\frac{(s-1)}{2}+1$ and output degree-d PSS of these vectors of multiplication-triples.

To achieve a constant communication overhead, we need to guarantee that there are sufficiently many PSS in the instances of VSS, Beaver's multiplication and polynomial verification. It turns out that if D starts the protocol with n^3 batches of vectors of multiplication-triples, then we will be able to achieve a constant overhead. The protocol will then output PSS of $n^3 \cdot (\frac{(s-1)}{2}+1)$ vectors of multiplication-triples, each consisting of p multiplication-triples. Hence the total number of multiplication-triples which are output on behalf of D will be $\Theta(n^5)$. The protocol will require a constant expected number of rounds and incur a communication of $\mathcal{O}(n^5 \log |\mathbb{F}|)$ bits. Thus the communication overhead of the protocol will be a constant.

Perfectly-Secure Triple-Extraction with a Constant Overhead (Protocol Π_{TripExt}).

Given degree-d PSS of vectors of multiplication-triples $(\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)})$ for $i \in \{1, \ldots, n\}$, where P_i knows $(\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)})$ and which remains random for the adversary if P_i is honest, the perfectly-secure triple-extraction proceeds as follows: PSS of the input multiplication-triples are transformed into degree-d PSS of vectors of correlated multiplication-triples $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}, \boldsymbol{z}^{(i)})$ for $i \in \{1, \ldots, n\}$, such that there exist vectors $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ of polynomials of degree $\frac{n-1}{2}, \frac{n-1}{2}$ and $n-1$ respectively, passing through the transformed vectors of multiplication-triples. Note that $\boldsymbol{Z}(\cdot) = \boldsymbol{X}(\cdot)\boldsymbol{Y}(\cdot)$ will hold as all the transformed triples will be multiplication-triples. The transformation happens by running an instance of perfectly-secure Beaver's multiplication-protocol. Note that adversary will know at most t vectors of points on the polynomials, leaving $\frac{n-1}{2}+1-t$ "degree-of-freedom". Consequently, the parties extend the polynomials at $\frac{n-1}{2}+1-t = \frac{t+s+1}{2}$ new points and output degree-d PSS of these vectors of points, which will be multiplication-triples and which will be random for the adversary.

To achieve a constant communication overhead, the above protocol is executed in parallel on n^3 batches. This will ensure that there are sufficiently many products to be computed through perfectly-secure Beaver's protocol and maintain a constant communication overhead. Accordingly, the protocol will output degree-d PSS of $n^3 \cdot (\frac{t+s+1}{2})$ vectors of p random multiplication-triples. Hence the protocol outputs $\Theta(n^5)$ random multiplication-triples. The protocol will involve a constant expected number of rounds and incur a communication of $\mathcal{O}(n^5 \log |\mathbb{F}|)$ bits. Hence the communication overhead of the protocol will be a constant.

Perfectly-Secure Preprocessing Phase with a Constant Overhead (Protocol Π_{Prep}).

In the preprocessing phase protocol, every party acts as a dealer and verifiably generates degree-d PSS of vectors of random multiplication-triples using protocol Π_{TripSh}. The parties then run instances of Π_{TripExt} to extract degree-d PSS of vectors of multiplication-triples which remain random for the adversary. To achieve a constant communication overhead, we need to ensure each party secret-shares sufficiently many vectors of multiplication-triples and that there are sufficiently many vectors while running instances of Π_{TripExt}. It turns out that each party P_i has to generate PSS of $\Theta(c_M n^4)$ vectors of random multiplication-triples through Π_{TripSh}, where c_M is the number of multiplication gates in `ckt`. The parties then run $c_M n$ instances of Π_{TripExt} to extract $\Theta(c_M n^5)$ vectors of random multiplication-triples, where each vector consists of p elements. Overall the protocol will require a constant expected number of rounds and incur a total communication of $\mathcal{O}(c_M n^6)$ field elements, thus guaranteeing that the communication overhead of the protocol is a constant.

Note that the above analysis also determines the size of the inputs over which the SIMD circuit has to be evaluated (see the next protocol) to overall achieve a constant communication overhead. Namely, the SIMD circuit has to be evaluated over a batch of $\Theta(n^6)$ inputs. This will require to generate $\Theta(n^6)$ random multiplication-triples per multiplication-gate from the preprocessing phase and so we can deploy

our preprocessing phase protocol to generate degree-d PSS of $\Theta(c_M n^5)$ vectors of random multiplication-triples, where each vector has $\Theta(n)$ field elements.

Perfectly-Secure Circuit-Evaluation with a Constant Overhead (Protocol Π_{ckteval}).

The circuit-evaluation protocol for evaluating the SIMD circuit is straight forward. Unlike the statistical case where the SIMD circuit has to be evaluated over batches of large inputs, here we require the parties to evaluate circuit over batches of $\Theta(n^6)$ inputs to achieve a constant communication overhead. The parties first run the perfectly-secure pre-processing phase protocol to generate degree-d PSS of $\Theta(c_M n^5)$ vectors of random multiplication-triples, each consisting of p elements from $\mathbb{F}$. Every party P_i then generates degree-d PSS of its $\Theta(n^6)$ inputs to the circuit. The parties then start evaluating each gate in the circuit over a batch of $\Theta(n^6)$ inputs where the inputs will be degree-d packed-secret-shared. The linear gates are evaluated non-interactively, while multiplication gates are evaluated using perfectly-secure Beaver's protocol by deploying the secret-shared vectors of multiplication-triples generated in the pre-processing phase. Once the vectors of function outputs are ready in a packed-secret-shared fashion, they are publicly reconstructed. The protocol will require $\mathcal{O}(D_M)$ expected number of rounds. We require $|\mathbb{F}| \geq 3n$ which will ensure we have sufficiently many distinct elements to be utilized during polynomial-verification, triple-sharing and triple-extraction protocols. We summarize the result in Theorem B.2.

Theorem B.1. *For an SIMD circuit* $\mathtt{ckt}$ *over a finite field* $\mathbb{F} \geq 3n$ *consisting of* c_O *output gates and having multiplicative depth* D_M*, there exists a perfectly-secure MPC protocol for* n *parties tolerating* t *corruptions where* $n = 3t + s$*. The protocol evaluates the circuit over a batch of* $\Theta(n^6)$ *inputs from* $\mathbb{F}$*. The protocol requires* $\mathcal{O}(D_M)$ *expected number of rounds and incurs a communication of* $\mathcal{O}(|\mathtt{ckt}| + c_O \cdot n)$ *field elements, provided* $s = \Theta(n)$*.*

B.2 Perfectly-secure MPC in the Asynchronous Network

In the asynchronous setting, we focus on the case where $n = 4t + s$ and $s = \Theta(n)$. As in the synchronous model, we provide instantiations of the core building blocks with perfect security, enabling the blueprint depicted in Fig. 1 to be realized with $\mathcal{O}(1)$ overhead. Analogous to the prior setting, we set the packing parameter $p = \frac{s}{4} + 1$, which results in $d = t + p - 1 = t + \frac{s}{4}$ and $d + p - 1 = t + \frac{s}{2}$. As before, we describe only the high-level functionality of these building blocks, as their construction follows directly from well-established primitives. Note that p, d and $d + p - 1$ are all $\Theta(n)$.

Perfectly-secure AVSS with a Constant Overhead. Choudhury et al. [24] proposed an asynchronous verifiable secret sharing protocol with perfect security for the optimal resilience threshold of $n \geq 4t+1$ that has an overhead of $\mathcal{O}(n)$ for sharing a single secret. Their construction relies on a bivariate polynomial of degree-$(2t, t)$, which enables sharing $\mathcal{O}(n)$ secrets simultaneously. From the perspective of amortized cost per secret, this is asymptotically equivalent to using a bivariate polynomial of degree-(ℓ, t) for $\ell \in \{p - 1, d, d + p - 1\}$. We observe that a simplified version of their protocol under a relaxed threshold of $n \geq 4t + s$ immediately yields a verifiable secret sharing protocol with an overhead of $\mathcal{O}(1)$ for sharing a secret. This is obtained by choosing a bivariate polynomial of degree-(ℓ, ℓ) which supports the simultaneous sharing of $\mathcal{O}(n^2)$ secrets, thereby resulting in constant-overhead secret sharing in the amortized sense.

In their protocol, there exists a dealer D with L vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)} \in \mathbb{F}^p$. The protocol can verifiably generate $\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(L)} \rangle_\ell$ for $\ell = 2t$. The verifiability ensures that even if D is corrupt, it has generated $\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(L)} \rangle_\ell$ for vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)} \in \mathbb{F}^p$ held by D. The protocol requires a constant expected number of rounds and incurs a communication of $\mathcal{O}(Lp \log |\mathbb{F}| + n^4 \log |\mathbb{F}|)$ bits. Consequently, if $L = n^3$, then the communication overhead of the protocol with be a constant, as $p = \Theta(n)$. At a high-level, their protocol works as follows. The dealer D essentially selects sufficient number of random bivariate polynomials of degrees $2t$ in x and t in y, embeds the elements of its input vectors $\mathbf{s}^{(1)}, \ldots, \mathbf{s}^{(L)}$ in these polynomials and then verifiably distributes the rows and column polynomials of these bivariate polynomials. The verifiability guarantees that even if a potentially *corrupt* D has distributed inconsistent rows and column polynomials to some parties, they can recover their "correct" polynomials consistent on unique bivariate polynomials of appropriate degrees. Once this is done, the parties output shares lying on their row/column polynomials. The mechanism to ensure verifiability first identifies a set of "sufficiently" many honest parties that have received consistent row and column polynomials that would

define unique bivariate polynomials. Subsequently, these parties assist the remaining parties to recover their column polynomial, followed by the row polynomial on the same bivariate polynomials.

We observe that instead of using bivariate polynomials of degree-$(2t, t)$ which can pack values only along one dimension, with the relaxed threshold, we can use bivariate polynomials of degree-(ℓ, ℓ) where $\ell \in \{p-1, d, d+p-1\}$. This allows us to pack along both, the x and y dimension, thus ensuring $\mathcal{O}(n^2)$ secrets are packed in a single bivariate. Moreover, by following the same protocol as in [24], with parameters adjusted to match the number of parties in our setting, verifiability is maintained even in the presence of a potentially corrupt dealer. The increased number of honest parties under our threshold assumption enables the identification of a larger subset of parties that have received consistent row and column evaluations. These honest parties can then assist the remaining parties in recovering their respective row and column polynomials simultaneously. Here as well, if $\ell \in \{d, d+p-1\}$, then the view of the adversary in the protocol remains independent of D's inputs for an *honest* D. We refer the readers to [24] for complete formal details.

Perfect-Secure Designated Reconstruction with a Constant Overhead. Given $\langle \mathbf{s} \rangle_\ell$ where $\ell \in \{p-1, d, d+p-1\}$ and a designated party $P_R \in \mathcal{P}$, one can use the following procedure to let P_R reconstruct $\mathbf{s}$. Every party sends its share of $\langle \mathbf{s} \rangle_\ell$ to P_R, who applies the *online error-correction* (OEC) [17], obtain the underlying sharing-polynomial and outputs $\mathbf{s}$. More specifically, for $z = 0, \ldots, t$, P_R waits to receive $\ell + t + 1 + z$ points from parties and applies the RS error-correction on the received shares to error-correct up to z errors. If it does not recover a unique polynomial, then it proceeds to the next iteration. Otherwise, it verifies whether the recovered polynomial is consistent with the shares of at least $\ell + t + 1$ parties whose shares were considered during RS error-correction. If this holds, then it outputs $\mathbf{s}$ obtained from this polynomial. Otherwise, it proceeds to the next iteration.

Since $n = 4t + s$ and ℓ could be at most $t + \frac{s}{2}$, it follows that P_R will always be able to error-correct up to t errors once it receives at least $3t + s$ shares eventually. Note that in this setting as well, we do not need any MACs to identify the incorrect shares, as there is sufficient redundancy to error-correct up to t errors. The protocol will require a single round and a communication of $\mathcal{O}(n \log |\mathbb{F}|)$ bits. Since $|\mathbf{s}| = p = \Theta(n)$, the communication overhead of the protocol is constant.

Perfectly-Secure Degree Reduction with a Constant Overhead. This can be achieved by exactly following the blueprint discussed in Section 2 and outlined in Fig 1. Moreover, similar to the statistical case in the asynchronous network, the degree reduction is robust for the degree $\ell \in \{d, d+p-1\}$. In particular, given inputs $\langle \mathbf{s}^{(1)} \rangle_\ell, \ldots, \langle \mathbf{s}^{(t+s)} \rangle_\ell$, where $\ell \in \{d, d+p-1\}$, the goal is to generate $\langle \mathbf{s}^{(1)} \rangle_{p-1}, \ldots, \langle \mathbf{s}^{(t+s)} \rangle_{p-1}$ with a constant communication overhead. For this, the parties first define a vector of degree-$(t+s-1)$ polynomials $\mathbf{S}(X) = \mathbf{s}^{(1)} + \mathbf{s}^{(2)} \cdot X + \ldots + \mathbf{s}^{(t+s)} \cdot X^{t+s-1}$. The parties then locally compute $\langle \mathbf{S}(i) \rangle_\ell$ and robustly reconstruct $\mathbf{S}(i)$ towards P_i. Party P_i then generates $\langle \mathbf{S}(i) \rangle_{p-1}$ using the perfect AVSS protocol. This is followed by parties executing an instance of ACS to agree on a subset of $n - t \geq 3t + s$ parties that actually shared their $\mathbf{S}(i)$ values. This is followed by the parties error-correcting up to t incorrect vectors of points on the polynomials in $\mathbf{S}(X)$ and computing $\langle \mathbf{s}^{(1)} \rangle_{p-1}, \ldots, \langle \mathbf{s}^{(t+s)} \rangle_{p-1}$.

To achieve $\mathcal{O}(1)$ communication overhead, we need to ensure that each P_i has at least n^3 vectors of size p elements each for redistribution through VSS.[32]. Hence, we need to run the steps of above degree-reduction protocol for n^3 batches, where in each batch there are $t + s$ vectors. Thus the protocol will take inputs $\{\langle \mathbf{s}^{(L,M)} \rangle_\ell\}$ and output $\{\langle \mathbf{s}^{(L,M)} \rangle_{p-1}\}$, where $L \in \{1, \ldots, n^3\}$ denotes the batch number and $M \in \{1, \ldots, t+s\}$ denotes the input within a specific batch. Hence the input will consists of $n^3(t+s)p = \Theta(n^5)$ elements from $\mathbb{F}$. The protocol will involve nL instances of designated private reconstruction and n instances of AVSS, each for sharing L vectors. Additionally, it requires one instance of ACS which can be invoked using the protocol from [5] with a communication of $\mathcal{O}(n^5 \log n)$ bits in constant expected number of rounds. The total communication cost of the protocol will be $\mathcal{O}(n^2 L \log |\mathbb{F}| + nLp \log |\mathbb{F}| + n^5 \log |\mathbb{F}|) = \mathcal{O}(n^5 \log |\mathbb{F}|)$ bits. The protocol will require a constant expected number of rounds.

Verifiably Generating Double-Packed-Secret-Sharing with Perfect Security. Similar to the case of perfectly-secure protocol in the synchronous network, we need a protocol Π_{Double}, which enables a designated dealer $D \in \mathcal{P}$ to verifiably generate double-packed-secret-sharing (DPSS) of its vectors of inputs, which remain private if D is *honest*. This protocol will be then used to generate DPSS of random

[32] Recall that perfect AVSS achieves $\mathcal{O}(1)$ overhead only if the designated dealer has at least n^3 vectors for packed-secret-sharing.

vectors with perfect security. The protocol in the asynchronous network follows that in the synchronous network closely, with the only difference in the number of parties from whom we wait for confirmation of their vector of values. Instead of waiting for $n-t$ parties to confirm, we can now proceed if we get confirmation from up to $n-2t$ parties. This still guarantees that at least $n-3t \geq t+s$ honest parties confirmed positively, thus ensuring correctness of the shared pairs [33].

Similar to the synchronous case, to ensure constant overhead, we need to ensure that D has at least n^3 vectors for secret-sharing so that it can use the perfect AVSS for sharing them. We also need to ensure that we run the above steps for several batches of input vectors of D in parallel, so that the cost of parties publicly acknowledging the positive confirmation through broadcast is amortized. Consequently, we run the steps of the above protocol for n^2 batches, where in each batch D will have $(t+s)$ pairs of vectors each consisting of p elements. There will be $\Theta(n^3)$ instances of private reconstruction in the protocol, incurring a communication of $\mathcal{O}(n^4 \log|\mathbb{F}|)$ bits. Additionally the parties will broadcast $\mathcal{O}(n)$ bits. Note that each party performs its verification of vectors for all the n^2 batches and then broadcasts once. The protocol will output DPSS of $n^2 s$ pairs of vectors on behalf of D, which will be random if D is honest. Hence, the communication overhead of the protocol will be $\mathcal{O}(1)$, as output consists of $\Theta(n^4)$ field elements. The protocol will require a constant expected number of rounds.

Random Double-Packed-Secret-Sharing with Perfect Security and a Constant Overhead (Protocol Π_{RandPair}). Given protocol Π_{Double}, we generate DPSS of random (unknown to any party) vectors of elements using the same technique as in the perfect synchronous setting. Namely, each party P_i acts as a dealer and invokes an instance of Π_{Double} for generating DPSS of $n^2 s$ pairs of random vectors of elements. To account for potentially corrupt parties which do not perform the sharing and avoid endless wait, parties run an instance of ACS and identify a set of $n-t \geq 3t+s$ parties that actually generated DPSS. The DPSS generated by these parties are then grouped into $n^2 s$ batches, where the j^{th} batch consists of the j^{th} DPSS generated by all the parties in the set of $n-t$ parties identified by ACS. Due to the relaxed threshold in this setting, $n-t$ parties' DPSS considered in each batch is equivalent to n parties' DPSS considered in the perfectly-secure synchronous setting. Observe that there are at least $n-2t$ DPSS generated by the honest parties, which are random for the adversary. Consequently, by using randomness-extraction techniques based on polynomial interpolation, parties can locally extract DPSS of $n-2t$ random vectors from the j^{th} batch.

The protocol will involve n instances of Π_{Double} and a single instance of ACS, which will cost $\mathcal{O}(n^5 \log|\mathbb{F}|)$ bits. The protocol will output DPSS of $n^2 s \cdot (n-2t) = \Theta(n^4)$ random vectors, each consisting of p elements from $\mathbb{F}$. Hence the communication overhead of the protocol will be a constant. The protocol will require a constant expected number of rounds.

Perfectly-Secure Beaver Multiplication with a Constant Overhead (Protocol Π_{Beaver}). Similar to the case of perfect protocol in the synchronous setting (and statistical protocol in the asynchronous network), performing Beaver's multiplication over PSS is straightforward. This is because we can perform degree-reduction, both for degree-d as well as degree-$(d+p-1)$ PSS. To achieve $\mathcal{O}(1)$ communication overhead, we ensure that we have "sufficiently" many pairs of vectors to be multiplied, so that we can satisfy the constraints on the minimum number of inputs required in the instances of degree-reduction to achieve a constant overhead.

Specifically, the protocol will take degree-d PSS of $n^3(t+s)$ pairs of vectors as input and will securely compute degree-d PSS of (component-wise) product of these vectors. As before, there will be two instances of degree-reduction, one for reducing the degree of sharing from d to $p-1$ and the second, for reducing the degree of sharing from $d+p-1$ to $p-1$. Each of these instances will incur a communication cost of $\mathcal{O}(n^5 \log|\mathbb{F}|)$ bits. We will also need to generate DPSS of $n^3(t+s) = \Theta(n^4)$ vectors of random elements which cost $\mathcal{O}(n^5 \log|\mathbb{F}|)$ bits of communication. So the communication overhead of the protocol will be constant. This is because the input size (namely the number of field elements whose product is computed) is $\Theta(n^5)$. The protocol will require a constant expected number of rounds.

Perfectly-Secure Polynomial Verification with a Constant Overhead (Protocol Π_{PolyVer}).

The perfectly-secure protocol for polynomial verification in the asynchronous proceeds very closely to its synchronous counterpart with minor changes. We avoid repeating the protocol steps to avoid repetition and only describe the changes to be incorporated to adapt the protocol to this setting. Firstly,

[33] Note that waiting for confirmation from $n-t$ parties is a stronger requirement that still guarantees the required properties, since for an honest dealer, eventually $n-t$ honest parties will confirm positively.

the polynomials $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ are vectors of p polynomials of degree-ℓ, ℓ and 2ℓ respectively, where $\ell \geq t$ and *unlike* synchronous network, we have $2\ell + t + 1 \leq n - t$. Now similar to the previous case, each party P_i performs verifies $\boldsymbol{Z}(i) = \boldsymbol{X}(i) \times \boldsymbol{Y}(i)$ (component-wise) and publicly announces the result, and parties publicly verify the check for those who complain. If the complain of some party is genuine, then the dealer is discarded. Otherwise, as long as the relation is verified for $n - t \geq 2\ell + t + 1$ parties (either by parties announcing the success of verification or by public reconstruction), parties proceed to compute the output as described. The rest of the protocol proceeds exactly as in the synchronous network, with batching of many polynomials for ensuring a constant overhead.

Perfectly-Secure Verifiable Triple-Sharing with a Constant Overhead (Protocol Π_{TripSh}). The perfectly-secure triple sharing protocol is exactly the same as its synchronous counterpart, with the minor modification being that the polynomial verification invoked within it now corresponds to the perfectly-secure asynchronous polynomial verification described before. Hence, we avoid describing the steps again to avoid repetition.

Perfectly-Secure Triple-Extraction with a Constant Overhead (Protocol Π_{TripExt}). Unlike the synchronous network, in this setting, up to t parties may not share their triples using verifiable triple sharing protocol. To account for this while avoiding endless wait in a synchronous network, the triple extraction protocol takes as input the triples shared by up to $n - t$ parties. Without loss of generality, let $P_1, \ldots, P_{n-t}$ be the parties who shared their triples. Then the triple extraction protocol proceeds as follows. Given degree-d PSS of vectors of multiplication-triples $(\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)})$ for $i \in \{1, \ldots, n-t\}$, where P_i knows $(\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)})$ and which remains random for the adversary if P_i is honest, the perfectly-secure triple-extraction proceeds as follows: PSS of the input multiplication-triples are transformed into degree-d PSS of vectors of correlated multiplication-triples $(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)}, \boldsymbol{z}^{(i)})$ for $i \in \{1, \ldots, n-t\}$, such that there exist vectors $\boldsymbol{X}(\cdot), \boldsymbol{Y}(\cdot)$ and $\boldsymbol{Z}(\cdot)$ of polynomials of degree $\frac{n-t-1}{2}$, $\frac{n-t-1}{2}$ and $n-t-1$ respectively, passing through the transformed vectors of multiplication-triples. The transformation happens as in the synchronous variant. Note that adversary will know at most t vectors of points on the polynomials, leaving $\frac{n-t-1}{2} + 1 - t$ "degree-of-freedom". Consequently, the parties extend the polynomials at $\frac{n-t-1}{2} + 1 - t = \frac{t+s+1}{2}$ new points and output degree-d PSS of these vectors of points, which will be multiplication-triples and which will be random for the adversary.

To achieve a constant communication overhead, the above protocol is executed in parallel on n^3 batches and will output degree-d PSS of $n^3 \cdot (\frac{t+s+1}{2})$ vectors of p random multiplication-triples. Hence the protocol outputs $\Theta(n^5)$ random multiplication-triples and incurs a communication of $\mathcal{O}(n^5 \log |\mathbb{F}|)$ bits in a constant expected number of rounds.

Perfectly-Secure Preprocessing Phase with a Constant Overhead (Protocol Π_{Prep}).

Similar to the synchronous case, during preprocessing, every party acts as a dealer and verifiably generates degree-d PSS of vectors of random multiplication-triples using protocol Π_{TripSh}. Further, to avoid endless wait due to t corrupt parties which may not initiate triple sharing, parties run an instance of ACS to identify a set of (at least) $n - t$ parties who indeed complete their triple sharing instances. Parties then run instances of Π_{TripExt} to extract degree-d PSS of vectors of multiplication-triples which are random and unknown to any party. To achieve a constant communication overhead, we need to ensure each party secret-shares sufficiently many vectors of multiplication-triples and that there are sufficiently many vectors while running instances of Π_{TripExt}. It turns out that each party P_i has to generate PSS of $\Theta(c_M n^4)$ vectors of random multiplication-triples through Π_{TripSh}, where c_M is the number of multiplication gates in `ckt`. The parties then run $c_M n$ instances of Π_{TripExt} to extract $\Theta(c_M n^5)$ vectors of random multiplication-triples, where each vector consists of p elements. The complete protocol requires a constant expected number of rounds and incurs a total communication of $\mathcal{O}(c_M n^6)$ field elements, thus guaranteeing that the communication overhead of the protocol is constant.

As in the synchronous case, this determines the size of the inputs over which the SIMD circuit has to be evaluated achieve a constant communication overhead. Via the same analysis as earlier, the SIMD circuit will be evaluated over a batch of $\Theta(n^6)$ inputs, hence our preprocessing phase protocol can be instantiated to generate degree-d PSS of $\Theta(c_M n^5)$ vectors of random multiplication-triples, where each vector has $\Theta(n)$ field elements.

Perfectly-Secure Circuit-Evaluation with a Constant Overhead (Protocol Π_{ckteval}).

The circuit-evaluation protocol for evaluating the SIMD circuit is straight forward where the SIMD circuit has to be evaluated over batches of $\Theta(n^6)$ inputs to achieve a constant communication overhead.

Parties first run the perfectly-secure pre-processing phase protocol to generate degree-d PSS of $\Theta(c_M n^5)$ vectors of random multiplication-triples, each consisting of p elements from $\mathbb{F}$. Every party P_i then generates degree-d PSS of its $\Theta(n^6)$ inputs to the circuit. Here as well, to account for asynchrony, parties execute an instance of the ACS protocol to identify a set of $n - t$ parties which shared their inputs that will be considered in the execution. For the rest of the parties, assume a default sharing of all-zero vectors as input. The parties then start evaluating each gate in the circuit over a batch of $\Theta(n^6)$ inputs where the inputs will be degree-d packed-secret-shared. Linear gates are evaluated non-interactively. Further, multiplication gates are evaluated using the asynchronous version of perfectly-secure Beaver's protocol that consumes the secret-shared vectors of multiplication-triples generated in the pre-processing phase. Once the packed-secret-shared vectors of output are ready, they are publicly reconstructed. The protocol requires $\mathcal{O}(D_M)$ expected number of rounds. We require $|\mathbb{F}| \geq 3n$ which will ensure we have sufficiently many distinct elements to be utilized during polynomial-verification, triple-sharing and triple-extraction protocols. We summarize the result in Theorem B.2.

Theorem B.2. *For an SIMD circuit* $\texttt{ckt}$ *over a finite field* $\mathbb{F} \geq 3n$ *consisting of* c_O *output gates and having multiplicative depth* D_M*, there exists a perfectly-secure asynchronous MPC protocol for* n *parties tolerating* t *corruptions where* $n = 4t + s$*. The protocol evaluates the circuit over a batch of* $\Theta(n^6)$ *inputs from* $\mathbb{F}$*. The protocol requires* $\mathcal{O}(D_M)$ *expected number of rounds and incurs a communication of* $\mathcal{O}(|\texttt{ckt}| + c_O \cdot n)$ *field elements, provided* $s = \Theta(n)$*.*